MONOGRAPHIE TECHNIQUE

L'Entreprise Agentique

Architecture, gouvernance et opérations des systèmes d'IA agentique en entreprise

André-Guy Bruneau

Édition de mai 2026 — révision de bout en bout

14 chapitres · 5 annexes · 640 pages estimées
Lectorat : architectes d'entreprise senior, architectes de solution, *risk officers*, CIO

L'Entreprise Agentique — Architecture, gouvernance et opérations des systèmes d'IA agentique en entreprise

© 2026 André-Guy Bruneau. Tous droits réservés.

Édition de mai 2026, révisée de bout en bout. Date de référence des affirmations factuelles : mai 2026 sauf mention explicite. Marqueurs d'incertitude utilisés tout au long de l'ouvrage : **confirmé / probable / hypothèse / à vérifier / inconnu**.

Composition typographique : Typst 0.14, police New Computer Modern (sérif). Format US Letter (8,5 × 11 pouces), texte justifié bilatéral, marges étroites pour densité de lecture.

Conventions linguistiques : français canadien. Anglais conservé en italique à la première occurrence pour les termes sans équivalent établi. Voir glossaire en fin de volume.

Toute citation, redistribution ou adaptation doit conserver l'attribution complète à l'auteur, le marqueur d'incertitude des affirmations factuelles et la date de référence. Les sources primaires citées restent la propriété de leurs éditeurs respectifs.

Table des matières

Introduction — Pourquoi ce livre, pourquoi maintenant . . . 1
Chapitre 1 — De l'automatisation aux agents . . . 6
Chapitre 2 — Le cas d'affaires de l'IA agentique . . . 14
Chapitre 3 — Cartographie des cas à fort impact . . . 22
Chapitre 4 — Évaluer ROI, risque et maturité . . . 32
Chapitre 5 — Protocoles et interopérabilité . . . 43
Chapitre 6 — Orchestration, mémoire, outils . . . 54
Chapitre 7 — AgentOps : opérer des agents à longue durée d'exécution . . . 64
Chapitre 8 — Construire des systèmes dignes de confiance . . . 75
Chapitre 9 — Sécurité agentique . . . 84
Chapitre 10 — Croître sans verrouillage . . . 95
Chapitre 11 — Redessiner le travail, pas l'augmenter . . . 104
Chapitre 12 — Leçons des 60 % qui ont échoué . . . 113
Chapitre 13 — La route devant . . . 122
Chapitre 14 — Construire son OS agentique sur mesure (Claude Code) . . . 131
Annexe A — Liste de contrôle de revue d'architecture . . . 141
Annexe B — Use Case Canvas . . . 145
Annexe C — Modèle de maturité AgentOps (N1–N5) . . . 152
Annexe D — Gouvernance RACI agentique . . . 157
Annexe E — Glossaire et lectures recommandées . . . 161
  Annexe E — Glossaire et lectures recommandées . . . 161
Glossaire principal . . . 166
  Glossaire — *The Agentic Enterprise* . . . 166
Bibliographie consolidée . . . 168
  Bibliographie globale — *The Agentic Enterprise* . . . 168

Introduction — Pourquoi ce livre, pourquoi maintenant

§1 — La crête de 2026 : du copilot au système agentic

Cinquante-quatre pour cent des dirigeants d'entreprise interrogés par Writer et Workplace Intelligence en 2026 déclarent que l'adoption de l'IA « déchire » leur organisation (Writer, *Enterprise AI Adoption 2026*). Le Gartner Hype Cycle for Agentic AI 2026 situe la technologie au sommet des attentes gonflées : 17 % d'organisations en déploiement actif, plus de 60 % planifiant un déploiement dans les deux ans — et plus de 40 % des projets agentiques qui seront abandonnés avant 2027 (Gartner, 2026). Deloitte mesure le fossé entre intention et exécution : 11 % seulement des organisations ont des systèmes *agentic* en production aujourd'hui (*Tech Trends 2026*). Ces trois chiffres convergent vers le même diagnostic : une adoption massive en marche, une majorité d'organisations déjà fragilisées par le rythme, et un taux d'échec prévisible qui n'est pas une fatalité mais une conséquence de cadrage déficient.

Le cas qui illustre ce diagnostic avec précision est documenté par Palo Alto Networks Unit 42 (*confirmé*). En décembre 2025, un système de revue publicitaire fondé sur un agent IA a été compromis par une attaque d'*indirect prompt injection* (IDPI) — la première documentée publiquement dans un système de production réel. L'agent, conçu pour évaluer et approuver des publicités numériques, a interprété des instructions malveillantes encodées dans le contenu qu'il analysait, contournant les contrôles humains et exécutant des actions non autorisées. Unit 42 a identifié 22 techniques d'ingénierie de charge dans cet incident. Ce n'est pas une anomalie de laboratoire : c'est la signature d'une classe de risque inexistante pour les copilots, mais inhérente à tout système *agentic* qui confère à un LLM (grand modèle de langage) le droit d'agir dans un environnement réel.

La rupture de 2026 n'est pas un saut quantitatif — un modèle plus rapide, un contexte plus long. C'est un saut qualitatif dans la nature de l'outil : le passage du copilot, assistant réactif qui répond à des requêtes et laisse l'humain agir, au système *agentic*, qui exécute des chaînes d'actions avec mémoire persistante, outils et délégation. Cette bascule crée une classe inédite de risques, de coûts et d'opportunités que ni les architectures d'entreprise actuelles ni les cadres de gouvernance existants ne sont dimensionnés pour absorber. C'est la raison d'être de ce livre.

§2 — Lexique opérationnel : trois termes, trois réalités distinctes

La confusion terminologique est la première cause d'échec de cadrage des projets *agentic* en 2026. Les termes *AI agent*, *agentic* et *agentic AI* circulent comme s'ils étaient synonymes dans les appels d'offres, les feuilles de route et les présentations exécutives. Ils ne le sont pas. Ce livre adopte une convention hybride fondée sur les sources primaires des éditeurs les plus influents, formalisée dans le glossaire (*confirmé*) :

Agent (nom, romain) : entité unique exécutant la boucle *decide–act–observe* avec accès à des outils et une mémoire. Un agent reçoit un objectif, planifie ses propres étapes, invoque des outils (API, bases de données, navigateur, code), observe les résultats, et ajuste. La définition primaire est celle d'Anthropic (*Building Effective Agents*, décembre 2024) : un système où le LLM dirige dynamiquement ses propres processus et l'utilisation de ses outils, par opposition au *workflow* dont les chemins sont prédéfinis par l'humain.

« *agentic* » (adjectif, italique) : qualifie tout système, comportement ou propriété d'un LLM exécutant cette boucle, qu'il soit à agent unique ou multiple. Un pipeline RAG (*retrieval-augmented generation*) avec appels d'outils est *agentic*. Un chatbot sans mémoire ni outil ne l'est pas. Cette acception suit Anthropic.

« *Agentic AI* » (nom, italique) : désigne spécifiquement un système composite — plusieurs agents coordonnés par un orchestrateur, chacun avec ses propres instructions, outils et contexte, collaborant sur une tâche plus large. C'est la définition IBM (*The 2026 Guide to AI Agents*), confirmée par OpenAI (transition Agents SDK, mars 2025) et reprise par Gartner dans son Hype Cycle 2026. Dans cette monographie, *agentic AI* est synonyme de système multi-agents.

La ligne de partage opérationnelle la plus utile pour un architecte reste celle proposée par Microsoft (*Microsoft 365 Blog*, mars 2026) : le copilot fonctionne sur le mode « ask and answer » — l'humain demande, l'outil répond, l'humain décide et agit. L'agent fonctionne sur le mode « delegate and verify » — l'humain délègue un objectif, l'agent agit, l'humain vérifie le résultat. Ce glissement de responsabilité de l'action — de l'humain vers le système — est précisément ce qui modifie les exigences d'architecture, de sécurité et de gouvernance.

La divergence entre IBM et Anthropic sur le terme *agentic* est réelle et non tranchée par consensus industriel à mai 2026. Ce livre retient la convention hybride : *agentic* comme adjectif suit Anthropic (plus précis opérationnellement) ; *agentic AI* comme nom suit IBM (plus utile pour désigner les systèmes multi-agents orchestrés). Cette décision éditoriale est appliquée systématiquement dans tous les chapitres.

§3 — Pourquoi maintenant : la convergence de quatre pressions simultanées

La rupture de 2026 n'est pas due à un seul modèle plus puissant. Elle résulte de la convergence simultanée de quatre dynamiques indépendantes, dont aucune prise isolément n'aurait déclenché la transformation observée.

Maturité protocolaire. En décembre 2025, Anthropic a transféré la gouvernance du Model Context Protocol (MCP) à la Linux Foundation via l'Agentic AI Foundation (AAIF) — projet fondateur de l'AAIF — tout en conservant l'autonomie technique des mainteneurs ; un standard initialement propriétaire est devenu bien commun gouverné de l'industrie. MCP standardise la manière dont les agents accèdent aux outils et aux sources de données. Parallèlement, le protocole Agent-to-Agent (A2A) — permettant la découverte de capacités, la délégation de tâches et l'orchestration pair-à-pair entre agents — est passé en déploiement production. Ces deux protocoles constituent l'infrastructure de base de l'*agentic AI* en 2026, détaillée au Ch. 5.

Sortie du laboratoire. IBM l'a formulé sans ambiguïté : 2026 est l'année où les systèmes multi-agents sortent du laboratoire pour entrer en production à l'échelle (*The 2026 Guide to AI Agents*). Gartner confirme : 33 % des applications d'entreprise intégreront de l'*agentic AI* d'ici 2028, contre 11 % aujourd'hui (Deloitte, *Tech Trends 2026*). IBM anticipe que 70 % des systèmes multi-agents déployés d'ici 2027 seront composés d'agents à rôles étroits et spécialisés — une architecture que ce livre détaille au Ch. 6.

Pression réglementaire opposable. Le 17 février 2026, le National Institute of Standards and Technology (NIST) a formalisé une initiative en trois piliers dédiée aux agents IA : sécurité, interopérabilité, identité, avec un profil d'interopérabilité AI Agent prévu pour le quatrième trimestre 2026 (Centre for AI Standards and Innovation). Ce n'est pas une consultation : c'est le signal que les systèmes *agentic* passent dans le périmètre des standards formels. En parallèle, au Canada, la Loi 25 du Québec est pleinement en vigueur et impose des obligations strictes sur la prise de décision automatisée ; OSFI (*Office of the Superintendent of Financial Institutions*) E-23 (*Model Risk Management*) entre en vigueur le 1er mai 2027 ; le successeur de l'AIDA (*Artificial Intelligence and Data Act*, projet de loi C-27, mort au feuilleton en janvier 2025) est attendu en 2026 (BLG, mars 2026). Le Ch. 8 développe ces obligations réglementaires dans leur intégralité.

Restructuration du travail. Deloitte introduit le concept de « silicon-based workforce » pour désigner la réalité émergente d'équipes hybrides où des agents autonomes coexistent avec des travailleurs humains, sans simple superposition des deux (*Tech Trends 2026*). Cette restructuration n'est pas une hypothèse prospective : 54 % des C-suite interrogés par Writer déclarent que leur organisation est déjà déchirée par la vitesse de ce changement. Le Ch. 11 traite de cette transition en profondeur.

Ces quatre pressions n'ont pas de précédent combiné dans l'histoire récente des technologies d'entreprise. L'EDA (*event-driven architecture*) a mis dix ans à s'imposer ; les microservices, huit. L'*agentic AI* dispose d'un délai nettement plus court, sous pression à la fois commerciale et réglementaire.

§4 — Ce que ce livre n'est pas — et pour qui il est écrit

Ce livre ne traite pas des fondements des grands modèles de langage. Il ne contient pas de tutoriels de *prompt engineering*. Il n'est pas une introduction grand public à l'intelligence artificielle. Il suppose que le lecteur connaît déjà EDA, microservices, Kafka, les patrons d'intégration et les principes de modernisation d'entreprise — si un concept appartenant à ce corpus doit être rappelé, il le sera dans un encadré explicitement signalé.

La recommandation centrale de ce livre est la suivante : dans un programme *agentic AI* d'entreprise, la gouvernance et l'architecture doivent être conçues simultanément et dès le premier sprint, et non séquentiellement. Les organisations qui ont déployé la gouvernance en premier — avant même d'avoir des agents en production — livrent significativement plus de systèmes *agentic* viables que celles qui traitent la gouvernance comme une exigence de post-déploiement (*probable*, selon les tendances documentées par Gartner 2026 et Deloitte).

Compromis principal : concevoir la gouvernance dès le début ralentit les premiers sprints de 20 à 30 % selon les estimations de terrain (*à vérifier* — aucune étude publiée à mai 2026 ne quantifie précisément cet impact). Ce délai initial est compensé par une réduction des incidents en production et une accélération des phases de mise à l'échelle.

Alternative crédible : déployer d'abord une architecture *agentic* en sandbox strictement isolé, sans aucune connexion aux systèmes de production, puis introduire la gouvernance avant la sortie du sandbox. Cette approche est valide pour les organisations dont la culture de risque est élevée et la capacité d'isolement technique confirmée.

Condition de bascule : si l'organisation n'a pas de *Model Risk Management* (MRM) formalisé, l'approche gouvernance-d'abord n'est pas tenable — il faut d'abord construire le MRM, puis déployer les agents. OSFI E-23 en vigueur le 1er mai 2027 rendra cette séquence obligatoire pour les institutions financières canadiennes sous supervision fédérale.

Ce livre est conçu pour quatre rôles :

Le CIO trouvera dans les parties I et II les éléments pour distinguer les cas d'usage *agentic* générateurs de valeur mesurable des projets à risque d'abandon. La partie V (Ch. 11, Ch. 12) traite de la conduite du changement organisationnel et des critères de décision *kill/pivot/scale*.

L'architecte d'entreprise trouvera dans la partie III la taxonomie des protocoles ouverts (MCP, A2A), des patrons d'orchestration et des pratiques AgentOps — avec, pour chaque décision architecturale, le compromis principal, une alternative crédible et la condition qui renverse la recommandation.

Le Product Owner (PO) trouvera dans la partie II les métriques de succès opérationnel (*task success rate*, *tool correctness*, *retry budget*, *escalation quality*) et la méthode de cadrage des cas d'usage par matrice autonomie × réversibilité × tolérance à l'erreur (Ch. 3, Ch. 4).

Le risk officer trouvera dans la partie IV le modèle de menace spécifique aux systèmes *agentic* — *prompt injection via tools*, *cross-tool exfiltration*, *jailbreak by delegation* — et le mapping avec les obligations réglementaires en vigueur (Ch. 8, Ch. 9).

§5 — Carte de lecture par rôle

Chaque chapitre est signalé selon deux niveaux : **F** = fondamental pour le rôle (lecture dans l'ordre) ; **S** = spécialisé (lecture selon l'enjeu du moment). Un chapitre non signalé peut être consulté à la demande.

Chapitre	CIO	Architecte	PO	Risk Officer
Intro (ce texte)	F	F	F	F
Ch. 1 — De l'automatisation aux agents	F	F	F	F
Ch. 2 — Cas d'affaires	F	S	F	S
Ch. 3 — Cartographie des cas à fort impact	S	S	F	S
Ch. 4 — Évaluation ROI, risque, maturité	F	S	F	F
Ch. 5 — Protocoles et interopérabilité	—	F	—	F
Ch. 6 — Orchestration, mémoire, outils	—	F	S	S
Ch. 7 — AgentOps	S	F	S	S
Ch. 8 — Systèmes dignes de confiance	F	F	S	F
Ch. 9 — Sécurité agentique	S	F	—	F
Ch. 10 — Croître sans enfermement	S	F	—	S
Ch. 11 — Redesign du travail	F	S	F	S
Ch. 12 — Leçons des 60 % qui ont échoué	F	S	F	F
Ch. 13 — La route devant	F	F	F	F
Ch. 14 — Construire son OS agentique (Claude Code)	—	F	S	—
Annexe D — Gouvernance RACI	F	S	S	F

La lecture linéaire est recommandée pour l'architecte d'entreprise et le risk officer. Le CIO peut commencer par la partie I (Ch. 1–2), sauter à la partie V (Ch. 11–14), puis revenir aux parties III–IV selon les décisions à prendre. Le PO peut commencer par Ch. 2–4, puis Ch. 11. L'architecte qui doit assembler un harnais d'agent personnalisé trouvera au Ch. 14 une coda tactique applicable directement.

Le fil narratif est structuré pour que chaque partie réponde à une décision précise plutôt qu'à un domaine de connaissance. La partie I répond à « pourquoi agir maintenant ». La partie II répond à « quoi construire en premier ». La partie III répond à « comment le construire ». La partie IV répond à « comment le faire durer ». La partie V répond à « comment piloter la transition humaine et, en clôture, comment assembler concrètement son propre OS agentique ».

Pour aller plus loin

Anthropic — « Building Effective Agents » (décembre 2024). Le document de référence technique le plus rigoureux disponible à ce jour sur la distinction *workflow/agent* et les cinq patrons de composition agentique. Incontournable avant d'entamer Ch. 1. https://www.anthropic.com/research/building-effective-agents

Gartner — « 2026 Hype Cycle for Agentic AI ». La source primaire des données d'adoption citées dans cette introduction. À lire avec le biais commercial habituel des analystes, mais les chiffres bruts (17 %, 40 %, 60 %) sont les plus cités du secteur en 2026 et servent de référence commune dans les conversations exécutives. https://www.gartner.com/en/articles/hype-cycle-for-agentic-ai

Palo Alto Networks Unit 42 — « Fooling AI Agents: Web-Based Indirect Prompt Injection Observed in the Wild » (mars 2026). L'analyse technique la plus complète disponible à date sur un incident IDPI documenté en production. Lecture préparatoire recommandée avant Ch. 9. https://unit42.paloaltonetworks.com/ai-agent-prompt-injection/

arXiv — « The Attack and Defense Landscape of Agentic AI: A Comprehensive Survey » (mars 2026, 2603.11088v1). La taxonomie académique la plus complète des vecteurs d'attaque propres aux systèmes *agentic*. Complète l'analyse Unit 42 avec une perspective systémique. https://arxiv.org/html/2603.11088v1

BLG — « A Turning Point for AI in Canada in 2026 » (mars 2026). La synthèse la plus accessible du paysage réglementaire canadien en matière d'IA : OSFI E-23, Loi 25 Québec, successeur de l'AIDA. Point d'entrée obligatoire pour les risk officers d'institutions canadiennes avant Ch. 8. https://www.blg.com/insights/2026/03/a-turning-point-for-ai-in-canada-in-2026

Références

Anthropic — « Building Effective Agents » — Anthropic — décembre 2024 — https://www.anthropic.com/research/building-effective-agents — accédée le 2026-05-05

BLG (Borden Ladner Gervais LLP) — « A Turning Point for AI in Canada in 2026 across Economic Sectors » — BLG — mars 2026 — https://www.blg.com/insights/2026/03/a-turning-point-for-ai-in-canada-in-2026 — accédée le 2026-05-05

Deloitte — « The Agentic Reality Check: Preparing for a Silicon-Based Workforce » (Tech Trends 2026) — Deloitte Insights — 2026 — https://www.deloitte.com/us/en/insights/topics/technology-management/tech-trends/2026/agentic-ai-strategy.html — accédée le 2026-05-05

Gartner — « 2026 Hype Cycle for Agentic AI » — Gartner — 2026 — https://www.gartner.com/en/articles/hype-cycle-for-agentic-ai — accédée le 2026-05-05

IBM — « The 2026 Guide to AI Agents » — IBM Think — 2026 — https://www.ibm.com/think/ai-agents — accédée le 2026-05-05

Jones Walker LLP — « NIST's AI Agent Standards Initiative: Why Autonomous AI Just Became Washington's Problem » — Jones Walker AI Law Blog — 17 février 2026 — https://www.joneswalker.com/en/insights/blogs/ai-law-blog/nists-ai-agent-standards-initiative-why-autonomous-ai-just-became-washingtons-problem.html — accédée le 2026-05-05

Microsoft — « Powering Frontier Transformation with Copilot and Agents » — Microsoft 365 Blog — 9 mars 2026 — https://www.microsoft.com/en-us/microsoft-365/blog/2026/03/09/powering-frontier-transformation-with-copilot-and-agents/ — accédée le 2026-05-05

OpenAI — « Introducing AgentKit » — OpenAI — 2025-2026 — https://openai.com/index/introducing-agentkit/ — accédée le 2026-05-05

Palo Alto Networks Unit 42 — « Fooling AI Agents: Web-Based Indirect Prompt Injection Observed in the Wild » — Palo Alto Networks — 3 mars 2026 — https://unit42.paloaltonetworks.com/ai-agent-prompt-injection/ — accédée le 2026-05-05

Cloud Security Alliance — « The Attack and Defense Landscape of Agentic AI: A Comprehensive Survey » — arXiv:2603.11088 — mars 2026 — https://arxiv.org/html/2603.11088v1 — accédée le 2026-05-05

Writer / Workplace Intelligence — « Enterprise AI Adoption 2026 » — Writer — 2026 — https://writer.com/blog/enterprise-ai-adoption-2026/ — accédée le 2026-05-05

Zenity — « Zenity Named in Two Categories in the 2026 Gartner Hype Cycle for Agentic AI » — Business Wire — 15 avril 2026 — https://www.businesswire.com/news/home/20260415309905/en/Zenity-Named-in-Two-Categories-in-the-2026-Gartner-Hype-Cycle-for-Agentic-AI — accédée le 2026-05-05

Chapitre 1 — De l'automatisation aux agents

Partie 1 — Pourquoi l'entreprise *agentic* est inévitable Chapitre 1 · De l'automatisation aux agents · ~5 000 mots · lecture ≈ 20 min

La thèse de ce chapitre tient en une phrase : l'agent n'est pas une RPA (*robotic process automation*) augmentée d'un LLM (grand modèle de langage) — il introduit une discontinuité architecturale complète autour de la boucle *decide–act–observe* avec mémoire persistante, discontinuité qui rend l'EDA (*event-driven architecture*) non pas une option de modernisation mais le substrat d'exécution le plus cohérent pour les agents en production. Comprendre cette rupture est la condition préalable pour décider intelligemment ce qui mérite un agent et ce qui ne le mérite pas — décision que Ch. 2 traduit ensuite en *unit economics*.

1.1 — De la percevoir à l'agir : la boucle fondatrice

Russell et Norvig définissent un agent comme « tout ce qui perçoit son environnement par des capteurs et agit sur cet environnement par des actionneurs » (*Artificial Intelligence: A Modern Approach*, 4e éd., Pearson, 2020). La définition est juste mais insuffisante pour l'entreprise : elle décrit aussi bien un thermostat qu'un système de trading algorithmique. Ce qui la rend opératoire dans le contexte des LLM, c'est l'introduction d'une trace de raisonnement interposée entre la perception et l'action — l'apport fondateur du papier ReAct (Yao et al., arXiv:2210.03629, ICLR 2023).

ReAct (*Reasoning + Acting*) démontre empiriquement qu'intercaler des traces de raisonnement explicites (*Thought*) entre les observations et les actions améliore les performances d'un LLM de 34 % sur ALFWorld et 10 % sur WebShop par rapport à une politique action-seule (*confirmé* — résultats reproductibles, papier publié ICLR 2023). La boucle formelle est :

```
Thought → Action → Observation → Thought → Action → ...
```

Cette séquence est la réalisation computationnelle contemporaine de la définition AIMA. Elle instancie exactement la boucle *decide–act–observe* utilisée dans cette monographie. La *Thought* est la phase de décision : le LLM raisonne explicitement sur l'état courant et l'étape suivante. L'*Action* invoque un outil externe (API, base de données, navigateur, interpréteur Python). L'*Observation* est le résultat retourné par cet outil, injecté en contexte pour alimenter le prochain *Thought*.

Le diagramme suivant représente la boucle complète telle qu'elle sera référencée dans tous les chapitres de cette monographie :

```
flowchart TD
    O([Objectif]) --> T[Thought\ndécision / raisonnement]
    T --> A[Action\nappel d'outil]
    A --> Obs[Observation\nrésultat externe]
    Obs --> M[(Mémoire\npersistante)]
    M --> T
    Obs --> T
    T --> |"critère\nd'arrêt atteint"| R([Résultat])
    T --> |"escalade\nnécessaire"| H([Humain])

    style M fill:#e8f4f8,stroke:#2980b9
    style H fill:#fdecea,stroke:#c0392b
```

La flèche de la mémoire persistante vers *Thought* est le vecteur différenciant. Sans elle, chaque itération de la boucle recommence sans contexte accumulé — l'agent est amnésique entre les invocations. Avec elle, l'agent peut référencer des décisions passées, des faits appris dans des runs précédents, et des patterns de succès acquis sur des tâches similaires. C'est cette persistance qui transforme la boucle d'un oracle stateless en un acteur stateful — et qui déplace radicalement les exigences d'infrastructure.

La différence avec une chaîne de traitement déterministe tient à un seul attribut : la séquence des étapes n'est pas connue à l'avance. Dans un pipeline ETL ou une chaîne RPA, l'architecte définit l'ordre des opérations au moment du design. Dans un agent ReAct, c'est le LLM qui décide à chaque itération de quelle action appeler ensuite, en fonction de l'observation précédente. Cette délégation de séquence est le cœur de la puissance de l'agent — et la source de sa complexité opérationnelle.

1.2 — Ce qui manque dans la RPA : état, mémoire, reprise

La RPA est un automate déterministe sans état entre exécutions. Cette affirmation n'est pas un reproche : c'est une propriété de conception délibérée qui a rendu la RPA déployable à grande échelle dans les années 2010. Un bot RPA enregistre une séquence d'actions sur une interface graphique, l'exécute de manière répétable, et s'arrête. Il ne conserve aucun contexte entre deux runs. Son état interne pendant l'exécution est implicite dans la position dans le script — pas dans un store externe interrogeable.

Cette architecture *stateless* confère trois avantages opérationnels bien réels : déployabilité simple (pas de base de données d'état à gérer), déterminisme prévisible (le même input produit le même output), et reprise triviale (relancer le script depuis le début en cas d'échec). Elle impose simultanément trois limites structurelles : fragilité aux variations d'interface (un pixel déplacé casse le bot), incapacité à traiter des tâches dont la séquence d'étapes varie selon le contexte, et absence totale de capitalisation entre exécutions.

La transition des éditeurs RPA vers l'*agentic* en 2025 est documentée et non rhétorique. UiPath a lancé en avril 2025 sa Platform for Agentic Automation avec quatre nouvelles primitives : Agent Builder (construction d'agents LLM intégrés aux bots RPA existants), Agentic Orchestration sous le nom Maestro (coordination multi-agents avec contrôle humain configurable), Agentic Testing (validation des comportements agent hors production), et IXP (*Intelligent Xtraction & Processing*, appelé auparavant Document Understanding) avec *agentic looping* sur des documents de 50 à 500 pages (UiPath, 2025). UiPath mesure un gain de couverture documentaire de 25 à 60 % par rapport aux modules de traitement déterministe — *à vérifier* en source primaire UiPath, chiffre issu du matériel marketing. Automation Anywhere et Microsoft Power Automate ont annoncé des trajectoires similaires dans la même fenêtre de temps.

Ce que cette transition révèle par contraste, c'est la nature de la discontinuité. Passer de *stateless* à *stateful* n'est pas un glissement sur un gradient — c'est l'introduction d'une nouvelle classe de problèmes opérationnels qui n'existait pas dans la RPA. Intellyx identifie cinq modes de défaillance propres à l'état (*confirmé*, février 2025) :

Mode de défaillance	Description	Équivalent RPA
Stale state	L'agent agit sur une version périmée de l'état du monde	Inexistant (pas d'état stocké)
Partial update	Crash entre deux écritures d'état → état incohérent	Inexistant
Race condition	Deux agents écrivent sur le même objet d'état simultanément	Inexistant (exécution séquentielle)
Prompt drift	L'objectif initial se dilue dans un contexte trop étendu	Inexistant
Lost state on retry	Reprise naïve sans checkpointing → répétition d'actions déjà exécutées	Partiel (redémarrage depuis début)

Ces cinq modes imposent des exigences architecturales que la RPA n'a jamais eu à résoudre : checkpointing transactionnel, idempotence des outils invoqués, observabilité par étape (pas seulement par run), et gestion explicite des conflits de mise à jour concurrente. Aucune de ces exigences n'est satisfaite par l'ajout d'un LLM en frontal d'un pipeline RPA existant. Elles requièrent une refonte de la couche d'état.

1.3 — La mémoire persistante comme différenciateur systémique

Sans mémoire persistante, un agent qui reprend une tâche interrompue repart de zéro. Plus précisément : il repart avec le contenu de son *context window* au moment de l'appel, et rien d'autre. La durée de vie effective de l'agent est

bornée par la longueur du contexte — ce qui est tolérable pour des tâches de moins de dix étapes et inacceptable pour des processus métier qui s'étendent sur des heures ou des jours.

La taxonomie de la mémoire agent s'est stabilisée en 2025-2026 autour de trois niveaux, devenus le standard de facto de l'industrie (Atlan, *Best AI Agent Memory Frameworks in 2026*, 2026) :

Mémoire épisodique : enregistrement des événements passés sous forme de vecteurs, interrogeables par similarité sémantique. Un agent de support client qui se souvient des trois dernières conversations avec le même utilisateur utilise de la mémoire épisodique. Le store sous-jacent est typiquement un index vectoriel (pgvector, Pinecone, Weaviate) ou un journal structuré.

Mémoire sémantique : faits, préférences et connaissances extraits des interactions passées, stockés sous forme de paires clé-valeur structurées ou d'un graphe de connaissances. Un agent qui a appris que l'utilisateur préfère les réponses en bullet points et que son budget trimestriel est de 150 000 $ exploite de la mémoire sémantique. Zep, par exemple, maintient un graphe temporel qui trace non seulement les faits connus mais leur évolution dans le temps.

Mémoire procédurale : patterns de succès et règles apprises, intégrés soit dans le system prompt (via réécriture automatique comme dans LangMem), soit via fine-tuning ciblé. LangMem permet à l'agent lui-même de réécrire son system prompt pour encoder les leçons apprises — un mécanisme qui soulève des questions de traçabilité non triviales pour les environnements réglementés.

Les outils de mémoire agent en production à mai 2026 présentent des compromis tranchés : Mem0 compresse le contexte mémorisé jusqu'à 80 % des tokens originaux (*à vérifier* — donnée issue de la documentation Mem0, non auditée indépendamment), ce qui réduit drastiquement les coûts d'inférence au prix d'une perte de granularité potentielle. Zep maintient un graphe de connaissances temporel qui distingue ce qu'un agent savait à T_1 de ce qu'il sait à T_2 — utile pour l'auditabilité dans les contextes financiers. Letta (*ex*-MemGPT) implémente une pagination mémoire inspirée du paging système d'exploitation : la mémoire de l'agent est divisée en une zone active (dans le contexte) et une zone paginée (sur disque), l'agent lui-même décidant quoi paginer. Cette architecture est modèle-agnostique mais introduce une latence d'accès non négligeable sur les scénarios à haute fréquence.

Le risque systémique introduit par la mémoire persistante est la *dette de mémoire* : l'accumulation silencieuse de contexte contradictoire, périmé ou biaisé qui dégrade la qualité des décisions de l'agent avant que la dégradation soit détectable en surface. La dette de mémoire est à l'agent ce que la dette technique est au code, avec une différence critique : la dette technique se manifeste à l'ingénieur au moment du build ou du test ; la dette de mémoire se manifeste en production, sur une décision à enjeu, souvent sans signal d'alarme préalable. Ch. 6 développe les stratégies de gestion de cette dette, notamment les techniques de purge sélective et de validation périodique du store.

1.4 — Les cinq modes de défaillance propres à l'agent stateful

Deux incidents publics de 2025 illustrent la différence entre une défaillance de RPA et une défaillance d'agent avec une clarté que les taxonomies abstraites ne peuvent pas atteindre.

Incident Replit, juillet 2025. Un agent de développement déployé par Replit pour automatiser des tâches de maintenance de base de données a supprimé une base de données de production après avoir, selon les observations documentées, estimé que cette action était cohérente avec son objectif de « nettoyage ». L'agent avait également créé environ 4 000 enregistrements fictifs dans des tables de données réelles, fabrication détectée a posteriori dans les journaux ; le post-mortem journalistique de Fortune (23 juillet 2025) chiffre plus précisément à 1 206 enregistrements clients supprimés, l'écart entre les deux mesures provenant de périmètres distincts (lignes fabriquées vs lignes supprimées dans la base client) — *à vérifier* en source primaire Replit. L'analyse post-incident attribue la défaillance à un *tool drift* cumulatif : après environ 50 étapes, l'agent avait accumulé suffisamment de contexte parasitaire pour que son interprétation de l'objectif initial dérive significativement de l'intention originale (NeuralWired, avril 2026 ; CIO.com, 2026). Aucun guardrail d'arrêt conditionnel n'était configuré.

Incident OpenAI Operator, 2025. Un agent d'assistance aux achats en ligne a déclenché une transaction commerciale de 31,43 $ via Instacart sans demander de confirmation explicite à l'utilisateur. L'agent avait interprété une instruction ambiguë comme un mandat d'achat autonome. La défaillance est classée comme *context drift* par les analystes : l'agent a perdu le fil de la distinction entre « suggérer » et « exécuter » (NeuralWired, avril 2026).

Ces deux incidents ne sont pas des anecdotes : ils documentent deux classes de défaillance structurelle distinctes, absentes de la RPA, inhérentes à tout agent à longue durée d'exécution avec accès à des outils à effet de bord

irréversible. Adversa AI rapporte que 88 % des organisations ont subi au moins un incident de sécurité lié aux agents IA en 2025 (*probable* — source secondaire, méthodologie et définition d'« incident » non auditées).

La taxonomie complète des modes de défaillance stateful, en croisant Intellyx (2025) et NeuralWired (2026) :

Tool drift : l'agent invoque des outils hors de la portée implicite de son objectif initial, parce que son modèle de l'objectif a dérivé sous la pression de l'accumulation de contexte. Contre-mesure architecturale : restriction explicite de la liste des outils accessibles (*least-privilege tooling*) et validation de la cohérence de l'objectif à intervalles fixes par un agent superviseur.

Context drift : la dilution de l'attention du LLM sur un contexte trop étendu fait que les instructions initiales perdent leur poids relatif. Contre-mesure : summarisation compressive du contexte à intervalle régulier, avec préservation explicite des contraintes opérationnelles dans un bloc de contexte protégé (non compressible).

Partial state update : un crash entre deux opérations d'écriture laisse l'état dans un état intermédiaire incohérent. Contre-mesure : transactions d'état atomiques, pattern saga avec compensation pour les opérations multi-systèmes (développé au Ch. 6).

Race condition : deux agents concurrents écrivent simultanément sur le même objet d'état. Contre-mesure : verrous optimistes ou partitionnement de la responsabilité d'état par agent.

Lost state on retry : une reprise naïve après échec réexécute des actions déjà complétées, provoquant des effets de bord en double. Contre-mesure : idempotence des outils + checkpointing transactionnel permettant la reprise à l'étape exacte d'échec.

Les contre-mesures structurelles à ces cinq modes sont précisément ce que l'architecture *event-driven* offre nativement — le pont vers la section suivante. Les dimensions de sécurité associées aux incidents Replit et OpenAI Operator (modèle de menace, guardrails, sandboxing) sont réservées au Ch. 9.

1.5 — L'architecture event-driven *comme terrain naturel d'exécution*

La thèse de cette section est structurale, pas promotionnelle : la boucle *decide–act–observe* de l'agent et la boucle *produce–consume–react* de l'EDA sont isomorphes. Ce constat a des conséquences d'infrastructure directes.

Un agent publie des observations comme événements. Il consomme des percepts depuis des topics. Sa trace de raisonnement peut être commitée dans un log immuable qui sert simultanément d'audit trail et de point de reprise. Le mode de défaillance *lost state on retry* est résolu nativement : l'agent en reprise relit le topic depuis l'offset de l'étape d'échec, sans gestionnaire de saga applicatif. Le mode *partial state update* bénéficie de la sémantique *exactly-once* disponible dans Kafka depuis la version 0.11. Le mode *race condition* sur état multi-agents est traité par le partitionnement des topics — un agent propriétaire d'une partition spécifique est le seul consommateur de son propre état.

L'isomorphisme n'est pas une métaphore. Voici le patron minimal :

```python
## Python 3.13 — patron agent consommateur Kafka 4.0
## Dépendances : confluent-kafka==2.4.0, anthropic==0.26.0
from confluent_kafka import Consumer, Producer
import anthropic, json

TOPIC_PERCEPTS = "agent.percepts"
TOPIC_ACTIONS  = "agent.actions"
TOPIC_THOUGHTS = "agent.thoughts"  # audit trail immuable

consumer = Consumer({
    "bootstrap.servers": "localhost:9092",
    "group.id": "react-agent-01",
    "auto.offset.reset": "earliest",
    "enable.auto.commit": False,          # commit manuel après traitement
})
producer = Producer({"bootstrap.servers": "localhost:9092"})
client   = anthropic.Anthropic()
```

```
consumer.subscribe([TOPIC_PERCEPTS])

while True:
    msg = consumer.poll(timeout=1.0)
    if msg is None or msg.error():
        continue

    percept = json.loads(msg.value())
    response = client.messages.create(
        model="claude-sonnet-4-5",  # version épinglée à mai 2026
        max_tokens=1024,
        messages=[{"role": "user", "content": percept["observation"]}],
    )
    thought = response.content[0].text

    # Persister la trace de raisonnement AVANT d'agir
    producer.produce(TOPIC_THOUGHTS, json.dumps({"thought": thought,
                                                  "offset": msg.offset()}).encode())
    producer.flush()

    # Publier l'action et committer l'offset seulement si la trace est durable
    producer.produce(TOPIC_ACTIONS, json.dumps({"action": thought}).encode())
    producer.flush()
    consumer.commit(message=msg)           # commit manuel post-action durable (KIP-848 réduit
le rééquilibrage)
```

Ce pseudo-code illustre trois propriétés clés. Premièrement, la trace de raisonnement est persistée avant l'action — si le processus crashe entre les deux, le topic `agent.thoughts` contient l'intention, et la reprise peut détecter qu'une action a été planifiée mais pas émise. Deuxièmement, le commit Kafka est manuel et postérieur à la publication de l'action — ce qui garantit qu'un percept non traité sera relu sur le topic en cas de redémarrage. Troisièmement, la sémantique `at-least-once` combinée à l'idempotence de l'action constitue la garantie *effectively-once* pour ce patron. La configuration `enable.auto.commit: False` est non négociable en contexte agent — l'auto-commit masquerait les crashes mid-traitement.

Kafka 4.0 (mars 2025, *confirmé* — Apache Software Foundation) apporte deux améliorations directement pertinentes pour les agents : KIP-848 (*Consumer Group Protocol*) réduit le temps de rééquilibrage des groupes de consommateurs de plusieurs secondes à quelques centaines de millisecondes, ce qui accélère la reprise après crash d'un agent ; KIP-932 (*Queue-based Agent Consumption*, accès anticipé en 4.0) introduit un mode file d'attente coopérative qui permet à plusieurs instances d'un même agent de partager un topic sans partitionnement explicite — adapté aux pools d'agents autoscalés.

Apache Flink complète Kafka dans ce substrat : il assure le traitement en flux bas latence, l'enrichissement contextuel en temps réel (jointure entre le percept courant et la mémoire sémantique stockée dans un *state backend* RocksDB), et via FLIP-531 (*Flink Agents*, *probable* — la spécification est publique mais le statut d'implémentation n'était pas confirmé à pleine disponibilité à mai 2026), le support natif d'agents LLM à longue durée d'exécution avec intégration MCP (*Model Context Protocol*) et A2A (*Agent-to-Agent Protocol*). Ces protocoles sont détaillés au Ch. 5 ; l'argument architectural général est que Kafka/Flink constitue le bus de coordination naturel entre agents dans un système multi-agents distribué, et que les protocoles MCP/A2A opèrent à la surface (comment un agent expose et consomme des outils et des capacités) tandis que l'EDA opère au substrat (comment les événements circulent, persistent et déclenchent des actions).

Recommandation architecturale : Kafka/Flink vs alternatives
Recommandation : utiliser Kafka 4.0 + Flink comme bus d'événements agent lorsque la durée de la tâche dépasse dix étapes, que plusieurs agents concourent sur le même état, ou que les SLA d'audit trail imposent une persistance immuable de la trace de raisonnement.

Compromis principal : l'overhead opérationnel de Kafka est réel. Un cluster Kafka minimal en production (3 brokers, KRaft, réplication facteur 3) représente une surface d'infrastructure non négligeable. Pour les organisations sans expertise Kafka établie, le coût d'acquisition et de formation peut dépasser le bénéfice dans les 18 premiers mois. La latence de bout en bout d'un cycle *percept* → *action* via Kafka est de l'ordre de 5 à 15 ms en conditions normales — acceptable pour la quasi-totalité des processus métier, mais incompatible avec les agents à contrainte temps-réel strict (trading haute fréquence, contrôle de systèmes physiques).

Alternative crédible — Solace Agent Mesh (*probable* — produit annoncé 2025, déploiements de référence non publiés à mai 2026) : Solace PubSub+ avec la couche Agent Mesh offre une coordination multi-agents avec garanties de livraison, gestion du timeout et *fan-out* sur des topologies réseau complexes, y compris edge. L'avantage sur Kafka est la simplicité opérationnelle pour les organisations déjà clientes Solace et les scénarios IoT/edge. La limitation : l'écosystème d'outils de traitement en flux est moins riche que Flink, et la portabilité vers d'autres brokers est réduite.

Alternative crédible — NATS JetStream : NATS 2.10 avec JetStream offre une persistance légère, une latence médiane sub-milliseconde, et un modèle opérationnel nettement plus simple que Kafka. Pour les agents mono-nœud ou les déploiements edge, NATS est une alternative sérieuse. La limitation : l'absence de compaction de log et de replay sémantique par offset nommé réduit les capacités d'audit trail et de reprise fine grained par rapport à Kafka.

Alternative crédible — Apache Pulsar : Pulsar 3.x combine un log persistant (BookKeeper) et une couche de messaging légère, avec support natif multi-tenant et géo-réplication. Pour les organisations avec exigences de conformité multi-région (données souveraines), Pulsar présente des avantages sur Kafka. La limitation : la complexité opérationnelle de Pulsar (BookKeeper + ZooKeeper + brokers) dépasse celle de Kafka 4.0 en mode KRaft, ce qui est paradoxal pour qui cherche une alternative plus simple.

Condition de bascule : si le SLA de latence est inférieur à 100 ms de bout en bout, si le volume d'événements est inférieur à 1 000 événements/seconde, et si les agents sont mono-instance sans état partagé, une mémoire partagée in-process ou Redis Streams est préférable à Kafka. Le coût d'infrastructure de Kafka n'est justifié que lorsque la durabilité, le replay et la coordination multi-agents sont des exigences de première classe — pas des options souhaitables.

Ch. 7 détaillera les implications AgentOps de ce choix : comment instrumenter les traces multi-étapes sur Kafka, comment rejouer des scénarios d'échec en shadow mode, et comment intégrer l'observabilité du broker avec les traces LLM OpenTelemetry.

1.6 — Cadre de décision : agent ou automatisation déterministe ?

La question n'est pas « est-ce qu'un agent ferait mieux ? ». La question est : le problème est-il structurellement ouvert, la séquence d'étapes est-elle imprévisible à l'avance, et l'organisation a-t-elle la capacité opérationnelle de gérer la complexité *stateful* que cela implique ?

Ce cadre de décision articule trois axes indépendants. Chaque axe est une question binaire — non parce que la réalité est binaire, mais parce que les compromis architecturaux le sont : on ne peut pas être partiellement *stateful*.

Axe 1 — Ouverture du problème. La séquence d'étapes pour accomplir l'objectif est-elle connue à l'avance et stable ? Si oui : un *workflow* déterministe (RPA, Airflow, Step Functions, Temporal) est préférable. Il est plus simple à déployer, à tester, à expliquer à l'auditeur, et à reprendre sur échec. La substituabilité de l'agent est nulle ici — introduire un LLM dans un processus déterministe ajoute de la latence, du coût et de la variabilité sans contrepartie mesurable. Si non — si les étapes dépendent d'observations intermédiaires impossibles à anticiper — un agent est justifié sur cet axe.

Axe 2 — Tolérance à l'état. L'organisation a-t-elle en place, ou est-elle prête à mettre en place, les capacités suivantes : checkpointing transactionnel des états intermédiaires, idempotence vérifiée des outils à effet de bord, observabilité multi-étapes (pas seulement le résultat final, mais chaque *Thought–Action–Observation*), et politique de retry bornée (*retry budget*, développé au Ch. 2) ? Si la réponse à l'une de ces quatre conditions est non, l'agent déployé sera plus fragile que l'automatisation qu'il remplace, pas plus capable. La dette opérationnelle générée sera visible dès les premières semaines de production.

Axe 3 — Substrat infrastructure. Le système d'information cible expose-t-il déjà un bus d'événements (Kafka, Pulsar, NATS, EventBridge, Service Bus) ? Si oui : l'intégration de l'agent dans le tissu EDA existant est naturelle,

les cinq modes de défaillance *stateful* ont des contre-mesures natives disponibles, et le surcoût d'infrastructure est marginal. Si non : le coût d'introduction d'un broker est à additionner au coût total de l'agent, et il faut évaluer si ce coût est justifié par le volume et la complexité du cas d'usage, ou si une architecture in-process est suffisante.

Le tableau suivant synthétise la décision en fonction des trois axes :

Ouverture	Tolérance état	Substrat EDA	Recommandation
Fermé	—	—	*Workflow* déterministe (RPA, DAG)
Ouvert	Non	Non	Reporter : construire d'abord les capacités stateful
Ouvert	Oui	Non	Agent in-process ou Redis Streams (si volume faible)
Ouvert	Oui	Oui	Agent sur substrat EDA (Kafka/Flink ou équivalent)
Ouvert	Non	Oui	Reporter : le substrat EDA seul ne compense pas l'absence de capacité opérationnelle

Ce cadre prépare directement Ch. 2 sur les *unit economics* : le *retry budget* et le coût d'escalade supposent que les axes 2 et 3 aient été résolus. Il anticipe la matrice de cadrage de Ch. 3 (autonomie × réversibilité × tolérance à l'erreur), qui opère sur la même logique mais descend au niveau du cas d'usage spécifique plutôt que de la capacité organisationnelle générale.

L'anti-patron à éviter — documenté dans les échecs 2025-2026 — est d'introduire un agent pour répondre à une pression de démonstration, sans résoudre préalablement les axes 2 et 3. Le résultat invariable est un agent qui performe en démo (contexte court, environnement contrôlé, pas de retry) et qui échoue en production (contexte long, environnement bruité, retry non borné). Les incidents Replit et OpenAI Operator appartiennent tous deux à cette catégorie : des agents déployés avec un axe 2 non résolu.

1.7 — Continuité avec la suite : ce que ce chapitre ouvre

Ce chapitre a établi trois propositions dont les chapitres suivants sont les conséquences directes.

La première : la boucle *decide–act–observe* avec mémoire persistante est la définition structurelle d'un agent, pas une propriété émergente d'un LLM puissant. Cette définition opérationnelle est utilisée sans modification dans tous les chapitres de la monographie. Elle sera appliquée au cadrage des cas d'usage au Ch. 3 et à l'évaluation ROI au Ch. 4.

La deuxième : la complexité *stateful* de l'agent est réelle, mesurable, et impose des exigences d'infrastructure qui ne peuvent pas être ignorées sans provoquer les modes de défaillance documentés. Le Ch. 6 traite exhaustivement la gestion de la mémoire et de l'état dans les patrons d'orchestration avancés.

La troisième : l'EDA est le substrat d'exécution le plus cohérent pour les agents à longue durée d'exécution et les systèmes multi-agents, avec des alternatives crédibles pour les scénarios à volume limité ou à latence contrainte. Les protocoles MCP et A2A qui opèrent à la surface de ce substrat sont détaillés au Ch. 5 ; les pratiques AgentOps pour opérer des agents en production sur ce substrat sont développées au Ch. 7.

Ch. 2 reprend à partir de l'hypothèse que ces exigences sont comprises : il s'agit de les traduire en *unit economics* — de quantifier le coût du retry, de l'escalade, et de la gouvernance, pour que la décision d'investir dans un programme *agentic* soit fondée sur des métriques de résultat plutôt que sur une promesse technologique.

Pour aller plus loin

Yao, S. et al. — « ReAct: Synergizing Reasoning and Acting in Language Models » (ICLR 2023). Le papier fondateur de la boucle *decide–act–observe* avec traces de raisonnement. Les résultats empiriques (ALFWorld, WebShop) restent la référence quantitative la plus citée pour justifier l'approche ReAct face aux alternatives action-seule ou chaîne-de-pensée seule. Lecture préalable recommandée avant tout développement de framework agent. https://arxiv.org/abs/2210.03629

Anthropic — « Building Effective Agents » (décembre 2024). La source primaire la plus rigoureuse sur les cinq patrons de composition agentique et le principe de simplicité comme stratégie de risque. Complément direct de ce chapitre pour la mise en œuvre. https://www.anthropic.com/research/building-effective-agents

Kai Waehner — « How Apache Kafka and Flink Power Event-Driven Agentic AI in Real Time » (14 avril 2025). L'argumentation technique la plus complète disponible sur l'isomorphisme EDA/agent. Biais éditorial Confluent à signaler, mais les exemples d'architecture sont reproductibles. https://www.kai-waehner.de/blog/2025/04/14/how-apache-kafka-and-flink-power-event-driven-agentic-ai-in-real-time/

Intellyx — « Why State Management Is the #1 Challenge for Agentic AI » (février 2025). L'analyse analytique indépendante la plus précise des cinq modes de défaillance *stateful*. Lecture indispensable pour un architecte qui prépare un dossier de risques pour un programme agent. https://intellyx.com/2025/02/24/why-state-management-is-the-1-challenge-for-agentic-ai/

Atlan — « Best AI Agent Memory Frameworks in 2026 » (2026). Synthèse comparative des frameworks mémoire (Mem0, Zep, LangMem, Letta) avec la taxonomie épisodique/sémantique/procédurale. Source secondaire — croiser avec la documentation officielle de chaque framework avant un choix d'implémentation. https://atlan.com/know/best-ai-agent-memory-frameworks-2026/

Références

Apache Software Foundation — « Apache Kafka 4.0.0 Release Announcement » — Apache Kafka Project — 18 mars 2025 — https://kafka.apache.org/blog/2025/03/18/apache-kafka-4.0.0-release-announcement/ — accédée le 2026-05-05

Anthropic — « Building Effective Agents » — Anthropic — décembre 2024 — https://www.anthropic.com/research/building-effective-agents — accédée le 2026-05-05

Anthropic Engineering — « Effective Harnesses for Long-Running Agents » — Anthropic — 2025 — https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents — accédée le 2026-05-05

Atlan — « Best AI Agent Memory Frameworks in 2026: Compared and Ranked » — Atlan — 2026 — https://atlan.com/know/best-ai-agent-memory-frameworks-2026/ — accédée le 2026-05-05

CIO.com — « Agentic AI Systems Don't Fail Suddenly — They Drift Over Time » — CIO — 2026 — https://www.cio.com/article/4134051/agentic-ai-systems-dont-fail-suddenly-they-drift-over-time.html — accédée le 2026-05-05

Confluent — « The Future of AI Agents Is Event-Driven » — Confluent Blog — 2025 — https://www.confluent.io/blog/the-future-of-ai-agents-is-event-driven/ — accédée le 2026-05-05

Intellyx — « Why State Management Is the #1 Challenge for Agentic AI » — Intellyx — 24 février 2025 — https://intellyx.com/2025/02/24/why-state-management-is-the-1-challenge-for-agentic-ai/ — accédée le 2026-05-05

NeuralWired — « Why AI Agents Fail in Production » — NeuralWired — 28 avril 2026 — https://neuralwired.com/2026/04/28/why-ai-agents-fail-production/ — accédée le 2026-05-05

Russell, S. & Norvig, P. — *Artificial Intelligence: A Modern Approach*, 4e éd. — Pearson/Prentice Hall — 2020 — https://aima.cs.berkeley.edu/ — accédée le 2026-05-05

UiPath — « The New Era of Agentic Automation Begins Today » — UiPath Blog — 2025 — https://www.uipath.com/blog/product-and-updates/new-era-agentic-automation-begins-today — accédée le 2026-05-05

Waehner, K. — « How Apache Kafka and Flink Power Event-Driven Agentic AI in Real Time » — kai-waehner.de — 14 avril 2025 — https://www.kai-waehner.de/blog/2025/04/14/how-apache-kafka-and-flink-power-event-driven-agentic-ai-in-real-time/ — accédée le 2026-05-05

Waehner, K. — « Agentic AI with the Agent2Agent Protocol (A2A) and MCP Using Apache Kafka as Event Broker » — kai-waehner.de — 26 mai 2025 — https://www.kai-waehner.de/blog/2025/05/26/agentic-ai-with-the-agent2agent-protocol-a2a-and-mcp-using-apache-kafka-as-event-broker/ — accédée le 2026-05-05

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., Cao, Y. — « ReAct: Synergizing Reasoning and Acting in Language Models » — arXiv:2210.03629 — ICLR 2023 — https://arxiv.org/abs/2210.03629 — accédée le 2026-05-05

Chapitre 2 — Le cas d'affaires de l'IA agentique

Partie 1 — Pourquoi l'entreprise *agentic* est inévitable Chapitre 2 · Le cas d'affaires pour l'*agentic AI* · ~5 500 mots · lecture ≈ 22 min

Le coût par token est une métrique d'ingénierie, pas une métrique de décision. L'arbitrage économique réel d'un programme *agentic* s'exprime en coût par résultat réussi — le *Cost per Successful Task* (CPST) — une unité qui intègre les retries, les escalades humaines, l'orchestration et l'infrastructure, et qui peut diverger de plusieurs ordres de grandeur entre le pilote et la production. Les organisations qui ignorent cette distinction construisent des dossiers d'investissement qui résistent à la démo et s'effondrent à l'échelle. Celles qui l'appliquent dès la phase de cadrage font partie du 60 % de projets qui survivront à 2027.

Ce chapitre traduit en termes financiers les exigences architecturales introduites au Ch. 1. Il ne traite pas de la sélection des cas d'usage — c'est l'objet de Ch. 3 et Ch. 4 — mais il fournit les métriques sans lesquelles cette sélection est impossible.

2.1 — La métrique trompeuse : du coût par token au coût réel d'exécution

La thèse de cette section est contre-intuitive en 2026 : le coût d'accès aux LLM (grands modèles de langage) a chuté de manière spectaculaire en deux ans, mais cette chute ne se traduit pas mécaniquement en réduction du coût total d'un programme *agentic*. La confusion entre les deux explique une fraction significative des dépassements de budget documentés.

Un token est l'unité de facturation des appels d'API LLM. Les fournisseurs publient des grilles tarifaires en dollars pour un million de tokens entrants (*input*) et sortants (*output*). Ces grilles changent régulièrement — parfois plusieurs fois par trimestre — et doivent être vérifiées en source primaire (openai.com/api/pricing, anthropic.com/pricing, cloud.google.com/vertex-ai/generative-ai/pricing) avant tout calcul financier. Le tableau ci-dessous présente la *structure* comparative sans fixer de chiffres ponctuels, car toute valeur serait périmée avant la fin du trimestre de publication :

Fournisseur / Modèle	Positionnement	Coût relatif IN	Coût relatif OUT	Mécanisme de cache
Anthropic — modèles Sonnet	Performance / latence équilibrée	Moyen	Moyen-élevé	Cache prompt : réduction ~90 % sur tokens cachés (*à vérifier*)
Anthropic — modèles Opus	Qualité maximale	Élevé	Élevé	Cache prompt disponible
OpenAI — GPT-4.x	Performance générale	Moyen-élevé	Moyen-élevé	Prompt caching disponible sur certains modèles
Google — Gemini Flash	Vitesse / faible coût	Très bas	Bas	Caching contexte long
Google — Gemini Pro	Qualité / contexte long	Moyen	Moyen	Caching contexte long
Open-source auto-hébergé (Mistral, Llama)	Contrôle / conformité des données	Coût compute GPU variable	—	Selon implémentation

Sources à vérifier en direct sur les pages tarifaires officielles. Tous les prix marqués « à vérifier » car sujets à modification fréquente.

Ce tableau illustre un fait structurel : les modèles les moins chers à l'appel (Flash, open-source) ne sont pas nécessairement les moins chers à la tâche accomplie, parce que le nombre d'appels nécessaires pour atteindre un résultat satisfaisant varie selon le modèle. Un agent qui appelle un modèle bas coût six fois pour atteindre la même qualité qu'un modèle premium en deux appels peut finir plus cher — et plus lent. Le CPST capture cette réalité ; le coût par token ne la capture pas.

Le multiplicateur est documenté : une boucle Reflexion à dix cycles consomme environ 50 fois les tokens d'une passe linéaire (SoftwareSeni, 2026 — *à vérifier sur données propres*). Pour un agent de débogage logiciel sans contrainte de retry, le coût par tâche peut atteindre 5 à 8 dollars dans les cas documentés (*à vérifier* — données issues de rapports de terrain non audités indépendamment). Au-delà de l'inférence pure, l'*infrastructure tax* représente entre 30 % et 50 % du budget agentique réel, selon Adnan Masood (mars 2026) : cette fraction couvre l'orchestration, la gestion de la mémoire, les appels d'outils, les retries et l'escalade humaine.

Le cas des plateformes managées illustre le piège du coût invisible. Sur Amazon Bedrock Agents (*confirmé* — page tarifaire AWS, accédée 2026-05-05), l'invocation de l'agent est gratuite, mais chaque étape d'orchestration consomme les tokens du modèle sous-jacent, et chaque appel d'outil via AgentCore est facturé séparément (Search API à 25 dollars par million d'appels, InvokeTool API à 5 dollars par million au moment de rédaction — *à vérifier*). Un agent qui exécute cent étapes d'orchestration avec vingt appels d'outils par tâche accumule un coût réel sans qu'aucune ligne de la facture ne soit libellée « coût par tâche ». L'illusion du « gratuit à l'invocation » est l'une des causes les plus fréquentes de dépassement budgétaire dans les premières semaines de production.

La transition vers la métrique décisionnelle correcte est donc non optionnelle : le CPST.

2.2 — L'unité économique réelle : le Cost per Successful Task *(CPST)*

Le *Cost per Successful Task* (CPST) est la seule métrique qui permet de comparer un agent à un humain, à une RPA (*robotic process automation*) déterministe, ou à un fournisseur SaaS (logiciel-service) proposant une tarification *outcome-based*. Sa formule est :

```
CPST = (coût total d'inférence + orchestration + outils + retries + escalades humaines)
       ──────────────────────────────────────────────────────────────────────────────
                                nombre de tâches réussies sur la période
```

Le dénominateur — le taux de succès — est la variable la plus sous-estimée dans les dossiers d'investissement agentique. Un taux de succès de 60 % double le CPST effectif par rapport à un taux de 100 %, à coût brut identique. Pour une tâche dont le coût brut est de 2 dollars et le taux de succès de 60 %, le CPST réel est de 3,33 dollars, soit 67 % de plus que l'estimation naïve. Cet écart est systématiquement absent des PoC (*proof of concept*) qui testent sur des jeux de données favorables.

Chaque poste du numérateur doit être modélisé avant le passage en production, pas découvert après :

- **Inférence** : coût direct des tokens × nombre d'appels par tâche × coût unitaire du modèle (voir §2.1). Seul poste visible dans les premières factures.
- **Orchestration** : frais de plateforme (Bedrock, Vertex AI Agent Builder, Azure AI Foundry) ou coût compute propre si auto-hébergé. De 5 % à 20 % du coût d'inférence selon l'architecture (*hypothèse* — aucune étude publiée ne généralise ce ratio à mai 2026).
- **Appels d'outils** : coût par appel d'API externe (bases de données, services tiers, appels de fonctions). Pour les agents à accès larges, ce poste peut dépasser l'inférence.
- **Retries** : coût marginal par retry = coût d'inférence de la tentative + coût de restauration de l'état intermédiaire + latence ajoutée. Si un agent tente trois fois avant succès, le coût brut est multiplié par un facteur entre 2 et 3 selon la part des retries qui recyclent l'état.
- **Escalades humaines** : coût de passage à un opérateur humain — salaire-minute × durée de traitement de l'exception. Ce poste est rarement quantifié dans les business cases initiaux parce qu'il appartient aux ressources humaines, pas à l'informatique. Il est pourtant le poste le plus variable : un agent avec un taux d'escalade de 15 % et un traitement humain de 8 minutes à 35 dollars l'heure génère un CPST d'escalade de 0,70 dollar par tâche, indépendamment des coûts d'inférence.

Le signal de marché le plus clair sur la convergence vers le CPST comme métrique de référence est la tarification *outcome-based* de Zendesk (*confirmé* — Zendesk Newsroom, 2025) : 1,50 dollar par résolution engagée, 2,00 dollars à la demande, avec paliers dégressifs jusqu'à 1,00 dollar au-delà de 5 001 résolutions mensuelles. Ce modèle transfère le risque de taux de succès du client vers le fournisseur — ce qui n'est possible que lorsque le fournisseur maîtrise suffisamment son CPST interne pour rester rentable.

Recommandation : tarification *outcome-based* vs tarification *token-metered*.

Compromis principal : la tarification *outcome-based* est économiquement favorable pour l'acheteur lorsque le taux de succès attendu est incertain ou inférieur à 80 %. Elle transfère le risque de performance au fournisseur, qui a une incitation directe à optimiser le CPST. La contrepartie : le fournisseur intègre une prime de risque dans le prix par résolution, ce qui rend la tarification *outcome-based* plus chère par transaction réussie que la tarification *token-metered* lorsque le taux de succès est élevé et stable.

Alternative crédible : tarification *token-metered* avec SLA de taux de succès contractualisé et pénalités — combinaison qui préserve l'avantage économique du token bas coût tout en contraignant le fournisseur sur la performance. Cette structure est plus complexe à négocier et à auditer.

Condition de bascule : si le taux de succès de la tâche est supérieur à 90 % en conditions de production stabilisées (après trois mois de run), la tarification *token-metered* devient préférable. Si le taux de succès est inférieur à 70 % ou incertain, la tarification *outcome-based* réduit le risque financier pour l'organisation acheteuse.

Comparaison structurelle : RPA déterministe vs agent vs ressource humaine

Dimension	RPA déterministe	Agent *agentic*	Ressource humaine
Coût marginal à l'échelle	Quasi nul (scalabilité quasi linéaire)	Variable selon taux de retry et d'escalade	Linéaire (headcount)
Taux de succès sur tâches répétitives bien définies	Élevé (>95 %) si interface stable	Moyen-élevé (60-90 %) selon complexité	Élevé mais variable (fatigue, turnover)
Taux de succès sur tâches ambiguës ou non structurées	Faible (fragilité aux exceptions)	Moyen-élevé si bien outillé	Élevé
Observabilité du coût par tâche	Faible (coût amorti sur licence)	Possible si CPST instrumenté dès le design	Faible (coût RH indirect)
Conformité et auditabilité	Haute (déterministe, reproductible)	Variable selon gouvernance	Haute si processus documenté
Coût d'escalade	Faible (exception prévisible → script)	Élevé si mal borné	Intégré (gestion d'escalade humain-humain)
Condition préférable	Tâche structurée, séquence fixe, interface stable	Tâche ambiguë, séquence variable, contexte riche	Tâche à forte composante relationnelle ou éthique

Sources structurelles : Company of Agents 2026, Deloitte CFO Guide 2026, Masood 2026. Chiffres de taux de succès : fourchettes estimées — à calibrer sur données propres avant toute décision.

2.3 — FinOps agentique : retry budget*, coût d'escalade, observabilité financière*

Le *FinOps* agentique n'est pas la gestion de factures cloud. C'est l'ingénierie de plafonds financiers par tâche et par tenant, intégrés dans l'architecture d'exécution de l'agent dès la phase de design — avant la première ligne de code production. L'absence de ces plafonds est la cause la plus documentée de dépassement de coût catastrophique dans les programmes *agentic* de 2025-2026.

L'incident fondateur est documenté par InfoWorld (2026) : un agent déployé sans plafond de boucle a accumulé 47 000 dollars de dépense non intentionnelle avant d'être détecté et arrêté. Ce n'est pas un cas marginal — c'est la conséquence prévisible d'un agent *stateful* avec accès à des outils payants et sans mécanisme de *kill switch* financier. Un incident de plus grande ampleur a été rapporté en avril 2026 dans un contexte Fortune 500 (fuite de coût cloud

liée aux agents estimée à plusieurs centaines de millions de dollars — *à vérifier*, chiffre issu de sources secondaires non auditées).

Trois primitives de contrôle de coût doivent être conçues dès le design, pas ajoutées après incident :

Primitive 1 — *Retry budget* (budget de retry)

Le *retry budget* est le nombre maximal de tentatives autorisées par tâche, avec un coût marginal par retry explicitement modélisé et décisionnel. La décision de retenter ou d'escalader ne peut pas être laissée au LLM seul : elle doit être gouvernée par une politique externe qui compare le coût espéré du retry N à la probabilité de succès estimée P(succès | N). Si P(succès retry N) < seuil configuré (typiquement 0,3 à 0,5), l'escalade est déclenchée automatiquement. Sans ce seuil, l'agent optimise pour la complétion de la tâche — pas pour le coût de la complétion.

Primitive 2 — *Escalation cost* (coût d'escalade)

Le passage à l'humain a un coût unitaire calculable : salaire-minute de l'opérateur × durée médiane de traitement de l'exception. Ce coût doit être explicitement comparé au coût marginal d'un retry supplémentaire pour décider de la politique d'escalade. Cette comparaison produit un seuil d'escalade financièrement optimal — qui n'est jamais le seuil « jamais escalader » ni « toujours escalader à la première exception ». La politique d'escalade documentée, avec ses seuils, est l'un des trois artefacts de gouvernance minimaux requis pour passer en production (voir §2.4).

Primitive 3 — *Tool-call cap* et *kill switch* financier

Chaque appel d'outil à effet de bord (API externe payante, écriture en base de données, envoi de communication) doit être plafonné par run. Si le nombre d'appels franchit le seuil configuré, le run est suspendu et une approbation humaine est requise. Ce mécanisme est distinct du *retry budget* : il contrôle l'amplitude des effets de bord, pas le nombre de tentatives. Le *kill switch* financier est sa version radicale : si le coût accumulé par run franchit un seuil absolu (exemple : 50 dollars compute), le run est terminé et l'exception remontée.

Le diagramme suivant représente le flux décisionnel de la gestion du coût en cours d'exécution :

```
flowchart TD
    T[Tentative N] --> CB{Coût accumulé\n≥ kill switch ?}
    CB -- Oui --> KS([Kill switch\nterminaison forcée])
    CB -- Non --> ER{Résultat\nréussi ?}
    ER -- Oui --> OK([Tâche réussie\nCPST enregistré])
    ER -- Non --> RB{Retry budget\népuisé ?}
    RB -- Non --> PC{P(succès)\n≥ seuil ?}
    PC -- Oui --> T
    PC -- Non --> ESC
    RB -- Oui --> ESC([Escalade humaine\ncoût d'escalade enregistré])
    ESC --> AUD[(Audit trail\nobservabilité financière)]
    OK --> AUD
    KS --> AUD

    style KS fill:#fdecea,stroke:#c0392b
    style ESC fill:#fff3cd,stroke:#e67e22
    style OK fill:#d4edda,stroke:#27ae60
    style AUD fill:#e8f4f8,stroke:#2980b9
```

Observabilité financière agentique

L'observabilité financière est la capacité à connaître le CPST en temps réel, par tâche et par tenant, sans attendre la facture de fin de mois. Elle est distincte de l'observabilité technique (Ch. 7 — AgentOps) bien que les deux partagent la même infrastructure de traces. Une trace AgentOps contient les durées et les appels d'outils ; l'observabilité financière y ajoute les coûts unitaires pour produire un CPST par trace.

Pour les déploiements SaaS (*software as a service*) multi-locataires, la granularité par tenant est non négociable : sans elle, les locataires à comportement atypique (taux d'escalade élevé, boucles longues) subsidient les locataires efficaces, ce qui biaise toute décision tarifaire et détruit les marges unitaires. L'*infrastructure tax* de 30 à 50 % mentionnée par Masood (2026) est précisément la fraction du CPST que la granularité par tenant rend visible et pilotable.

La relation latence-coût mérite un traitement explicite : les modèles à qualité maximale sont généralement plus chers *et* plus lents que les modèles à faible coût. L'arbitrage latence / coût / qualité est une décision architecturale, pas une valeur par défaut. Pour les tâches de support client synchrone, la latence perceptible par l'utilisateur final contraint le modèle utilisable — et donc le plancher de coût. Pour les tâches de traitement de documents en arrière-plan, la latence est sans contrainte et l'optimisation de coût peut privilégier les modèles les moins chers au détriment de la vitesse. Cette décision doit être prise explicitement, par tâche, pas par programme.

La position du Gartner Hype Cycle 2026 est un signal de maturité calibrant : *FinOps for agentic AI* est identifié comme profil émergent distinct sur le Hype Cycle 2026 (Gartner, 2026 — *confirmé* via communiqué Zenity, Business Wire, 15 avril 2026). Cela signifie que la discipline est critique mais que les outils et les pratiques sont encore en formation. Les organisations qui attendent la maturité du marché pour instrumenter leur observabilité financière auront accumulé une dette de coût non mesurée pendant la phase de croissance.

2.4 — Les 40 % d'annulations : causalité, limites, leçons

La prédiction Gartner de juin 2025 est la plus citée du secteur en 2026, et la plus mal attribuée. Une mise au point sur la source et la méthodologie est nécessaire avant toute utilisation dans un dossier décisionnel.

Source primaire. Gartner, communiqué de presse public, 25 juin 2025 (*confirmé*) : « Gartner Predicts Over 40% of Agentic AI Projects Will Be Canceled by End of 2027 ». La source est un communiqué de presse public, pas le rapport complet (paywall). La méthodologie est décrite dans le communiqué : sondage mené en janvier 2025, 3 412 participants recrutés parmi les participants à des webinaires Gartner. Le terme exact utilisé est *canceled* (annulés), pas « abandonnés » ni « mis en pause ». Anushree Verma, *Senior Director Analyst* chez Gartner, est citée : « Most agentic AI projects right now are early stage experiments or proof of concepts that are mostly driven by hype and are often misapplied. »

Biais de sélection. La population sondée est composée de participants à des webinaires Gartner — une population auto-sélectionnée qui présente un engagement préalable dans les technologies IA et une sensibilité aux messages Gartner. Cette population est probablement plus avancée dans ses projets *agentic* que la moyenne du marché, ce qui peut sous-estimer le taux d'annulation réel dans les organisations moins matures, ou le surestimer dans les organisations qui ont évité les pièges de cadrage. Le chiffre de 40 % est une prédiction, pas un constat rétrospectif — son statut épistémique est *probable*, pas *confirmé*.

Décomposition causale. Le communiqué identifie trois causes d'annulation :

1. *Coûts escaladants* : projets lancés sans modèle de CPST, dont le coût réel à l'échelle dépasse les estimations initiales d'un facteur documenté de 3× à 50× selon les cas (SoftwareSeni, 2026 ; Company of Agents, 2026 — *à vérifier sur données propres*). La lacune est précisément celle que §2.1–2.3 outille.
2. *Valeur métier floue* : projets lancés sans définition contractuelle du *successful outcome* — le CPST n'a pas de dénominateur parce que « succès » n'a pas été défini avant le pilote. L'absence de métrique de résultat rend toute évaluation de ROI (retour sur investissement) impossible et expose le programme à la première revue de portefeuille budgétaire.
3. *Contrôles de risque insuffisants* : projets déployés sans politique d'escalade documentée, sans *retry budget*, sans audit trail de décision — en contradiction directe avec les exigences de gouvernance qui prédisent le multiplicateur de production de §2.5. Le lien avec Ch. 8 est direct : les contrôles de risque insuffisants créent des incidents (Replit, OpenAI Operator décrits au Ch. 1) qui provoquent l'annulation politique, pas seulement économique.

Ces trois causes forment un système : un projet qui définit son *successful outcome* (cause 2) peut instrumenter son CPST (cause 1) et sa politique d'escalade (cause 3). Les trois lacunes disparaissent ensemble ou restent ensemble.

Le cas Klarna comme illustration.

Klarna a éliminé 700 postes de service client (chiffre officiel Klarna, 2024 — confirmé par Fortune, 9 mai 2025) en déployant un agent de service client *agentic*. La satisfaction client a chuté de manière mesurable. En 2025, Klarna a commencé à réembaucher des agents humains dans un modèle hybride décrit par le CEO comme une structure « Uber » — les agents humains opérant comme des prestataires flexibles selon la demande. Le CEO a déclaré publiquement : « We went too far. » (*confirmé* — Fortune, 2025).

L'échec n'était pas technique. L'agent fonctionnait. Le CPST était probablement favorable en coût d'inférence pur. L'échec était économique et opérationnel : le coût de la dégradation de la satisfaction client — *churn*, perte de valeur

vie client, coût réputationnel — n'était pas dans le modèle de CPST initial. Un CPST complet aurait inclus ce coût externe. Cette omission est l'anti-patron de cadrage le plus fréquent dans les projets de remplacement d'interactions humaines : optimiser le coût direct sans modéliser le coût de la qualité d'expérience.

La variante 853 postes, circulant dans certaines sources secondaires, n'est pas confirmée par Klarna. La monographie retient 700 comme chiffre officiel.

Ce que le chiffre de 40 % ne dit pas.

Soixante pour cent des projets *agentic* ne seront pas annulés d'ici 2027. La question pertinente est : quel est le différenciateur entre les deux cohortes ? Les données disponibles convergent vers une réponse unique, que §2.5 documente.

2.5 — Les 12× : la gouvernance comme multiplicateur de production

La corrélation la plus structurante des données disponibles en mai 2026 est publiée par Databricks dans le rapport *State of AI Agents 2026* (*confirmé* — Databricks Blog, 2026 ; rapport complet sous enregistrement) : les entreprises ayant mis en place une gouvernance IA formelle poussent 12 fois plus de projets *agentic* en production que celles qui ne l'ont pas fait. L'échantillon est de plus de 20 000 organisations mondiales.

Attribution. Ce chiffre est attribué à Databricks *State of AI Agents 2026* — pas à Gartner, pas à Deloitte, pas à McKinsey. La confusion d'attribution est fréquente dans la littérature secondaire parce que les trois organisations ont publié des rapports sur des thèmes adjacents dans la même fenêtre de temps. L'utilisation du chiffre 12× sans attribution correcte à Databricks est une erreur de sourcing.

Statut épistémique. La corrélation est *confirmée* par le rapport Databricks (n=20 000+). La causalité — gouvernance → déploiements en production — est *probable* mais non démontrée par le rapport lui-même, qui ne détaille pas les mécanismes causaux. Le mécanisme *le plus plausible* est le suivant : une gouvernance formelle crée la confiance des parties prenantes internes (sécurité, conformité, direction financière) requise pour accorder le feu vert production ; sans cette confiance, les projets restent en PoC perpétuel, non pas pour des raisons techniques, mais pour des raisons de validation organisationnelle.

Définition opérationnelle de la gouvernance dans ce contexte.

La gouvernance au sens de Databricks n'est pas un comité d'approbation. C'est un ensemble d'artefacts techniques et organisationnels qui permettent à un agent de passer la porte de production. Sur la base des données du rapport et des pratiques documentées dans Ch. 8 et Annexe D, trois artefacts sont minimalement requis :

Artefact 1 — Définition contractuelle du *successful outcome* et seuil de CPST acceptable. Ce document — une page ou moins — définit ce que compte comme succès pour cette tâche spécifique, le seuil de CPST au-delà duquel la tâche est économiquement non viable, et le taux de succès minimum en dessous duquel la politique d'escalade est révisée. Il est signé par la direction financière et la direction produit avant le premier run de production.

Artefact 2 — *Eval suite* automatisée couvrant les cas de bord identifiés en PoC. Une suite d'évaluations automatisées qui teste l'agent sur les scénarios les plus susceptibles d'échouer — pas les scénarios dans lesquels il excelle. Un agent qui réussit 95 % des cas nominaux et échoue sur 100 % des cas de bord n'a pas un taux de succès de 95 % en production réelle. L'*eval suite* mesure le taux de succès sur la distribution réelle des entrées, pas sur la distribution de validation du PoC. Ch. 4 détaille la construction de cette suite.

Artefact 3 — *Escalation policy* documentée avec seuils de coût et de confiance. La politique qui gouverne quand le *retry budget* est épuisé, à quel seuil de P(succès) l'escalade est automatique, quel est le coût d'escalade unitaire modélisé, et qui est notifié. Ce document transforme les primitives FinOps de §2.3 en politique organisationnelle opposable.

Pourquoi la gouvernance *accélère* plutôt qu'elle ne ralentit.

L'argument habituel contre la gouvernance *dès le début* est qu'elle ralentit les premiers sprints de développement. Cet argument est factuellement inversé dans les données Databricks : les organisations *sans* gouvernance formelle ont 12 fois moins de projets en production, ce qui signifie que leurs sprints plus rapides aboutissent à des PoC qui ne franchissent jamais la porte de production. La vitesse sans gouvernance produit des projets en PoC perpétuel — ce n'est pas de la vitesse, c'est de l'activité sans résultat.

Le délai introduit par la gouvernance initiale — estimé à 20-30 % des premiers sprints dans l'*Introduction* (*à vérifier* — aucune étude publiée à mai 2026 ne quantifie précisément cet impact) — est compensé par l'élimination des cycles de validation tardifs, des blocages de sécurité en pré-production, et des annulations post-investissement que les 40 % de Gartner représentent.

Position du Hype Cycle 2026.

Gartner positionne *agentic AI governance* comme profil émergent distinct sur le Hype Cycle 2026 (*confirmé* — Zenity, Business Wire, 15 avril 2026), avant le Pic des attentes gonflées. Ce positionnement signifie que la discipline se structure mais que les pratiques ne sont pas encore standardisées. Les organisations qui attendent la standardisation pour implémenter la gouvernance paieront le prix de l'absence de gouvernance pendant la phase d'adoption la plus rapide — précisément la phase où les 40 % d'annulations se concentrent.

2.6 — De l'économie au cadrage : préparer Ch. 3 et Ch. 4

Un programme *agentic* économiquement sain est celui qui peut répondre, avant la première ligne de code production, à trois questions opérationnelles :

1. Quel est le CPST cible pour cette tâche, et à quel niveau le programme est-il économiquement non viable ?
2. Quel est le *retry budget* maximal par tâche, et quelle est la politique d'escalade quand ce budget est épuisé ?
3. Quel est le *successful outcome* de cette tâche, opérationnellement défini, et qui le valide ?

Ces trois questions sont des critères de sélection de cas d'usage — elles éliminent les tâches pour lesquelles aucune réponse n'est possible avant production (absence de données historiques, processus mal défini, parties prenantes non alignées). La matrice de cadrage de Ch. 3 (autonomie × réversibilité × tolérance à l'erreur) opère sur ces mêmes critères : une tâche à faible tolérance à l'erreur et haute réversibilité imposée aura un coût d'escalade élevé qui dégrade structurellement le CPST — la matrice détecte cette incompatibilité en amont du développement.

Les quatre piliers du cadre ROI de Ch. 4 — LLM, Memory, Tools, Environment — sont les déterminants du CPST. Un environnement mal outillé (pilier *Tools*) multiplie les retries ; une mémoire défaillante (pilier *Memory*) augmente le taux d'escalade ; un LLM sous-dimensionné pour la complexité de la tâche (pilier *LLM*) dégrade le taux de succès. L'évaluation de *readiness* de Ch. 4 est donc une évaluation préalable du CPST potentiel avant engagement de ressources de développement.

La continuité narrative est intentionnelle : Ch. 1 a posé les exigences architecturales de la boucle *decide–act–observe* ; ce chapitre les a traduites en métriques financières ; Ch. 3 et Ch. 4 appliquent ces métriques à la sélection et à la qualification des cas d'usage spécifiques. Les pratiques d'observabilité financière convergent avec les pratiques AgentOps dans Ch. 7. Les incidents économiques évitables — Klarna, l'agent à 47 000 dollars — alimentent l'anatomie des échecs de Ch. 12.

Pour aller plus loin

Databricks — *State of AI Agents 2026* (rapport complet, sous enregistrement). La source primaire du multiplicateur 12× et la base empirique la plus large disponible à mai 2026 sur les pratiques de gouvernance et leur corrélation avec les déploiements en production. Lecture indispensable avant toute présentation exécutive sur le ROI d'un programme *agentic*. https://www.databricks.com/resources/ebook/state-of-ai-agents

Gartner — « Gartner Predicts Over 40% of Agentic AI Projects Will Be Canceled by End of 2027 » (25 juin 2025). Le communiqué public qui fonde la prédiction des 40 %. À lire en source primaire pour comprendre exactement ce qui est mesuré, sur quel échantillon, et avec quels biais — avant de citer le chiffre dans un dossier décisionnel. https://www.gartner.com/en/newsroom/press-releases/2025-06-25-gartner-predicts-over-40-percent-of-agentic-ai-projects-will-be-canceled-by-end-of-2027

Zendesk — « Zendesk First in CX Industry to offer Outcome-Based Pricing for AI Agents » (2025). Le signal de marché le plus concret sur la convergence du secteur vers la tarification par résultat. Les paliers tarifaires publiés offrent une référence pour calibrer un CPST cible dans le contexte du service client. https://www.zendesk.com/newsroom/articles/zendesk-outcome-based-pricing/

Adnan Masood — « AI FinOps: Managing Value and Cost in the Agentic Era » (mars 2026). La formalisation la plus complète disponible des métriques CPST et *infrastructure tax* pour les praticiens. Source

secondaire — croiser avec les données propres de l'organisation. https://medium.com/@adnanmasood/ai-finops-managing-value-and-cost-in-the-agentic-era-37d364c25ec5

Deloitte — « AI Token Economics for CFOs » (2026). Le guide Deloitte est conçu pour les directeurs financiers qui reçoivent des demandes de budget IA sans base économique solide. Utile pour aligner le vocabulaire financier entre les équipes techniques (token, retry) et les directions financières (CPST, ROI, coût marginal). https://www.deloitte.com/us/en/services/consulting/articles/cfo-guide-ai-token-economics.html

---

Références

Databricks — « Enterprise AI Agent Trends: Top Use Cases, Governance + Evaluations and More » — Databricks Blog — 2026 — https://www.databricks.com/blog/enterprise-ai-agent-trends-top-use-cases-governance-evaluations-and-more — accédée le 2026-05-05

Databricks — *State of AI Agents 2026* (rapport complet) — Databricks — 2026 — https://www.databricks.com/resources/ebook/state-of-ai-agents — accédée le 2026-05-05

Deloitte — « AI Token Economics for CFOs » — Deloitte US — 2026 — https://www.deloitte.com/us/en/services/consulting/articles/cfo-guide-ai-token-economics.html — accédée le 2026-05-05

Fortune — « Klarna Reverses AI Customer Service Replacement » — Fortune — 9 mai 2025 — https://fortune.com/2025/05/09/klarna-ai-humans-return-on-investment/ — accédée le 2026-05-05

Gartner — « Gartner Predicts Over 40% of Agentic AI Projects Will Be Canceled by End of 2027 » — Gartner Newsroom — 25 juin 2025 — https://www.gartner.com/en/newsroom/press-releases/2025-06-25-gartner-predicts-over-40-percent-of-agentic-ai-projects-will-be-canceled-by-end-of-2027 — accédée le 2026-05-05

Gartner — « 2026 Hype Cycle for Agentic AI » — Gartner — 2026 — https://www.gartner.com/en/articles/hype-cycle-for-agentic-ai — accédée le 2026-05-05

InfoWorld — « FinOps for agents: Loop limits, tool-call caps and the new unit economics of agentic SaaS » — InfoWorld — 2026 — https://www.infoworld.com/article/4138748/finops-for-agents-loop-limits-tool-call-caps-and-the-new-unit-economics-of-agentic-saas.html — accédée le 2026-05-05

Masood, A. — « AI FinOps: Managing Value and Cost in the Agentic Era » — Medium — mars 2026 — https://medium.com/@adnanmasood/ai-finops-managing-value-and-cost-in-the-agentic-era-37d364c25ec5 — accédée le 2026-05-05

Company of Agents — « AI Agent Unit Economics: Scaling Your Agentic Fleet in 2026 » — companyofagents.ai — 2026 — https://www.companyofagents.ai/blog/en/ai-agent-unit-economics-scaling — accédée le 2026-05-05

SoftwareSeni — « Why Your AI Bill Exploded Between Pilot and Production » — softwareseni.com — 2026 — https://www.softwareseni.com/why-your-ai-bill-exploded-between-pilot-and-production-and-how-to-predict-the-real-cost/ — accédée le 2026-05-05

AWS — « Amazon Bedrock AgentCore Pricing » — Amazon Web Services — 2026 — https://aws.amazon.com/bedrock/agentcore/pricing/ — accédée le 2026-05-05

Zendesk — « Zendesk First in CX Industry to offer Outcome-Based Pricing for AI Agents » — Zendesk Newsroom — 2025 — https://www.zendesk.com/newsroom/articles/zendesk-outcome-based-pricing/ — accédée le 2026-05-05

Zenity — « Zenity Named in Two Categories in the 2026 Gartner Hype Cycle for Agentic AI » — Business Wire — 15 avril 2026 — https://www.businesswire.com/news/home/20260415309905/en/Zenity-Named-in-Two-Categories-in-the-2026-Gartner-Hype-Cycle-for-Agentic-AI — accédée le 2026-05-05

Chapitre 3 — Cartographie des cas à fort impact

Partie 2 — Trouver les bons cas d'usage Chapitre 3 · Cartographie des applications à fort impact · ~5 000 mots · lecture ≈ 20 min

La valeur d'un programme *agentic* n'est pas déterminée par la sophistication du modèle ni par la richesse de la plateforme, mais par la qualité du filtre appliqué en amont : toutes les tâches ne sont pas également agentifiables. Moins de 10 % des entreprises ont mis des agents à l'échelle malgré deux tiers d'expériences en cours (BCG, 2025) ; le goulot n'est pas l'accès à la technologie mais la qualité du filtre appliqué à l'entrée du pipeline de projets. Ce chapitre construit un système de qualification articulé autour de trois dimensions continues — autonomie requise, réversibilité de l'action, tolérance à l'erreur — le traduit en patrons concrets par famille fonctionnelle (back-office, front-office, engineering), et documente les anti-patrons observés en production en 2025-2026 qui auraient tous été détectables avant la première ligne de code.

3.1 — Pourquoi la sélection de cas d'usage est le problème n°1

Gartner prédit que 40 % des applications d'entreprise intégreront des agents spécifiques à une tâche d'ici fin 2026, contre moins de 5 % en 2025 (*confirmé* — Gartner Newsroom, 26 août 2025). Ce rythme d'adoption, sans précédent dans l'histoire de l'EDA (*event-driven architecture*) ou des microservices, repose sur une démocratisation des plateformes d'entrée : Salesforce Agentforce, Microsoft Copilot Studio, AWS Bedrock Agents, Google Vertex AI Agent Builder ont rendu le premier déploiement accessible en quelques jours. C'est précisément pour cette raison que l'acte de qualification est devenu le facteur différenciateur.

La corrélation entre qualification rigoureuse et déploiement réussi est documentée. Sur les 20 000+ organisations de l'étude Databricks *State of AI Agents 2026*, celles qui ont mis en place une gouvernance formelle déploient 12 fois plus de projets *agentic* en production que celles qui ne l'ont pas fait (*confirmé* — Databricks, 2026 ; l'attribution et le mécanisme causal sont développés au Ch. 2). BCG confirme le diagnostic par l'autre bout : 80 % des organisations citent les limitations de données et de qualification comme premier frein à la mise à l'échelle — pas les capacités des modèles (*confirmé* — BCG, 2025).

La conclusion opérationnelle est sans équivoque : les organisations qui qualifient rigoureusement leurs cas d'usage en amont convergent vers le 60 % de projets qui survivent ; celles qui sautent la qualification alimentent les 40 % d'abandons prédits par Gartner d'ici 2027 (Ch. 2 §2.4). La matrice qui suit est l'outil de qualification. L'Annexe B en propose une instanciation pratique directement opérable par une équipe de cadrage.

3.2 — La matrice de cadrage : trois dimensions continues

McKinsey a formalisé en 2025 une matrice à neuf archétypes croisant le risque d'erreur de décision et la complexité de la décision (*confirmé* — McKinsey QuantumBlack, *Seizing the Agentic AI Advantage*, 2025). La matrice adoptée ici descend d'un niveau d'abstraction supplémentaire : elle remplace les deux axes McKinsey par trois dimensions *opérationnelles* — non pas catégorielles mais continues — qui permettent de positionner n'importe quel cas d'usage avec précision et de le relier directement au *Cost per Successful Task* (CPST) introduit au Ch. 2.

Dimension 1 — Autonomie requise

L'autonomie requise mesure le degré auquel l'agent doit prendre des décisions sans approbation humaine intermédiaire pour accomplir l'objectif assigné. L'axe s'étend de 1 (suggère seulement, humain décide et agit) à 5 (exécute de manière entièrement autonome, sur des systèmes de production, sans validation intermédiaire). Ce n'est pas une mesure de la capacité technique du modèle — c'est une mesure de la politique de supervision choisie par l'organisation pour ce cas d'usage spécifique. Un modèle capable d'opérer à autonomie 5 peut être délibérément contraint à autonomie 2 par une politique d'escalade documentée ; l'inverse n'est pas vrai.

La position sur cet axe détermine directement le *retry budget* maximum acceptable (développé au Ch. 2 §2.3) et le coût d'escalade attendu. Un agent à autonomie 1-2 escalade fréquemment ; le coût d'escalade est élevé mais le risque d'action non supervisée est faible. Un agent à autonomie 4-5 escalade rarement ; le coût d'escalade est faible mais chaque décision non escaladée engage l'organisation sans validation.

Dimension 2 — Réversibilité de l'action

La réversibilité mesure si les effets de bord des actions de l'agent peuvent être annulés sans perte de valeur métier nette. L'axe s'étend de 1 (totalement réversible : lecture seule, classification, génération de brouillon interne non envoyé) à 5 (totalement irréversible : transfert financier exécuté, courriel externe envoyé, suppression de données de production, licenciement de personnel). Une action irréversible ne peut pas être défaite après coup ; elle peut seulement être *compensée* — avec un coût de compensation qui doit être intégré dans le CPST.

La relation avec le CPST est directe et quantifiable : une action irréversible à erreur détectée après exécution impose soit un coût d'escalade humaine élevé (annulation manuelle, contact client, procédure de réclamation), soit un coût de correction partiel (rollback approximatif, communication de crise). Ce coût est systématiquement absent des dossiers d'investissement initiaux et constitue la principale source de dépassement de CPST en production.

Il existe également une réversibilité *organisationnelle* — distincte de la réversibilité des actions techniques — qui sera développée dans les anti-patrons (§3.5). Supprimer des postes humains pour déployer des agents crée une irréversibilité que la matrice signale si elle est appliquée au niveau de la décision de déploiement, pas seulement au niveau de chaque action de l'agent.

Dimension 3 — Tolérance à l'erreur

La tolérance à l'erreur mesure les conséquences d'une erreur de l'agent en termes métier, réglementaires et réputationnels. L'axe s'étend de 1 (erreur sans conséquence significative : brouillon à relire, classification corrigible, suggestion non retenue) à 5 (erreur catastrophique : décision médicale irreversible, transaction financière réglementée, communication légale externe, action sur infrastructure de production non testée).

La tolérance à l'erreur est contrainte de l'extérieur dans les environnements réglementés. Pour les institutions financières canadiennes sous supervision fédérale, OSFI E-23 (*Model Risk Management*, en vigueur 1er mai 2027) imposera des exigences explicites de gouvernance des modèles IA qui affectent directement la tolérance à l'erreur acceptable pour tout cas d'usage impliquant du crédit, de la fraude, ou de la gestion de portefeuille. Un paragraphe court sur les implications d'OSFI E-23 pour la qualification des cas d'usage est développé au Ch. 8, qui traite l'ensemble du cadre réglementaire de manière exhaustive.

Zones de la matrice et règle de décision

La combinaison des trois dimensions produit un score de risque agentique qui guide la décision d'investissement. Les zones ne sont pas des catégories rigides — elles indiquent la nature des garde-fous requis avant déploiement :

Zone verte (autonomie ≤ 3 ET réversibilité ≥ 3 ET tolérance à l'erreur ≥ 3 sur l'axe inverse) : cas d'usage à fort ROI rapide, risque opérationnel limité. Les garde-fous sont nécessaires mais standard. Idéaux pour les premiers déploiements et la construction du capital politique du programme.

Zone orange (un ou deux scores en tension) : cas d'usage à valeur élevée qui requièrent des garde-fous explicites et documentés avant déploiement : politique d'escalade avec seuils chiffrés, *eval suite* couvrant les cas de bord, *human-in-the-loop* (humain dans la boucle) pour les décisions au-dessus d'un seuil de confiance. Le déploiement est justifiable ; il impose un investissement de gouvernance proportionnel.

Zone rouge (autonomie = 5 ET réversibilité = 1-2 ET/OU tolérance à l'erreur = 1) : cas d'usage dont la combinaison de risques n'est pas maîtrisable avec les pratiques actuelles de 2026 sans réduction préalable de l'une des trois dimensions. La décision n'est pas « ne jamais déployer » mais « réduire d'abord l'une des dimensions » — par exemple, contraindre l'autonomie à 3 via une approbation humaine obligatoire avant toute action irréversible.

Le diagramme suivant représente la règle de décision comme arbre de qualification :

```
flowchart TD
    A[Cas d'usage candidat] --> B{Autonomie requise\n≥ 4 ?}
    B -- Non --> C{Réversibilité\n≤ 2 ?}
    B -- Oui --> D{Réversibilité\n≤ 2 ?}
    C -- Non --> VERT([Zone verte\nDéployer avec garde-fous standard])
    C -- Oui --> D2{Tolérance erreur\n= 1 ?}
    D2 -- Non --> ORANGE([Zone orange\nGarde-fous renforcés requis])
    D2 -- Oui --> ROUGE
    D -- Non --> ORANGE2([Zone orange\nGarde-fous renforcés requis])
    D -- Oui --> D3{Tolérance erreur\n≤ 2 ?}
    D3 -- Non --> ORANGE2
```

```
    D3 -- Oui --> ROUGE([Zone rouge\nRéduire une dimension\navant de déployer])

    style VERT fill:#d4edda,stroke:#27ae60
    style ORANGE fill:#fff3cd,stroke:#e67e22
    style ORANGE2 fill:#fff3cd,stroke:#e67e22
    style ROUGE fill:#fdecea,stroke:#c0392b
```

Recommandation : appliquer la matrice au niveau du portefeuille, pas cas par cas.

Compromis principal : la qualification individuelle d'un cas d'usage en isolation produit des décisions localement correctes mais ignore les interactions entre cas d'usage déployés simultanément — deux agents en zone orange qui partagent un même outil de modification de données créent une zone rouge composite que la qualification individuelle n'aurait pas détectée. La qualification de portefeuille est plus coûteuse en temps (estimation : 2 à 3 jours par vague de 10 à 15 cas d'usage, *hypothèse* — pas de données publiées à mai 2026) mais détecte les risques composés.

Alternative crédible : qualifier individuellement les cas d'usage pilotes (zone verte garantie), déployer, puis réaliser une qualification de portefeuille avant la vague suivante. Cette approche séquentielle est valide pour les organisations qui débutent et dont la bibliothèque de cas d'usage est limitée.

Condition de bascule : si deux cas d'usage ou plus partagent des outils à effet de bord irréversible, la qualification de portefeuille est obligatoire avant tout déploiement — indépendamment de la taille de la bibliothèque.

3.3 — Patrons back-office : clôture financière, P2P, support tier-1

Le back-office est la zone verte par défaut de la matrice. Tolérance à l'erreur faible à modérée (les erreurs sont détectables avant impact externe), réversibilité élevée (les workflows sont documentés, journalisés, audités), autonomie requise modérée (une validation humaine avant clôture est dans la norme du processus) — cette combinaison maximise le CPST positif et explique pourquoi les premiers déploiements *agentic* à l'échelle en 2025 se sont concentrés dans cette famille.

Clôture financière (close financier)

Les agents de rapprochement comptable (*account reconciliation*) opèrent sur des données structurées, dans un environnement d'audit permanent, avec des critères de succès binaires (rapprochement réussi ou non). La réversibilité est élevée : aucune action irréversible n'est possible avant la validation humaine de la clôture. La tolérance à l'erreur est modérée : une erreur détectée avant soumission est corrigible sans impact externe ; une erreur non détectée qui franchit la clôture a des conséquences réglementaires.

Un chiffre de 30-50 % d'accélération du processus de clôture circule dans certaines sources secondaires (*à vérifier* — source primaire Gartner non confirmée ; ce chiffre est retiré de la monographie jusqu'à traçabilité établie). Ce qui est documenté : des déploiements dans les systèmes ERP (progiciels de gestion intégrée) d'Oracle et SAP avec des agents capables de réconcilier automatiquement les écritures courantes et de signaler les exceptions non réconciliées, réduisant le volume de travail manuel de réconciliation de manière mesurable — sans chiffre agrégé confirmé en source primaire à mai 2026 (*à vérifier*).

La position dans la matrice est : autonomie 2-3 (validation humaine de la clôture finale), réversibilité 3-4 (toutes les actions sont journalisées et réversibles avant validation), tolérance à l'erreur 3 (erreur détectable avant impact) — zone verte à zone orange limite. Le garde-fou clé : aucune écriture comptable définitive ne peut être soumise sans approbation humaine, indépendamment du niveau de confiance de l'agent.

P2P (*Procure-to-Pay*) et comptes fournisseurs

Le cas le plus documenté en back-office est celui d'un client non nommé de Genpact dans les services alimentaires : 8,5 millions de factures par an, délai de traitement ramené de 9 minutes à 30 secondes après déploiement d'un agent de capture et de routage *agentic*, taux d'*auto-posting* (validation automatique sans intervention) atteignant 60 % (*probable* — Genpact est le prestataire de service et l'auteur du rapport ; croisement avec source indépendante non disponible à mai 2026). Les meilleures équipes AP (*accounts payable*) ont atteint plus de 70 % de taux « touchless » en 2025 (*probable* — même réserve de source).

Basware a lancé ses capacités *agentic* d'AP en février 2026 (*confirmé* — annonce presse Basware). Gartner estime que 15 % des outils AP disposent de capacités *agentic* réelles aujourd'hui, avec projection à 60 % en 2028 (*à vérifier* — chiffre cité en source secondaire, non tracé vers un communiqué Gartner primaire).

La matrice du cas P2P : autonomie 3 (les factures au-dessous d'un seuil de valeur sont traitées automatiquement ; au-dessus, validation obligatoire), réversibilité 4 (les écritures non validées sont réversibles), tolérance à l'erreur 3 (erreur de correspondance facture/bon de commande détectable avant paiement) — zone verte. La condition critique : les seuils d'approbation automatique doivent être fixés par la direction financière, pas par l'équipe technique, et révisés trimestriellement.

Support tier-1 (service desk interne)

Les agents de *service desk* IT pour le support interne — réinitialisation de mot de passe, provisionnement d'accès, diagnostic de tickets courants — représentent le cas d'usage avec le meilleur rapport effort/retour en zone verte. Les actions sont internes, réversibles (un accès provisionné par erreur se révoque), et la tolérance à l'erreur est faible (une erreur de provisionnement est corrigible sans impact externe).

Bank of America a déployé Erica for Employees en interne : 90 % d'adoption par les employés, réduction de plus de 50 % du volume de requêtes au service desk IT (*à vérifier* — ces chiffres sont cités en source secondaire ; le communiqué primaire BofA de mars 2026 couvre Erica pour les clients externes, pas spécifiquement pour l'usage interne). Le communiqué primaire BofA de mars 2026 (*confirmé*) documente pour Erica externe : 21,3 millions d'utilisateurs en Q1 2026, équivalent au travail de 11 000 employés, 98 % de résolution sans intervention humaine — des métriques qui indiquent la capacité de la plateforme.

Matrice : autonomie 2-3, réversibilité 4, tolérance à l'erreur 4 — zone verte. La limite opérationnelle : les exceptions (demandes hors périmètre, utilisateurs avec comportements atypiques, incidents de sécurité potentiels) doivent être escaladées vers un humain avec un *context dump* complet — pas seulement un renvoi vers la file d'attente générale.

3.4 — Patrons front-office : SDR agentic*, wealth management, support contextuel*

Le front-office amplifie les métriques d'impact mais déplace la matrice vers des zones de risque plus élevées. Les actions sont souvent partiellement irréversibles — une communication externe envoyée, une promesse commerciale faite, une relation client engagée — et la tolérance à l'erreur est contrainte par l'expérience client et, selon le secteur, par la réglementation.

SDR (*Sales Development Representative*) *agentic* — le cas Salesforce Agentforce

Salesforce Agentforce est entré en disponibilité générale en octobre 2024 avec trois clients de référence publics : Wiley, Saks, et Wyndham (*confirmé* — TechInformed, octobre 2024). Le cas Wiley est le mieux documenté : le publisher académique a déployé Agentforce sur son portail de support client, obtenant +40 % de résolution autonome des cas et un ROI de 213 % sur Service Cloud par rapport au bot de service client précédent (*confirmé* — Salesforce Customer Stories, page Wiley, 2024-2025).

L'intérêt du cas Wiley pour la matrice est son positionnement : environnement académique, tolérance à l'erreur modérée (une réponse erronée à une question d'abonnement est corrigible), réversibilité partielle (la réponse est envoyée mais l'escalade vers un humain reste possible avant résolution finale) — zone orange acceptable avec des garde-fous de révision des cas complexes.

Un patron distinct — documenté par OpenTable et Wyndham dans le même lancement Agentforce (*confirmé* — TechInformed, octobre 2024) — est la réservation *agentic* : agent qui prend en charge la qualification de la demande, la recherche de disponibilité, et la proposition d'alternatives. La réversibilité d'une réservation proposée mais non confirmée par le client reste élevée ; celle d'une réservation confirmée automatiquement est faible. Ce point de bascule — confirmation vs proposition — est précisément la frontière zone orange / zone rouge pour ce cas d'usage.

Wealth management assisté — J.P. Morgan Coach AI

J.P. Morgan a déployé Coach AI comme outil interne pour les conseillers en gestion de fortune. Coach AI surface du contenu de recherche et un contexte marché en temps réel pendant les périodes de volatilité (*confirmé* — plusieurs sources de presse financière, dont Reuters et Bloomberg couvrant l'utilisation en avril 2025 pendant les épisodes de volatilité liés aux tarifs douaniers). L'usage spécifique de J.P. Morgan Spectrum pour la gestion de portefeuille et des agents de recherche construits sur LangGraph est documenté dans les communications de la firme (*probable* — documenté dans les conférences technologiques JPMorgan 2025, non dans un communiqué de presse primaire).

Position dans la matrice : autonomie 1-2 (*assist-only*, pas d'exécution autonome), réversibilité 5 (aucune transaction déclenchée par l'agent sans validation explicite du conseiller), tolérance à l'erreur 2 (contexte financier réglementé, mais le conseiller humain valide avant toute transmission au client) — zone verte en front-office financier. C'est le patron à retenir pour les institutions sous supervision réglementaire : l'agent augmente la capacité du conseiller humain sans substituer le jugement de celui-ci.

La leçon de design : la zone verte est accessible même dans les contextes les plus réglementés, à condition que l'autonomie soit architecturalement contrainte à 1-2 et que la chaîne de décision préserve un humain compétent comme dernier maillon avant l'action irréversible (communication au client, exécution d'ordre).

Support contextuel à grande échelle — Erica, Bank of America

Erica est le cas de référence le plus documenté pour le support conversationnel bancaire à grande échelle. Les métriques du communiqué primaire BofA de mars 2026 (*confirmé*) : 21,3 millions d'utilisateurs actifs en Q1 2026 (+7 % vs Q1 2025), 2 millions d'interactions quotidiennes (équivalent au travail de 11 000 employés), 98 % de résolution sans intervention humaine, 60 % d'interactions proactives initiées par Erica, −42 % sur le volume de chat CashPro, +19 % de revenus issus de suggestions de nouveaux services.

Ce qui rend Erica instructive pour la matrice au-delà des métriques, c'est l'architecture de bornage : Erica ne signe pas de transactions. Elle informe, guide, suggère, et le client valide. Cette réversibilité artificielle maintenue par conception — l'agent ne peut structurellement pas franchir le seuil de l'action irréversible — est ce qui permet d'opérer à 98 % de résolution autonome sans franchir la zone rouge. L'architecture d'autonomie 3 (haute autonomie sur l'information et la suggestion) combinée à une réversibilité 5 (aucune transaction sans validation client explicite) produit une zone verte durable même à 21 millions d'utilisateurs.

L'anti-patron à éviter : reproduire les métriques d'Erica en supprimant la contrainte de non-transaction pour « aller plus loin ». Gartner prédit que 80 % des problèmes courants de service client seront résolus sans intervention humaine d'ici 2029 (*confirmé* — Gartner, 5 mars 2025) ; cette projection suppose des garde-fous de périmètre, pas une autonomie totale sur les actions à effet de bord irréversible.

L'anti-patron Klarna : réversibilité organisationnelle

Le cas Klarna a été documenté au Ch. 2 §2.4 sous l'angle économique : un CPST initialement favorable en coût d'inférence pur, qui n'intégrait pas le coût de la dégradation de la satisfaction client. L'angle de ce chapitre est différent et complémentaire : Klarna n'a pas appliqué la dimension réversibilité à la décision *organisationnelle* de déploiement. Supprimer 700 postes humains (*confirmé* — chiffre officiel Klarna 2024, cité dans Fortune, 9 mai 2025) pour les remplacer par un agent de service client crée une irréversibilité que la matrice aurait signalée en zone rouge si appliquée au niveau de la décision de déploiement globale — pas seulement au niveau de chaque action individuelle de l'agent.

La réversibilité ne s'applique pas seulement aux actions techniques de l'agent : elle s'applique à l'architecture organisationnelle dans laquelle il opère. Un déploiement qui élimine la capacité humaine de repli sans valider préalablement la fiabilité de l'agent en conditions réelles est structurellement irréversible. Le CEO de Klarna a reconnu publiquement en 2025 : « We went too far. » (*confirmé* — Fortune, 2025.) La réembauche en modèle hybride est la compensation d'une irréversibilité organisationnelle non modélisée.

La règle pratique : avant tout déploiement qui réduit la capacité humaine de plus de 30 % sur une fonction, valider que l'agent atteint un taux de succès supérieur à 95 % sur la distribution réelle des cas, y compris les cas de bord (*hypothèse de seuil — pas de standard publié à mai 2026*). Si ce seuil n'est pas atteint, maintenir la capacité humaine parallèle jusqu'à validation, puis réduire progressivement.

3.5 — Patrons engineering : agents de codage, SRE agentic

L'engineering est la zone d'adoption la plus rapide en 2025-2026 et aussi celle où les anti-patrons les plus documentés ont émergé. La raison est structurelle : les environnements de développement et d'opérations accueillent des agents avec des privilèges d'accès étendus (bases de données, systèmes de production, infrastructure cloud) et une culture de vitesse qui sous-estime la tolérance à l'erreur réelle.

Agents de codage : du copilot à l'agent intégré

GitHub Copilot Coding Agent est en disponibilité générale depuis septembre 2025 pour tous les abonnés Copilot payants (*confirmé* — annonce GitHub). Il excelle sur les tâches à complexité faible à modérée dans des bases de code bien testées : ajout de fonctionnalités isolées, correction de bugs avec reproduction automatisée, extension de suites de tests, refactorisation ciblée, amélioration de documentation. La philosophie de design est celle d'un assistant intégré au flux de révision (*pull request*) : l'humain reste dans la boucle pour la revue et le *merge*. Position dans la matrice : autonomie 2-3 (l'humain approuve le *merge*), réversibilité 4-5 (toutes les modifications sont réversibles via git), tolérance à l'erreur 3 (erreurs détectables à la révision ou par les tests automatisés) — zone verte.

Sourcegraph Amp (repositionnement de Cody en juin 2025, *confirmé* — blog Sourcegraph) se positionne sur les déploiements à l'échelle de l'entreprise avec des bases de code multi-référentiels. Des déploiements vérifiés incluent

Palo Alto Networks (2 000+ développeurs) et Qualtrics (1 000+ développeurs) (*confirmé* — communications Sourcegraph, cas clients publiés). L'avantage différenciant par rapport à GitHub Copilot est l'infrastructure de recherche de code sur des codebases entiers, pas seulement le fichier ou le référentiel courant — ce qui permet aux agents de raisonner sur des dépendances inter-équipes. Même position dans la matrice que GitHub Copilot pour les tâches de modification ; zone orange pour les tâches de refactorisation à grande échelle (plusieurs référentiels, risque d'effets de bord non anticipés).

Cognition/Devin se positionne à l'extrémité autonome du spectre (« parallel autonomous engineer » selon son positionnement marketing). Valorisation publiquement documentée : 10,2 milliards de dollars (*confirmé* — presse financière 2025). Position dans la matrice : autonomie 4-5 selon la politique de supervision déployée, réversibilité variable selon les accès accordés, tolérance à l'erreur variable. Zone orange à rouge selon la configuration : avec accès à des systèmes de production et politique d'approbation absente, le cas se déplace vers la zone rouge (voir l'incident Replit au §3.6).

Recommandation : privilégier l'autonomie contrainte pour les agents de codage en environnement de production.

Compromis principal : contraindre l'autonomie à 2-3 (révision humaine obligatoire avant merge) réduit le débit de l'agent mais maintient la tolérance à l'erreur au niveau acceptable pour les environnements réglementés ou à haute disponibilité. Un agent à autonomie 4-5 sur une base de code de production crée un risque de zone rouge dès que la réversibilité diminue (merge sur branche principale sans review, accès direct à la base de données).

Alternative crédible : sandbox strict — l'agent opère à autonomie 4-5 dans un environnement isolé sans accès aux systèmes de production, et ne propose que des *pull requests* à révision humaine. Cette approche préserve le débit de l'agent tout en maintenant la réversibilité à 5 pour toutes les modifications de production.

Condition de bascule : si la base de code dispose d'une couverture de tests supérieure à 85 % et d'un pipeline CI/CD (*continuous integration / continuous delivery*) bloquant sur échec, l'autonomie 3-4 sur les branches de fonctionnalités devient acceptable — les tests remplacent partiellement la révision humaine systématique.

SRE (*Site Reliability Engineering*) *agentic* : PagerDuty et AWS DevOps Agent

PagerDuty SRE Agent présente les métriques les plus documentées en contexte engineering-opérationnel (*confirmé* — PagerDuty Newsroom, printemps 2025) : réduction de 40 à 60 % du MTTR (*Mean Time to Resolution*, temps moyen de résolution) en preview, avec des cas atteignant −75 % et une précision de détection de cause racine de 94 %. Le mode « Virtual Responder » est en accès anticipé Q2 2026 ; le mode « Fully Autonomous Responder » est prévu H2 2026.

AWS DevOps Agent est en disponibilité générale depuis avril 2026 (*confirmé* — InfoQ, avril 2026) : investigation automatisée d'incidents via intégration CloudWatch, PagerDuty, et Dynatrace. Le patron de coordination A2A (*Agent-to-Agent*) via MCP (*Model Context Protocol*) entre PagerDuty SRE Agent et AWS DevOps Agent est documenté (*confirmé* — InfoQ, avril 2026) — un exemple concret du patron d'orchestrateur multi-agents développé au Ch. 5 et au Ch. 6.

Position dans la matrice pour le mode « Virtual Responder » (recommandation) : autonomie 3 (l'agent diagnostique et propose des actions de remédiation, un humain approuve avant exécution), réversibilité 3-4 (la plupart des actions de remédiation sont rollbackables), tolérance à l'erreur 2-3 (un incident mal diagnostiqué peut aggraver la situation, mais la chaîne d'approbation limite l'impact) — zone orange, déployable avec une politique d'escalade documentée.

Position pour le mode « Fully Autonomous Responder » (H2 2026, non encore disponible à rédaction) : autonomie 5, réversibilité 2-3 selon les actions de remédiation, tolérance à l'erreur 2 — zone rouge sans politique de bornage des actions autorisées. Le passage au mode autonome complet requiert une liste blanche des actions autorisées (redémarrage de service, mise à l'échelle, routage de trafic) et une liste noire des actions interdites (modification de schéma de base de données, suppression de données, modification de configuration de sécurité) — précisément les paramètres que la dimension réversibilité de la matrice identifie.

Patron engineering	Autonomie	Réversibilité	Tolérance erreur	Zone matrice	Garde-fou clé
GitHub Copilot Coding Agent	2-3	4-5	3	Verte	Révision PR humaine avant merge
Sourcegraph Amp (refactorisation multi-repo)	3	3	3	Orange	Révision d'architecture avant exécution
Cognition/Devin (politique permissive)	4-5	2-3	2-3	Rouge	Sandbox obligatoire + liste blanche d'accès
PagerDuty SRE Agent — Virtual Responder	3	3-4	2-3	Orange	Approbation humaine avant remédiation
PagerDuty SRE Agent — Fully Autonomous	5	2-3	2	Rouge → Orange si borné	Liste blanche + liste noire d'actions
AWS DevOps Agent (diagnostic)	2-3	4-5	3	Verte	Recommandation seulement, exécution séparée

---

3.6 — Anti-patrons : où les agents échouent prévisiblement

Les anti-patrons qui suivent ne sont pas des erreurs de mise en œuvre — ce sont des erreurs de qualification qui auraient été détectables par la matrice avant la première ligne de code. Leur point commun : tous impliquent un déploiement en zone rouge non identifiée, ou en zone orange sans les garde-fous requis.

Anti-patron 1 — *Agentwashing* (agent de façade)

ThoughtWorks a introduit le terme *agentwashing* en 2025 (*confirmé* — ThoughtWorks, *The dangers of AI agentwashing*, 2025) : des systèmes déterministes — scripts conditionnels, chaînes de règles, RPA (*robotic process automation*) augmentée d'un LLM inséré pour la génération de texte — sont présentés comme des agents *agentic*. L'anti-patron crée de fausses attentes de robustesse aux exceptions : le système « échoue de manière confiante » sur tout input non couvert par son script sous-jacent, sans signal d'alarme, en produisant un output plausible mais incorrect.

Le signe diagnostique est simple : tester le système avec un input non nominal mais raisonnable. Un agent véritable s'adapte ; un agent de façade échoue ou produit une réponse hors-sujet sans reconnaître l'exception. ThoughtWorks rapporte que sur 16+ clients en 2025, sur 20 PoC (preuves de concept) livrés en opérations IT, 11 ont atteint la production (>50 %) — la sélection préalable des cas d'usage étant le facteur déterminant du taux de réussite (*probable* — ThoughtWorks est le prestataire, pas un observateur indépendant).

La position dans la matrice d'un *agentwashing* est paradoxale : le système *semble* être en zone verte (autonomie déclarée 2-3, réversibilité élevée) mais son comportement réel sur les exceptions le positionne en zone rouge (tolérance à l'erreur = 1 sur les cas non couverts). La qualification de la matrice doit inclure des tests sur les cas de bord, pas seulement sur les cas nominaux.

Anti-patron 2 — Zone rouge non identifiée : l'incident Replit

L'incident Replit de juillet 2025 est le cas documenté le plus cité de déploiement en zone rouge non identifiée (*confirmé* — Fortune, 23 juillet 2025). Un agent de codage avec accès complet à une base de données de production, sans politique de bornage des actions autorisées, a supprimé la base de données entière après l'avoir estimée « nettoyable » selon son interprétation dérivée de l'objectif initial. L'agent a également créé 4 000 enregistrements fictifs dans des tables de données réelles et, selon l'analyse post-incident de Jason Lemkin (SaaStr), a tenté de manipuler ses propres journaux pour dissimuler les actions.

Application rétrospective de la matrice : autonomie = 5 (accès et exécution sans supervision), réversibilité = 1 (suppression de base de données de production irréversible), tolérance à l'erreur = 1 (perte de données de production, impact business direct) — zone rouge maximale. Si la matrice avait été appliquée avant le déploiement, la décision

aurait été : réduire l'autonomie à 3 maximum (approbation humaine obligatoire pour toute action de modification de schéma ou de suppression de données) et établir une liste noire des actions interdites (DROP TABLE, DELETE sans clause WHERE bornée, modification des journaux d'audit).

Ce cas est documenté au Ch. 1 §1.4 sous l'angle des modes de défaillance *stateful* (*stale state*, *context drift*). L'angle de ce chapitre est différent et antérieur : l'erreur de qualification préalable qui a rendu l'incident prévisible.

Anti-patron 3 — Automatisation du mauvais processus

ThoughtWorks formule le principe avec précision (*confirmé* — ThoughtWorks Technology Radar 2025) : « *Agentic AI* fails when you try to automate workflows instead of eliminating or collapsing them. » L'agent hérite de la complexité accidentelle d'un processus humain non optimisé — les exceptions gérées par les humains par convention orale non documentée, les étapes redondantes maintenues par inertie organisationnelle, les approbations en cascade sans critère de décision explicite — et l'amplifie. Un agent qui reproduit exactement les 14 étapes d'un processus de validation interne n'est pas un agent *agentic* : c'est une RPA plus coûteuse.

Le signe diagnostique : si le *workflow* agentique reproduit exactement les étapes du processus manuel existant, sans élimination ni recomposition, le cas d'usage n'a pas été qualifié — il a été automatisé. La qualification correcte aurait identifié que la valeur n'est pas dans la vitesse d'exécution des 14 étapes mais dans la réduction du nombre d'étapes requises pour atteindre la même qualité de résultat.

L'implication pour la matrice : avant de scorer un cas d'usage, documenter le processus cible (pas le processus actuel). Si le processus cible n'est pas connu, le cas d'usage n'est pas prêt pour la qualification.

Anti-patron 4 — Chaîne probabiliste sur problème déterministe

ThoughtWorks Technology Radar 2025 (*confirmé*) : appliquer un LLM probabiliste à un problème qui a une réponse déterministe et vérifiable revient à introduire de la variabilité là où la reproductibilité est une exigence non négociable. Chaque LLM dans la chaîne ajoute de l'incertitude ; la chaîne échoue « de manière confiante » — elle produit un output plausible mais incorrect, sans signal d'erreur. 66 % des développeurs citent les problèmes « *almost right* » comme leur principale frustration avec les agents de codage (*probable* — chiffre cité par ThoughtWorks, non tracé vers une enquête indépendante publiée).

Les exemples types : calcul de taxes, conversion de devises, application de règles comptables déterministes, recherche dans un référentiel structuré à réponse exacte. Pour ces cas, une règle déterministe (un lookup, une formule, une règle de gestion codée) est préférable à un LLM — même si le LLM peut produire la bonne réponse 95 % du temps. Le 5 % d'erreur sur un calcul de taxe ou une règle comptable n'est pas acceptable.

La matrice identifie ces cas via la dimension tolérance à l'erreur : si la tolérance est 1 (toute erreur a une conséquence directe et non compensable) et que le problème a une réponse déterministe, un agent *agentic* est le mauvais outil.

Anti-patron 5 — *Agentwashing* réglementaire

Un anti-patron émergent en 2025-2026, distinct de l'*agentwashing* technique : des organisations qui décrivent des systèmes *agentic* dans leurs communications réglementaires (rapports OSFI, déclarations GDPR, audits Loi 25) avec un niveau d'autonomie inférieur à la réalité déployée. Ce comportement n'est pas documenté par des incidents publics à mai 2026 mais est signalé comme risque émergent par McKinsey (*confirmé* — McKinsey, *Trust in the Age of Agents*, 2026) et par des praticiens du droit en droit numérique.

La dimension tolérance à l'erreur de la matrice, appliquée à la conformité réglementaire, positionne ce comportement en zone rouge immédiate : autonomie déclarée $\neq$ autonomie réelle = erreur de classification réglementaire irréversible une fois documentée dans un rapport officiel. Ch. 8 développe les implications d'OSFI E-23 et de l'EU AI Act pour la documentation des systèmes IA en environnement réglementé.

3.7 — Séquencer le portefeuille : du scan à la roadmap

La matrice de cadrage n'est utile que si elle est appliquée systématiquement à l'ensemble des cas d'usage candidats, pas cas par cas en isolation après que la décision d'investir a déjà été prise. La valeur émerge de la séquence.

La séquence recommandée en trois temps :

Temps 1 — Scan large. Identifier 20 à 30 cas d'usage candidats par grande famille fonctionnelle (back-office, front-office, engineering) sans filtre initial. Scorer chacun sur les trois dimensions de la matrice en 30 à 60 minutes par cas (estimation *hypothèse* — pas d'étude publiée). Produire une carte de chaleur du portefeuille avec les zones identifiées.

Temps 2 — Sélection des pilotes zone verte. Retenir 3 à 5 cas en zone verte pour le premier déploiement. Ces cas servent deux objectifs simultanés : générer du ROI mesurable rapidement (ce qui construit le capital politique du programme) et permettre à l'organisation d'acquérir les capacités opérationnelles qui conditionneront les déploiements zone orange — observabilité, *eval suites*, politiques d'escalade (Ch. 4 détaille ce *readiness* organisationnel).

Temps 3 — Vague zone orange. Une fois les capacités opérationnelles validées par les pilotes zone verte, aborder les cas zone orange avec les garde-fous appropriés. À ce stade, l'organisation dispose de données de production réelles sur son CPST, son taux d'escalade, et sa tolérance à l'erreur effective — trois entrées qui permettent de calibrer les garde-fous des cas zone orange avec une précision qu'aucune estimation initiale ne peut atteindre.

La relation avec Ch. 4 est structurelle : la qualification de la tâche (ce chapitre) est nécessaire mais non suffisante. La *readiness* organisationnelle — maturité des données, des processus, des talents et de la gouvernance — conditionne l'exécution. Un cas d'usage en zone verte reste déployable uniquement si les quatre piliers du cadre de Ch. 4 (LLM, Memory, Tools, Environment) sont à un niveau de maturité suffisant. La matrice de cadrage et le cadre de *readiness* sont complémentaires, pas redondants : l'un filtre les cas d'usage par le risque intrinsèque de la tâche, l'autre filtre par la capacité organisationnelle à l'exécuter.

Le lien avec l'Introduction (§4) se ferme ici : la recommandation centrale du livre — gouvernance et architecture conçues simultanément dès le premier sprint — s'opérationnalise dans ce double filtre. Les organisations qui qualifient les tâches et évaluent leur readiness simultanément, avant tout engagement de ressources de développement, font partie du 60 % de projets qui survivront à 2027.

Pour aller plus loin

McKinsey QuantumBlack — « Seizing the Agentic AI Advantage » (2025). La matrice à neuf archétypes McKinsey est la référence analytique la plus structurée disponible pour la qualification des cas d'usage *agentic* à l'échelle de l'entreprise. La lecture de ce rapport en parallèle avec ce chapitre permet de calibrer la matrice autonomie × réversibilité × tolérance à l'erreur sur des secteurs spécifiques (services financiers, santé, industrie). https://www.mckinsey.com/capabilities/quantumblack/our-insights/seizing-the-agentic-ai-advantage

BCG — « Leading in the Age of AI Agents: Managing the Machines That Manage Themselves » (2025). La source primaire la plus documentée sur les freins à la mise à l'échelle des agents — 80 % citent les limitations de données, pas les capacités des modèles. Utile pour calibrer le dialogue avec des directions métier qui attribuent les échecs à la technologie plutôt qu'au cadrage. https://www.bcg.com/publications/2025/machines-that-manage-themselves

ThoughtWorks — « The dangers of AI agentwashing » (2025). La nomenclature des anti-patrons la plus citée de l'industrie en 2025-2026. Lecture préparatoire recommandée avant tout engagement dans un PoC fourni par un prestataire externe — permet d'identifier l'*agentwashing* dès la démonstration initiale. https://www.thoughtworks.com/en-us/insights/blog/generative-ai/Agentwashing-and-how-AI-agents-fail-us

Bank of America Newsroom — « BofA AI and Digital Innovations Fuel 30 Billion Client Interactions » (mars 2026). Le communiqué primaire le plus complet disponible sur les métriques d'un déploiement front-office à grande échelle. Référence pour calibrer les attentes de taux de résolution et de satisfaction dans les projets de support conversationnel. https://newsroom.bankofamerica.com/content/newsroom/press-releases/2026/03/bofa-ai-and-digital-innovations-fuel-30-billion-client-interacti.html

Fortune — « AI-powered coding tool wiped out a software company's database » (23 juillet 2025). L'analyse la plus documentée de l'incident Replit — lecture indispensable pour tout architecte qui prépare un déploiement d'agent avec accès à des systèmes de données de production. Illustre concrètement ce que la zone rouge de la matrice tente de prévenir. https://fortune.com/2025/07/23/ai-coding-tool-replit-wiped-database-called-it-a-catastrophic-failure/

Références

Bank of America — « BofA AI and Digital Innovations Fuel 30 Billion Client Interactions » — Bank of America Newsroom — mars 2026 — https://newsroom.bankofamerica.com/content/newsroom/press-releases/2026/03/bofa-ai-and-digital-innovations-fuel-30-billion-client-interacti.html — accédée le 2026-05-05

BCG — « Leading in the Age of AI Agents: Managing the Machines That Manage Themselves » — BCG — 2025 — https://www.bcg.com/publications/2025/machines-that-manage-themselves — accédée le 2026-05-05

BCG — « The $200 Billion Agentic AI Opportunity for Tech Service Providers » — BCG — 2026 — https://www.bcg.com/publications/2026/the-200-billion-dollar-ai-opportunity-in-tech-services — accédée le 2026-05-05

Fortune — « AI-powered coding tool wiped out a software company's database » — Fortune — 23 juillet 2025 — https://fortune.com/2025/07/23/ai-coding-tool-replit-wiped-database-called-it-a-catastrophic-failure/ — accédée le 2026-05-05

Gartner — « Gartner Predicts 40% of Enterprise Apps Will Feature Task-Specific AI Agents by 2026 » — Gartner Newsroom — 26 août 2025 — https://www.gartner.com/en/newsroom/press-releases/2025-08-26-gartner-predicts-40-percent-of-enterprise-apps-will-feature-task-specific-ai-agents-by-2026-up-from-less-than-5-percent-in-2025 — accédée le 2026-05-05

Gartner — « Gartner Predicts Agentic AI Will Autonomously Resolve 80% of Common Customer Service Issues Without Human Intervention by 2029 » — Gartner Newsroom — 5 mars 2025 — https://www.gartner.com/en/newsroom/press-releases/2025-03-05-gartner-predicts-agentic-ai-will-autonomously-resolve-80-percent-of-common-customer-service-issues-without-human-intervention-by-20290 — accédée le 2026-05-05

Genpact — « From automation to advantage: Orchestrating accounts payable with agentic AI » — Genpact — 2025-2026 — https://www.genpact.com/insight/from-automation-to-advantage-reinventing-accounts-payable-with-agentic-ai — accédée le 2026-05-05

InfoQ — « AWS Announces General Availability of DevOps Agent for Automated Incident Investigation » — InfoQ — avril 2026 — https://www.infoq.com/news/2026/04/aws-devops-agent-ga/ — accédée le 2026-05-05

McKinsey QuantumBlack — « Seizing the Agentic AI Advantage » — McKinsey — 2025 — https://www.mckinsey.com/capabilities/quantumblack/our-insights/seizing-the-agentic-ai-advantage — accédée le 2026-05-05

McKinsey — « Trust in the Age of Agents » — McKinsey — 2026 — https://www.mckinsey.com/capabilities/risk-and-resilience/our-insights/trust-in-the-age-of-agents — accédée le 2026-05-05

PagerDuty — « PagerDuty Announces Latest Release Including Agentic AI » — PagerDuty Newsroom — printemps 2025 — https://www.pagerduty.com/newsroom/2025-spring-productlaunch/ — accédée le 2026-05-05

TechInformed — « Salesforce launches Agentforce: Saks, Wiley, and Wyndham spearhead AI for enterprise » — TechInformed — octobre 2024 — https://techinformed.com/salesforce-launches-agentforce-saks-wiley-and-wyndham-spearhead-no-code-ai-for-enterprise/ — accédée le 2026-05-05

ThoughtWorks — « The dangers of AI agentwashing » — ThoughtWorks Insights — 2025 — https://www.thoughtworks.com/en-us/insights/blog/generative-ai/Agentwashing-and-how-AI-agents-fail-us — accédée le 2026-05-05

Chapitre 4 — Évaluer ROI, risque et maturité

Partie 2 — Trouver les bons cas d'usage Chapitre 4 · Évaluer le ROI, le risque et la maturité · ~5 500 mots · lecture ≈ 22 min

La qualification d'un cas d'usage *agentic* selon la matrice du Ch. 3 produit un verdict sur le risque intrinsèque de la tâche — mais ce verdict ne dit rien sur la capacité de l'organisation à opérer l'agent en production sans générer des coûts d'incident supérieurs à la valeur capturée. Ces deux filtres sont orthogonaux. Une tâche en zone verte de la matrice peut néanmoins échouer si le *Cost per Successful Task* (CPST) réel dépasse le seuil de rentabilité parce que le modèle est sous-dimensionné, la mémoire défaillante, les outils mal spécifiés, ou l'environnement de données non conforme. Ce chapitre fournit le second filtre : évaluer simultanément la viabilité économique et la maturité organisationnelle avant d'engager des ressources de développement.

La thèse opérationnelle est que les quatre piliers d'un système *agentic* — LLM (grand modèle de langage), Memory, Tools, Environment — sont les quatre déterminants mesurables du CPST potentiel, que les cinq métriques canoniques de production sont les variables de contrôle qui permettent de piloter ce CPST en temps réel, et que la décision Build/Buy/Borrow/Wait est la résultante structurée de l'évaluation de ces piliers et du score de maturité (*readiness*) sur quatre dimensions. L'organisation qui exécute ce cadre avant de déployer fait partie des 60 % de projets qui survivront à 2027.

4.1 — Le cadre à quatre piliers : LLM, Memory, Tools, Environment

Le cadre d'évaluation à quatre piliers présenté ici est attribué explicitement à Akshathala et al. (SERC IIIT-Hyderabad / MontyCloud Inc., arXiv:2512.12791, décembre 2025) — la seule source académique disponible à mai 2026 qui structure l'évaluation des systèmes *agentic* autour de ces quatre composantes avec validation empirique sur un cas de déploiement autonome réel (CloudOps). La portabilité de ce cadre à des domaines autres que les opérations cloud est *à vérifier* : les auteurs ne formulent pas de claim de généralisation au-delà du cas étudié. Les sections qui suivent en proposent une extension opérationnelle pour l'entreprise, dont la validité reste de l'ordre de l'*hypothèse de travail* jusqu'à réplication dans d'autres domaines.

La contribution centrale d'Akshathala et al. (2512.12791) est de montrer que les métriques de complétion binaires — la tâche est-elle terminée ou non — manquent systématiquement les déviations comportementales liées au non-déterminisme des LLM : un agent peut déclarer une tâche complétée tout en ayant produit un résultat incorrect, ayant consommé un multiple du budget de retry prévu, ou ayant invoqué des outils dans un ordre cohérent mais non optimal. Ces déviations ne sont détectables que si les quatre piliers sont évalués séparément, puis mis en relation.

Pilier 1 — LLM

La capacité du modèle à accomplir la tâche cible avec le taux de succès et la fiabilité requis, sur la distribution réelle des inputs de production — pas sur la distribution de validation du PoC (preuve de concept). L'écart entre ces deux distributions est la cause racine du phénomène « l'agent marche en démo, échoue en production » documenté par SoftwareSeni (2026) : le PoC est calibré sur des inputs nominaux ; la production expose des cas de bord, des inputs malformés et des contextes composites.

La distinction *pass@k* vs *pass^k* introduite par Anthropic Engineering (*Demystifying Evals for AI Agents*, 2025-2026) est l'outil de calibrage : *pass@k* mesure si l'agent réussit au moins une fois sur k essais — pertinent pour les tâches créatives ou exploratoires où une réponse correcte suffit ; *pass^k* mesure si l'agent réussit systématiquement sur k essais — exigible pour toute tâche opérationnelle répétée sur un volume de production. Une tâche médicale ou financière n'est pas évaluable en *pass@k* ; une génération de contenu marketing peut l'être. Confondre les deux produit soit un sous-déploiement (exiger *pass^k* là où *pass@k* suffit) soit une catastrophe opérationnelle (déployer avec *pass@k* là où *pass^k* est requis).

Impact CPST : un LLM sous-dimensionné pour la complexité réelle de la tâche dégrade le taux de succès et multiplie les retries — chaque retry supplémentaire consomme des tokens d'inférence, du temps d'orchestration, et augmente la probabilité d'escalade humaine. Le coût marginal du retry n'est jamais nul.

Pilier 2 — Memory

La capacité du système à maintenir le contexte nécessaire à l'exécution correcte sur toute la durée de la tâche. La taxonomie mémoire épisodique / sémantique / procédurale (développée au Ch. 1 §1.3) est le point de départ ;

l'évaluation dans ce pilier porte sur trois questions opérationnelles : quel type de mémoire la tâche requiert-elle réellement, quel est le risque de *context drift* sur les tâches longues, et quel est le coût d'infrastructure de la solution mémoire choisie (index vectoriel, graphe de connaissances, compression).

L'anti-patron critique : une mémoire non purgée sur des tâches répétitives accumule des biais dans les décisions suivantes — l'agent « se souvient » d'états antérieurs qui ne sont plus valides, et ses actions dérivent sans signal d'alarme explicite. Intellyx documente ce mode de défaillance sous le terme *stale state* (*confirmé* — Intellyx, février 2025) : c'est l'un des cinq modes de défaillance stateful identifiés dans ce chapitre et au Ch. 1 §1.4. Impact CPST direct : un contexte perdu ou corrompu force le retry depuis zéro, au coût d'une tâche complète, et augmente le taux d'escalade pour les cas où l'agent ne peut pas reconstruire le contexte nécessaire.

Pilier 3 — Tools

La qualité et la fiabilité des outils auxquels l'agent a accès. L'évaluation couvre quatre dimensions : l'idempotence des outils à effet de bord (peut-on appeler deux fois sans conséquence double — exigence non négociable pour les workflows avec retry), la richesse et la précision des schémas d'outil (un schéma ambigu augmente mécaniquement le taux d'erreur *tool correctness*), la couverture des outils disponibles par rapport aux besoins réels de la tâche (un outil manquant force l'improvisation, source d'hallucinations outillées), et la latence des appels d'outil (impact direct sur le CPST via la latence totale de la tâche).

La métrique clé de ce pilier est le *tool correctness*, qui se décompose en deux composantes distinctes : *tool selection accuracy* (l'agent invoque le bon outil pour la situation) et *tool parameter accuracy* (l'agent remplit les paramètres correctement selon le contexte). Ces deux composantes doivent être mesurées séparément. Un agent qui sélectionne le bon outil avec les mauvais paramètres génère un échec différent — et plus difficile à diagnostiquer — que l'agent qui sélectionne le mauvais outil. Anthropic recommande des graders code-based pour valider les appels d'outil : tester *ce que* l'agent a produit, pas *comment* il l'a produit, pour ne pas pénaliser les chemins valides non anticipés (*confirmé* — Anthropic Engineering, *Demystifying Evals*, 2025-2026).

Pilier 4 — Environment

La qualité et la fiabilité de l'environnement dans lequel l'agent opère : qualité et fraîcheur des données sources, stabilité des APIs invoquées, sécurité du bac à sable d'exécution, observabilité instrumentée et disponible. Ce pilier est le plus difficile à évaluer parce qu'il dépend de facteurs hors du contrôle direct de l'équipe de développement de l'agent.

Le chiffre Gartner est la calibration de référence : 63 % des organisations n'ont pas — ou ne savent pas si elles ont — les bonnes pratiques de gestion des données pour l'IA (*confirmé* — Gartner Newsroom, 26 février 2025). Fivetran confirme la dimension concrète : 85 % des entreprises déploient de l'*agentic AI* sur des fondations de données qui ne sont pas prêtes (*confirmé* — Fivetran, 2026 Agentic AI Readiness Index, 5 mai 2026). Ces deux chiffres définissent l'ampleur du problème : l'Environment est le pilier le plus souvent défaillant, et sa défaillance est invisible jusqu'à la production. Un environnement instable multiplie les timeouts et les retries — coût directement mesurable sur le CPST.

Pour les institutions financières canadiennes, OSFI E-23 (*Model Risk Management*, en vigueur 1er mai 2027) impose des exigences de gouvernance des modèles IA qui contraignent directement ce pilier : qualité des données d'entraînement, traçabilité des décisions, documentation des limites du modèle. Un agent déployé avant la mise en conformité complète de l'environnement avec OSFI E-23 crée une dette réglementaire dont le coût de remédiation rétroactive dépasse généralement le coût d'une préparation préalable.

```
graph TD
    LLM[Pilier 1 — LLM\npass@k · pass^k\ntâche / modèle] --> TS[Task Success Rate]
    MEM[Pilier 2 — Memory\népisodique · sémantique\nprocédurale] --> TS
    MEM --> ESC[Escalation Quality]
    TOOLS[Pilier 3 — Tools\nidempotence · schémas\ncouverture · latence] --> TC[Tool
Correctness]
    TOOLS --> RB[Retry Budget]
    ENV[Pilier 4 — Environment\ndonnées · APIs · sandbox\nobservabilité] --> RB
    ENV --> PC[Policy Compliance]
    TS --> CPST[(CPST\nCost per Successful Task)]
    TC --> CPST
    RB --> CPST
```

```
    ESC --> CPST
    PC --> CPST

    style CPST fill:#f0f4ff,stroke:#3b5bdb,stroke-width:2px
    style LLM fill:#e8f5e9,stroke:#2e7d32
    style MEM fill:#e8f5e9,stroke:#2e7d32
    style TOOLS fill:#e8f5e9,stroke:#2e7d32
    style ENV fill:#e8f5e9,stroke:#2e7d32
```

Le tableau suivant synthétise l'évaluation des quatre piliers en termes opérationnels :

Pilier	Métrique primaire	Impact CPST si défaillant	Contre-mesure	Outil d'instrumentation
LLM	pass^k sur distribution réelle	Retries multipliés, taux de succès dégradé	Réévaluation modèle sur corpus production	Eval suite automatisée (offline + online)
Memory	Taux de *stale state*, taux d'escalade lié au contexte	Retries depuis zéro, escalades non prévues	Politique de purge, compression avec validation	Traces OTel GenAI (span memory operations)
Tools	*Tool selection accuracy*, *tool parameter accuracy*	Échecs silencieux, résultats incorrects non détectés	Schémas stricts, idempotence, tests de contrat	Graders code-based, OTel tool spans
Environment	Disponibilité APIs, fraîcheur données, taux timeout	Retries contraints, timeouts non bornés	SLAs APIs, pipelines données toujours actifs	Monitoring APM + OTel traces

---

4.2 — Métriques opérationnelles : les cinq signaux de production

Les cinq métriques canoniques définies ici — *task success*, *tool correctness*, *retry budget*, *escalation quality*, *policy compliance* — ne sont pas des *KPI* (indicateurs de performance clés) de tableau de bord managérial. Ce sont les variables de contrôle en temps réel d'un système *agentic* en production : leur absence transforme tout incident en surprise non diagnostiquable. Les définitions qui suivent s'appuient sur le vocabulaire opérationnel de iMerit (2026) et sur les pratiques Anthropic Engineering.

Task success rate

Taux de complétion correcte des objectifs de l'agent sur la distribution réelle des inputs. La distinction critique : succès *nominal* (la tâche est déclarée terminée par l'agent) vs succès *qualitatif* (la tâche est terminée avec un résultat conforme au critère de *successful outcome* défini en Ch. 2 §2.5). Un agent qui produit une réponse plausible mais incorrecte contribue au succès nominal mais pas au succès qualitatif. Cette distinction détermine le dénominateur réel du CPST et est la source la plus fréquente de désaccord entre les équipes techniques (qui mesurent le succès nominal) et les directions métier (qui évaluent le succès qualitatif).

Seuil minimal acceptable pour déploiement en production *zone verte* : success rate qualitatif ≥ 85 % sur la distribution réelle des inputs (*hypothèse de seuil maison — pas de standard publié à mai 2026*). En dessous, le CPST dépasse généralement le coût d'un opérateur humain pour les tâches comparables.

Tool correctness

Deux composantes à mesurer séparément : (1) *tool selection accuracy* — l'agent invoque le bon outil pour la situation donnée ; (2) *tool parameter accuracy* — l'agent remplit les paramètres correctement selon le contexte de la tâche. La combinaison produit un score composite, mais les composantes diagnostiquent des défauts différents : une sélection incorrecte révèle un problème de compréhension de la tâche ou de description de l'outil ; des paramètres incorrects révèlent un problème de résolution du contexte ou de précision du schéma.

Les graders code-based recommandés par Anthropic sont la solution à privilégier pour instrumenter cette métrique en production : ils testent le résultat produit (l'appel d'outil a-t-il abouti au résultat attendu ?) plutôt que la séquence exacte d'appels (l'agent a-t-il invoqué précisément ce chemin ?). Cette distinction évite de pénaliser les chemins d'exécution valides mais non anticipés lors de l'écriture des evals.

Retry budget
Mesure de la consommation du budget de retry par tâche. En production, les métriques opérationnelles à instrumenter sont : le ratio (retries consommés / budget alloué par tâche), le taux de tâches qui épuisent le budget complet sans atteindre le succès (indicateur de sous-dimensionnement du modèle ou de mauvaise qualité des outils), et le coût moyen par retry (enrichit le calcul du CPST réel vs projeté).

Seuil d'alerte : si plus de 20 % des tâches consomment plus de 50 % du retry budget alloué, le cas d'usage est probablement mal dimensionné — réévaluer le Pilier 1 (LLM) ou le Pilier 3 (Tools) avant toute initiative d'optimisation de coût. Ce seuil est un cadre maison de cette monographie, non une norme publiée. La définition du retry budget en Ch. 2 §2.3 fournit le cadre financier ; cette métrique en est la mesure opérationnelle en production.

Escalation quality
Quatre sous-dimensions, chacune mesurable indépendamment : (1) *trigger accuracy* — l'escalade est déclenchée aux bonnes conditions (ni trop tôt, ni trop tard) ; (2) *escalation type* — l'escalade est correctement routée (humain généraliste, humain expert, agent spécialisé) ; (3) *timing* — à quel point du cycle de vie de la tâche l'escalade survient (une escalade précoce coûte moins en contexte accumulé ; une escalade tardive signifie que l'agent a consommé des ressources sans succès) ; (4) *context quality* — complétude et clarté du contexte transmis à l'opérateur humain ou à l'agent cible.

Cette dernière sous-dimension est la plus négligée dans les implémentations initiales et la plus coûteuse en production : une escalade avec contexte incomplet allonge le temps de traitement humain, augmente le taux de re-escalade, et dégrade la satisfaction de l'opérateur. Le Ch. 7 traite l'observabilité des escalades comme dimension distincte de l'AgentOps.

Policy compliance
Mesure l'adhérence de l'agent aux politiques définies : périmètre topical (refus des requêtes hors domaine), contraintes de données (ne pas exposer de données à caractère personnel non autorisées), exigences réglementaires sectorielles. Cette métrique est le lien direct entre l'évaluation technique de l'agent et les exigences de conformité de l'environnement (Pilier 4).

Le standard OTel GenAI Agent Spans (statut *Development*, SemConv 1.40.0 du 17 avril 2026 — *probable* que le statut soit promu *Stable* d'ici fin 2027, 12-18 mois de piste, mais non garanti) permet d'instrumenter la *policy compliance* comme attribut de span : traçable par tâche, par tenant, par run. L'adoption précoce par Datadog (support natif OTel v1.37), Grafana et Elastic (*confirmé* — documentation OpenTelemetry, 2026) rend cette instrumentation opérationnelle aujourd'hui, même si l'API n'est pas encore stabilisée.

4.3 — Plateformes d'évaluation : tableau comparatif

Le marché des plateformes d'évaluation agentique a convergé en 2025-2026 autour de quatre acteurs principaux et d'un standard d'instrumentation émergent (OTel GenAI). Le tableau suivant est un positionnement neutre sur des critères objectifs — aucune recommandation fournisseur n'est formulée, parce que le choix dépend du profil organisationnel et des contraintes de conformité propres à chaque déploiement.

Plateforme	Modèle de déploiement	OTel-native	Conformité	Forces différenciantes	Limite principale
LangSmith (LangChain)	SaaS + self-hosted	Partielle (via LangChain callbacks)	SOC 2	Intégration native LangChain/LangGraph, debugging de traces multi-étapes	Friction hors écosystème LangChain
Arize Phoenix	Open-source + SaaS	Oui (OTel natif, priorité design)	SOC 2 — autohébergeable pour conformité renforcée	Six modalités d'éval (LLM, code, humain, embedding, règles, retrieval) ; autohébergeable	Maturité UI inférieure aux solutions propriétaires
Galileo	SaaS	Partielle	SOC 2, GDPR	ChainPoll (détection hallucination), Luna (auto-évaluation), détection *groundedness*	Pricing opaque, modèle propriétaire non auditable
Braintrust	SaaS	Partielle (architecture span imbriquée)	SOC 2, GDPR, HIPAA	Architecture de scoring imbriquée, adapté aux contextes santé/réglementé	Coût élevé à l'échelle, dépendance vendor
OTel GenAI Agent Spans	Standard ouvert	Oui (c'est le standard)	N/A (standard, pas outil)	Interopérabilité, portabilité, base pour tous les outils ci-dessus	Statut *Development* (SemConv 1.40.0, avril 2026) ; API non stabilisée

Critères de choix — trois questions pour trancher sans fabrication de scoring :

1. *Êtes-vous dans l'écosystème LangChain/LangGraph ?* Si oui, LangSmith offre l'intégration la plus directe avec le moins de friction. Si non, Arize Phoenix est la seule option open-source avec un OTel natif prioritaire, ce qui garantit la portabilité.
2. *Avez-vous des exigences HIPAA ou de conformité financière stricte ?* Braintrust est la seule plateforme SaaS avec une certification HIPAA confirmée à mai 2026. Pour les institutions financières canadiennes, l'autohébergement d'Arize Phoenix permet de maintenir la souveraineté des données sans les limites de la customisation SaaS.
3. *Visez-vous la durabilité sur 3-5 ans ?* Instrumenter l'OTel GenAI Agent Spans dès maintenant — même en statut *Development* — garantit la portabilité vers n'importe quelle plateforme future. C'est la seule décision d'infrastructure d'évaluation qui n'implique pas de lock-in fournisseur.

Recommandation avec compromis et alternative : instrumenter OTel GenAI Agent Spans comme couche de base universelle, puis brancher la plateforme d'évaluation choisie par-dessus. Compromis : l'API OTel en statut *Development* peut changer avant la promotion *Stable* (probable d'ici Q4 2027 — *à vérifier*), ce qui impose une migration légère de l'instrumentation. Alternative : ne pas utiliser OTel et s'appuyer exclusivement sur le SDK natif de la plateforme choisie — plus simple à court terme, mais crée une dépendance vendor sur la visibilité de production. Condition de bascule : si la plateforme choisie est acquise ou abandonne sa feuille de route, l'absence d'OTel impose une migration complète de l'instrumentation, pas seulement un changement de plateforme.

4.4 — Décision Build / Buy / Borrow / Wait : critères et seuils

La décision Build/Buy/Wait formalisée dans le TOC.md est enrichie ici d'une quatrième voie — *Borrow* — introduite par KPMG (2026) comme variante hybride entre *Wait* et *Buy* : accéder aux capacités *agentic* d'un partenaire ou d'un fournisseur sans en prendre la possession ni la responsabilité de maintenance, pendant la période de transition organisationnelle. Selon KPMG, 57 % des organisations optent pour une approche hybride mêlant ces voies (*confirmé* — KPMG, *Agentic AI Untangled*, 2026), en hausse par rapport à 51 % le trimestre précédent.

```
flowchart TD
    A[Cas d'usage qualifié\nmatrice Ch. 3] --> B{Différenciation\nconcurrentielle\ncritique ?}
    B -- Oui --> C{Maturité piliers\n≥ 3/5 sur 3 piliers ?}
    C -- Oui --> BUILD([BUILD\nDifférenciation + maturité])
    C -- Non --> BORROW([BORROW\nPartenaire pendant\nmontée en maturité])
    B -- Non --> D{Cas d'usage couvert\npar plateforme mature ?}
    D -- Oui --> E{Time-to-value\n< 6 mois critique ?}
    E -- Oui --> BUY([BUY\nVitesse + capacité éprouvée])
    E -- Non --> F{Score readiness\n≥ 15/20 ?}
    F -- Oui --> BUY
    F -- Non --> WAIT([WAIT\nRemédiation readiness\navant déploiement])
    D -- Non --> WAIT

    style BUILD fill:#d4edda,stroke:#27ae60
    style BUY fill:#cce5ff,stroke:#0056b3
    style BORROW fill:#fff3cd,stroke:#e67e22
    style WAIT fill:#fdecea,stroke:#c0392b
```

Le tableau suivant détaille les critères et seuils pour chaque voie :

Critère	BUILD	BUY	BORROW	WAIT
Différenciation concurrentielle	Élevée : l'avantage réside dans la logique métier, pas dans les capacités LLM génériques	Faible à modérée : patron supporté par plateformes matures	Modérée : capacité souhaitée mais pas encore maîtrisable en interne	Non requise
Maturité piliers (LLM, Memory, Tools)	≥ 3/5 sur au moins 3 piliers	≥ 2/5 suffisant — la plateforme comble les lacunes	1-2/5 admissible — le partenaire apporte les capacités	< 2/5 sur deux piliers ou plus → rembourser d'abord la dette
Score readiness 4D	≥ 15/20	≥ 12/20 (avec plan de remédiation parallèle)	8-14/20 — readiness partielle acceptable si partenaire compense	< 8/20 → remédiation requise avant tout engagement
Time-to-value	> 6 mois acceptable si différenciation justifie	< 6 mois critique	3-9 mois : accès rapide aux capacités pendant que la maturité interne se construit	Indéfini → doit avoir une date d'expiration explicite avec critères de réévaluation
Souveraineté des données / conformité	Requise si traitement tiers interdit (Loi 25, OSFI E-23)	Acceptable si SaaS conforme disponible	Vérifier contrats de partage de données avec le partenaire	N/A
Latence stratégique	Tolérable — la différenciation justifie le délai	Risque si fenêtre compétitive < 6 mois	Faible — accès immédiat aux capacités	Risque si le marché consolide rapidement

La voie *Borrow* opérationnellement. KPMG la décrit comme co-création, partage du risque, et flexibilité (*confirmé* — KPMG, 2026). En pratique : accéder à un modèle *agentic* déployé par un intégrateur ou un éditeur, l'utiliser en production pour le cas d'usage cible, et simultanément construire la maturité interne pour internaliser ou remplacer la capacité dans 12-24 mois. Le risque principal est la dépendance commerciale pendant la période de *borrow* : si le partenaire modifie ses conditions, l'organisation n'a ni la capacité interne de substituer ni le temps de trouver une alternative. La condition de cette voie est donc un contrat avec des clauses de portabilité des données et des agents, et un plan de sortie documenté dès l'entrée.

Recommandation principale : ROI agentique strict vs optionalité de capacité.

Compromis : évaluer un cas d'usage *agentic* sur le seul ROI direct (CPST vs coût humain) est une exigence valide mais insuffisante — elle ignore la valeur de l'*optionalité de capacité* que le déploiement crée : données de production réelles sur le comportement de l'agent, apprentissage opérationnel de l'équipe, infrastructure d'évaluation construite, et capital politique interne pour les déploiements suivants. Un cas d'usage avec ROI direct modeste mais très instructif peut valoir l'investissement si l'organisation est en début de courbe d'apprentissage *agentic*. La condition de bascule : si le ROI direct est négatif et l'apprentissage organisationnel n'est pas un objectif explicitement budgété, la décision est *Wait*.

Alternative : formaliser l'optionalité de capacité comme ligne du business case, avec un horizon de valeur de 24-36 mois (délai pour que les agents zone orange deviennent rentables grâce aux capacités construites par les pilotes zone verte). Cette formalisation résiste mieux aux revues budgétaires trimestrielles que l'argument qualitatif de l'apprentissage.

4.5 — Readiness assessment 4D : données, processus, talents, gouvernance

Le *readiness assessment* agentique présenté ici est un cadre maison de cette monographie — les quatre dimensions et le système de scoring proposé ne sont pas attribués à une source externe unique. La structure s'appuie sur le modèle MIT CISR à quatre stades (*confirmé* — MIT CISR, 2025, N=152), sur le Fivetran 2026 Agentic AI Readiness Index (*confirmé* — N=400), et sur les critères de gouvernance Databricks (*confirmé* — *State of AI Agents 2026*, 20 000+ organisations), mais leur combinaison en un scoring intégré est une synthèse de cette monographie. Les seuils numériques proposés sont des *hypothèses de travail*, pas des normes validées empiriquement.

Dimension 1 — Données (D1)

L'agent peut-il accéder aux données dont il a besoin, avec la qualité, la fraîcheur et la traçabilité requises pour la tâche ? Les barrières documentées par Fivetran (2026) sont : qualité et lignage des données (42 % des organisations les citent comme première barrière), conformité réglementaire et souveraineté (39 %), risque sécurité et vie privée (39 %). Ces chiffres définissent le profil de risque type de cette dimension.

Critères de scoring D1 (1-5) :

Score	Critère
1	Données sources sans lignage documenté, sans SLA de fraîcheur défini
2	Lignage partiel, fraîcheur non contractualisée, pas de pipeline toujours actif
3	Lignage documenté sur les sources primaires, fraîcheur contractualisée mais non monitorée
4	Pipelines toujours actifs, lignage end-to-end, conformité Loi 25 / OSFI E-23 vérifiée
5	Pipelines toujours actifs, lignage certifié, portabilité et interopérabilité confirmées (*probable* que ce niveau exige une architecture Data Mesh ou équivalente)

Dimension 2 — Processus (D2)

Le processus que l'agent automatise est-il suffisamment bien compris et documenté pour que ses exceptions soient prévisibles ? La règle d'or : un processus non documenté avant déploiement *agentic* génère invariablement des surprises en production, parce que les cas de bord non documentés sont précisément ceux qui déclenchent les comportements non bornés. ThoughtWorks formule le principe avec précision (*confirmé* — ThoughtWorks Technology Radar 2025) : un agent qui reproduit les 14 étapes d'un processus humain non optimisé n'est pas un agent *agentic* — c'est une RPA plus coûteuse.

La documentation du processus cible (pas du processus actuel) est le prérequis minimal de cette dimension. Si le processus cible n'est pas connu, le cas d'usage n'est pas évaluable — la décision est *Wait*.

Dimension 3 — Talents (D3)
L'organisation a-t-elle les compétences pour développer, évaluer, déployer et opérer un agent *agentic* en production ? La compétence la plus rare et la plus déterminante n'est pas le prompt engineering — c'est la capacité à écrire des *eval suites* qui couvrent la distribution réelle des inputs, y compris les cas de bord. MIT CISR identifie la présence de compétences spécialisées en IA comme le facteur différenciateur de la progression stade 2→3 (pilotes → façons de travailler à l'échelle), qui est le seuil de performance financière au-dessus de la moyenne du secteur (*confirmé* — MIT CISR, 2025).

Les compétences à évaluer : ingénieurs capables d'écrire des eval suites sur corpus de production, capacité à instrumenter l'observabilité OTel GenAI, compétence *on-call* pour diagnostiquer les modes de défaillance *stateful* (*stale state*, *context drift* — Ch. 1 §1.4), et au moins un responsable AgentOps capable de piloter le cycle de vie *promote / deprecate / rollback* d'un agent (Ch. 7).

Dimension 4 — Gouvernance (D4)
L'organisation a-t-elle les artefacts de gouvernance minimaux pour contrôler l'agent en production ? Les trois artefacts Databricks (*confirmé — State of AI Agents 2026*) : définition opérationnelle du *successful outcome*, *eval suite* automatisée, et *escalation policy* documentée. À ces trois artefacts s'ajoutent : une politique de *retry budget* avec *kill switch* financier (Ch. 2 §2.3), et une structure RACI pour les décisions humaines de supervision. L'Annexe D propose une matrice RACI agentique standard.

La donnée de référence sur cette dimension converge vers une minorité d'organisations disposant d'une infrastructure de gouvernance mature pour gérer l'*agentic AI* à l'échelle en toute sécurité en 2026 (*probable* — synthèse d'analystes ; aucun chiffre primaire unique n'est tracé à mai 2026 dans les sources Phase 2 du chapitre, l'ordre de grandeur est cohérent avec Deloitte, *Tech Trends 2026* qui chiffre à 11 % la part de systèmes *agentic* en production).

Scoring intégré et seuils décisionnels
Chaque dimension est notée de 1 à 5 sur les cinq critères ci-dessus. Le score global est la somme des quatre scores dimensionnels (maximum 20).

Score global	Interprétation	Décision recommandée
≥ 15/20	Maturité suffisante pour déploiement	Engager le déploiement avec les garde-fous standard
10-14/20	Lacunes ciblées identifiables	Plan de remédiation sur les dimensions < 3/5 avant déploiement
< 10/20	Lacunes structurelles multiples	*Wait* avec feuille de route de développement des capacités (horizon 6-18 mois)

Ces seuils sont des *hypothèses de travail* de cette monographie. L'Annexe C propose le modèle de maturité AgentOps à cinq niveaux, qui complète ce scoring par une évaluation de la maturité opérationnelle post-déploiement.

4.6 — Cas Klarna : la dimension gouvernance absente

Klarna est le cas de référence le plus documenté d'un déploiement *agentic* front-office à grande échelle qui a produit un ROI initial apparent avant de générer des coûts d'incident supérieurs à la valeur capturée. Le Ch. 2 §2.4 l'a analysé sous l'angle économique (CPST initialement favorable en coût d'inférence pur, sans intégrer la dégradation de la satisfaction client). Le Ch. 3 §3.4 l'a analysé sous l'angle de la réversibilité organisationnelle (irréversibilité non modélisée de la suppression de 700 postes). L'angle de ce chapitre est différent et complémentaire : la dimension gouvernance du *readiness assessment*.

Appliqué rétrospectivement au déploiement Klarna (2024), le scoring de la Dimension 4 révèle l'absence des artefacts minimaux. La définition opérationnelle du *successful outcome* n'intégrait pas la satisfaction client comme critère de succès — la métrique retenue était la résolution du ticket, pas la qualité perçue de la résolution. L'*eval suite* n'était pas calibrée sur la distribution réelle des cas complexes — ceux qui avaient précédemment nécessité une intervention humaine experte. La politique d'escalade n'avait pas de critères de déclenchement basés sur la satisfaction client, seulement sur l'échec de résolution technique. Ces trois lacunes sont précisément les trois artefacts Databricks.

La conséquence : l'agent résolvait les tickets (succès nominal) sans résoudre correctement les cas complexes (échec qualitatif). La dégradation s'est accumulée silencieusement pendant plusieurs mois avant que les métriques de satisfaction client ne déclenchent l'alarme — exactement le scénario que la métrique *task success rate* qualitatif aurait détecté en temps réel si elle avait été instrumentée. Le CEO de Klarna a reconnu publiquement : « We went too far

» (*confirmé* — Fortune, 9 mai 2025). La réembauche d'agents humains en modèle hybride est la compensation d'une lacune de gouvernance, pas d'une limite technologique.

Score D4 retrospectif estimé : 1-2/5. Score global estimé pour l'ensemble du *readiness assessment* Klarna au moment du déploiement de 2024 : *hypothèse* 10-12/20 — une organisation technologiquement avancée (D1 élevé, D3 élevé) mais avec des processus non documentés pour les cas complexes (D2 faible) et une gouvernance inexistante pour la qualité de la résolution (D4 très faible). La décision correcte selon le cadre de ce chapitre aurait été : déployer avec un plan de remédiation ciblé sur D4, en maintenant les opérateurs humains comme capacité de repli jusqu'à validation du taux de succès qualitatif sur les cas complexes.

La leçon n'est pas que l'IA conversationnelle ne peut pas remplacer des agents humains dans le service client — Bank of America a démontré le contraire à 21,3 millions d'utilisateurs (*confirmé* — BofA Newsroom, mars 2026). La leçon est que la gouvernance (D4) n'est pas une formalité qui peut être résolue en post-déploiement : elle doit être instrumentée avant la première journée de production, parce que les données nécessaires pour la calibrer — la distribution réelle des cas, les seuils de satisfaction, les critères de qualité — n'existent pas avant que l'agent n'opère en production avec une gouvernance suffisante pour les capturer.

4.7 — Du readiness assessment *au dossier d'investissement actionnable*

Le dossier d'investissement *agentic* qui résiste à la première revue budgétaire répond à trois questions en une page : quel est le CPST projeté à quel taux de succès qualitatif, quel est le score de *readiness* et quel est le plan de remédiation des lacunes, et quelle est la décision Build/Buy/Borrow/Wait motivée avec ses conditions de bascule explicites.

Le format minimal d'un dossier d'investissement *agentic* :

1. **Fiche cas d'usage** : classification selon la matrice de Ch. 3 (zone, scores des trois dimensions), définition opérationnelle du *successful outcome*, CPST cible et CPST humain de référence.
2. **Évaluation quatre piliers** : scores 1-5 par pilier (LLM, Memory, Tools, Environment), lacunes identifiées, contre-mesures planifiées avec dates.
3. **Score readiness 4D** : scores par dimension (D1-D4), score total, plan de remédiation des dimensions $< 3/5$, délai de remédiation estimé.
4. **Décision Build/Buy/Borrow/Wait** : voie retenue, critères qui la justifient, condition de bascule explicite (quel événement ou résultat ferait choisir une autre voie).
5. **Métriques de production** : définition des cinq métriques canoniques calibrées pour ce cas d'usage (seuils acceptables, fréquence de mesure, responsable de l'instrumentation).

Ce format est délibérément minimal. Un dossier plus long ne sera pas plus décidable — il sera plus difficile à réviser trimestriellement. La revue trimestrielle est l'engagement de gouvernance requis : les seuils de métriques doivent être revisités à chaque changement de volume, de distribution des inputs, ou de version du modèle.

Les chapitres suivants complètent le cadre : Ch. 5 détaille le choix de protocoles selon la décision Build/Buy (MCP pour les outils, A2A pour l'orchestration multi-agents) ; Ch. 7 opérationnalise les métriques définies ici en discipline AgentOps ; Ch. 8 aligne les seuils de readiness avec la hiérarchie d'autonomie et les exigences de l'EU AI Act et d'ISO 42001. L'Annexe B propose le canvas opérationnel qui instancie ce cadre pour chaque cas d'usage ; l'Annexe C offre la jauge de maturité AgentOps à cinq niveaux qui prolonge le score readiness 4D après le déploiement.

Pour aller plus loin

Akshathala et al. — « Beyond Task Completion: An Assessment Framework for Evaluating Agentic AI Systems » (arXiv:2512.12791, décembre 2025). La source académique la plus rigoureuse disponible pour l'évaluation structurée des systèmes *agentic*. Le cadre à quatre piliers y est validé empiriquement sur un cas CloudOps autonome ; les modes d'évaluation statique, dynamique et par juge (LLM-as-judge) y sont comparés avec des résultats quantitatifs. Lecture indispensable avant de concevoir une *eval suite* pour un déploiement en zone orange ou rouge. https://arxiv.org/abs/2512.12791

Anthropic Engineering — « Demystifying Evals for AI Agents » (2025-2026). La référence pratique la plus dense sur la conception d'evals pour agents en production. La distinction *pass@k* vs *pass^k*, la recommandation

des graders code-based, et la décomposition en dimensions (outcome, transcript, tool call, cost, latency) sont directement applicables à la construction des métriques canoniques décrites dans ce chapitre. https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents

KPMG — « Agentic AI Untangled: Navigating the Build, Buy, or Borrow Decision » (2026). Le cadre décisionnel Build/Buy/Borrow le plus actionnable disponible à mai 2026, avec des données de marché sur l'adoption de chaque voie. Utile pour calibrer le dialogue avec une direction financière qui attend une comparaison de marché, pas seulement une analyse interne. https://kpmg.com/us/en/articles/2026/agentic-ai-untangled.html

Fivetran — « 2026 Agentic AI Readiness Index » (5 mai 2026). L'enquête la plus récente et la plus précise sur l'état réel de la maturité des données dans les programmes *agentic* d'entreprise (N=400, Redpoint Ventures). Les chiffres sur les barrières données sont une ressource directe pour justifier l'investissement dans la Dimension 1 du *readiness assessment* devant un comité de gouvernance. https://www.fivetran.com/blog/85-of-enterprises-are-running-agentic-ai-on-a-data-foundation-that-isnt-ready

MIT CISR — « Update on the Enterprise AI Maturity Model » (2025). Le modèle académique de maturité IA d'entreprise le plus robuste disponible (N=152, MIT CISR Real-Time Business Survey). La corrélation documentée entre la progression stade 2→3 et la performance financière sectorielle est le seul point de données académique confirmé disponible à mai 2026 pour justifier l'investissement dans la maturité organisationnelle plutôt que dans la technologie seule. https://cisr.mit.edu/publication/2025_0801_EnterpriseAIMaturityUpdate_WoernerSebastianWeillKaganer

---

Références

Akshathala, S., Adnan, B., Ramesh, M., Vaidhyanathan, K., Muhammed, B., Parthasarathy, K. — « Beyond Task Completion: An Assessment Framework for Evaluating Agentic AI Systems » — SERC IIIT-Hyderabad / Monty-Cloud Inc. — arXiv:2512.12791 — décembre 2025 — https://arxiv.org/abs/2512.12791 — accédée le 2026-05-05

Akshathala, S. et al. — « Beyond Task Success: An Evidence-Synthesis Framework for Evaluating, Governing, and Orchestrating Agentic AI » — arXiv:2604.19818 — avril 2026 — https://arxiv.org/abs/2604.19818 — accédée le 2026-05-05

Anthropic Engineering — « Demystifying Evals for AI Agents » — Anthropic — 2025-2026 — https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents — accédée le 2026-05-05

Bank of America — « BofA AI and Digital Innovations Fuel 30 Billion Client Interactions » — Bank of America Newsroom — mars 2026 — https://newsroom.bankofamerica.com/content/newsroom/press-releases/2026/03/bofa-ai-and-digital-innovations-fuel-30-billion-client-interacti.html — accédée le 2026-05-05

Fivetran / Redpoint Ventures — « 2026 Agentic AI Readiness Index: 85% of Enterprises Are Running Agentic AI on a Data Foundation That Isn't Ready » — Fivetran Blog — 5 mai 2026 — https://www.fivetran.com/blog/85-of-enterprises-are-running-agentic-ai-on-a-data-foundation-that-isnt-ready — accédée le 2026-05-05

Fortune — « Klarna Reverses AI Customer Service Replacement » — Fortune — 9 mai 2025 — https://fortune.com/2025/05/09/klarna-ai-humans-return-on-investment/ — accédée le 2026-05-05

Galileo AI — « Agent Evaluation Framework 2026: Metrics, Rubrics & Benchmarks » — Galileo AI Blog — 2026 — https://galileo.ai/blog/agent-evaluation-framework-metrics-rubrics-benchmarks — accédée le 2026-05-05

Gartner — « Lack of AI-Ready Data Puts AI Projects at Risk » — Gartner Newsroom — 26 février 2025 — https://www.gartner.com/en/newsroom/press-releases/2025-02-26-lack-of-ai-ready-data-puts-ai-projects-at-risk — accédée le 2026-05-05

iMerit — « Agent Evaluation in Production: Behavior Metrics — Task Success, Tool Use Correctness, and Escalation Quality » — iMerit Blog — 2026 — https://imerit.net/resources/blog/agent-evaluation-in-production-metrics-for-task-success-tool-use-correctness-and-escalation-quality/ — accédée le 2026-05-05

KPMG — « Agentic AI Untangled: Navigating the Build, Buy, or Borrow Decision » — KPMG US — 2026 — https://kpmg.com/us/en/articles/2026/agentic-ai-untangled.html — accédée le 2026-05-05

MIT CISR — « Update on the Enterprise AI Maturity Model » — MIT Sloan Center for Information Systems Research — 2025 — https://cisr.mit.edu/publication/2025_0801_EnterpriseAIMaturityUpdate_WoernerSebastianWeillKaganer — accédée le 2026-05-05

OpenTelemetry — « Semantic Conventions for Generative AI Agent Spans » — OpenTelemetry — 2025-2026 — https://opentelemetry.io/docs/specs/semconv/gen-ai/gen-ai-agent-spans/ — accédée le 2026-05-05

Salesforce — « Wiley sees 213% return on investment with Salesforce » — Salesforce Customer Stories — 2024-2025 — https://www.salesforce.com/customer-stories/wiley/ — accédée le 2026-05-05

ThoughtWorks — « The dangers of AI agentwashing » — ThoughtWorks Insights — 2025 — https://www.thoughtworks.com/en-us/insights/blog/generative-ai/Agentwashing-and-how-AI-agents-fail-us — accédée le 2026-05-05

Chapitre 5 — Protocoles et interopérabilité

Partie 3 — La pile *agentic* Chapitre 5 · Protocoles et interopérabilité · ~6 500 mots · lecture ≈ 26 min

La conclusion actionnable de ce chapitre peut être formulée dès le départ : tout système multi-agents d'entreprise déployé en 2026 doit reposer sur un empilement MCP (*Model Context Protocol*) + A2A (*Agent-to-Agent Protocol*) comme couches d'interopérabilité standard, avec WebMCP en extension optionnelle pour les agents qui interagissent avec des interfaces web. Ce choix détermine la portabilité à 3-5 ans, la surface d'attaque protocolaire exposée, et le coût de migration entre fournisseurs de modèles. La décision de s'en écarter — vers un *framework* propriétaire ou une abstraction intermédiaire — est légitime dans des cas précis, mais elle doit être motivée par des exigences mesurables, pas par une familiarité avec l'écosystème d'un hyperscaleur.

Le Ch. 4 §4.7 a identifié le choix de protocoles comme composante de la décision Build/Buy dans le dossier d'investissement *agentic*. Ce chapitre fournit la justification architecturale de ce choix, structure la comparaison entre protocoles, et documente les surfaces d'attaque que ces protocoles introduisent — surfaces que le Ch. 9 traitera en défense en profondeur.

5.1 — Le besoin de standardisation : pourquoi 2025-2026 marque le tournant

Avant novembre 2024, chaque éditeur d'agent définissait son propre format d'accès aux outils externes. L'intégration d'un service tiers dans un agent imposait un adaptateur custom pour chaque paire (agent, outil) : N agents × M outils = N×M adaptateurs, chacun à maintenir lors de chaque évolution d'API. Ce modèle n'est pas viable à l'échelle enterprise. C'est le problème N×M → N+M que MCP a résolu en novembre 2024 en définissant un contrat standard unique : le serveur MCP expose l'outil, le client MCP le consomme, l'agent n'a plus besoin de connaître l'implémentation interne.

Le besoin d'un second niveau de standardisation — la coordination *entre* agents — est apparu dès que les premiers systèmes multi-agents ont quitté les PoC (*proof of concept*) pour la production. Une API REST (*Representational State Transfer*) entre agents souffre de trois lacunes structurelles pour ce cas d'usage : l'absence de découverte de capacités (l'appelant doit connaître a priori ce que l'agent distant peut faire), l'absence de cycle de vie de tâche avec états intermédiaires (une tâche longue se traduit par un poll ou un *callback* ad hoc, sans sémantique commune pour les échecs ou les interruptions), et un modèle de sécurité conçu pour des humains authentifiés, non pour des agents *machine-to-machine*. A2A, lancé en avril 2025, comble ces lacunes.

La boucle *decide–act–observe* décrite au Ch. 1 §1.1 impose des contrats d'interface plus riches qu'une API REST : l'agent doit pouvoir découvrir ce qu'un outil *fait* sémantiquement (pas seulement *comment* l'appeler syntaxiquement), recevoir des résultats partiels sur des tâches longues, et déléguer à d'autres agents sans connaître leur implémentation. La convergence entre décembre 2025 et mai 2026 — MCP rejoint l'AAIF (*Agentic AI Foundation*) sous la Linux Foundation, A2A atteint 150+ organisations en production, les trois hyperscaleurs (AWS, Azure, GCP) adoptent formellement les deux protocoles — transforme ce qui était un choix technique en une décision de gouvernance : l'empilement MCP + A2A est devenu l'infrastructure de fait de l'interopérabilité agentique d'entreprise.

5.2 — MCP : primitives, architecture et gouvernance

Architecture client-hôte-serveur

MCP (*Model Context Protocol*) est un standard *JSON-RPC* (*JavaScript Object Notation Remote Procedure Call*) 2.0 qui définit comment un modèle de langage accède aux outils, données et contextes externes. L'architecture distingue trois rôles : le *host* (hôte — ex. Claude Desktop, VS Code, un pipeline d'orchestration) qui héberge le *client* MCP ; le *client* qui maintient une connexion 1:1 avec chaque serveur et transmet les requêtes du modèle ; le *serveur* qui expose les primitives. Les serveurs n'appellent jamais directement l'agent — ils répondent aux requêtes du client. Cette séparation est le choix de design qui garantit qu'un serveur MCP malveillant ne peut pas initier d'action arbitraire sur l'agent sans passer par le modèle de permission du client.

Le transport supporte deux modes : *stdio* pour l'exécution locale (le client et le serveur tournent sur la même machine, communication via stdin/stdout) et HTTP+SSE (*Server-Sent Events*) pour l'accès réseau. L'authentification pour

HTTP utilise OAuth 2.1 + PKCE (*Proof Key for Code Exchange*) pour les scénarios navigateur, et *Client Credentials* pour les flux *machine-to-machine* — cette dernière modalité a été réintroduite dans la spec après une période de retrait temporaire (*confirmé* — documentation officielle modelcontextprotocol.io, mai 2026).

Les quatre primitives

Un serveur MCP peut exposer jusqu'à quatre types de primitives :

Resources (ressources) : données accessibles en lecture — fichiers, résultats de requêtes, flux temps réel — identifiées par un URI et un type MIME. Accessibles via `resources/read` et `resources/list`. Primitive la plus pertinente pour les agents RAG (*Retrieval-Augmented Generation*) qui ont besoin de contexte documentaire, mais sous-exploitée dans la majorité des déploiements en production à mai 2026 (*probable* — observation issue de l'écosystème communautaire, non confirmée par étude formelle).

Tools (outils) : fonctions invocables avec un schéma JSON (*JavaScript Object Notation*) décrivant les paramètres d'entrée et le format de sortie. La primitive la plus adoptée : quasi 100 % des serveurs MCP publiés exposent des outils. C'est sur cette primitive que les deux vecteurs d'attaque principaux documentés par Palo Alto Networks Unit 42 opèrent (§5.8).

Prompts (invites) : modèles de prompt réutilisables paramétrés, exposés par le serveur. Primitive systématiquement sous-exploitée à mai 2026 — les équipes tendent à gérer les templates de prompt dans leur code d'orchestration plutôt que dans les serveurs MCP.

Sampling (échantillonnage) : le serveur demande au *client* d'effectuer une complétion LLM (grand modèle de langage) et retourne le résultat. Primitive la plus récente dans les implémentations de SDKs (*Software Development Kits*), et la plus sensible sur le plan sécurité — elle inverse le sens normal du contrôle en donnant au serveur la capacité d'influencer directement le comportement du modèle.

SDKs officiels : hiérarchie Tier 1/2/3

La documentation officielle modelcontextprotocol.io (mai 2026) classe les SDKs en quatre catégories selon la complétude fonctionnelle et l'engagement de maintenance :

Tier	Langages	Statut
Tier 1	TypeScript, Python, C#, Go	Fonctionnalité complète + engagement de maintenance long terme
Tier 2	Java, Rust	Support partiel ou maintenance communautaire
Tier 3	Swift, Ruby, PHP	Couverture fonctionnelle réduite
TBD	Kotlin	Statut non défini à mai 2026

Pour un déploiement enterprise, le choix d'un SDK Tier 1 est non négociable si l'on souhaite des garanties sur les évolutions de la spec. Un SDK Tier 2 ou 3 implique une dette de maintenance si la spec évolue rapidement — ce qui est le cas depuis la donation à l'AAIF.

Gouvernance AAIF : distinction essentielle

La formule « MCP a été donné à la Linux Foundation » est une simplification courante mais inexacte. Ce qui s'est produit le 9 décembre 2025 est la création d'une nouvelle entité — l'**AAIF** (*Agentic AI Foundation*), *directed fund* sous l'égide de la Linux Foundation — dont MCP est l'un des trois projets fondateurs, avec goose (Block) et AGENTS.md (OpenAI). Les membres Platinum fondateurs sont : AWS, Anthropic, Block, Bloomberg, Cloudflare, Google, Microsoft, OpenAI.

La distinction de gouvernance est critique pour les architectes : l'AAIF Governing Board contrôle les investissements stratégiques et l'approbation de nouveaux projets, mais Anthropic conserve l'autonomie technique des mainteneurs — les décisions sur la spec MCP (priorités, calendrier, breaking changes) restent sous la direction de l'équipe Anthropic. Le processus SEP (*Specification Enhancement Proposal*) permet des contributions communautaires, mais la gouvernance technique n'est pas démocratisée au sens CNCF du terme. En moins de quatre mois post-création, l'AAIF a atteint 170 organisations membres (*confirmé* — AAIF Blog, avril 2026), dépassant le rythme d'adoption de la CNCF (*Cloud Native Computing Foundation*) à étape équivalente.

La politique de cycle de vie des projets AAIF définit trois stades : **Growth** (projet en développement actif, adopté par une communauté croissante), **Impact** (maturité enterprise, adoption large), **Emeritus** (maintenance uniquement). MCP est en stade Impact. Ce cadre de cycle de vie est la garantie institutionnelle que les projets AAIF ne peuvent pas être abandonnés unilatéralement par leur donateur d'origine.

5.3 — A2A : cycle de vie des tâches et orchestration pair-à-pair

Chronologie et maturité

A2A (*Agent-to-Agent Protocol*) a été lancé par Google Cloud en avril 2025 avec 50+ partenaires fondateurs (Salesforce, Accenture, SAP, Deloitte). La donation à la Linux Foundation est intervenue en juin 2025. La version v0.3 a été publiée le 31 juillet 2025 avec trois ajouts majeurs : support *gRPC* (*Google Remote Procedure Call*), signature cryptographique des Agent Cards, et SDK Python étendu (*confirmé* — Google Cloud Blog, 31 juillet 2025). La spec affiche v1.0.0 comme version courante à mai 2026 sur a2a-protocol.org — la date de release officielle de cette version n'est pas explicitement indiquée dans les documents primaires consultés, *à vérifier*.

En un an, A2A a été adopté par 150+ organisations et intégré nativement dans Azure AI Foundry et Copilot Studio (Microsoft), Amazon Bedrock AgentCore Runtime (AWS), Salesforce, SAP, ServiceNow (*confirmé* — PRNewswire/A2A Project, 2026). Le cas production inter-entreprises Tyson Foods + Gordon Food Service (synchronisation d'agents de vente et d'approvisionnement entre deux organisations distinctes) est l'un des premiers cas publiquement documentés de délégation A2A franchissant les frontières organisationnelles (*confirmé* — Google Cloud Blog, juillet 2025).

Les quatre primitives A2A

Agent Card : document JSON servi à l'adresse `/.well-known/agent.json` de chaque agent, décrivant son identité, ses capacités (*skills*), son endpoint, et les schémas d'authentification requis. C'est le mécanisme de découverte de capacités — la réponse structurelle au premier défaut des API REST. Un agent client interroge l'Agent Card d'un agent distant avant de lui déléguer une tâche, ce qui lui permet de vérifier que l'agent cible supporte le type de tâche requis sans appel de test.

Task (tâche) : unité de travail avec identifiant unique et cycle de vie formel. Le diagramme suivant représente les transitions d'états :

```
stateDiagram-v2
    [*] --> SUBMITTED : création de la tâche
    SUBMITTED --> WORKING : prise en charge par l'agent
    WORKING --> COMPLETED : succès
    WORKING --> FAILED : erreur non récupérable
    WORKING --> INPUT_REQUIRED : l'agent attend une entrée du client
    WORKING --> AUTH_REQUIRED : l'agent demande une autorisation supplémentaire
    WORKING --> CANCELED : annulation par le client
    WORKING --> REJECTED : refus de l'agent (capacité insuffisante, politique)
    INPUT_REQUIRED --> WORKING : input fourni
    AUTH_REQUIRED --> WORKING : autorisation accordée
    COMPLETED --> [*]
    FAILED --> [*]
    CANCELED --> [*]
    REJECTED --> [*]
```

Les états terminaux sont COMPLETED, FAILED, CANCELED et REJECTED. La distinction entre FAILED (erreur interne) et REJECTED (refus délibéré de l'agent) est opérationnellement significative : un REJECTED déclenche une recherche d'agent alternatif, un FAILED déclenche un retry ou une escalade. Les états INPUT_REQUIRED et AUTH_REQUIRED introduisent des points d'intervention humaine dans le cycle de vie — ce qui connecte A2A directement au modèle HITL (*Human-in-the-Loop*) de Ch. 8.

Message : unité de communication multi-parts entre client et agent, supportant texte, fichiers et données structurées. Les messages permettent des échanges interactifs au sein du cycle de vie d'une tâche — par exemple, demander des clarifications avant de produire un résultat.

Artifact (artefact) : sorties de la tâche, composées de *Parts*. Un artefact peut être un document, un résultat structuré, une image ou tout autre type de livrable de l'agent cible.

Transports et sécurité

A2A supporte trois *bindings* de transport : JSON-RPC 2.0 / HTTP (binding principal, recommandé pour la compatibilité maximale), *gRPC* (ajouté en v0.3, recommandé pour la performance dans les scénarios haute fréquence), et HTTP+JSON/*REST* pour la compatibilité avec les infrastructures qui ne supportent pas JSON-RPC. Le *streaming* de mises à jour de progression s'appuie sur les *Server-Sent Events* (SSE).

Sur la sécurité, la spec définit plusieurs mécanismes : OAuth 2.0, OpenID Connect, API key, HTTP auth, *mTLS* (*mutual Transport Layer Security*). La spec ne les impose pas — elle définit les modalités supportées mais laisse le choix à l'implémenteur. Cette flexibilité est une limite de gouvernance : un déploiement A2A qui n'impose pas OAuth 2.0 ou *mTLS* entre agents n'est pas protégé par le protocole lui-même.

5.4 — WebMCP : la couche navigateur (statut expérimental, mai 2026)

WebMCP n'est pas un protocole indépendant de MCP. C'est une couche d'exposition des primitives MCP (tools uniquement, dans la version actuelle) via l'API navigateur `navigator.modelContext`, développée conjointement par Microsoft Edge (Patrick Brosset, auteur principal) et Google Chrome, au sein du Web Machine Learning Working Group du W3C. La distinction est architecturalement critique : un serveur MCP existant ne devient pas automatiquement accessible via WebMCP — il faut une implémentation explicite de `navigator.modelContext` côté page web.

L'API côté développeur web comprend quatre méthodes : `navigator.modelContext.provideContext()` pour enregistrer un ensemble de contextes en masse, `registerTool()` / `unregisterTool()` pour la gestion individuelle des outils exposés, et `agent.requestUserInteraction()` pour déclencher une confirmation humaine avant qu'un agent exécute une action sur la page. Ce dernier appel est le mécanisme principal de maintien du contrôle humain dans la boucle pour les interactions web — il traduit architecturalement les principes du Ch. 8 dans la couche navigateur.

Le statut de support par navigateur à mai 2026 :

Navigateur	Version	Statut
Chrome	146	Stable, implémentation complète
Edge	147	Support ajouté mars 2026
Firefox	—	En développement (estimation 8-12 semaines pour stable — *à vérifier*)
Safari / WebKit	—	Participation au W3C, aucun engagement public de *timeline*

La spec W3C est au stade *Candidate Recommendation* — stable pour implémentation mais non ratifiée formellement. La recommandation finale est prévue Q3 2026. L'absence d'engagement public d'Apple sur WebKit est documentée mais ne préjuge pas d'une adoption future.

Recommandation avec compromis : WebMCP est pertinent uniquement pour les agents qui interagissent structurellement avec des interfaces web (automatisation de SaaS sans API, *web scraping* structuré, actions sur formulaires). Pour les systèmes multi-agents d'entreprise *back-end*, WebMCP ne s'applique pas. Compromis : le périmètre limité aux *tools* (pas de *resources* ni *prompts* dans la version actuelle) réduit l'utilité pour des agents RAG ou des agents d'orchestration complexes. Alternative : un *framework* d'automatisation web (Playwright, Puppeteer) avec un serveur MCP *tools* wrappant les actions navigateur — approche plus mature à mai 2026 que WebMCP, mais qui ne bénéficie pas du modèle de permission navigateur natif. Condition de bascule : si les *resources* et *prompts* sont ajoutés à la spec WebMCP et que Safari implémente le standard, le rapport coût/bénéfice en faveur de WebMCP bascule significativement.

5.5 — La pile complète : MCP + A2A + WebMCP

L'architecture d'un système multi-agents d'entreprise complet s'organise en couches complémentaires et non concurrentes. Le diagramme suivant représente la pile de référence :

```
graph TD
    subgraph Agent_Runtime["Agent Runtime (orchestrateur)"]
        ORCH[Agent orchestrateur\nboucle decide–act–observe]
    end

    subgraph MCP_Layer["Couche MCP – accès outils et données"]
        MCP_S1[Serveur MCP\noutils métier]
        MCP_S2[Serveur MCP\nbase de données]
```

```
        MCP_S3[Serveur MCP\nAPIs externes]
    end

    subgraph A2A_Layer["Couche A2A — coordination inter-agents"]
        AG1[Agent spécialisé A\nAgent Card /.well-known]
        AG2[Agent spécialisé B\nAgent Card /.well-known]
        AG3[Agent spécialisé C\nAgent Card /.well-known]
    end

    subgraph WebMCP_Layer["Couche WebMCP — interfaces web (optionnel)"]
        WEB[navigator.modelContext\nChrome 146 · Edge 147]
    end

    subgraph EDA["Substrat EDA — Apache Kafka (Ch. 1 §1.5)"]
        KAFKA[Bus d'événements\nKafka 4.0]
    end

    ORCH -->|JSON-RPC / stdio| MCP_S1
    ORCH -->|JSON-RPC / stdio| MCP_S2
    ORCH -->|JSON-RPC / stdio| MCP_S3
    ORCH -->|A2A Task delegation| AG1
    ORCH -->|A2A Task delegation| AG2
    ORCH -->|A2A Task delegation| AG3
    AG1 -->|MCP tools| MCP_S1
    AG2 -->|MCP tools| MCP_S2
    ORCH -.->|optionnel : agents web| WEB
    ORCH --> KAFKA
    AG1 --> KAFKA
    AG2 --> KAFKA

    style MCP_Layer fill:#e8f5e9,stroke:#2e7d32
    style A2A_Layer fill:#e3f2fd,stroke:#1565c0
    style WebMCP_Layer fill:#fff9c4,stroke:#f9a825
    style EDA fill:#fce4ec,stroke:#c62828
```

La règle de composition est simple : MCP gère l'accès aux outils et aux données pour chaque agent individuellement ; A2A gère la délégation de tâches entre agents ; WebMCP étend MCP vers les interfaces web. Un agent peut être à la fois *client* A2A (il délègue des tâches à d'autres agents) et *serveur* A2A (il accepte des délégations d'un orchestrateur), tout en utilisant MCP pour accéder à ses propres outils. Le substrat EDA (*event-driven architecture*) décrit au Ch. 1 §1.5 — Apache Kafka 4.0 comme bus d'événements — constitue la couche de transport asynchrone *sous* les protocoles applicatifs, et ne duplique pas leur rôle.

5.6 — Tableau comparatif : MCP, A2A, ACP, UCP

Les quatre protocoles de l'écosystème agentique 2026 occupent des couches distinctes et ne sont pas concurrents. La question « MCP ou A2A ? » est mal posée — la bonne question est « quelle couche dois-je instrumenter pour ce besoin précis ? »

Critère	MCP	A2A	ACP (historique)	UCP
Couche	Accès outils et données (agent → ressource)	Coordination inter-agents (agent → agent)	Communication inter-agents open (absorbé)	Commerce IA (transactions Google)
Transport principal	JSON-RPC 2.0 / stdio / HTTP+SSE	JSON-RPC 2.0 / HTTP, gRPC, REST	REST/HTTP + SSE	Non public
Primitives clés	Resources, Tools, Prompts, Sampling	Agent Card, Task, Message, Artifact	REST endpoints, SSE streaming	Offres, négociation prix, paiement
Gouvernance	AAIF (projet fondateur), Anthropic maintient l'autonomie technique	Linux Foundation (depuis juin 2025)	Absorbé dans A2A sous LF	Google (propriétaire)
Maturité production	Élevée : 110M+ téléchargements/mois, Uber 60 000+ exécutions/semaine	Moyenne–élevée : 150+ organisations, 3 hyperscaleurs	N/A — absorbed	Faible : spécialisé commerce Google
Surface d'attaque connue	Élevée : tool poisoning (Unit 42), RCE by design (OX Security)	Modérée : usurpation Agent Card sans signature	N/A	Non documentée publiquement
Cas d'usage enterprise généraliste	Universel — tout accès outil	Tout scénario multi-agents	N/A (use A2A)	Hors périmètre sauf e-commerce Google

ACP (*Agent Communication Protocol*, IBM Research) : développé par IBM pour la communication inter-agents HTTP-natif REST, multi-frameworks, multimodal. Absorbé dans le projet A2A sous gouvernance Linux Foundation à mai 2026 (*confirmé* — IBM Research). IBM maintient le BeeAI Framework et Platform comme implémentations primaires s'appuyant sur la spec A2A. ACP n'est plus un protocole concurrent — le distinguer dans une comparaison est historiquement pertinent (il a existé comme initiative indépendante en 2024-2025) mais architecturalement dépassé.

UCP (*Universal Commerce Protocol*, Google) : protocole de transactions IA à vocation commerce, conçu pour les agents opérant dans l'écosystème Google (Shopping graph, marchands intégrés). Couvre la négociation de prix, les offres et la confirmation de paiement entre agents. Périmètre très spécifique — hors périmètre pour les architectures enterprise généralistes à l'exception des acteurs de l'e-commerce fortement intégrés dans l'écosystème Google.

5.7 — Gouvernance et maturité : AAIF, Linux Foundation, NIST

Tableau de maturité différenciée

Critère	MCP	A2A	WebMCP
Volume téléchargements / mois	110M+ (AAIF Blog, avril 2026 — chiffre annoncé au MCP Dev Summit NYC ; DigitalApplied citait 97M en mars 2026, croissance ~13 % en un mois)	Non publié	N/A (API navigateur)
Déploiement production enterprise	Confirmé (Uber, banques, éditeurs SaaS)	Confirmé (Azure AI Foundry, Bedrock AgentCore, Salesforce, SAP)	Expérimental
Surface d'attaque connue	Documentée, non corrigée par le protocole	Partielle (Agent Card sans signature dans v < 0.3)	Non documentée (trop récent)
Statut spec	Stabilisée, processus SEP actif	v1.0.0 affichée, date officielle *à vérifier*	W3C *Candidate Recommendation*, non ratifiée
Gouvernance	AAIF / LF, Anthropic conserve autonomie technique	Linux Foundation	W3C (WG Machine Learning)
Horizon de stabilité	3-5 ans (confirmé institutionnellement)	2-4 ans (*probable*)	1-2 ans (ratification W3C attendue Q3 2026)

AAIF : gouvernance du cycle de vie

Le modèle de gouvernance AAIF distingue l'AAIF Governing Board (stratégie, budget, approbation de nouveaux projets, membres fondateurs Platinum) du Technical Steering Committee (cycle de vie des projets selon les trois stades Growth / Impact / Emeritus). Cette séparation de gouvernance est empruntée à la CNCF et a pour effet qu'une organisation peut entrer dans le Governing Board sans obtenir de droit de veto sur les décisions techniques. Pour les architectes, cette nuance est importante : rejoindre l'AAIF comme membre garantit une voix dans les décisions stratégiques, pas un contrôle sur la direction de la spec.

Les événements structurants du calendrier AAIF 2026 : MCP Dev Summit NYC (2-3 avril 2026, 110M téléchargements annoncés) ; AGNTCon + MCPCon Europe (Amsterdam, 17-18 septembre 2026) ; AGNTCon + MCPCon North America (San Jose, 22-23 octobre 2026). Ces événements constituent les points de publication des évolutions majeures de la spec.

NIST AI Agent Standards Initiative

L'initiative NIST (*National Institute of Standards and Technology*) AI Agent Standards a été annoncée officiellement le 17 février 2026 (*confirmé* — NIST/CAISI). Elle s'articule en trois piliers : (1) leadership américain dans les organismes internationaux de normalisation — ISO/IEC JTC 1 — pour prévenir la fragmentation géopolitique des standards (UE, Chine ont leurs propres initiatives) ; (2) développement de protocoles *open-source* co-investi avec la NSF (*National Science Foundation*) ; (3) recherche fondamentale en sécurité, identité et méthodologies d'évaluation de l'interopérabilité.

Les livrables à court terme confirmés dans le texte primaire NIST : RFI sécurité agent (échéance 9 mars 2026), concept paper Identity and Authorization (échéance 2 avril 2026), sessions d'écoute sectorielles. Un profil d'interopérabilité AI Agent est prévu Q4 2026 (*probable* — confirmé dans des sources secondaires, non explicité dans le texte primaire NIST). Ce profil sera la première spécification normative externe aux éditeurs sur laquelle les architectures enterprise seront évaluées. La **recommandation opérationnelle** est d'instrumenter les systèmes pour ce profil dès aujourd'hui — pas d'attendre sa publication pour concevoir l'architecture. Le *readiness assessment* de Ch. 4 §4.5, Dimension 4 (Gouvernance), doit intégrer la conformité protocolaire comme critère de scoring d'ici Q4 2026.

5.8 — Surface d'attaque protocolaire : ce que MCP n'impose pas

Les vulnérabilités les plus graves de MCP à mai 2026 ne sont pas des bugs d'implémentation dans un SDK spécifique. Ce sont des conséquences de décisions de design dans la spec elle-même, confirmées comme telles par Anthropic. Ce chapitre présente les vecteurs ; le modèle de défense complet est au Ch. 9.

Tool poisoning

Le vecteur *tool poisoning* repose sur la capacité d'un serveur MCP malveillant (ou compromis) d'injecter des instructions dans la description textuelle d'un outil (*tool description*, champ texte libre dans le schéma MCP). Ces instructions sont invisibles à l'utilisateur qui voit l'interface graphique du *host*, mais visibles au modèle de langage qui traite la description complète. Un serveur météo peut ainsi embarquer dans sa description la commande d'exfiltrer les données d'un autre serveur MCP légitime simultanément connecté — avec rien de plus qu'un webhook gratuit comme destination (*confirmé* — Palo Alto Networks Unit 42, 2025-2026). La spec MCP ne définit aucun contrôle obligatoire sur le contenu des descriptions d'outils.

Injection via sampling

La primitive *sampling* inverse la direction du contrôle : le serveur demande au *client* d'effectuer une complétion LLM et reçoit le résultat. Un serveur qui contrôle à la fois le prompt soumis au modèle via *sampling* et le traitement de la réponse peut injecter des instructions persistantes qui modifient le comportement de l'agent sur des requêtes ultérieures — un vecteur distinct des injections classiques de *prompt injection* (*confirmé* — Palo Alto Networks Unit 42). Ce vecteur est nul si la primitive *sampling* n'est pas implémentée, critique s'il l'est.

Vulnérabilité architecturale RCE dans les SDKs

En avril 2026, OX Security a publié la documentation d'une vulnérabilité *RCE* (*Remote Code Execution*) affectant les SDKs officiels MCP — TypeScript, Python, Java, Rust. Ce n'est pas un bug de code corrigeable par un patch : c'est une décision de design dans la gestion des appels inter-SDKs. Les tests OX ont compromis 9 registres MCP sur 11 testés lors d'une injection d'essai. Anthropic a confirmé que le comportement est « *by design* » et a décliné de modifier le protocole (*confirmé* — OX Security, avril 2026). La mitigation DNS rebinding annoncée au MCP Dev Summit d'avril 2026 (Jonathan Leitschuh) est une correction au niveau SDK, pas une correction protocolaire — elle réduit la surface d'attaque dans les SDKs Tier 1 mais ne résout pas le problème architectural.

Supply chain des registres MCP

Le modèle de distribution des serveurs MCP via des registres tiers (Smithery, mcp.so, et neuf autres) introduit un vecteur de supply chain documenté. Deux incidents publics rapportés (*à vérifier* — sources primaires non tracées dans le bloc Notes de recherche du chapitre) : Postmark (septembre 2025, injection dans le serveur MCP Postmark permettant de mettre en copie les e-mails envoyés via l'agent) et Smithery (octobre 2025, registre compromis avec exposition d'applications et de tokens API). La modélisation générale de cette classe d'attaque (ASI04 — *supply chain* agentique) est traitée au Ch. 9.

Surface d'attaque A2A

A2A présente une surface d'attaque différente et plus limitée. La principale menace est l'usurpation d'Agent Card : un agent malveillant peut se déclarer capable d'exécuter des tâches qu'il ne peut pas exécuter, ou usurper l'identité d'un agent légitime, si les Agent Cards ne sont pas signées cryptographiquement. La version v0.3+ d'A2A introduit la signature des Agent Cards — mais la spec ne l'impose pas. La recommandation est de configurer la validation de signature comme exigence de politique interne dès le déploiement.

Implication pour l'architecte : les décisions de sécurité protocolaire pour MCP et A2A ne peuvent pas être déléguées au protocole lui-même. Elles doivent être prises au niveau de l'architecture : isolation des serveurs MCP tiers dans un périmètre de confiance séparé des serveurs internes, vérification des Agent Cards avec signature obligatoire, politique d'autorisation *per-task* pour les appels inter-agents. Le modèle de confiance à deux niveaux (serveurs internes vs tiers) adopté par Uber pour ses 10 000+ services internes via MCP est le patron de référence documenté à mai 2026 (*confirmé* — AAIF Blog, avril 2026).

5.9 — Recommandation architecturale : construire sur l'empilement ouvert

Extrait de code : appel MCP tool en TypeScript

L'extrait suivant illustre un appel MCP *tool* minimal depuis un client TypeScript 5.6, SDK MCP Tier 1 épinglé à `@modelcontextprotocol/sdk@1.x` :

```
// TypeScript 5.6 · @modelcontextprotocol/sdk@1.x (Tier 1)
import { Client } from "@modelcontextprotocol/sdk/client/index.js";
```

```
import { StdioClientTransport } from "@modelcontextprotocol/sdk/client/stdio.js";

const transport = new StdioClientTransport({
  command: "npx",
  args: ["-y", "@internal/crm-mcp-server"],
});

const client = new Client({ name: "orchestrator", version: "1.0.0" });
await client.connect(transport);

// Découverte des outils disponibles
const { tools } = await client.listTools();

// Invocation d'un outil
const result = await client.callTool({
  name: "get_customer_profile",
  arguments: { customer_id: "C-84921" },
});

console.log(result.content);
await client.close();
```

L'empilement en 23 lignes : transport stdio pour un serveur local, découverte des outils disponibles avant invocation, appel structuré avec arguments typés. La primitive *sampling* n'est pas exposée dans cet extrait — décision délibérée pour minimiser la surface d'attaque (§5.8). La version SDK est épinglée explicitement : toute mise à jour vers un SDK Tier 2 ou 3 doit faire l'objet d'une révision de sécurité explicite.

Recommandation principale avec compromis, alternative et condition de bascule

Recommandation : adopter MCP + A2A comme couches d'interopérabilité standard de tout système multi-agents d'entreprise déployé en 2026. MCP pour l'accès aux outils et données de chaque agent, A2A pour la délégation de tâches entre agents. Ne pas choisir : la question « MCP ou A2A » est structurellement mal posée — ils répondent à des besoins orthogonaux.

Compromis : standardisation ne garantit pas sécurité. MCP présente une surface d'attaque documentée et non entièrement résolue au niveau du protocole (tool poisoning, RCE by design). A2A impose une complexité opérationnelle (gestion des Agent Cards, cycle de vie des tâches, authentification inter-agents) absente des RPC simples. Le coût de démarrage est plus élevé que pour une solution propriétaire.

Alternative crédible : déploiement sur un *framework* propriétaire d'orchestration — AWS Bedrock Agents, Azure AI Foundry agents natifs, Google Vertex AI Agent Builder. Ces plateformes sont plus simples à déployer, disposent d'intégrations IAM (*Identity and Access Management*) natives, et exposent moins de configuration de sécurité protocolaire à l'architecte. L'alternative hybride MCP-natif vs *gateway abstraction layer* (LiteLLM, OpenRouter) permet d'accéder aux outils MCP sans exposer directement les primitives au modèle — couche de médiation qui absorbe une partie de la surface d'attaque au prix d'un *hop* supplémentaire.

Condition de bascule vers le propriétaire : (1) le SLA de latence inter-agents est < 50 ms et les SDKs MCP Tier 1 ne peuvent pas l'atteindre dans le contexte d'exécution cible ; (2) les exigences de conformité imposent un contrôle total de la couche transport non compatible avec la spec MCP actuelle (ex. résidence des données en territoire strictement contrôlé avec chiffrement de bout en bout non délégable à un serveur tiers) ; (3) l'équipe ne dispose pas des compétences de sécurité pour gérer correctement la surface d'attaque MCP — dans ce cas, l'alternative propriétaire est moins risquée que l'empilement ouvert mal sécurisé.

Condition de bascule vers la *gateway* abstraction : si l'organisation opère des agents multi-fournisseurs (Anthropic + OpenAI + Mistral dans le même système) et souhaite router dynamiquement par coût, capacité ou conformité géographique, une couche LiteLLM/OpenRouter *devant* les serveurs MCP est le bon choix — elle permet de changer de fournisseur de modèle sans modifier les serveurs MCP ni la logique d'agent, ce qui est la promesse centrale de portabilité de Ch. 10.

Transition vers Ch. 6
Les protocoles MCP et A2A définissent les contrats d'interface. Ils ne prescrivent pas comment orchestrer plusieurs agents en séquence, en parallèle, ou en graphe ; comment structurer la mémoire persistante entre les appels ; comment concevoir les outils pour l'idempotence et les contrats d'effet de bord. Ce sont les questions que Ch. 6 traite comme discipline distincte : les protocoles sont l'infrastructure, les patrons d'orchestration sont l'architecture.

Pour les équipes de sécurité, le modèle de menace complet des protocoles — avec les patterns de défense en profondeur pour tool poisoning, sampling injection, et supply chain — est au Ch. 9. L'Annexe A propose la checklist de révision d'architecture protocolaire pour les déploiements en zone orange et rouge de la matrice de Ch. 3.

Pour aller plus loin

Linux Foundation / AAIF — « Linux Foundation Announces the Formation of the Agentic AI Foundation » (9 décembre 2025). La source primaire de référence sur la structure de gouvernance AAIF, la liste des membres Platinum fondateurs et les trois projets fondateurs. Lecture obligatoire avant tout dialogue avec un juriste ou un responsable de conformité sur les implications de la donation MCP. https://www.linuxfoundation.org/press/linux-foundation-announces-the-formation-of-the-agentic-ai-foundation

a2a-protocol.org — « Agent2Agent (A2A) Protocol Specification v1.0.0 ». La référence normative pour implémenter A2A : définition complète des primitives, cycle de vie des tâches, transports et schémas d'authentification. À lire avant de choisir un SDK ou de définir les politiques de signature des Agent Cards. https://a2a-protocol.org/latest/specification/

OX Security — « The Mother of All AI Supply Chains » (avril 2026). La documentation technique la plus rigoureuse disponible sur la vulnérabilité architecturale RCE des SDKs MCP et la réponse d'Anthropic. Lecture obligatoire pour les équipes de sécurité avant tout déploiement de serveurs MCP exposés à des sources tierces. https://www.ox.security/blog/the-mother-of-all-ai-supply-chains-critical-systemic-vulnerability-at-the-core-of-the-mcp/

NIST / CAISI — « Announcing the AI Agent Standards Initiative » (17 février 2026). La source primaire pour les trois piliers de l'initiative et les livrables à court terme. À surveiller pour Q4 2026 et le profil d'interopérabilité AI Agent. https://www.nist.gov/news-events/news/2026/02/announcing-ai-agent-standards-initiative-interoperable-and-secure

Patrick Brosset (Microsoft Edge) — « WebMCP updates, clarifications, and next steps » (23 février 2026). La seule source primaire de l'auteur de la spec WebMCP expliquant la relation architecturale entre WebMCP et MCP, le périmètre *tools-only* actuel, et le statut du processus W3C. https://patrickbrosset.com/articles/2026-02-23-webmcp-updates-clarifications-and-next-steps/

Références

AAIF — « MCP Is Now Enterprise Infrastructure: Everything That Happened at MCP Dev Summit North America 2026 » — Agentic AI Foundation Blog — avril 2026 — https://aaif.io/blog/mcp-is-now-enterprise-infrastructure-everything-that-happened-at-mcp-dev-summit-north-america-2026/ — accédée le 2026-05-05

Brosset, P. (Microsoft Edge) — « WebMCP updates, clarifications, and next steps » — patrickbrosset.com — 23 février 2026 — https://patrickbrosset.com/articles/2026-02-23-webmcp-updates-clarifications-and-next-steps/ — accédée le 2026-05-05

DEV Community / AI Agent Economy — « WebMCP in 2026: Which Browsers Support navigator.modelContext? » — DEV Community — 2026 — https://dev.to/ai-agent-economy/webmcp-in-2026-which-browsers-support-navigatormodelcontext-complete-compatibility-status-1oe4 — accédée le 2026-05-05

DigitalApplied — « AI Agent Protocol Ecosystem Map 2026 » — DigitalApplied Blog — mars 2026 — https://www.digitalapplied.com/blog/ai-agent-protocol-ecosystem-map-2026-mcp-a2a-acp-ucp — accédée le 2026-05-05

Google Cloud — « Agent2Agent protocol (A2A) is getting an upgrade » — Google Cloud Blog — 31 juillet 2025 — https://cloud.google.com/blog/products/ai-machine-learning/agent2agent-protocol-is-getting-an-upgrade — accédée le 2026-05-05

IBM Research — « Agent Communication Protocol (ACP) » — IBM Research — 2025-2026 — https://research.ibm.com/projects/agent-communication-protocol — accédée le 2026-05-05

Linux Foundation — « Linux Foundation Announces the Formation of the Agentic AI Foundation (AAIF) » — Linux Foundation Press — 9 décembre 2025 — https://www.linuxfoundation.org/press/linux-foundation-announces-the-formation-of-the-agentic-ai-foundation — accédée le 2026-05-05

modelcontextprotocol.io — « SDKs — Model Context Protocol » — documentation officielle — mai 2026 — https://modelcontextprotocol.io/docs/sdk — accédée le 2026-05-05

NIST / CAISI — « Announcing the AI Agent Standards Initiative for Interoperable and Secure Innovation » — NIST — 17 février 2026 — https://www.nist.gov/news-events/news/2026/02/announcing-ai-agent-standards-initiative-interoperable-and-secure — accédée le 2026-05-05

OX Security — « The Mother of All AI Supply Chains: Critical, Systemic Vulnerability at the Core of Anthropic's MCP » — OX Security Blog — avril 2026 — https://www.ox.security/blog/the-mother-of-all-ai-supply-chains-critical-systemic-vulnerability-at-the-core-of-the-mcp/ — accédée le 2026-05-05

Palo Alto Networks Unit 42 — « New Prompt Injection Attack Vectors Through MCP Sampling » — Unit 42 Blog — 2025-2026 — https://unit42.paloaltonetworks.com/model-context-protocol-attack-vectors/ — accédée le 2026-05-05

PRNewswire / A2A Project — « A2A Protocol Surpasses 150 Organizations, Lands in Major Cloud Platforms, and Sees Enterprise Production Use in First Year » — PRNewswire — 2026 — https://www.prnewswire.com/news-releases/a2a-protocol-surpasses-150-organizations-lands-in-major-cloud-platforms-and-sees-enterprise-production-use-in-first-year-302737641.html — accédée le 2026-05-05

A2A Protocol — « Agent2Agent (A2A) Protocol Specification v1.0.0 » — a2a-protocol.org — mai 2026 — https://a2a-protocol.org/latest/specification/ — accédée le 2026-05-05

Chapitre 6 — Orchestration, mémoire, outils

Partie 3 — La pile *agentic* Chapitre 6 · Orchestration, mémoire et outils · ~6 200 mots · lecture ≈ 24 min

La conclusion de ce chapitre tient en une contrainte de conception : orchestration, mémoire et outils doivent être conçus ensemble, pas séquentiellement. Le choix d'un patron d'orchestration contraint le modèle de mémoire requis — un *swarm* décentralisé a besoin d'une mémoire partagée cohérente que n'exige pas un *supervisor* centralisé. Le modèle de mémoire contraint à son tour le design des outils — un outil qui modifie un état partagé entre agents doit garantir l'idempotence ou documenter explicitement ses effets de bord, faute de quoi la tolérance aux pannes de l'orchestrateur devient un vecteur de corruption des données. La majorité des défaillances *agentic* en production ne proviennent pas d'une mauvaise sélection de modèle : elles proviennent de la méconnaissance de ces dépendances mutuelles.

Le Ch. 5 a établi que MCP (*Model Context Protocol*) et A2A (*Agent-to-Agent Protocol*) définissent les contrats d'interface de la pile *agentic*. Ces protocoles ne prescrivent pas comment orchestrer plusieurs agents en séquence, en parallèle ou en graphe, ni comment structurer la mémoire persistante entre les appels, ni comment concevoir les outils pour qu'ils puissent être réessayés sans risque. Ces trois questions sont l'objet de ce chapitre.

6.1 — De la boucle à l'orchestration : ce que les protocoles ne prescrivent pas

La boucle *decide–act–observe* décrite au Ch. 1 §1.1 est l'atome de comportement agentique. L'orchestration est la molécule : elle définit comment plusieurs atomes interagissent, dans quel ordre, avec quelle logique de branchement et de reprise. MCP définit comment chaque atome accède à ses outils. A2A définit comment un agent en délègue à un autre. Ni l'un ni l'autre ne prescrit la topologie de composition — séquentielle, parallèle, conditionnelle, cyclique, hiérarchique — ni la logique de reprise sur erreur : après quel délai, avec quel contexte restauré, avec quelle limite de tentatives.

Cette lacune n'est pas un défaut protocolaire : c'est un choix de séparation des préoccupations. MCP et A2A sont des standards d'interface, pas des *frameworks* d'orchestration. Le bon niveau pour décider de la topologie reste l'architecte, qui doit choisir parmi une taxonomie de patrons selon les contraintes de son système — contraintes de latence, de criticité des effets de bord, de nombre d'agents, de dynamicité de la tâche. Cette décision précède et contraint le choix du *framework* d'implémentation.

Une précision terminologique s'impose. Les éditeurs de *frameworks* et la littérature académique utilisent des taxonomies différentes pour les mêmes patrons : MAF 1.0 (*Microsoft Agent Framework*) distingue cinq patrons natifs (*Sequential*, *Concurrent*, *Group Chat*, *Handoff*, *Magentic*), où *Magentic* étend le supervisor classique d'un *task ledger* dynamique ; ce qu'OpenAI Swarm appelait *handoff* est identique au mécanisme de transfert de contrôle d'un *swarm* LangGraph. Ce chapitre adopte une taxonomie composite à cinq patrons qui transcende ces dénominations de *frameworks* et permet de choisir un patron avant de choisir un *framework*.

6.2 — Taxonomie des cinq patrons d'orchestration

Les cinq patrons

Supervisor : un agent orchestrateur central reçoit la tâche et délègue à des agents *workers* spécialisés via des appels explicites. L'orchestrateur décide à chaque étape quel *worker* invoquer, avec quel sous-objectif, et intègre les résultats. Le routage est entièrement centralisé dans le LLM de supervision. Les avantages sont la lisibilité du flux de décision et la simplicité du débogage — toute décision de délégation passe par un point unique d'observation. Le goulot d'étranglement est symétrique à cet avantage : sur des tâches longues à haute fréquence, l'orchestrateur devient le facteur limitant de la latence. LangGraph expose ce patron via la bibliothèque `langgraph-supervisor-py` ; MAF 1.0 fournit une variante enrichie sous le nom *Magentic Pattern*, qui ajoute un *task ledger* dynamique au supervisor classique (*confirmé* — Microsoft Foundry Blog, avril 2026).

Swarm : chaque agent décide lui-même de passer le contrôle à un autre agent (*handoff*) en fonction de son propre état d'exécution — l'observation d'une condition, la détection d'une capacité manquante, l'atteinte d'un résultat partiel. Il n'y a pas d'orchestrateur permanent : le contrôle circule entre agents selon les transferts. Ce patron est

adapté aux tâches exploratoires dont la prochaine étape ne peut pas être connue avant que l'étape courante ait produit un résultat. Son risque principal est la formation de cycles si les conditions de transfert ne sont pas bornées. OpenAI Swarm (octobre 2024, éducatif) puis Agents SDK (mars 2025, production) ont popularisé ce patron avec les primitives *Handoffs* ; LangGraph 0.3.x le propose nativement (*à vérifier* — version exacte sur PyPI langgraph-checkpoint à mai 2026).

Hierarchical : un arbre de superviseurs, chaque superviseur gérant une équipe de *workers*. Les *workers* d'un niveau peuvent eux-mêmes être des superviseurs du niveau inférieur. Ce patron est la seule option viable au-delà de 20-30 agents dans des domaines multiples — l'alternative du *supervisor* unique devient une limitation de fenêtre de contexte dès que le nombre d'agents et d'outils dépasse la capacité de représentation en mémoire de travail du LLM orchestrateur. Google ADK (*Agent Development Kit*, avril 2025) expose un arbre hiérarchique natif avec support A2A ; CrewAI v1.12 propose un processus hiérarchique avec manager automatique, délégation et validation des résultats (*confirmé* — gurusup.com, 2026).

Graph-based : les agents et fonctions sont des nœuds dans un graphe orienté ; les arêtes représentent des flux de données avec conditions de déclenchement. Un exécuteur n'est activé que lorsque ses *inputs* sont prêts — ce qui permet l'exécution parallèle de branches indépendantes naturellement. Le *pipeline* est un graphe dégénéré sans cycle ni branchement conditionnel. MAF 1.0 propose un *Workflow* typé à arêtes explicites (*confirmé* — Microsoft Foundry Blog, avril 2026) ; LangGraph est fondé sur le concept de *StateGraph* avec *reducers* pour la résolution de conflits d'état.

Mesh : chaque agent peut communiquer directement avec chaque autre agent sans passer par un orchestrateur centralisé ni un transfert de contrôle formel. La coordination émerge des interactions locales. Ce patron correspond au paradigme *puppeteer* décrit dans le papier NeurIPS 2025 « Multi-Agent Collaboration via Evolving Orchestration » (arXiv:2505.19591) : un orchestrateur entraîné par renforcement (*RL*) dirige dynamiquement les agents en réponse à l'état courant de la tâche, surpassant l'orchestration statique avec des coûts computationnels réduits (*confirmé* — NeurIPS 2025). En pratique, le patron *mesh* sans mécanisme de coordination explicite produit des systèmes difficiles à déboguer et à auditer — il est pertinent dans les contextes de recherche ou pour des systèmes à agents très homogènes avec des interfaces strictement définies.

Tableau comparatif

Patron	Latence de routage	Précision du routage	Débogage	Tolérance aux pannes	Cas d'usage enterprise typique	Framework représentatif
Supervisor	Haute (chaque délégation = appel LLM)	Élevée (LLM décide)	Excellent	Moyen (SPOF orchestrateur)	Workflows définis, < 10 agents	LangGraph supervisor, MAF *Magentic Pattern*
Swarm	Faible (transfert direct)	Variable (selon agent source)	Difficile (flux non déterministe)	Bon (pas de SPOF central)	Tâches exploratoires, agents homogènes	OpenAI Agents SDK, LangGraph swarm
Hierarchical	Modérée	Élevée	Bon (par niveau)	Bon (isolation par équipe)	Systèmes > 20 agents, multi-domaines	Google ADK, CrewAI v1.12, MAF
Graph-based	Faible (parallélisme natif)	Élevée (arêtes typées)	Très bon (graphe visible)	Excellent (isolation de branche)	ETL agentique, pipelines de traitement	LangGraph StateGraph, MAF Workflow
Mesh / puppeteer	Très faible	Variable	Très difficile	Variable	R&D, agents homogènes à haute fréquence	arXiv:2505.19591 (recherche)

Recommandation avec compromis, alternative et condition de bascule

Recommandation : démarrer avec le patron *supervisor* pour tout nouveau système multi-agents en production, quelle que soit la cible à terme. Le *supervisor* offre le meilleur rapport complexité opérationnelle / visibilité du flux de décision lors de la phase d'exploration. Lorsque les tests révèlent que la latence de l'orchestrateur est le facteur limitant, migrer vers *graph-based* pour les workflows structurés ou *swarm* pour les tâches exploratoires.

Compromis : le *supervisor* crée un point de défaillance unique (*SPOF — single point of failure*) et sa latence de routage est proportionnelle au nombre d'étapes. À 50+ agents ou en scénario haute fréquence, il est intenable.

Alternative crédible : le patron *hierarchical* est l'alternative naturelle au *supervisor* monolithique pour les systèmes à grande échelle. Il distribue la charge de décision sans sacrifier la traçabilité des délégations.

Condition de bascule : si la topologie de la tâche n'est pas connue à l'avance (les prochaines étapes dépendent des résultats intermédiaires et ne peuvent être prédéfinies), le *swarm* ou le *mesh* deviennent les patrons appropriés — le *supervisor* ne peut pas déléguer ce qu'il ne peut pas planifier.

6.3 — Panorama des frameworks d'orchestration (mai 2026)

Les cinq *frameworks* d'orchestration enterprise et la bibliothèque de validation qui les complète sont comparés ci-dessous. Le choix d'un *framework* doit suivre, et non précéder, le choix du patron d'orchestration — un *framework* est une implémentation opinionée d'un ou plusieurs patrons, pas une décision architecturale neutre.

Critère	LangGraph	MAF 1.0	CrewAI v1.12	OpenAI Agents SDK	Google ADK	Pydantic AI
Patron primaire	Graph-based, supervisor, swarm	Graph-based, supervisor, swarm, hierarchical	Séquentiel, hierarchical, Flows	Swarm (Handoffs)	Hierarchical	Validation (pas orchestration)
Verrouillage fournisseur	Faible (multi-LLM)	Faible (multi-provider)	Faible (OpenAI-compatible)	Fort (OpenAI)	Modéré (Gemini/GCP préféré)	Nul
Interopérabilité MCP/A2A	MCP via LangChain tools	Native MCP + A2A (*confirmé*)	MCP partiel	MCP via SDK	A2A natif	N/A
Mémoire intégrée	LangMem, checkpointing PostgreSQL/Redis	Intégration Semantic Kernel Memory	Qdrant Edge natif	Assistants API (sunset 26 août 2026)	Via Vertex AI	N/A
Observabilité	LangSmith	Azure Monitor, logs MAF	Dashboards CrewAI	Tracing OpenAI	Cloud Trace GCP	N/A
Statut mai 2026	Actif, LangGraph 0.3.x (*à vérifier — PyPI*)	GA avril 2026, LTS confirmé	Actif, v1.12	Actif, production	Actif (depuis avril 2025)	Actif, usage validation uniquement

MAF 1.0 et AutoGen : AutoGen est en maintenance seule depuis la convergence vers MAF 1.0 (*confirmé* — Microsoft Foundry Blog, avril 2026). MAF 1.0 est le successeur supporté, issu de la fusion d'AutoGen et de Semantic Kernel. Tout nouveau projet qui aurait ciblé AutoGen doit cibler MAF 1.0. Le code AutoGen existant reste fonctionnel mais ne reçoit plus de nouvelles fonctionnalités.

OpenAI Agents SDK : l'Assistants API est en *sunset* le 26 août 2026 (*confirmé* — OpenAI). Les équipes qui utilisent l'Assistants API doivent migrer vers l'Agents SDK avant cette date. Le verrouillage fournisseur est le principal facteur limitant pour les architectures multi-fournisseurs — voir Ch. 10.

Pydantic AI : ce n'est pas un *framework* d'orchestration. C'est une bibliothèque de validation typée des *outputs* d'agents et de structuration des appels LLM, utilisée à l'intérieur d'un agent ou d'un *framework* d'orchestration,

jamais à la place. Sa présence dans ce tableau reflète une confusion fréquente dans les appels d'offres — la clarifier permet d'éviter un non-alignement entre équipes.

6.4 — Memory engineering : taxonomie et implémentation

Taxonomie cognitive primaire

La grille épisodique / sémantique / procédurale, héritée des neurosciences cognitives (Tulving, 1972 ; Squire, 1987), est la taxonomie opérationnellement la plus utile pour un architecte parce qu'elle détermine directement le choix de backend, la stratégie d'éviction et l'exposition aux risques de dérive. Elle n'est pas une convention pédagogique.

Mémoire épisodique : traces séquentielles et datées des interactions passées — conversations, résultats d'outils, décisions prises, contexte de session. Les épisodes sont associés à un moment et à un état particulier du monde. L'accès est temporel : « qu'est-ce que l'agent a fait lors de la dernière session avec ce client ? » Backend naturel : base vectorielle (*embedding* + recherche approximative par similarité) — Qdrant, pgvector, Pinecone. Le risque propre à cette couche est le *context rot* : à mesure que le volume d'épisodes croît, la précision de la récupération se dégrade et les épisodes anciens ou contradictoires polluent les récupérations. C'est la forme de dette de mémoire la plus commune en production (*confirmé* — documentation Anthropic, 2025).

Mémoire sémantique : faits, préférences, entités, relations — indépendants du temps et de l'épisode dans lequel ils ont été appris. « Ce client préfère les livrables en français » est un fait sémantique : il est vrai maintenant sans référence à la conversation qui l'a révélé. La distinction critique avec la mémoire épisodique est l'indépendance temporelle et la généralisation. Backend naturel : graphe de connaissances (Neo4j, Zep) ou base vectorielle avec *chunking* sémantique. La dimension temporelle de la mémoire sémantique — un fait peut changer au fil du temps — exige un mécanisme de *fact versioning* que les bases vectorielles simples ne fournissent pas nativement.

Mémoire procédurale : instructions d'action, *workflows*, politiques, règles de comportement — comment l'agent doit opérer dans des situations données. « Toujours valider l'e-mail avant d'envoyer » est une règle procédurale. LangMem implémente cette catégorie via la réécriture directe du *system prompt* par l'agent lui-même (*confirmé* — Mem0, State of AI Agent Memory 2026) : l'agent modifie ses propres instructions à mesure qu'il apprend de nouvelles procédures. Le risque spécifique est la dérive procédurale — sans validation des réécritures, l'agent peut s'auto-programmer des comportements non voulus sur des runs longs.

Le diagramme suivant représente le cycle de vie d'un épisode de mémoire, de sa création à sa consolidation ou son éviction :

```
flowchart LR
    A([Interaction / résultat d'outil]) --> B[Encodage\n— épisodique brut]
    B --> C{Pertinence\n≥ seuil ?}
    C -->|Oui| D[Consolidation\nsémantique / procédurale]
    C -->|Non| E[Éviction\ntemporelle ou LRU]
    D --> F[(Mémoire\npersistante)]
    F -->|Récupération| G[Contexte\nagent actif]
    G --> A
    E --> H[Archive\ncompressée]
    H -->|Audit| I([AgentOps\nCh. 7])

    style F fill:#e8f5e9,stroke:#2e7d32
    style I fill:#fce4ec,stroke:#c62828
```

Correspondance avec la taxonomie académique

> **Encadré — Correspondance académique arXiv:2512.13564**
>
> Hu et al. (47 auteurs, décembre 2025) proposent une taxonomie alternative articulée sur le *support* de la mémoire plutôt que sur sa fonction cognitive : (1) **token-level** — informations stockées dans la fenêtre de contexte active du LLM, volatiles à chaque appel ; (2) **paramétrique** — connaissances intégrées dans les poids du modèle via entraînement ou fine-tuning, non modifiables à l'exécution ; (3) **latente** — représentations internes des couches cachées du réseau, accessibles uniquement via des techniques d'interprétabilité.

La taxonomie fonctionnelle affinée distingue *factual memory* (faits du monde), *experiential memory* (résultats de tâches passées) et *working memory* (contexte actif courant). La correspondance avec la grille cognitive : la mémoire épisodique ≈ *experiential* (token-level + stockage externe) ; la mémoire sémantique ≈ *factual* (paramétrique + graphe externe) ; la mémoire procédurale ≈ *factual* paramétrique + *working* token-level. La taxonomie de Hu et al. est plus précise pour la recherche ; la grille cognitive est plus opératoire pour la conception d'architecture.

Panorama des outils (mai 2026)

Outil	Type de mémoire primaire	Architecture	Différenciation clé	Cas d'usage fort
Mem0	Épisodique + sémantique	Cloud + self-hosted	Compression 80 % tokens, graphe de faits extraits, 50 000+ développeurs	Agents de support / personnalisation longue durée
Zep Cloud	Sémantique (graphe temporel)	Cloud	Graphe de connaissances temporel — tracking des faits qui *changent* dans le temps + recherche vectorielle hybride	Entités enterprise avec relations évoluant (CRM, finance)
Letta (ex-MemGPT)	Épisodique + procédurale	Self-hosted	Blocs mémoire éditables, runtime stateful, pagination OS-inspired	Agents à identité persistante, contrainte forte sur la fenêtre de contexte
LangMem	Procédurale (réécriture system prompt)	Self-hosted (LangGraph natif)	Agent réécrit ses propres instructions	Équipes LangGraph, budget infrastructure maîtrisé
OpenAI Memory	Épisodique cross-sessions (GPT-4o)	Propriétaire OpenAI	Gérée par le fournisseur, hors SDK ouvert	Hors périmètre multi-fournisseurs (*à vérifier* — beta, fonctionnalités non stables à mai 2026)
Anthropic Memory tool	Épisodique (mémoire cross-sessions Claude)	Propriétaire Anthropic	Gérée par le fournisseur, hors SDK ouvert	Hors périmètre multi-fournisseurs (*à vérifier* — fonctionnalités non stables à mai 2026)

Note sur OpenAI Memory et Anthropic Memory tool : ces deux produits relèvent de la catégorie « mémoire épisodique gérée par le fournisseur ». Ils simplifient le déploiement pour des systèmes mono-fournisseur mais excluent structurellement toute portabilité. Pour une architecture multi-fournisseurs ou auto-hébergée, Mem0, Zep ou Letta sont les alternatives à privilégier — voir Ch. 10 pour le cadre de décision *lock-in*.

6.5 — Dette de mémoire : symptômes, mesure et contre-mesures

Définition opérationnelle

La dette de mémoire est l'écart cumulé entre ce que l'agent croit vrai — dans ses couches épisodique, sémantique et procédurale — et ce qui est vrai dans l'environnement réel, résultant de l'accumulation de mémoires non validées, contradictoires ou périmées. Elle est distincte de la dette technique au sens de Ward Cunningham : elle n'est pas le résultat d'un raccourci de code, mais d'un désalignement croissant entre la représentation interne de l'agent et l'état du monde.

Symptômes et modes de défaillance

Cinq modes de défaillance couvrent la majorité des incidents de mémoire documentés en production :

Stale state : l'agent agit sur des informations périmées — un statut de commande changé depuis la dernière récupération, un prix mis à jour entre deux sessions. Le risque augmente avec la durée entre les récupérations mémoire et la volatilité des données de référence.

Partial updates : une mise à jour partielle de la mémoire laisse des fragments cohérents avec l'ancien état et d'autres avec le nouvel état — l'agent prend des décisions contradictoires selon quelle portion de mémoire il consulte en premier.

Race conditions : dans un système multi-agents avec mémoire partagée, deux agents lisent simultanément le même état, modifient chacun leur copie, et persistent leurs versions — l'une écrase l'autre sans conflit détecté. Sans verrouillage ou résolution de conflits, la mémoire partagée est structurellement corrompue.

Prompt drift : sur des runs longs (au-delà de 50 interactions — *confirmé*, NeuralWired/CIO.com, 2026, cité au Ch. 1), la dilution progressive de l'objectif initial dans un contexte croissant dégrade la cohérence des décisions. L'agent continue d'exécuter des actions localement cohérentes qui ne convergent plus vers l'objectif original.

Lost state on retry : lors d'une reprise après erreur, l'état mémoire au moment de l'échec n'est pas restauré — l'agent repart d'un état antérieur et reproduit les actions déjà exécutées, potentiellement avec des effets de bord irréversibles.

Quantification de l'ampleur

IBM Research (2025) a quantifié le risque de débordement de fenêtre de contexte dans un flux de production en Materials Science : les sorties d'outils atteignent 2 millions d'éléments cumulés au fil de l'exécution. L'approche directe — passer l'ensemble des sorties en contexte — requiert 20 822 181 tokens et échoue (dépassement de fenêtre). Le *Memory Pointer Pattern* — stocker les sorties en mémoire externe et ne passer que des pointeurs en contexte — réduit la consommation à 1 234 tokens, soit une réduction de 16 800× (*probable* — IBM Research, 2025, accédé via mem0.ai et sparkco.ai ; texte primaire IBM non vérifié directement). Ce chiffre est spécifique au domaine Materials Science et ne se généralise pas directement à d'autres flux — il vaut comme ordre de grandeur sur l'ampleur possible du problème, pas comme valeur de référence universelle.

Contre-mesures

Compression sémantique : plutôt que de stocker les sorties d'outils brutes en mémoire épisodique, extraire les faits saillants et les stocker sous forme compressée. Mem0 annonce une réduction de 80 % de la consommation de tokens par cette approche (*confirmé* — Mem0, State of AI Agent Memory 2026). Le compromis est la perte des détails bruts — certains agents ont besoin des données complètes pour un audit ou un rejeu.

Éviction temporelle et LRU : définir une politique d'expiration pour les épisodes — les interactions antérieures à N jours ou N sessions sont archivées plutôt que récupérées activement. L'alternative est l'éviction *Least Recently Used (LRU)* sur la taille de la mémoire active. Les deux stratégies doivent coexister : l'éviction temporelle gère l'obsolescence, LRU gère le volume.

Memory Pointer Pattern : ne stocker en contexte actif que des identifiants de références mémoire (pointeurs), pas le contenu. Le contenu est récupéré à la demande lors de l'exécution. Ce patron est la réponse la plus économe en tokens pour les flux à haute densité de données d'outils.

Summarization hiérarchique : à intervalles réguliers (toutes les N sessions ou N tokens consommés), un LLM secondaire produit un résumé structuré des épisodes récents et remplace les épisodes bruts par ce résumé. Anthropic documente cette stratégie dans « Effective context engineering for AI agents » (2025) comme mécanisme de gestion de la fenêtre de contexte sur les agents longue durée.

Validation par second LLM : avant de persister une mise à jour mémoire procédurale (réécriture de *system prompt* via LangMem), soumettre la nouvelle version à un LLM de validation qui vérifie la cohérence avec les contraintes de politique définies. Coût : une inférence supplémentaire par mise à jour procédurale. Bénéfice : protection contre la dérive procédurale non supervisée.

Le lien avec Ch. 7 est direct : les *memory diffs* de l'observabilité AgentOps — la différence entre l'état mémoire avant et après une session — sont l'instrument de mesure de la dette de mémoire en production. Sans observabilité des diffs mémoire, la dette s'accumule invisiblement jusqu'à se manifester comme une défaillance comportementale inexpliquée.

6.6 — Tool design : idempotence, contrats d'effet de bord, schémas robustes

Idempotence : définition et techniques

Un outil *idempotent* produit le même résultat quelle que soit le nombre de fois où il est appelé avec les mêmes paramètres — appeler `create_order(order_id="ORD-7821", amount=150)` une, deux ou cinq fois ne crée qu'une seule commande de 150 $. Cette propriété est non négociable dans un système *agentic* parce que les orchestrateurs retentent les appels échoués par conception — si l'outil n'est pas idempotent, chaque *retry* produit un effet de bord additionnel.

Trois techniques couvrent la majorité des cas :

Clé d'idempotence (*idempotency key*) : le client génère un identifiant unique pour chaque intention d'action (UUID v4 ou ULID), le transmet dans chaque appel, et le serveur garantit qu'un appel portant un identifiant déjà traité retourne le résultat du premier appel sans re-exécution. Stripe, Adyen et la quasi-totalité des APIs de paiement implémentent ce patron. La clé d'idempotence doit figurer dans le schéma de l'outil MCP comme paramètre obligatoire pour les actions à effets irréversibles.

Vérification d'état avant écriture (*check-then-act*) : avant d'écrire, vérifier si l'action a déjà été exécutée en consultant l'état actuel. Si la commande existe déjà avec le statut COMPLETED, retourner le résultat existant sans re-créer. Attention : ce patron suppose que la vérification et l'écriture sont atomiques ou que le risque de *race condition* est acceptable dans le contexte.

Opérations upsert plutôt qu'insert : remplacer les opérations d'insertion pure par des *upserts* (mise à jour si l'entrée existe, insertion sinon). Un *upsert* sur une clé primaire stable est idempotent par construction, à condition que la clé primaire soit déterministe depuis les paramètres de l'appel.

La distinction REST est utile comme point de départ : les méthodes GET, HEAD, OPTIONS et PUT sont idempotentes par définition de la spécification HTTP (RFC 9110) ; POST ne l'est pas. Un outil MCP qui wrapped un POST doit implémenter l'idempotence explicitement.

Contrats d'effet de bord

Un *side-effect contract* est la déclaration explicite, dans la description de l'outil, de chaque effet non réversible que son invocation produit dans l'environnement. L'agent doit connaître ces effets *avant* d'appeler l'outil, pour deux raisons architecturales : décider s'il doit demander une confirmation humaine (voir Ch. 8 sur HITL), et résister à une injection d'outil malveillante qui tenterait de déclencher des effets non déclarés (voir Ch. 9).

Les effets à déclarer explicitement incluent : écriture en base de données ou système de fichier, envoi d'e-mail ou de notification, déclenchement de paiement ou de transaction financière, appel d'API externe facturée, modification d'un état partagé entre agents, action irréversible (suppression de données, fermeture d'un compte). Le format proposé est un champ `side_effects` dans la description textuelle de l'outil, complémentaire au schéma JSON :

```python
## Python 3.13 · MCP SDK Python Tier 1 · clé d'idempotence + side_effects contract
from mcp.server.fastmcp import FastMCP
import uuid

mcp = FastMCP("payment-server")

@mcp.tool(
    description="""
    Initiate a payment transaction.

    SIDE EFFECTS (irreversible):
    - Charges the customer's payment method immediately upon success.
    - Sends a confirmation email to the customer.
    - Creates an immutable audit record in the ledger.

    IDEMPOTENCY: Pass a stable idempotency_key (UUID) — repeated calls
    with the same key return the original result without re-charging.

    Use only when the user has explicitly confirmed the amount and recipient.
    """,
```

```
)
def initiate_payment(
    customer_id: str,
    amount_cents: int,
    currency: str,
    idempotency_key: str,
) -> dict:
    # Idempotence check: return existing result if key already processed
    existing = ledger.get(idempotency_key)
    if existing:
        return {"status": "already_processed", "transaction": existing}

    transaction = payment_gateway.charge(customer_id, amount_cents, currency)
    ledger.store(idempotency_key, transaction)
    return {"status": "success", "transaction": transaction}
```

Ce patron en 23 lignes illustre la convergence des trois propriétés : description guidante avec effets de bord déclarés, vérification d'idempotence avant action, retour structuré permettant à l'orchestrateur de distinguer un succès neuf d'une action déjà exécutée.

Schémas robustes

La spécification MCP exige que chaque outil expose son schéma de paramètres en JSON Schema — c'est la base syntaxique. Mais Anthropic documente explicitement que les descriptions textuelles libres guident le comportement de l'agent plus efficacement que le schéma JSON seul (*Writing effective tools for AI agents*, 2025) : le LLM utilise la description pour décider *quand* appeler l'outil et *comment* interpréter les résultats, pas seulement *comment* former les paramètres.

Quatre pratiques de design d'outils sont documentées (*confirmé* — Anthropic Engineering, 2025) :

Descriptions guidantes : la description doit répondre à la question « quand l'agent doit-il appeler cet outil et dans quel contexte ? » — pas seulement « que fait cet outil ? ». Une description qui énonce les préconditions, les contraintes et les cas d'usage appropriés réduit les appels redondants et les erreurs de paramètres.

Exemples d'utilisation intégrés : inclure dans la description un ou deux exemples d'appel typique avec les valeurs de paramètres attendues. Les LLM généralisent efficacement depuis des exemples concrets pour des types d'arguments non familiers.

Réponses d'erreur actionnables : les erreurs retournées par l'outil doivent être structurées et contenir des instructions de correction — pas des codes d'erreur opaques. « `400: invalid_param` » ne permet pas à l'agent de se corriger ; « `Le paramètre currency doit être un code ISO 4217 sur 3 lettres majuscules (ex. CAD, USD, EUR)` » le permet.

Chargement à la demande (*on-demand tool loading*) : exposer au LLM uniquement les outils pertinents pour la tâche courante, pas l'ensemble du catalogue. Un catalogue de 100+ outils dans la fenêtre de contexte active dégrade les performances de sélection et consomme des tokens inutilement. MAF 1.0 et LangGraph supportent tous deux la résolution dynamique du sous-ensemble d'outils applicable à chaque étape.

La version courante de référence pour la formalisation des schémas est OpenAPI 3.2.0 (OAS 3.2, septembre 2025), qui ajoute le support natif des *Server-Sent Events* (SSE) et JSON Lines pour le *streaming*, et la compatibilité JSON Schema complète (*confirmé* — OpenAPI Initiative, 2025). MCP utilise JSON Schema 2020-12 pour les schémas d'outils — les deux sont compatibles au niveau des types primitifs et des contraintes.

6.7 — Recommandation architecturale et transition vers Ch. 7

La règle de composition des trois couches

L'architecture interne d'un système *agentic* enterprise résulte de trois décisions couplées. Elles ne peuvent pas être prises indépendamment :

1. **Patron d'orchestration → contrainte sur la mémoire.** Un *swarm* décentralisé exige une mémoire partagée cohérente accessible à chaque agent sans passer par un orchestrateur central. Un *supervisor* peut se contenter

d'une mémoire locale à l'orchestrateur. Un système *hierarchical* exige une isolation de mémoire par équipe pour éviter que les agents d'un domaine contaminent la mémoire d'un autre.

2. **Modèle de mémoire → contrainte sur les outils.** Si la mémoire épisodique est partagée entre agents, les outils qui la modifient doivent garantir l'idempotence et documenter leurs effets de bord pour éviter les *race conditions*. Si la mémoire est isolée par session, les contraintes sont moins strictes — mais les outils à effets irréversibles (paiement, suppression) restent non négociablement idempotents.
3. **Design des outils → rétroaction sur le patron.** Un catalogue d'outils non idempotents limite le patron d'orchestration utilisable aux séquences sans *retry* — ce qui exclut tout patron tolérant aux pannes.

Recommandation principale

Recommandation : pour les systèmes *agentic* d'entreprise en production en 2026, combiner le patron *supervisor* avec une taxonomie mémoire à deux couches (épisodique vectorielle + sémantique en graphe) et des outils systématiquement idempotents avec contrats d'effet de bord déclarés. Cette combinaison offre le meilleur équilibre entre débogage, auditabilité et tolérance aux pannes.

Compromis principal : le patron *supervisor* ne passe pas à l'échelle au-delà de 20-30 agents sans dégradation de latence. La couche sémantique en graphe (Zep ou Neo4j) a un coût opérationnel significativement supérieur à une base vectorielle simple et requiert une expertise de modélisation de graphe que peu d'équipes possèdent en 2026.

Alternative crédible : mémoire comme vecteur uniquement (*memory-as-vector*) avec Mem0 ou pgvector est plus simple à opérer et couvre 80 % des cas d'usage si les entités du domaine ne changent pas fréquemment dans le temps. La mémoire comme graphe (*memory-as-graph*) avec Zep est l'alternative pertinente uniquement quand le suivi de l'évolution temporelle des faits est une exigence fonctionnelle — typiquement dans les domaines CRM, finance ou gouvernance réglementaire où les faits ont une date d'entrée en vigueur.

Condition de bascule : si le système dépasse 30 agents actifs ou si les domaines des agents sont suffisamment distincts pour que leur mémoire ne doive jamais interagir, migrer vers *hierarchical* avec isolation de mémoire par équipe. Si la topologie de la tâche n'est pas connue à l'avance au moment du design, préférer *graph-based* ou *swarm*.

Anti-patrons documentés

Orchestrateur unique sur 50+ agents : le *supervisor* monolithique sur un grand nombre d'agents accumule une fenêtre de contexte qui dépasse rapidement les capacités opérationnelles du LLM orchestrateur et produit une dégradation des décisions de routage. La solution structurelle est le patron *hierarchical* avec décomposition en équipes de 5-10 agents.

Mémoire sans politique d'éviction : une base vectorielle qui croît sans limite produit à terme une dégradation des performances de récupération (augmentation du bruit dans les résultats) et une escalade des coûts d'indexation. Définir une politique d'éviction est une décision architecturale, pas une optimisation postérieure.

Outils sans idempotence dans des workflows de paiement ou d'écriture : dans tout workflow où un *retry* est possible (tous les workflows tolérants aux pannes), un outil non idempotent à effet irréversible est une bombe à retardement opérationnelle. La détection se fait en Phase 4 de révision d'architecture — voir Annexe A pour la checklist.

Transition vers Ch. 7

Ce chapitre a construit le plan de construction interne d'un système *agentic* : les patrons qui organisent la coordination, la mémoire qui accumule le contexte et l'apprentissage, les outils qui agissent sur l'environnement avec des propriétés de sûreté vérifiables. La question qui suit est opérationnelle : comment observer, mesurer et opérer ce système une fois en production ? C'est l'objet de Ch. 7 — AgentOps comme discipline distincte de MLOps, centrée sur les traces multi-étapes, les *tool spans*, les *memory diffs* et le cycle de vie *promote/deprecate/rollback* d'un agent.

Pour les équipes de sécurité, les contrats d'effet de bord décrits en §6.6 sont la première ligne de défense contre l'exploitation d'outils par *tool poisoning* — un agent qui connaît les effets déclarés peut refuser d'exécuter une action dont les effets dépassent le contrat. Le modèle de menace complet est au Ch. 9.

Pour aller plus loin

Hu, Y. et al. — « Memory in the Age of AI Agents: A Survey » — arXiv:2512.13564 — décembre 2025. La référence académique la plus exhaustive et récente (47 auteurs) sur la taxonomie et les mécanismes de mémoire agentique. Lecture indispensable avant de concevoir la couche mémoire d'un système multi-agents ou d'évaluer un outil de mémoire. https://arxiv.org/abs/2512.13564

Microsoft Foundry Blog — « Introducing Microsoft Agent Framework: The Open-Source Engine for Agentic AI Apps » — avril 2026. La documentation d'annonce de MAF 1.0 décrit les cinq patrons

d'orchestration implémentés, le *Magentic Pattern* avec *task ledger* dynamique, et l'interopérabilité native MCP + A2A. Lecture obligatoire avant de choisir MAF ou de migrer depuis AutoGen. https://devblogs.microsoft.com/foundry/introducing-microsoft-agent-framework-the-open-source-engine-for-agentic-ai-apps/

Anthropic Engineering — « Writing effective tools for AI agents » — 2025. La doctrine primaire sur le design d'outils pour agents : descriptions guidantes, on-demand loading, réponses d'erreur actionnables. Source de référence normative pour §6.6. https://www.anthropic.com/engineering/writing-tools-for-agents

Mem0 — « State of AI Agent Memory 2026 ». Les chiffres de marché, la différenciation entre Mem0, Zep, LangMem et Letta, et la quantification de la réduction de tokens. Point d'entrée pratique pour choisir un outil de mémoire en 2026. https://mem0.ai/blog/state-of-ai-agent-memory-2026

arXiv:2505.19591 — « Multi-Agent Collaboration via Evolving Orchestration » — NeurIPS 2025. La validation académique du paradigme *puppeteer* (orchestration RL-based dynamique) et sa supériorité sur les patrons statiques. Pertinent pour les équipes qui envisagent un patron *mesh* ou souhaitent comprendre les limites des patrons statiques. https://arxiv.org/html/2505.19591v1

Références

Anthropic Engineering — « Writing effective tools for AI agents » — Anthropic — 2025 — https://www.anthropic.com/engineering/writing-tools-for-agents — accédée le 2026-05-05

Anthropic Engineering — « Effective context engineering for AI agents » — Anthropic — 2025 — https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents — accédée le 2026-05-05

focused.io / DEV Community — « Multi-Agent Orchestration in LangGraph: Supervisor vs Swarm Tradeoffs and Architecture » — 2025-2026 — https://dev.to/focused_dot_io/multi-agent-orchestration-in-langgraph-supervisor-vs-swarm-tradeoffs-and-architecture-1b7e — accédée le 2026-05-05

gurusup.com — « Best Multi-Agent Frameworks in 2026 » — 2026 — https://gurusup.com/blog/best-multi-agent-frameworks-2026 — accédée le 2026-05-05

Hu, Y. et al. (47 auteurs) — « Memory in the Age of AI Agents: A Survey » — arXiv:2512.13564 — décembre 2025 — https://arxiv.org/abs/2512.13564 — accédée le 2026-05-05

IBM Research — quantification Memory Pointer Pattern — 2025 — référencé via mem0.ai et sparkco.ai — accédée le 2026-05-05

Mem0 — « State of AI Agent Memory 2026 » — mem0.ai — 2026 — https://mem0.ai/blog/state-of-ai-agent-memory-2026 — accédée le 2026-05-05

Mem0 — « Graph-Based Memory Solutions for AI Context: Top 5 Compared » — mem0.ai — janvier 2026 — https://mem0.ai/blog/graph-memory-solutions-ai-agents — accédée le 2026-05-05

Microsoft Foundry Blog — « Introducing Microsoft Agent Framework: The Open-Source Engine for Agentic AI Apps » — Microsoft — avril 2026 — https://devblogs.microsoft.com/foundry/introducing-microsoft-agent-framework-the-open-source-engine-for-agentic-ai-apps/ — accédée le 2026-05-05

OpenAI — « New tools for building agents » (Agents SDK) — OpenAI — mars 2025 — https://openai.com/index/new-tools-for-building-agents/ — accédée le 2026-05-05

OpenAPI Initiative — « OpenAPI Specification 3.2.0 » — septembre 2025 — https://spec.openapis.org/oas/v3.2.0.html — accédée le 2026-05-05

arXiv:2505.19591 — « Multi-Agent Collaboration via Evolving Orchestration » — NeurIPS 2025 — https://arxiv.org/html/2505.19591v1 — accédée le 2026-05-05

Chapitre 7 — AgentOps : opérer des agents à longue durée d'exécution

Partie 3 — La pile *agentic* Chapitre 7 · AgentOps : opérer des agents longue durée · ~6 500 mots · lecture ≈ 26 min

L'observabilité et le cycle de vie d'un agent *agentic* en production ne se gèrent pas avec les outils MLOps hérités — non pas parce que ceux-ci sont insuffisants, mais parce que l'objet opéré est structurellement différent. MLOps opère des artefacts statiques dont le comportement est déterminé par des poids de modèle et une distribution d'entraînement ; AgentOps opère des systèmes dont le comportement dépend à l'exécution des outils invoqués, de l'état mémoire accumulé, des délégations effectuées, des budgets consommés, et des décisions prises dans un environnement qui change. Un agent peut produire des effets irréversibles — e-mail envoyé, enregistrement modifié, commande exécutée — avant qu'un opérateur ait eu l'occasion d'observer sa trajectoire. L'incident Replit de juillet 2025, où un agent a supprimé 1 206 enregistrements de production malgré une instruction de gel explicite, illustre cette asymétrie avec précision (*confirmé* — Fortune, 23 juillet 2025 ; earezki.com, mars 2026) : les logs capturaient les sorties, pas les décisions. La conclusion est immédiate : instrumenter, évaluer et piloter le cycle de vie d'un agent avec la rigueur propre à AgentOps n'est pas une option de maturité avancée — c'est la condition minimale pour opérer sans risque de dommage irréversible.

7.1 — AgentOps vs. MLOps : le delta opérationnel

La progression MLOps → LLMOps → AgentOps n'est pas un continuum incrémental. Chaque transition introduit un changement de nature de l'objet opéré, pas seulement un élargissement du périmètre (Joshi, IJCAT 2025 ; Halacli.com, fév. 2026).

MLOps opère des artefacts déterministes conditionnellement à leurs poids : entraînement, validation, déploiement, monitoring de *distribution drift*. À la frontière droite de la chaîne, l'inférence produit une sortie — un score, un texte, une prédiction — et l'humain agit. Le modèle ne touche pas à l'environnement. La variable de rollback est un artefact de poids versionné ; la dérive se détecte par comparaison statistique de distributions.

LLMOps ajoute la gestion du prompt, de la fenêtre de contexte, du coût par token, et de la dérive de réponse au sens qualitatif. L'action reste humaine. La variable de rollback intègre désormais le prompt et ses variables de configuration.

AgentOps change l'objet opéré : l'agent agit. La dérive à surveiller n'est plus une distribution de prédictions mais un comportement d'exécution — quels outils sont invoqués, dans quel ordre, avec quel résultat, dans quels périmètres de permission. La variable de rollback est un artefact composite : prompt système, ensemble d'outils avec versions, configuration mémoire, périmètre de permission, seuils d'escalade. Un changement sur l'une de ces cinq composantes crée une nouvelle version de l'agent — et un rollback incomplet sur l'une d'elles laisse le système dans un état incohérent.

Dimension	MLOps	LLMOps	AgentOps
Unité d'observabilité	Prédiction	Réponse LLM	Trajectoire multi-étapes (spans imbriqués)
Signal de dérive	Distribution de features	Qualité de réponse	Comportement d'exécution, budget consommé
Artefact versionné	Poids du modèle	Poids + prompt	Tuple composite (prompt, outils, mémoire, permissions, seuils)
Variable de rollback	Version des poids	Version du prompt	Artefact composite complet
Budget à piloter	Coût de calcul	Coût par token	Retry budget + escalation cost + coût d'inférence
Périmètre de permission	Aucun (modèle passif)	Aucun (LLM passif)	Boundary active (outils réels avec effets de bord)

La formulation la plus précise du delta reste celle d'Intellibytes (2026) : « AgentOps governs AI that acts, not just AI that predicts or generates. » Les organisations qui disposent des trois disciplines intégrées opèrent avec moins d'incidents en production (*à vérifier* — Intellibytes cite 30-40 % d'amélioration sans source primaire identifiable ; le mécanisme causal est plausible mais le chiffre est à traiter comme indicatif).

7.2 — Anatomie d'une trace agentique : spans, events, diffs

Une trace agentique n'est pas une trace HTTP ni un pipeline ML linéaire : c'est un arbre de *spans* hétérogènes dont chaque nœud représente une catégorie d'opération distincte. Sans instrumenter les quatre catégories, la trace est aveugle sur les portions les plus risquées de l'exécution.

Les quatre catégories de spans

LLM spans : chaque appel au modèle de langage, avec le prompt en entrée, la réponse en sortie, les tokens consommés, la latence, le modèle utilisé. Ce span est la fondation de tout calcul de coût d'inférence.

Tool spans : chaque invocation d'un outil, avec le nom de l'outil, les paramètres en entrée (sérialisés), le résultat ou l'erreur en sortie, le code de statut, le retry count, les timestamps de début et de fin. Ce span est le signal primaire de *tool correctness* défini au Ch. 4 §4.3 — sélection correcte de l'outil et paramètres corrects sont mesurables séparément sur ce seul type de span.

Memory spans : récupérations et écritures en mémoire épisodique, sémantique et procédurale (voir Ch. 6 §6.5), avec les requêtes émises, les résultats retournés et les scores de pertinence. Le *memory diff* — comparaison de l'état mémoire avant et après une session — est l'instrument de mesure de la dette de mémoire en production.

Orchestration spans : décisions de délégation d'un agent superviseur à un agent worker — quel sous-agent a été activé, avec quel sous-objectif, quel résultat a été retourné, quelle décision a suivi.

```
sequenceDiagram
    participant O as Orchestration span
    participant L as LLM span
    participant T as Tool span
    participant M as Memory span

    O->>L: invoke_agent (session_id, goal)
    L-->>O: plan (steps, tool_calls)
    O->>M: memory_retrieve (query, top_k)
    M-->>O: retrieved_context (score: 0.87)
    O->>T: tool_call (search_db, params)
    T-->>O: result (200 OK, 4 records)
    O->>L: synthesize (context + result)
    L-->>O: response (draft action)
    O->>T: tool_call (write_record, params)
```

```
    T-->>O: result (201 Created)
    O->>M: memory_write (episodic, outcome)
```

Conventions OTel GenAI : statut Development

L'OpenTelemetry (*OTel*) Semantic Conventions (SemConv) 1.40.0 du 17 avril 2026 définit les attributs standardisés pour les *spans* d'agents GenAI (OpenTelemetry, 2026) :

- `gen_ai.agent.id` — identifiant unique de l'agent
- `gen_ai.agent.name` — nom lisible de l'agent
- `gen_ai.agent.description` — description de la capacité de l'agent
- `gen_ai.agent.version` — version de l'artefact agent
- Opérations définies : `create_agent`, `invoke_agent`
- Opt-in via variable d'environnement : `OTEL_SEMCONV_STABILITY_OPT_IN=gen_ai_latest_experimental`

> **Avertissement de stabilité.** Ces attributs sont labellisés *Development* dans SemConv 1.40.0 (OpenTelemetry, avril 2026). Le label *Development* indique que la spec est active mais pas encore stabilisée — les noms d'attributs, leur typage et leur sémantique peuvent changer dans une release ultérieure sans période de dépréciation. En mars 2026, la même spec était labellisée *experimental* ; la reclassification en *Development* ne représente pas une stabilisation : les deux labels signifient une API non contractuellement garantie. Recommandation : abstraire ces attributs derrière une bibliothèque d'instrumentation interne qui peut absorber les changements de nommage sans modifier l'ensemble du code instrumenté. Horizon de stabilisation : *probable* 12-18 mois selon la cadence historique d'OTel.

Adoption des conventions OTel GenAI à mai 2026 : Datadog LLM Observability (natif depuis OTel v1.37), Grafana (collecte traces dans Loki), Elastic (*à vérifier*). Les plateformes spécialisées (Arize Phoenix, Braintrust, Langfuse) exposent leurs propres schemas en complément ou en lieu et place des attributs OTel.

Extrait d'instrumentation Python 3.13 + OTel SDK 1.x

```python
## Python 3.13 — OpenTelemetry SDK 1.x
## AVERTISSEMENT : attributs gen_ai.* en statut Development (SemConv 1.40.0)
## Abstraire derrière une bibliothèque interne pour isoler des changements futurs.

from opentelemetry import trace
from opentelemetry.trace import SpanKind

tracer = trace.get_tracer("agentops.tool_span", "1.0.0")

def traced_tool_call(tool_name: str, params: dict, agent_id: str) -> dict:
    with tracer.start_as_current_span(
        "tool_call",
        kind=SpanKind.CLIENT,
        attributes={
            "gen_ai.agent.id": agent_id,        # Development — peut changer
            "gen_ai.agent.name": "invoice-agent",
            "tool.name": tool_name,
            "tool.params_hash": hash(str(params)),
            "tool.retry_count": 0,
        }
    ) as span:
        try:
            result = execute_tool(tool_name, params)
            span.set_attribute("tool.status", "ok")
            return result
        except Exception as exc:
            span.set_attribute("tool.status", "error")
```

```
            span.record_exception(exc)
            raise
```

7.3 — Plateformes AgentOps 2026 : tableau comparatif

Le marché s'est segmenté en trois catégories dont le choix engage des contraintes architecturales différentes : plateformes spécialisées *open-source*, plateformes spécialisées SaaS, et suites des hyperscaleurs. Le critère déterminant à 18 mois n'est pas la richesse fonctionnelle instantanée — toutes les plateformes majeures couvrent le tracing et les évaluations de base — mais la position sur deux axes structurels : *vendor lock-in* (OTel-natif vs schéma propriétaire) et *data residency* (self-hostable vs SaaS exclusif) (DigitalApplied, 2026).

Plateforme	Modèle	OTel-natif	Self-host	Traces/replay/eval	Intégration framework	Tarif indicatif
Arize Phoenix	Open-source	Oui (natif)	Oui (PostgreSQL + K8s)	Traces + 6 modalités eval	Framework-agnostique	Gratuit (self-host)
Langfuse	Open-source (racheté par ClickHouse jan. 2026)	Partiel	Oui	Traces + eval + datasets	Framework-agnostique	Gratuit (self-host) / SaaS pay-per-span
LangSmith	SaaS	Non (schéma LangChain)	Non	Traces + replay + eval	LangGraph natif (lock-in)	Par trace (pay-as-you-go)
Braintrust	SaaS	Partiel	Non	Traces + CI/CD gate + eval stat	Framework-agnostique	1M spans/mois gratuit, puis par span
Helicone	SaaS (proxy drop-in)	Non	Non	Traces + coût + latence	Proxy universel (< 2 min setup)	Pay-per-request
Datadog LLM Observability	SaaS (APM enterprise)	Oui (OTel v1.37)	Non	Traces + alerting	Natif Datadog stack	Par span (inclus Datadog)
AWS Bedrock AgentCore	Cloud managed	Non (CloudWatch)	Non (AWS)	Traces step-by-step + scoring	Natif AWS (Bedrock, Lambda)	Pay-per-use CloudWatch
Azure AI Foundry	Cloud managed	Non (Azure Monitor)	Non (Azure)	Traces + lifecycle complet	Natif Azure (Copilot Studio, A2A)	Pay-per-use Azure Monitor
Google Vertex AI Agent Builder	Cloud managed	Partiel (Cloud Trace)	Non (GCP)	Traces + A2A natif	Natif GCP + A2A	Pay-per-use Cloud Trace

Recommandation avec compromis. Pour une organisation sans stack cloud hyperscaleur consolidé, Arize Phoenix (self-host) couplé à Langfuse (gestion des datasets d'évaluation) représente la combinaison avec le moins de lock-in et le meilleur rapport données-souveraineté. Compromis principal : charge opérationnelle de l'hébergement (cluster Kubernetes + PostgreSQL à maintenir). Alternative crédible : Braintrust en SaaS si la gouvernance accepte que les données de traces passent chez un tiers SOC 2 / HIPAA / GDPR — Braintrust est la seule plateforme spécialisée certifiée sur les trois référentiels à mai 2026. **Condition qui renverse la recommandation** : si l'organisation est déjà cliente Datadog avec LLM Observability activé, la migration vers une plateforme séparée génère une duplication

d'instrumentation qui annule le bénéfice ; dans ce cas, rester sur Datadog + Arize Phoenix pour les évaluations approfondies est la décision rationnelle. Pour les organisations 100 % AWS ou Azure, les outils hyperscaleurs natifs (Bedrock AgentCore, Azure AI Foundry) offrent la meilleure intégration au cycle de vie CI/CD existant, au prix d'un lock-in de collecte de télémétrie.

7.4 — Cinq métriques canoniques instrumentées

Les cinq métriques canoniques introduites au Ch. 4 §4.3 — *task success*, *tool correctness*, *retry budget*, *escalation quality*, *policy compliance* — ne se mesureront en production que si chacune est instrumentée depuis un type de span distinct. Les regrouper dans un dashboard unique sans cette différenciation produit des indicateurs inutilisables pour le diagnostic opérationnel.

Métrique	Source de span	Méthode de scoring	Signal d'alerte
Task success	Span de session (niveau racine)	Grader code-based (résultat binaire structuré) ou LLM-as-judge	Taux < seuil sur fenêtre 24h
Tool correctness	Tool spans (sélection + paramètres)	Comparaison à la séquence attendue (golden trace)	Taux d'erreur de sélection > P95
Retry budget	Compteur retry par tool span + par session	Compteur direct (pas de scoring)	Dépassement de P95 du budget prévu
Escalation quality	Spans d'escalade humaine	Annotation humaine ou LLM-as-judge (trigger, timing, contexte)	Score < seuil sur fenêtre 7j
Policy compliance	Comparaison tool spans aux contrats d'effet de bord (Ch. 6 §6.6)	Règle déterministe (outil autorisé ? périmètre respecté ?)	Toute violation → alerte immédiate

La stratégie de dashboarding suit deux horizons temporels distincts. Le dashboard opérationnel temps réel couvre les métriques à faible latence de détection : *retry rate*, taux d'erreur par outil, latence P50/P95/P99 par session. Le dashboard qualité asynchrone couvre les métriques nécessitant agrégation ou annotation : *task success rate*, *escalation quality score* — calculés sur des fenêtres de 24h et 7 jours.

L'alerting distingue deux régimes. Les règles de seuil (*retry rate > 15 %*) détectent les dégradations aiguës — elles sont nécessaires mais insuffisantes. Les règles de dérive comportementale (*task success rate baisse de 5 % sur 7 jours glissants*) détectent les dégradations lentes, qui sont la signature de la dérive accumulée décrite par NeuralWired (avril 2026) : les agents dérivent après 50+ interactions sur des tâches avec état persistant (*confirmé* — cohérent avec Ch. 6 §6.5). En pratique, les équipes implémentent les règles de seuil mais pas les règles de dérive — laissant la catégorie de problème la plus insidieuse sans détection automatique.

7.5 — Évaluation en production : régression continue, replay, shadow runs

L'évaluation hors ligne sur jeux de tests synthétiques ou annotés manuellement ne peut pas anticiper la distribution réelle du trafic de production. Les trois techniques présentées ici s'appliquent à des stades différents du cycle de changement et sont complémentaires, non substituables (sakurasky.com, 2025-2026 ; Braintrust, 2026 ; Anthropic Engineering, 2025-2026).

Régression continue

Chaque *pull request* déclenche un run d'évaluation complet sur le dataset de tests canonique. Un CI/CD gate (*porte de qualité*) bloque le merge si les scores tombent sous les seuils définis. Braintrust implémente cette approche avec significance statistique — un score légèrement inférieur au seuil peut ne pas déclencher le blocage si la différence n'est pas statistiquement significative sur la taille d'échantillon disponible (Braintrust, 2026).

La difficulté principale du bootstrapping : les seuils pertinents ne peuvent être définis qu'à partir d'une baseline de production réelle. Les premières semaines d'un nouveau déploiement opèrent nécessairement sur des seuils calibrés sur données synthétiques ou estimées — le risque est un taux de faux positifs (blocages injustifiés) ou de faux négatifs (dérogations injustifiées) élevé pendant cette période de calibrage.

Replay déterministe
Le *replay* est une forme de *golden file testing* : les traces de production passées — interactions réelles avec résultats annotés comme corrects — servent de scénarios de test. Une nouvelle version de l'agent rejoue ces traces et ses sorties sont comparées aux goldens (sakurasky.com, 2025-2026).

Avantage structurel : les goldens capturent des distributions de cas de bord réels que les jeux de tests synthétiques ne couvrent pas. Risque principal : les goldens se périment si l'environnement change — APIs d'outils mises à jour, schémas modifiés, comportement de données amont modifié. Un golden basé sur la réponse d'un outil qui a changé d'interface produit des faux positifs ou des faux négatifs indépendamment de la qualité de la nouvelle version de l'agent. Maintenance des goldens : exiger que tout changement d'outil s'accompagne d'une mise à jour du corpus de goldens est la contre-mesure. En pratique, cette exigence est rarement formalisée.

Shadow runs
La nouvelle version de l'agent tourne en parallèle de la version courante sur le trafic de production réel ; ses sorties ne sont pas exposées aux utilisateurs ; la comparaison des décisions et trajectoires des deux versions se fait en tâche de fond (sakurasky.com, 2025-2026).

Coût : le doublement du volume d'inférence et d'exécution des outils pendant la période shadow est réel. La justification économique est que ce coût est inférieur au coût moyen d'un incident de production sur une version non validée — un agent qui prend une mauvaise décision sur 1 % du trafic de production sur un périmètre à effets irréversibles génère des coûts de remédiation qui dépassent structurellement le coût de deux semaines de shadow run. Le chiffrage précis dépend du périmètre opérationnel de l'agent ; en l'absence de données de remédiation historiques, le raisonnement reste qualitatif.

```
flowchart LR
    A[Pull Request] --> B{CI/CD gate\nrégression continue}
    B -- score OK --> C[Merge + déploiement shadow]
    B -- score dégradé --> D[Blocage — investigation]
    C --> E{Shadow run\n48-72h trafic réel}
    E -- comportement équivalent --> F[Canary 5-10 %]
    E -- divergence détectée --> G[Rollback shadow]
    F --> H{Métriques 24h\ncanary vs production}
    H -- métriques OK --> I[Promotion 100 %]
    H -- dégradation --> J[Revert canary]
    I --> K[Monitoring continu\nrègle dérive]
    K -- alerte dérive --> L{Diagnostic}
    L --> A
```

Boucle de feedback production → dataset
Les traces de production dont le score est sous le seuil doivent alimenter automatiquement le dataset de tests de développement — c'est la seule façon de maintenir la pertinence du jeu de tests au-delà des premiers mois. Anthropic Engineering (*Demystifying Evals for AI Agents*, 2025-2026) documente ce principe : « production monitoring samples live traffic, runs metrics asynchronously, sends alerts when scores fall below thresholds » — la valeur est dans la boucle fermée, pas dans le monitoring isolé.

7.6 — Plan de contrôle : permission boundaries, rate limits, budgets, kill switches

L'observabilité et le plan de contrôle sont deux couches distinctes du système AgentOps. L'observabilité collecte et analyse ; le plan de contrôle impose et applique. Cette distinction est critique : un tableau de bord qui montre qu'un agent dépasse son budget de retry n'arrête pas l'agent. Un plan de contrôle qui impose un plafond de retry arrête l'agent (Halacli.com, fév. 2026).

Permission boundaries
Chaque agent reçoit au moment de son instanciation un ensemble de permissions explicitement déclaré — liste des outils disponibles, périmètre de données accessible (tenant, environnement, périmètre géographique), identité des systèmes qu'il peut atteindre. Ces permissions ne doivent pas être inférées par le LLM à partir de sa capacité technique à appeler un outil : un agent techniquement capable d'écrire dans une base de production ne doit le pouvoir que si cette permission figure dans son périmètre déclaré. La séparation dev/prod dans les permissions est la leçon

opérationnelle primaire de l'incident Replit (*confirmé*). Le lien avec les *permission boundaries* en contexte sécurité est traité au Ch. 9.

Rate limits et retry budgets

Le *rate limit* plafonne le nombre d'appels à un outil ou à un système externe dans une fenêtre de temps donnée — il protège les systèmes aval d'un agent pathologique entrant dans une boucle d'invocation. Le *retry budget* plafonne le nombre de tentatives sur un outil en échec dans une session — au-delà du budget, l'agent escalade ou s'arrête plutôt que de consommer des ressources sans convergence. Ces deux paramètres sont des composantes du plan de contrôle au sens de la FinOps *agentic* : la distinction entre *retry budget* et *escalation cost* introduite à l'Introduction et développée au Ch. 4 se traduit ici en paramètres configurables, pas en métriques passives.

Escalation cost et décision d'escalade

L'escalade vers un humain a un coût direct (temps de l'opérateur) et un coût indirect (latence, rupture de service). Le plan de contrôle doit définir explicitement les conditions qui déclenchent l'escalade — ne pas laisser ce choix à la discrétion du LLM — et mesurer la qualité de chaque escalade (*escalation quality*, Ch. 4 §4.3) pour calibrer les seuils dans le temps.

Kill switches

Le *kill switch* est la capacité d'arrêt immédiat de tout agent actif, sans attendre le cycle de déploiement normal. Il est distinct du rollback de version : le rollback remplace une version par une précédente ; le kill switch arrête l'exécution sans remplacement. Le kill switch est nécessaire pour les incidents de sécurité (franchissement de périmètre, *prompt injection* détectée en cours d'exécution) où chaque action supplémentaire de l'agent aggrave le dommage. L'absence de kill switch est un anti-patron de gouvernance, pas seulement un anti-patron opérationnel — voir Ch. 8 §8.3 et Ch. 9.

Azure AI Foundry expose un kill switch one-click depuis la console de gestion (*confirmé* — TechCommunity, 2026). AWS Bedrock AgentCore expose une API d'arrêt d'agent via le plan de contrôle CloudWatch (*confirmé* — AWS News Blog, 2025). Pour les déploiements sur infrastructure propre, l'implémentation du kill switch doit être conçue dès le départ comme un composant de première classe — pas ajoutée en réponse à un incident.

7.7 — Cycle de vie : promote, deprecate, roll back

Un agent n'est pas un modèle ML avec une version de poids : c'est un artefact composite. Le versionner et le faire évoluer exige de traiter l'artefact composite comme une unité atomique.

Définition de l'artefact versionné

L'artefact agent versionné est le tuple : `(prompt_système_vN, outils_avec_versions, config_mémoire, périmètre_permissions, seuils_escalade)`.

Un changement sur l'un quelconque des cinq éléments crée une nouvelle version de l'agent. Cette définition a des conséquences opérationnelles directes : si un fournisseur d'outil met à jour son API (signature changée, comportement modifié), l'artefact agent doit être reversionné et revalidé — même si le prompt système n'a pas changé.

Promote : canary vers production

La stratégie *canary* dirige 5-10 % du trafic vers la nouvelle version pendant une fenêtre de validation de 24 heures minimum. Les critères de promotion sont : *task success rate* ≥ version précédente ± marge de bruit statistique, *retry rate* ≤ seuil défini, aucune violation de *policy compliance*. L'absence de critères formels de promotion est l'anti-patron le plus fréquent : les équipes promeuvent sur la base de tests manuels ou de sentiment de qualité, sans gate automatisé, et découvrent la dégradation en production complète. Le pipeline CI/CD présenté à la section précédente formalise ce gate.

Deprecate : sunset d'agent

Le *deprecate* d'un agent suit la même logique que le sunset d'une API : délai d'annonce aux consommateurs, migration progressive du trafic, archivage de la version retirée comme cible de rollback d'urgence pendant une période définie (minimum 30 jours en production enterprise est une pratique courante — *à vérifier* en l'absence de standard formalisé). La différence avec une API est que les consommateurs de l'agent peuvent être d'autres agents dans un système multi-agents — le Ch. 5 a établi que les *Agent Cards* A2A portent des informations de capacité et d'endpoint ; elles doivent porter la date de sunset et la référence à la version successeur.

Roll back : restaurer l'artefact composite

Chaque déploiement doit conserver l'artefact composite précédent comme cible de rollback immédiat. Azure AI Foundry propose ce mécanisme one-click (*confirmé* — TechCommunity, 2026). L'auto-revert déclenché si le taux

d'erreur de la nouvelle version dépasse un seuil dans les N premières minutes post-déploiement est la contre-mesure pour les bugs qui ne se manifestent qu'à l'échelle.

Un rollback partiel — revenir au prompt précédent sans revenir aux versions d'outils précédentes — est un anti-patron : il recrée un état composite incohérent que l'agent précédent n'a jamais exécuté et dont le comportement est imprévisible. L'artefact versionné doit être traité comme une unité ou pas traité.

Matrice décision signal → action

Signal	Fréquence/urgence	Action recommandée
Task success rate baisse graduelle (-5 % / 7j)	Basse urgence, signaux lents	Shadow run + investigation de la dérive mémoire (Ch. 6 §6.5)
Retry rate dépasse P95 sur outil spécifique	Urgence modérée	Canary revert + inspection du tool span (changement d'API ?)
Policy compliance violation isolée	Urgence modérée	Isolation de la session + audit de la trace complète
Policy compliance violation répétée	Urgence haute	Kill switch + investigation avant réactivation
Incident de sécurité confirmé (*prompt injection*, franchissement de périmètre)	Urgence critique	Kill switch immédiat → Ch. 9
Dégradation généralisée post-déploiement canary	Variable	Auto-revert canary si gate automatisé ; rollback manuel sinon

---

7.8 — Anti-patrons opérationnels

Les anti-patrons suivants sont distincts des anti-patrons organisationnels et économiques traités au Ch. 12. Ils se situent strictement dans le périmètre opérationnel — instrumenter, évaluer, piloter.

Observabilité des outputs sans observabilité du raisonnement

Instrumenter les sorties de l'agent (réponses finales, e-mails envoyés, enregistrements créés) sans capturer les *tool spans* et les *orchestration spans* intermédiaires. L'incident Replit est la manifestation canonique de cet anti-patron (*confirmé* — earezki.com, mars 2026 ; Fortune, juillet 2025) : les logs capturaient les sorties, pas les décisions intermédiaires qui ont conduit à la suppression des 1 206 enregistrements. La conséquence opérationnelle est l'incapacité à distinguer une divergence bénigne (chemins d'exécution différents pour le même résultat) d'une divergence dangereuse (chemins d'exécution menant à des résultats incorrects ou destructeurs). Le coût d'instrumenter les quatre catégories de spans est non nul — mais il est non comparable au coût de remédiation d'un incident sans trace.

Un point mérite clarification sur l'incident Replit : l'analyse disponible (earezki.com, mars 2026 ; Fortune, juillet 2025) documente les faits et la réponse Replit, mais ne constitue pas un post-mortem officiel Replit avec causes racines formellement validées. L'utilisation de cet incident comme illustration qualitative des conséquences d'une observabilité absente est solide ; toute conclusion causale précise doit être traitée comme *hypothèse*. Le Ch. 12 traitera les modes de défaillance organisationnels et économiques liés.

Versionnage partiel de l'artefact agent

Versionner le prompt système sans versionner les outils liés — ou versionner les outils sans recapturer la configuration mémoire. Ce pattern produit des rollbacks incomplets qui replacent le système dans un état incohérent que l'agent précédent n'a jamais exécuté. La probabilité de comportements imprévus après un rollback partiel est élevée sans pouvoir être estimée précisément.

Évaluations absentes ou non actionnables

Déployer un agent sans évaluation continue (*régression continue* au minimum) revient à opérer en aveugle sur la qualité. La version moins grave mais plus fréquente est d'avoir des dashboards d'évaluation sans seuils actionnables — des métriques informationnelles, pas des décisions. Un architecte senior qui consulte un dashboard sans seuil de déclenchement ne peut pas décider ; il peut seulement observer.

Kill switch absent

Opérer un agent avec des effets irréversibles sans capacité d'arrêt immédiat est un risque de gouvernance inacceptable, indépendamment de la qualité de l'observabilité. Un agent correctement instrumenté dont on observe en temps réel une dégradation vers un incident de sécurité, et qu'on ne peut arrêter qu'en suivant un processus de déploiement de

20 minutes, a une observabilité qui ne sert à rien. Le kill switch doit être conçu comme la première fonctionnalité opérationnelle d'un agent à effets irréversibles, pas comme la dernière.

Shadow runs non budgétés
Décider d'un shadow run sans avoir alloué le budget d'inférence supplémentaire. Le doublement du coût d'exécution pendant la période shadow surprend les équipes sans provision FinOps agentique — et le shadow run est annulé prématurément, avant d'avoir fourni des données de comparaison suffisantes pour une décision de promotion éclairée. La FinOps *agentic* (Introduction, Ch. 4) doit provisionner explicitement les coûts de validation pre-production.

7.9 — Recommandation et transition vers Ch. 8

Recommandation architecturale
La maturité AgentOps d'une organisation se mesure à sa capacité à répondre à trois questions en moins de cinq minutes : quel est le *task success rate* de l'agent sur les 24 dernières heures ; quelle version composite est en production ; quelle action corrective est disponible immédiatement. Les organisations incapables de répondre aux trois opèrent en aveugle — non parce qu'elles n'ont pas de monitoring, mais parce qu'elles confondent l'observabilité des outputs avec l'observabilité du comportement.

Recommandation principale : instrumenter les quatre catégories de spans dès le premier déploiement en production, avec un CI/CD gate sur *task success rate* et un kill switch fonctionnel. Ne pas attendre la maturité avancée pour mettre ces deux contrôles en place — ils constituent le plancher, pas le plafond.

Compromis principal : l'instrumentation complète des quatre catégories de spans augmente la latence et le coût par session. Sur des agents longue durée à haute fréquence, cet overhead est mesurable. La réduction de l'overhead passe par l'échantillonnage (*sampling*) sur les spans à faible valeur diagnostique — typiquement les LLM spans en régime nominal — tout en maintenant un taux d'échantillonnage de 100 % sur les tool spans et les orchestration spans, qui sont les sources primaires de signal d'incident.

Alternative crédible : pour un déploiement initial à risque limité (agents en lecture seule, effets réversibles), démarrer avec une instrumentation légère (tool spans uniquement + *task success* grader code-based) et itérer vers l'instrumentation complète au franchissement d'un seuil de volume ou de criticité opérationnelle.

Condition qui renverse la recommandation : si le modèle de données de l'organisation ne peut pas absorber les traces d'exécution des agents (données métier sensibles dans les paramètres d'outils, contraintes réglementaires sur la rétention des logs), l'instrumentation complète requiert une architecture de traces masquées ou chiffrées — complexité supplémentaire significative. Dans ce cas, l'instrumentation d'agrégats (compteurs, latences, statuts) sans conservation des payloads bruts peut être le point d'équilibre. Cette contrainte est courante dans les secteurs financier et de santé ; la Loi 25 (Québec) et l'EU AI Act imposent des obligations complémentaires sur les données enregistrées dans les systèmes de décision automatisée (*à confirmer* avec le juriste de l'organisation — les textes réglementaires n'abordent pas explicitement les traces OTel).

Le modèle de maturité AgentOps à cinq niveaux — du **N1** (Ad hoc, aucune observabilité) au **N5** (Optimisé : évaluation continue, cycle de vie automatisé et gouvernance de portefeuille) — est détaillé à l'Annexe C. La checklist d'architecture AgentOps couvrant les dimensions sécurité, observabilité et FinOps est à l'Annexe A.

Transition vers Ch. 8
AgentOps construit la couche d'infrastructure de confiance technique : tracer, évaluer, piloter le cycle de vie. Mais la confiance dans les décisions d'un agent — la capacité d'un auditeur externe, d'un régulateur, ou d'un utilisateur final à comprendre pourquoi un agent a pris une décision donnée — requiert une couche supplémentaire que l'observabilité technique ne fournit pas. Cette couche, c'est l'auditabilité des décisions, le modèle HITL (*human-in-the-loop*) opérationnel, et la conformité réglementaire au sens de l'EU AI Act et d'ISO/IEC 42001. C'est l'objet du Ch. 8. L'AgentOps fournit les traces ; le Ch. 8 détermine ce que ces traces doivent prouver.

Pour aller plus loin

1. **Joshi, S. — « LLMOps, AgentOps, and MLOps for Generative AI: A Comprehensive Review » — IJCAT Vol. 14 Issue 07 — 2025 — https://ijcat.com/archieve/volume14/issue7/ijcatr14071001.pdf.** Revue systématique de 100+ articles. Point d'entrée académique le plus complet sur les trois disciplines et leur articulation. Indispensable pour une organisation qui doit arbitrer entre les trois niveaux de maturité.

2. **OpenTelemetry — « Semantic Conventions for GenAI agent and framework spans » — SemConv 1.40.0 — https://opentelemetry.io/docs/specs/semconv/gen-ai/gen-ai-agent-spans/.** Référence normative primaire pour l'instrumentation. À lire avec les notes de stabilité (*Development* = API non garantie) — et à surveiller pour la progression vers *Stable*.
3. **Braintrust — « What is agent observability? » — https://www.braintrust.dev/articles/agent-observability-tracing-tool-calls-memory.** La définition opérationnelle la plus précise des *tool spans* et *memory traces* disponible à mai 2026, avec architecture concrète et cas d'usage CI/CD gate.
4. **sakurasky.com — « Trustworthy AI Agents: Deterministic Replay » — https://www.sakurasky.com/blog/missing-primitives-for-trustworthy-ai-part-8/.** Seule source articulant les trois techniques d'évaluation en production (régression continue, replay, shadow runs) avec leurs conditions d'application et leurs limites.
5. **Microsoft TechCommunity — « From Zero to Hero: AgentOps » — Azure AI Foundry Blog — https://techcommunity.microsoft.com/blog/azure-ai-foundry-blog/from-zero-to-hero-agentops-–end-to-end-lifecycle-management-for-production-ai-a/4484922.** La documentation de cycle de vie AgentOps la plus détaillée disponible publiquement chez un hyperscaleur à mai 2026 ; utile comme référence de mise en œuvre concrète, indépendamment du choix de plateforme.

Références

- Joshi, S. — « LLMOps, AgentOps, and MLOps for Generative AI: A Comprehensive Review » — International Journal of Computer Applications Technology and Research, Vol. 14, Issue 07 — 2025 — https://ijcat.com/archieve/volume14/issue7/ijcatr14071001.pdf — accédé le 2026-05-05
- Intellibytes — « What is AgentOps? The Ultimate 2026 Guide to AI Agent Operations » — Medium — 2026 — https://medium.com/@Intellibytes/what-is-agentops-the-ultimate-2026-guide-to-ai-agent-operations-544876848ddd — accédé le 2026-05-05
- N-iX — « AI agent observability: The new standard for enterprise AI in 2026 » — N-iX Engineering Blog — 2026 — https://www.n-ix.com/ai-agent-observability/ — accédé le 2026-05-05
- OpenTelemetry — « Semantic Conventions for GenAI agent and framework spans » — SemConv 1.40.0 — 17 avril 2026 — https://opentelemetry.io/docs/specs/semconv/gen-ai/gen-ai-agent-spans/ — accédé le 2026-05-05
- earezki.com — « AI Agent Observability: Lessons from the Replit Production Data Loss Incident » — 18 mars 2026 — https://earezki.com/ai-news/2026-03-18-the-ai-agent-that-defied-a-code-freeze-deleted-1200-customer-records-and-then-lied-about-it/ — accédé le 2026-05-05
- Microsoft — « From Zero to Hero: AgentOps — End-to-End Lifecycle Management for Production AI Agents » — Azure AI Foundry Blog, TechCommunity — 2026 — https://techcommunity.microsoft.com/blog/azure-ai-foundry-blog/from-zero-to-hero-agentops—end-to-end-lifecycle-management-for-production-ai-a/4484922 — accédé le 2026-05-05
- AWS — « Introducing Amazon Bedrock AgentCore: Securely deploy and operate AI agents at any scale » — AWS News Blog — octobre 2025 — https://aws.amazon.com/blogs/aws/introducing-amazon-bedrock-agentcore-securely-deploy-and-operate-ai-agents-at-any-scale/ — accédé le 2026-05-05
- Braintrust — « What is agent observability? Tracing tool calls, memory, and multi-step reasoning » — Braintrust Articles — 2026 — https://www.braintrust.dev/articles/agent-observability-tracing-tool-calls-memory — accédé le 2026-05-05
- DigitalApplied — « Agent Observability: LangSmith, Langfuse, Arize 2026 » — DigitalApplied Blog — 2026 — https://www.digitalapplied.com/blog/agent-observability-platforms-langsmith-langfuse-arize-2026 — accédé le 2026-05-05
- sakurasky.com — « Trustworthy AI Agents: Deterministic Replay » — 2025-2026 — https://www.sakurasky.com/blog/missing-primitives-for-trustworthy-ai-part-8/ — accédé le 2026-05-05
- Anthropic Engineering — « Demystifying evals for AI agents » — 2025-2026 — https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents — accédé le 2026-05-05
- Halacli.com — « From MLOps to AgentOps: The Next Evolution in AI Operations » — 15 février 2026 — https://www.halacli.com/19_2026-02-15-mlops-to-agentops — accédé le 2026-05-05

- NeuralWired — « Why AI Agents Fail in Production (2026 Fix Guide) » — 28 avril 2026 — https://neuralwired.com/2026/04/28/why-ai-agents-fail-production/ — accédé le 2026-05-05

Chapitre 8 — Construire des systèmes dignes de confiance

Partie 4 — Confiance, sécurité et durabilité Chapitre 8 · Bâtir des systèmes dignes de confiance · ~5 500 mots · lecture ≈ 22 min

La confiance dans un système *agentic* n'est pas une attestation de l'éditeur du modèle — c'est une propriété architecturale que l'organisation déploie délibérément, ou qu'elle ne déploie pas du tout. L'illustration la plus nette de cette distinction reste l'incident Replit de juillet 2025, déjà évoqué au Ch. 7 sous l'angle de l'observabilité : un agent a supprimé 1 206 enregistrements de production malgré une instruction de gel explicite, parce que les journaux capturaient les sorties mais pas les décisions. L'organisation ne pouvait ni l'arrêter avant l'action, ni l'expliquer après. Ce n'est pas un problème de qualité du modèle — c'est un problème d'architecture de gouvernance.

La thèse opérationnelle de ce chapitre est que la fiabilité d'un système *agentic* en entreprise résulte de quatre éléments structurants interdépendants : un modèle d'autonomie hiérarchique explicite qui contraint le périmètre d'action à chaque niveau ; un dispositif HITL (*human-in-the-loop*) opérationnel conçu autour de l'escalade des exceptions, non de l'approbation systématique ; une infrastructure d'auditabilité fondée sur des journaux de décision immuables et des actions justifiables ; et une conformité alignée sur le corpus réglementaire 2024-2027 — EU AI Act (règlement UE 2024/1689), ISO/IEC 42001, NIST AI RMF (Artificial Intelligence Risk Management Framework), OSFI E-23 et Loi 25 Québec. Ces quatre éléments ne s'additionnent pas : ils se conditionnent mutuellement. L'architecture d'autonomie détermine ce qui doit être escaladé ; l'escalade produit les événements qui alimentent les journaux ; les journaux constituent la preuve de conformité.

8.1 — Hiérarchie d'autonomie : quatre niveaux, quatre périmètres de permission

Le choix du niveau d'autonomie pour une tâche n'est pas une décision de confiance dans le modèle — c'est une décision de risque opérationnel fondée sur la réversibilité de l'action et la tolérance à l'erreur établies à l'évaluation initiale (voir la matrice du Ch. 3 §3.2). Confondre les deux produit des architectures de gouvernance systématiquement mal calibrées : soit des approbations humaines imposées là où le risque est faible (goulots d'étranglement sans bénéfice de sécurité), soit une autonomie accordée là où le risque l'interdit (incidents irréversibles sans traçabilité).

Le modèle à quatre niveaux présenté ici s'ancre sur le cadre publié par le Knight First Amendment Institute (2026) et le papier associé arXiv:2506.12469, qui définissent les niveaux d'autonomie par le rôle résiduel de l'humain. La différence est d'angle : le cadre Knight adopte la perspective des rôles utilisateurs (opérateur, collaborateur, consultant, approbateur, observateur) ; le modèle ci-dessous adopte la perspective des *périmètres de permission* de l'agent — angle plus directement opérationnel pour l'architecte de solution.

Niveau	Désignation	Définition opérationnelle	Périmètre de permission	Condition réglementaire EU AI Act
N1	Assistance	L'agent produit une recommandation ; l'humain décide et agit.	Lecture seule. Aucune invocation d'outil avec effet de bord.	Hors périmètre haute-risque dans la majorité des cas.
N2	Supervisé	L'agent propose un plan et attend approbation humaine avant chaque action irréversible.	Lecture + écriture conditionnelle (gate humain sur actions irréversibles).	Satisfait l'exigence de surveillance humaine effective (Art. 14) si le gate est documenté.
N3	Autonome borné	L'agent exécute dans un périmètre d'action défini ; il escalade sur sortie de périmètre.	Lecture + écriture libre dans le périmètre ; blocage + escalade hors périmètre.	Haute-risque (crédit, emploi, infrastructure) : Art. 14 requiert surveillance effective — escalade documentée satisfait cette exigence si le périmètre est auditable.
N4	Autonome	L'agent agit sans gate préalable dans son périmètre, avec journalisation immuable obligatoire et revue périodique.	Lecture + écriture libre ; journalisation immuable ; revue humaine post-facto.	Haute-risque en N4 : Art. 14 EU AI Act exige que la surveillance humaine soit « effective » — la journalisation seule ne suffit pas ; revue post-facto doit être démontrée.

Note : Le modèle à cinq niveaux du Knight First Amendment Institute (2026) offre une granularité supplémentaire utile pour la cartographie des rôles organisationnels. Pour les organisations dont la gouvernance distingue explicitement les rôles d'approbateur et d'observateur, ce cadre à cinq niveaux est à considérer comme complément.

Compromis principal — autonomie fixe vs *adaptive autonomy*. Un niveau fixe (N1 à N4 attribué par type de tâche) maximise la prévisibilité de la gouvernance et l'auditabilité : le périmètre de permission est stable, les politiques de journalisation sont déterministes, et les obligations réglementaires sont cartographiables sans ambiguïté. L'*adaptive autonomy* — où le niveau est ajusté dynamiquement au run-time selon un score de confiance de l'agent sur la tâche en cours — peut réduire les gates inutiles sur des tâches bien maîtrisées et déclencher des escalades précoces sur des tâches inhabituelles. En 2026, cette approche est implémentée dans certains frameworks via *confidence-driven escalation* (source 12, IJCT). Elle reste cependant difficile à auditer : si le niveau d'autonomie varie à l'exécution, la preuve de conformité doit démontrer que chaque ajustement dynamique respectait bien les règles de la *agent policy* — charge de preuve significativement plus élevée qu'avec un niveau fixe. **Alternative recommandée** : commencer avec des niveaux fixes (N1-N4) et introduire l'*adaptive autonomy* uniquement sur des domaines de tâches à volume élevé où le gain d'efficacité est mesurable et la politique de journalisation dynamique est instrumentée. **Condition de bascule** : si le taux de faux-positifs d'escalade (actions sûres escaladées inutilement) dépasse 10 % du volume, le niveau fixe est probablement trop prudent pour ce domaine — l'*adaptive autonomy* devient alors une alternative crédible, à condition d'auditer la politique de score de confiance elle-même.

8.2 — HITL opérationnel : humans set rules, agents execute, exceptions escalate

Un HITL efficace n'est pas un frein d'urgence activé après l'échec — c'est un mécanisme de délégation structurée où le domaine de l'agent et le domaine de l'humain sont définis a priori avec précision chirurgicale. La formulation *humans set rules, agents execute, exceptions escalate* est la syntaxe opérationnelle du HITL d'entreprise. Elle distingue trois responsabilités non permutables.

Humans set rules — les politiques (périmètre d'action, seuils de confiance, types d'actions irréversibles, domaines interdits) sont définies par des humains avant déploiement. Ce travail n'est pas délégable à l'agent lui-même. C'est le produit de la collaboration entre l'architecte de solution, le *risk officer* et le *compliance lead* — formalisé dans une *agent policy* versionnée, auditée et stockée en configuration-as-code. La *agent policy* est l'artefact de gouvernance central dont dépendent les deux responsabilités suivantes.

Agents execute — dans le périmètre défini par la *agent policy*, l'agent exécute sans intervention humaine. Toute intervention humaine dans ce périmètre est un coût pur sans bénéfice de sécurité. La discipline opérationnelle consiste à définir ce périmètre suffisamment précis pour que l'exécution autonome soit sûre, suffisamment large pour que la délégation soit utile. Un périmètre trop étroit engendre un taux d'escalade élevé qui annule le gain de productivité attendu.

Exceptions escalate — toute condition hors périmètre déclenche une escalade structurée. Le papier IJCT (2026) identifie trois triggers d'escalade distincts : un seuil de confiance de l'agent sur la tâche en cours (signal probabiliste) ; une règle de conformité encodée dans la *agent policy* (signal déterministe) ; un type d'action qualifiée d'irréversible par la politique (signal catégoriel). Ces trois triggers ne se substituent pas — un système qui n'implémente que le seuil de confiance laisse des actions réglementairement contraintes sans protection explicite.

Trois patterns d'intervention humaine

Le papier IJCT (2026) formalise trois modes de présence humaine dans la boucle d'exécution agentique, distincts par leur temporalité et leur coût opérationnel.

HITL-in-loop (synchrone) : l'agent s'arrête et attend une réponse humaine avant de continuer. La latence d'exécution inclut le temps de réponse humain — de quelques secondes à plusieurs heures selon le processus. Ce mode est applicable aux actions irréversibles à fort impact : virement au-delà d'un seuil, modification d'un contrat actif, envoi d'une communication externe au nom de l'organisation, décision affectant des droits individuels. C'est le seul mode compatible avec les exigences les plus strictes de surveillance effective au sens de l'EU AI Act Art. 14 pour des systèmes haute-risque N4.

HITL-on-loop (asynchrone) : l'agent continue, l'humain est notifié et peut intervenir dans une fenêtre de temps définie avant que l'action devienne irréversible. Ce mode est applicable aux actions réversibles à risque modéré où la vitesse d'exécution a de la valeur. La fenêtre d'intervention doit être explicitement définie dans la *agent policy* — une notification sans fenêtre définie n'est pas un HITL, c'est une alerte décorative.

HITL-over-the-loop (post-facto) : l'agent exécute, les journaux sont revus périodiquement par un humain. Ce mode est compatible avec N4 pour les actions réversibles ou dans des domaines hors haute-risque. Il n'est pas un HITL au sens de l'EU AI Act pour les décisions individuelles haute-risque — la revue post-facto ne satisfait pas l'obligation de surveillance effective préalable.

Connexion Loi 25 Québec (Art. 12.1) — risque de *compliance washing*. La Loi 25 impose, pour toute décision prise « exclusivement par traitement automatisé » produisant des effets significatifs sur une personne, une obligation d'information et un droit de révision par une personne physique. L'anti-patron consiste à positionner un humain nominal dans la boucle — un approbateur qui reçoit 200 notifications par heure et les valide sans examen réel — pour échapper à l'obligation légale sans en satisfaire l'esprit. Ce *compliance washing* HITL est identifiable par la mesure du taux de refus humain sur les escalades : un taux inférieur à 1 % sur un volume élevé est le signal statistique d'un humain nominal, pas d'un humain effectif. Les régulateurs québécois (Commission d'accès à l'information) ont la capacité d'examiner ces métriques lors d'une enquête.

Compromis. HITL-in-loop garantit la conformité la plus stricte mais génère des goulots à volume élevé. **Alternative** : HITL-on-loop avec fenêtre d'intervention courte (15 à 30 minutes pour les cas à risque modéré) couplé à un mécanisme de révision sur demande pour satisfaire Art. 12.1 Loi 25. **Condition de bascule** : si le taux d'escalade dépasse 5 % du volume de transactions, le périmètre d'action de l'agent est mal calibré — le problème est dans la *agent policy*, pas dans le HITL lui-même.

8.3 — Auditabilité : journaux de décision, actions justifiables, immuabilité

Un journal de décision agentique n'est pas un log d'application — c'est la reconstruction complète et fidèle de la chaîne causale qui a produit une action, opposable à un auditeur humain. Cette distinction d'intention a des conséquences concrètes sur ce qui est instrumenté, comment il est stocké, et qui y a accès.

Le FINOS AI Governance Framework (AIGF) v2.0, mitigation MI-21 « Agent Decision Audit and Explainability » (2026), formalise cinq propriétés d'un journal de décision auditoire, adoptées ici comme référence opérationnelle.

1 — Traçabilité. Toute action est reliée à un agent identifié (agent ID + version), un objectif reçu, une autorisation active au moment de l'action. Le journal capture : qui a décidé, pourquoi (objectif et politique appliquée), avec quelle autorisation (périmètre de permission au moment de l'action, version de la *agent policy*), quels outils ont été invoqués (*tool spans* avec paramètres d'entrée et résultats), dans quel état mémoire (*memory diff* pré/post-action). La convention OTel GenAI SemConv 1.40.0 (attributs `gen_ai.agent.id`, `gen_ai.agent.name`, `gen_ai.agent.version`) fournit le vocabulaire d'instrumentation standard, en statut *Development* à mai 2026 — les API ne sont pas encore stabilisées, ce qui impose une stratégie d'abstraction entre la couche d'instrumentation et le reste du système.

2 — Explicabilité. La logique de décision est rendue lisible pour un humain non technique. Cela exige que le raisonnement intermédiaire de l'agent soit journalisé, pas seulement l'output final. Les systèmes qui ne capturent que l'action sans le raisonnement qui l'a produite créent un déficit d'explicabilité documenté — un auditeur qui ne peut reconstituer le « pourquoi » d'une action ne peut pas évaluer si la *agent policy* a été respectée.

3 — Immuabilité. Les journaux sont horodatés cryptographiquement et stockés dans un système en écriture seule. Un journal modifiable n'est pas un journal d'audit — c'est un artefact susceptible d'être altéré après un incident, ce qui le prive de toute valeur probatoire.

4 — Autorisation vérifiable. Chaque action peut être confrontée à la *agent policy* en vigueur *au moment de l'action*. Le versionnement de la *agent policy* est un prérequis : sans version ancrée dans le journal, il est impossible de démontrer que l'action était autorisée par la politique active, et non par une version antérieure ou ultérieure.

5 — Reproductibilité. À partir des journaux, un auditeur doit pouvoir reconstituer le chemin de décision par *replay* déterministe. La technique du *replay* déterministe, présentée au Ch. 7 §7.5 dans le contexte des evals en production, reprend ici sa valeur probatoire en audit : si le *replay* produit un résultat différent du journal, le système a un problème de déterminisme qui affecte son auditabilité.

Distinction observabilité (Ch. 7) vs auditabilité (ce chapitre). L'observabilité opérationnelle — traces OTel, dashboards, alertes — est instrumentée pour le diagnostic en temps réel par les opérateurs. L'auditabilité est instrumentée pour la preuve a posteriori par des auditeurs et des régulateurs. Les deux réutilisent la même infrastructure de journalisation (spans OTel GenAI SemConv 1.40.0), mais les politiques de rétention, d'immuabilité et d'accès sont différentes. Traiter ces deux besoins comme un seul projet crée soit une sur-rétention coûteuse (conserver les traces opérationnelles éphémères avec les journaux d'audit permanents), soit une sous-protection (stocker les journaux d'audit dans des systèmes modifiables).

Actions justifiables. Une action est *justifiable* si, pour tout auditeur, l'agent peut démontrer que l'action était (a) dans le périmètre autorisé, (b) cohérente avec l'objectif reçu, (c) proportionnée à l'information disponible au moment de l'action. La justifiabilité n'exige pas que l'action soit optimale — elle exige qu'elle soit défendable dans le cadre de la politique. Un agent qui a pris une décision sous-optimale mais traçable, explicable et dans le périmètre est auditable. Un agent qui a pris une décision optimale mais non tracée ne l'est pas.

Compromis. La journalisation complète du raisonnement intermédiaire alourdit la latence d'inférence (chaque token de raisonnement stocké allonge le round-trip) et multiplie le volume de données d'audit. **Alternative** : journalisation sélective sur actions à risque élevé uniquement, avec stockage compressé pour les actions à risque faible. **Condition de bascule** : si un incident survient sur une action non journalisée exhaustivement, le coût de l'enquête — reconstitution manuelle, incertitude réglementaire, risque de sanction — dépasse structurellement le coût de la journalisation préventive complète. La décision de journalisation sélective doit elle-même être documentée, versionnée, et revue périodiquement par le *compliance lead*.

8.4 — Conformité : cartographie du corpus réglementaire 2024-2027

Un système *agentic* déployé par une institution financière canadienne au service de clients européens navigue simultanément dans quatre cadres réglementaires sans comité de coordination entre eux — la charge de l'alignement est entièrement sur l'organisation déployante. Le tableau ci-dessous cartographie les obligations principales ; il est un point de départ pour la conformité, non un substitut à un avis juridique.

Cadre	Périmètre d'applicabilité	Obligations principales pour systèmes agentiques	Dates d'applicabilité
EU AI Act (UE 2024/1689)	Systèmes déployés dans l'UE ou à des ressortissants UE	Surveillance humaine effective (Art. 14), transparence (Art. 13), journaux d'événements (Art. 12), interdiction systèmes inacceptables (Art. 5), obligations GPAI (Titre VIII)	Interdictions : 2 fév. 2025 ; GPAI (General-Purpose AI) : 2 août 2025 ; haute-risque générique : 2 août 2026 ; produits réglementés intégrés : 2 août 2027 ; autorités publiques : 2 août 2030
ISO/IEC 42001:2023	Volontaire — certifiable par tierce partie (ISO/IEC 42006:2025 pour organismes d'audit)	AIMS : politique IA, évaluation d'impact, contrôle des risques, révision de direction, audit annuel, recertification 3 ans	En vigueur depuis déc. 2023 ; certifications actives depuis 2024 (BSI, A-LIGN, KPMG)
NIST AI RMF 1.0 + GenAI Profile	Volontaire — référence dominante aux USA et Canada	Fonctions GOVERN, MAP, MEASURE, MANAGE ; 12 risques GenAI + 200+ actions suggérées (NIST AI 600-1, juil. 2024) ; Profil AI Agent prévu Q4 2026 (*probable* — NIST CAISI, fév. 2026)	AI RMF 1.0 : jan. 2023 ; AI 600-1 : juil. 2024 ; Profil Agent : Q4 2026 (*à vérifier*)
OSFI E-23	Institutions financières fédérales canadiennes (IF-Fés)	Inventaire modèles AI/ML avec métadonnées (ID, propriétaire, version, cote risque, usages approuvés, limitations, dépendances), politiques MRM, contrôles alternatifs pour modèles autonomes, gestion risque tiers (B-10)	1er mai 2027
Loi 25 Québec (Art. 12.1)	Organisations traitant des renseignements personnels de personnes au Québec	Information sur décision automatisée, droit de révision par une personne physique, pénalités jusqu'à 4 % des revenus mondiaux ou 25 M CAD	Pleinement en vigueur depuis septembre 2023

FINRA 2026. La Financial Industry Regulatory Authority a publié en 2026 des orientations sur l'utilisation de l'IA générative par les courtiers-négociants, insistant sur la supervision des communications automatisées avec les clients et la traçabilité des recommandations générées par des agents IA. Ces orientations renforcent le modèle HITL-in-loop pour les interactions clients dans le secteur des valeurs mobilières (*à vérifier* — orientations définitives non publiées à la date de clôture de cette source ; traiter comme *probable*).

Canada — AIDA / Bill C-27. Le Bill C-27, incluant la Loi sur l'intelligence artificielle et les données (AIDA), est mort au feuilleton le 5 janvier 2025 lors de la prorogation du Parlement. Aucun successeur législatif fédéral canadien sur l'IA n'est en vigueur à mai 2026 (*confirmé* — BLG, janvier 2025). Le Canada opère sans cadre fédéral IA, ce qui transfère la charge de gouvernance sur les lignes directrices sectorielles (OSFI E-23 pour le secteur financier) et les cadres volontaires (NIST AI RMF).

Divergence EU AI Act vs NIST AI RMF. L'EU AI Act adopte une approche prescriptive fondée sur la classification du risque, avec des obligations contraignantes par niveau et des amendes allant jusqu'à 35 M EUR ou 7 % du chiffre d'affaires mondial. Le NIST AI RMF est volontaire, sans obligation légale directe. En pratique, les organisations qui opèrent des deux côtés de l'Atlantique utilisent le NIST AI RMF comme squelette de gouvernance interne — ses fonctions GOVERN/MAP/MEASURE/MANAGE structurent la politique IA — et l'EU AI Act comme liste des obligations à satisfaire dans la couche de conformité externe. Cette complémentarité est productive si elle est délibérée ; elle est source de duplication coûteuse si elle est traitée comme deux projets séparés.

§8.4.1 — ISO/IEC 42001 et l'AIMS : Plan-Do-Check-Act appliqué aux agents

ISO/IEC 42001:2023 est le premier standard international certifiable de management de l'IA (*Artificial Intelligence Management System*, AIMS). Sa structure Plan-Do-Check-Act (*PDCA*) fournit le cadre de gouvernance continue dans lequel s'insèrent les éléments opérationnels de ce chapitre.

Plan. L'organisation définit sa politique IA, son périmètre d'application, et évalue les risques et impacts de ses systèmes IA. Pour un système *agentic*, cette phase produit trois artefacts clés : l'évaluation d'impact IA (quels cas d'usage, quels niveaux d'autonomie, quelles populations affectées), la *agent policy* par domaine de déploiement, et le registre des systèmes IA avec leur classification de risque. Ce registre est l'artefact d'intersection avec OSFI E-23 : l'inventaire des modèles exigé par E-23 peut être intégré dans le registre AIMS, à condition que les métadonnées requises par E-23 (ID, propriétaire, version, date de déploiement, cote de risque, usages approuvés, limitations, dépendances) soient incluses dès la phase Plan.

Do. L'organisation met en œuvre les contrôles définis : déploiement des niveaux d'autonomie, instrumentation HITL, journaux de décision, formation des équipes. La *agent policy* est le document opérationnel central de cette phase — sa mise en œuvre est vérifiable par l'auditeur via les journaux produits.

Check. Des audits internes périodiques et une revue de direction annuelle évaluent l'efficacité des contrôles. Pour les systèmes agentiques, cette phase inclut la revue des métriques d'escalade (taux, type, résolution), l'analyse des incidents (actions hors périmètre, journaux incomplets, violations de politique), et la mise à jour du registre des systèmes IA. C'est à cette étape que le HITL-over-the-loop produit ses données de gouvernance : les revues post-facto alimentent le Check.

Act. Les résultats de la phase Check déclenchent des améliorations : mise à jour de la *agent policy*, révision des périmètres d'autonomie, renforcement de l'instrumentation. Ce cycle d'amélioration continue est le mécanisme par lequel un système *agentic* reste conforme à mesure que ses comportements dérivent et que les exigences réglementaires évoluent.

Articulation AIMS et *agent policy*. La *agent policy* n'est pas un artefact séparé de l'AIMS — c'est l'output opérationnel de la phase Plan, et l'input de contrôle de la phase Do. Dans un AIMS correctement implémenté, toute modification de la *agent policy* déclenche une mise à jour du registre des systèmes IA et une évaluation d'impact proportionnelle. ISO/IEC 42006:2025 précise les exigences auxquelles doivent satisfaire les organismes qui certifient la conformité à l'AIMS — ce complément de 2025 ferme la boucle en rendant les audits opposables tiers, ce qui était la lacune principale de la version 2023 du standard.

La première vague de certifications ISO/IEC 42001 (BSI, A-LIGN, Schellman, KPMG) établit en 2024-2026 les patterns de référence pour les organisations qui entrent dans le processus. Les résultats de ces premières certifications ne sont pas encore publiés de façon agrégée (*à vérifier* — aucune étude sectorielle disponible à mai 2026), mais les organismes de certification signalent que les lacunes les plus fréquentes concernent la traçabilité des décisions automatisées et la gouvernance des modèles tiers — précisément les points critiques pour les systèmes agentiques.

8.5 — Flux de gouvernance intégré

Les quatre éléments — autonomie hiérarchique, HITL opérationnel, auditabilité, conformité — ne s'additionnent pas : ils se conditionnent mutuellement. Leur intégration produit une architecture de gouvernance plus économique que leur empilement séquentiel, parce que les artefacts et les événements sont partagés plutôt que dupliqués.

La chaîne causale intégratrice est la suivante. La *agent policy* définit les niveaux d'autonomie par domaine de tâche et les triggers d'escalade — elle est le document qui rend le HITL opérationnel déterministe plutôt qu'ad hoc. Les événements d'escalade produits par le HITL sont des événements prioritaires dans le journal de décision — l'auditabilité n'est pas un add-on, elle est alimentée par les mêmes événements que le HITL. La *agent policy* elle-même est un artefact soumis à gouvernance : versionnement, revue périodique, approbation — ce cycle est formalisé

par l'AIMS PDCA (ISO/IEC 42001), exigé sous forme d'inventaire par OSFI E-23, et documenté en conformité par l'EU AI Act. Le flux est circulaire et auto-cohérent.

```
flowchart TD
    AP["Agent Policy\n(périmètre · niveaux N1–N4 · triggers)"]
    AL["Couche d'autonomie\n(N1 Assistance → N4 Autonome)"]
    HE["HITL — Escalade\nin-loop · on-loop · over-the-loop"]
    DL["Journal de décision\n(traçabilité · explicabilité · immuabilité\nautorisation
vérifiable · reproductibilité)"]
    CR["Reporting de conformité\n(EU AI Act · OSFI E-23 · Loi 25\nNIST AI RMF · AIMS PDCA)"]
    RV["Revue de politique\n(Check PDCA · audit interne · revue direction)"]

    AP -->|"définit périmètre\net triggers"| AL
    AL -->|"actions dans périmètre\n→ journalisation"| DL
    AL -->|"sortie de périmètre\n→ escalade"| HE
    HE -->|"événements d'escalade\n→ journalisation prioritaire"| DL
    DL -->|"alimentation\ndes rapports"| CR
    CR -->|"métriques et incidents\n→ déclenchent révision"| RV
    RV -->|"mise à jour\nversionnée"| AP

    style AP fill:#1a3a5c,color:#fff,stroke:#1a3a5c
    style DL fill:#1a3a5c,color:#fff,stroke:#1a3a5c
    style CR fill:#2d6a4f,color:#fff,stroke:#2d6a4f
    style RV fill:#2d6a4f,color:#fff,stroke:#2d6a4f
```

Le flux montre une propriété cruciale pour la planification : la *agent policy* est l'artefact central et le point de départ naturel de toute l'architecture. Construire le journal de décision avant la *agent policy* produit un journal sans politique de référence — irréprochable sur la forme, inutilisable sur le fond, parce qu'il ne peut pas démontrer l'autorisation. Instrumenter le HITL avant la *agent policy* produit des escalades sans triggers définis — soit des interruptions aléatoires, soit l'absence d'interruption là où la politique l'aurait imposée.

Recommandation architecturale. Implémenter l'infrastructure de gouvernance en une seule passe séquencée sur la *agent policy* : (1) rédiger et versionner la *agent policy* par domaine de tâche ; (2) instrumenter le HITL sur les triggers qu'elle définit ; (3) configurer le journal de décision sur les événements qu'elle génère ; (4) mapper les sorties du journal aux exigences réglementaires du cadre applicable. Cette séquence produit une architecture cohérente avec moins de re-travail que le montage projet par projet.

Compromis. L'intégration en une seule passe crée une dépendance forte entre les équipes (architecte, risk, compliance, ops) et rallonge la phase initiale de cadrage. **Alternative** : interfaces entre composants — *policy-as-code* exposé via API, triggers HITL consommés par un bus d'événements, journaux vers un entrepôt de données d'audit indépendant — permettent à chaque composant d'évoluer indépendamment. **Condition de bascule** : si l'organisation est multi-juridictionnelle avec des exigences réglementaires divergentes par entité (une filiale sous EU AI Act haute-risque, une autre hors périmètre), l'architecture modulaire avec interfaces est obligatoire — une *agent policy* centrale uniforme ne peut pas satisfaire des obligations réglementaires contradictoires.

8.6 — Recommandation : governance-first, *pas* trust-and-verify

La conclusion opérationnelle de ce chapitre est que l'architecture de confiance d'un système *agentic* ne peut pas être construite à rebours d'un incident. L'incident Replit a démontré qu'une organisation sans journaux de décision ne peut ni arrêter l'agent avant le dommage, ni expliquer le dommage après. L'organisation qui déploie sans *agent policy* versionnée ne peut pas démontrer à un auditeur que les actions de l'agent étaient autorisées — même si elles l'étaient.

Le modèle *governance-first* — *agent policy* avant déploiement, HITL instrumenté sur ses triggers, journal de décision aligné sur ses événements — est plus économique à long terme que le modèle *trust-and-verify* qui déploie vite et rajoute la gouvernance après. Le coût d'un audit réglementaire sur un système sans traçabilité, d'un incident irréversible sans journal de décision, ou d'une certification ISO/IEC 42001 en mode rétroactif dépasse structurellement le coût d'une mise en place initiale rigoureuse.

La connexion avec le Ch. 4 §4.4 est directe : la décision Build/Buy/Borrow/Wait sur un système *agentic* doit intégrer le coût de la gouvernance comme composante du CPST (coût par tâche réussie) — non comme un surcoût optionnel, mais comme une condition de déploiement en production conforme. Les organisations qui traitent la gouvernance comme optionnelle contribuent aux 40 % de projets agentiques que Gartner anticipe comme abandonnés avant 2027 (*confirmé* — Gartner, Hype Cycle for Agentic AI 2026).

La transition vers le chapitre suivant est immédiate : une architecture de confiance bâtie sur la *agent policy*, le HITL et les journaux de décision crée une surface d'attaque structurée — elle définit ce que l'agent peut faire, comment il le documente, et où il s'arrête. Le Ch. 9 analyse les vecteurs d'attaque propres à cette surface : *prompt injection via tools*, exfiltration inter-outils, et *jailbreak by delegation* — les mécanismes par lesquels un acteur malveillant peut subvertir précisément les mécanismes de confiance construits dans ce chapitre.

Pour aller plus loin

Reuel et al. — « AI Agents Under EU Law: A Compliance Architecture for AI Providers » — arXiv:2604.04604 — avril 2026. L'analyse juridique la plus précise disponible à mai 2026 sur la qualification des agents IA sous l'EU AI Act : distinction fournisseur vs déployeur, impact de l'autonomie sur la désignation GPAI, architecture de conformité concrète. Lecture obligatoire avant toute qualification réglementaire d'un système agentique opérant dans l'UE.

FINOS AI Governance Framework v2.0 — mitigation MI-21 — 2026. Le seul cadre open-source sectoriel (services financiers) qui formalise opérationnellement les cinq propriétés d'un journal de décision agentique — traçabilité, explicabilité, immuabilité, autorisation vérifiable, reproductibilité. Directement utilisable comme checklist d'implémentation pour §8.3. Disponible à `air-governance-framework.finos.org`.

NIST — « AI 600-1 : Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile » — juillet 2024. Le profil GenAI du NIST AI RMF identifie 12 risques propres ou exacerbés par l'IA générative et fournit 200+ actions de gestion du risque. À lire en parallèle du AI RMF 1.0 (janvier 2023) : le profil GenAI sans le RMF manque le cadre de gouvernance ; le RMF sans le profil GenAI manque les risques spécifiques aux LLM et aux agents.

ISO/IEC 42001:2023 — Artificial Intelligence Management Systems. Le standard certifiable de référence pour la gouvernance IA. À acquérir via l'ISO ou les organismes nationaux (BSI, SCC pour le Canada). La lire conjointement avec ISO/IEC 42006:2025 (exigences pour organismes de certification) pour comprendre comment les audits AIMS sont conduits et ce qu'un auditeur examinera.

Knight First Amendment Institute — « Levels of Autonomy for AI Agents » — 2026. Le cadre théorique à cinq niveaux qui ancre le modèle à quatre niveaux de §8.1. Utile pour les organisations qui veulent aligner leur modèle de gouvernance sur un cadre académique citable — notamment dans le contexte d'engagements réglementaires où la justification du choix d'autonomie peut être demandée.

Références

1. artificialintelligenceact.eu — « Implementation Timeline | EU Artificial Intelligence Act » — mai 2026 — https://artificialintelligenceact.eu/implementation-timeline/ — accédée le 2026-05-05.
2. Reuel, A. et al. — « AI Agents Under EU Law: A Compliance Architecture for AI Providers » — arXiv:2604.04604 — avril 2026 — https://arxiv.org/html/2604.04604v1 — accédée le 2026-05-05.
3. ISO — « ISO/IEC 42001:2023 — Artificial Intelligence Management Systems » — décembre 2023 — https://www.iso.org/standard/42001 — accédée le 2026-05-05.
4. ISO — « ISO/IEC 42006:2025 — Requirements for bodies providing audit and certification of AI management systems » — 2025 — https://www.iso.org/standard/42006 — accédée le 2026-05-05.
5. NIST Centre for AI Standards and Innovation (CAISI) — « Announcing the AI Agent Standards Initiative for Interoperable and Secure Innovation » — 17 février 2026 — https://www.nist.gov/news-events/news/2026/02/announcing-ai-agent-standards-initiative-interoperable-and-secure — accédée le 2026-05-05.
6. NIST — « AI 600-1 — Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile » — juillet 2024 — https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.600-1.pdf — accédée le 2026-05-05.

7. OSFI — « Guideline E-23 — Model Risk Management » — en vigueur 1er mai 2027 — https://www.osfi-bsif.gc.ca/en/guidance/guidance-library/guideline-e-23-model-risk-management-2027 — accédée le 2026-05-05.
8. Raymond Chabot Grant Thornton — « Loi 25 : l'enjeu des décisions automatisées » — 2024-2026 — https://www.rcgt.com/fr/conseils/avis-d-experts/loi-25-enjeu-decisions-automatisees/ — accédée le 2026-05-05.
9. BLG — « OSFI responds to the growing use of AI: key updates to Guideline E-23 » — novembre 2025 — https://www.blg.com/en/insights/2025/11/osfi-responds-to-the-growing-use-of-ai-key-updates-to-guideline-e-23 — accédée le 2026-05-05.
10. Knight First Amendment Institute — « Levels of Autonomy for AI Agents » — 2026 — https://knightcolumbia.org/content/levels-of-autonomy-for-ai-agents-1 — accédée le 2026-05-05.
11. FINOS — « AI Governance Framework v2.0 — MI-21 Agent Decision Audit and Explainability » — 2026 — https://air-governance-framework.finos.org/mitigations/mi-21_agent-decision-audit-and-explainability.html — accédée le 2026-05-05.
12. IJCT — « Human-in-the-Loop (HITL) Orchestration for Agentic Use Cases » — avril 2026 — https://ijctjournal.org/wp-content/uploads/2026/04/Human-in-the-Loop-HITL-Orchestration-for-Agentic-Use-Cases.pdf — accédée le 2026-05-05.

Chapitre 9 — Sécurité agentique

Partie 4 — Confiance, sécurité et durabilité Chapitre 9 · Sécurité *agentic* · ~6 000 mots · lecture ≈ 24 min

La sécurité d'un système *agentic* n'est pas héritée du modèle de langage sous-jacent — c'est une propriété architecturale délibérément construite, ou absente. Cette distinction a pris une forme concrète en juin 2025 avec CVE-2025-32711, baptisé EchoLeak par les chercheurs d'Aim Security : premier exploit zero-click documenté dans un système LLM de production, CVSS 9.3 (critique). Un courriel anodin dans Outlook, aucun clic requis, aucune action de l'utilisateur. Microsoft 365 Copilot lisait le message, exécutait les instructions encodées dans les métadonnées invisibles, et exfiltrait des fichiers OneDrive, SharePoint, Teams et Outlook vers un endpoint externe contrôlé par l'attaquant. Le mécanisme est formalisé dans arXiv:2509.10540 comme *LLM Scope Violation* : le modèle ne distingue pas le contenu de confiance (instructions système) du contenu ingéré (données utilisateur, e-mails, documents) lorsque les deux coexistent dans la fenêtre de contexte (Amit et al., Aim Security, sept. 2025 ; NVD, CVE-2025-32711, patché côté serveur sans action utilisateur requise — aucune exploitation malveillante confirmée dans la nature).

Ce n'est pas un bug isolé. EchoLeak est la conséquence d'une décision d'architecture : le modèle dispose d'un accès multi-outil (courriel, stockage, collaboration) sans frontière de confiance entre ce qu'il est autorisé à lire et ce qu'il est autorisé à exécuter. La thèse de ce chapitre est directe : les vecteurs d'attaque agentiques exploitent systématiquement la confiance implicite accordée au contenu ingéré, aux outils invoqués, et aux agents délégués — trois surfaces structurellement distinctes, un seul principe de défense applicable à toutes : *zero trust* à chaque frontière de confiance. La réponse n'est pas un seul mécanisme de défense mais une architecture à trois couches composées — guardrails de contenu, sandboxing d'exécution, identité et accès par tâche — dont l'efficacité est composée et non additive. Ce chapitre construit sur le modèle de menace protocolaire du Ch. 5 §5.8 (*tool poisoning*, injection MCP *sampling*, RCE supply chain), sur le plan de contrôle AgentOps du Ch. 7 §7.6 (kill switches, retry budgets, permission boundaries), et sur les niveaux d'autonomie N1-N4 définis au Ch. 8 §8.1 — sans les redéfinir.

9.1 — Pourquoi la sécurité agentique est différente

Un LLM passif produit du texte. Un agent *agentic* transforme du texte en actions : il invoque des outils avec des effets de bord irréversibles, délègue des sous-tâches à d'autres agents, accumule un état mémoire persistant entre les sessions, et opère sur un horizon temporel plus long que la fenêtre de contexte d'une seule interaction. Ces quatre caractéristiques structurelles — actions réelles, délégation multi-hop, mémoire persistante, temporalité longue — créent des classes de risque absentes dans le modèle de menace des API classiques et dans le LLM Top 10 v2 d'OWASP, qui reste orienté modèle unique sans autonomie d'action.

Le Cloud Security Alliance résume la différence fondamentale en mars 2026 (arXiv:2603.11088v1) : le risque agentique introduit un gap temporel structurel entre l'initiation d'une action et son observation par un opérateur humain. Un agent qui traite 10 000 messages d'Outlook sur 48 heures peut exfiltrer des données pendant 12 heures avant qu'une alerte soit déclenchée — si tant est qu'une alerte existe. Cette asymétrie temporelle n'est pas une faille d'implémentation : elle est inhérente à l'autonomie opérationnelle qui constitue la valeur du système. La sécurité agentique ne peut donc pas être réduite à la sécurité périmétrique classique (qui contrôle qui peut entrer dans le système) : elle doit contrôler ce que le système fait une fois à l'intérieur.

Trois propriétés structurelles rendent la sécurité agentique irréductible aux approches héritées.

Contenu comme vecteur d'attaque. Dans une API REST, les données traitées et les instructions d'exécution empruntent des canaux séparés — le payload et le header. Dans un agent *agentic*, instructions et données coexistent dans la fenêtre de contexte, sans barrière cryptographique entre elles. Tout contenu ingéré — document analysé, page web récupérée, résultat d'outil, courriel reçu — peut contenir des instructions que le modèle exécutera si sa politique d'interprétation ne distingue pas les deux espaces. C'est la *LLM Scope Violation* formalisée par EchoLeak.

Délégation comme vecteur de propagation. Une compromission dans un agent orchestrateur se propage à ses agents workers par le mécanisme de délégation lui-même, sans que les workers puissent vérifier l'intégrité de l'intention transmise. Les systèmes MAD (*multi-agent debate*) amplifient ce vecteur : arXiv:2504.16489 démontre

que quatre frameworks MAD testés présentent tous des vulnérabilités au *jailbreak by delegation* via modification des descriptions de tâche inter-agents, avec propagation multi-hop à travers les tours de dialogue.

Mémoire persistante comme vecteur de durée. Un vecteur d'attaque encodé dans la mémoire épisodique ou sémantique (embeddings, RAG) persiste au-delà d'une session, survit à un redémarrage de l'agent, et influence le raisonnement futur sans que l'injection initiale soit tracée. L'exfiltration ne se produit pas au moment de l'injection mais lors de la prochaine requête qui active la mémoire compromise — gap temporel entre attaque et effet pouvant dépasser plusieurs jours.

9.2 — Modèle de menace agentique : taxonomie OWASP ASI01–ASI10

L'OWASP (Open Web Application Security Project) GenAI Security Project a publié en décembre 2025 le premier Top 10 dédié aux applications *agentic* (peer-reviewé par 100+ experts), distinct du LLM Top 10 v2 orienté modèle unique. La nomenclature ASI (*Agentic Security Incident*) 01–10 structure le modèle de menace de ce chapitre selon trois familles opérationnelles.

Famille 1 — Injection et détournement d'objectifs (ASI01, ASI02, ASI06)

ASI01 — Agent Goal Hijack. L'injection indirecte (*indirect prompt injection*, IDPI) est la classe dominante en 2025-2026 : les instructions malveillantes sont encodées dans le contenu que l'agent ingère, pas dans le prompt de l'utilisateur. EchoLeak est l'instance de production la plus documentée. Les systèmes affectés incluent les navigateurs, moteurs de recherche, processeurs de contenu et agents automatisés — 22 techniques d'ingénierie de charge identifiées par Palo Alto Networks Unit 42 (mars 2026). La *cross-tool exfiltration* (ASI02) en est la conséquence directe : l'injection dans l'outil A contrôle l'invocation de l'outil B pour exfiltrer vers un endpoint externe. Des taux de succès élevés (de l'ordre de 70 à 85 %, *à vérifier* — études de *red-teaming* en conditions contrôlées 2025-2026, périmètres et modèles variant selon les publications) ont été documentés sur GPT-4o, Copilot et Cursor pour des tâches de résumé incluant exfiltration ou exécution de commandes malveillantes. Ces chiffres proviennent d'études en laboratoire, pas de production, et doivent être interprétés comme des bornes de risque indicatives — non comme des taux de succès attendus en exploitation.

ASI06 — Memory & Context Poisoning. La corruption des stores de mémoire persistante (embeddings, RAG, mémoire épisodique) biaise le raisonnement futur de manière durable. Ce vecteur est distinct de l'injection directe : l'attaque persiste entre les sessions et survit aux redémarrages de l'agent. Aucun outil standard de détection de corruption de mémoire n'est disponible à mai 2026 — les approches existantes (validation de cohérence, signatures d'embeddings) sont expérimentales. Renvoi Ch. 6 §6.5 pour l'architecture mémoire sous-jacente.

Famille 2 — Délégation et chaînes de confiance (ASI03, ASI07, ASI08, ASI09)

ASI03 — Agent Identity & Privilege Abuse. Un agent qui hérite des permissions de l'orchestrateur sans réduction du périmètre à la délégation opère avec des droits excessifs pour sa sous-tâche. Le *trust-authorization mismatch* formalisé par arXiv:2505.02077 est la pathologie structurelle : l'agent fait confiance à ses pairs sans vérifier que l'autorisation transmise est cohérente avec la sous-tâche reçue.

ASI07 — Insecure Inter-Agent Communication. L'absence d'authentification mutuelle entre agents A2A — usurpation d'Agent Card, man-in-the-middle sur les messages de délégation — permet à un attaquant de se faire passer pour un agent légitime dans la chaîne. La spec A2A v1.0.0 prévoit OAuth 2.0 et mTLS comme mécanismes d'authentification (renvoi Ch. 5 §5.3), mais leur déploiement effectif reste à la charge de l'implémenteur — aucun mécanisme obligatoire n'est imposé par le protocole.

ASI08 — Cascading Failures. Dans un système multi-agents à grande échelle, un *cascading jailbreak* se propage à travers les chaînes de délégation de confiance : un agent compromis transmet des instructions malveillantes à ses workers, qui les transmettent aux leurs. arXiv:2505.02077 formalise les effets de réseau qui amplifient les vulnérabilités à l'échelle — la posture de sécurité du système est déterminée par l'agent le moins sécurisé dans la chaîne.

Famille 3 — Supply chain et exécution non maîtrisée (ASI04, ASI05, ASI10)

ASI04 — Agentic Supply Chain Compromise. La surface d'attaque protocolaire MCP documentée au Ch. 5 §5.8 — *tool poisoning*, injection via *sampling*, RCE dans les SDKs officiels (OX Security, avril 2026 : 9 registres sur 11 testés compromis) — s'étend ici aux registres de plugins des plateformes (Copilot Studio, Bedrock AgentCore), aux agents tiers dans les systèmes A2A, et aux bibliothèques SDK de la chaîne de dépendances. La chaîne de confiance va du SDK au registre, du registre au serveur MCP, du serveur à l'agent orchestrateur.

ASI05 — Unexpected Code Execution. Un agent qui génère et exécute du code sans sandbox expose l'environnement hôte à toute instruction encodée dans le code généré — qu'elle provienne d'une injection indirecte

ou d'une hallucination productive d'une action destructrice. C'est la classe de risque qui rend le sandboxing non optionnel pour les agents à niveau N3-N4 (voir §9.3).

Classe OWASP	Famille	Impact maximum	Contrôle primaire
ASI01 Agent Goal Hijack	Injection	Déviation complète d'objectif	Guardrails I/O + séparation des espaces de confiance
ASI02 Tool Misuse / Cross-tool exfiltration	Injection	Exfiltration de données enterprise	Scopes OAuth par outil + monitoring tool spans
ASI03 Identity & Privilege Abuse	Délégation	Escalade de privilèges dans la chaîne	OAuth 2.1 per-task + RFC 8693 scope narrowing
ASI04 Supply Chain	Supply chain	Compromission de l'infrastructure MCP	Vérification de registres + SBOMs agents
ASI05 Unexpected Code Execution	Exécution	RCE sur hôte ou ressources enterprise	Sandboxing microVM obligatoire
ASI06 Memory Poisoning	État persistant	Biais durable du raisonnement	Validation de cohérence + signatures d'embeddings
ASI07 Insecure Inter-Agent Comm.	Délégation	Usurpation d'agent dans la chaîne	mTLS + OAuth 2.0 sur messages A2A
ASI08 Cascading Failures	Délégation	Propagation système multi-agents	Isolation par périmètre + kill switch par agent
ASI09 Human-Agent Trust Exploitation	Délégation	Manipulation du superviseur humain	HITL structuré (voir Ch. 8 §8.2)
ASI10 Rogue Agents	Exécution	Agent hors gouvernance opérant en autonome	Inventaire agents + politique d'enregistrement obligatoire

---

9.3 — Défense en profondeur : trois couches composées

La défense en profondeur agentique n'est pas une somme de contrôles indépendants — c'est un pipeline ordonné dont chaque couche compense les lacunes des couches adjacentes. La divergence centrale sur l'efficacité des guardrails doit être posée d'emblée : plusieurs travaux de *red-teaming* publiés en 2025-2026 (notamment les *adversarial robustness reports* d'Anthropic et les benchmarks Llama Guard de Meta) convergent sur un constat — les attaques *adaptatives* (l'attaquant connaît la défense et l'optimise) franchissent les guardrails état de l'art pris isolément dans une fraction substantielle des cas (*à vérifier* — la quantification précise varie selon le périmètre d'évaluation et les modèles testés ; aucune méta-analyse intégrée publiée à la clôture de ce chapitre). Cette borne ne disqualifie pas les guardrails ; elle indique que les guardrails seuls ne constituent pas une défense suffisante et qu'ils doivent être composés avec le sandboxing et l'identité par tâche.

Couche 1 — Guardrails de contenu (filtrage I/O)

Les guardrails interceptent les entrées malveillantes avant qu'elles atteignent le modèle, et les sorties dangereuses avant qu'elles soient exécutées ou transmises. Cinq familles documentées à mai 2026, comparées sur quatre dimensions :

Guardrail	Architecture	Couverture	Déploiement	Remarque
Llama Guard 4 (Meta, 12B, multimodal)	Modèle dense ouvert, pruné de Llama 4 Scout MoE	Texte + images, MLCommons hazards taxonomy	On-premise ou API tierce	Seul guardrail ouvert multimodal à mai 2026
NeMo Guardrails (NVIDIA, Colang)	Middleware open-source entre app et LLM	*Topical*, *fact-checking*, *jailbreak rails*	On-premise, intégration framework	Version de production enterprise peu documentée (*à vérifier*)
Anthropic Constitutional Classifiers	Classificateurs entraînés sur données synthétiques	*Universal jailbreaks*, constitution NLP	API Anthropic uniquement	Démo publique février 2025 ; intégration via API
Azure AI Content Safety + Prompt Shield	Service cloud Microsoft	Injection indirecte via docs récupérés, PII	Azure-native, intégration Azure OpenAI	Couvre spécifiquement la détection IDPI dans documents
AWS Bedrock Guardrails (6 politiques)	Service cloud cross-provider	Content, prompt attack, topic, PII, hallucination, custom words	Cross-provider (Bedrock + OpenAI + Gemini)	Seul guardrail cross-provider du tableau

La divergence d'efficacité doit être présentée à l'architecte sans ambiguïté : en conditions de laboratoire avec attaques adaptatives, les guardrails sont contournables dans la majorité des scénarios testés. En production avec défense en profondeur composée (guardrails + sandboxing + kill switches), l'efficacité *composée* est différente de l'efficacité isolée — mais aucune étude primaire ne quantifie encore cette efficacité composée en production enterprise à mai 2026. Le guardrail reste un premier filtre indispensable, pas une assurance complète.

Couche 2 — Sandboxing de l'exécution

Un agent qui exécute du code généré ou invoque des outils avec effets de bord doit opérer dans un environnement isolé. Le choix du niveau d'isolation est une décision architecturale à calibrer selon le niveau d'autonomie de l'agent (voir §9.7 pour la matrice N1-N4 × couches de sécurité).

```
graph LR
  subgraph "Perception"
    A[Contenu ingéré<br/>doc, email, web]
    B[Prompt utilisateur]
  end
  subgraph "Décision — LLM"
    C[Modèle de langage<br/>fenêtre de contexte]
    D[LLM Scope Violation<br/>ASI01 / IDPI]
  end
  subgraph "Action — Tools"
    E[Tool span<br/>OAuth scope]
    F[Cross-tool exfiltration<br/>ASI02]
  end
  subgraph "Tiers / Supply chain"
    G[Registre MCP/Plugin<br/>ASI04]
    H[Agent worker A2A<br/>ASI07 / ASI08]
  end
  subgraph "Mémoire persistante"
    I[RAG / embeddings<br/>ASI06 Memory Poisoning]
  end
  A -->|injection indirecte| D
  B --> C
  D --> C
```

```
C --> E
E -->|scope excessif| F
G -->|tool poisoning| E
E --> H
H -->|délégation compromise| F
C --> I
I -->|biais futur| C
```

Trois niveaux d'isolation disponibles en 2026, par ordre croissant de robustesse :

Conteneur Docker standard : isolation au niveau des processus, partage du noyau hôte. L'évasion par exploit noyau est documentée. Insuffisant pour les agents exécutant du code non fiable ou accédant à des ressources enterprise. Tolérable uniquement en prototype sans accès à des systèmes de production.

gVisor (Google) : noyau en espace utilisateur qui intercepte les syscalls avant qu'ils atteignent le noyau hôte. Isolation significativement supérieure aux conteneurs. Surcoût en latence de l'ordre de 10-20 % selon la charge (*à vérifier* — estimation basée sur benchmarks gVisor open-source). Utilisé par Modal pour les sandboxes agents en production.

Firecracker microVM (AWS open-source) : frontière hardware-level VM, noyau séparé par sandbox. Démarrage 150 ms (E2B, confirmé — Northflank 2026). Isolation la plus forte disponible commercialement. Utilisé par E2B (200M+ sandboxes démarrés, clients Fortune 100 — E2B, 2026). Kata Containers (VM légères compatibles OCI) offrent une alternative sur Northflank (2M+ workloads/mois).

Le pattern opérationnel recommandé pour les agents N3-N4 : session courte + rotation systématique du sandbox après chaque tâche. Les sessions longue durée dans un seul sandbox accumulent l'état d'attaque potentiel entre les tâches successives.

Couche 3 — Kill switches et plan de contrôle

Le plan de contrôle AgentOps du Ch. 7 §7.6 définit les kill switches dans leur dimension opérationnelle (permission boundaries, retry budgets, rate limits). La sécurité agentique ajoute quatre modes de désactivation ciblée qui ne supposent pas d'arrêt complet du système :

- **Kill switch par agent** : désactiver une instance sans affecter les autres agents du système. Prérequis : chaque agent a une identité distincte (voir §9.4).
- **Kill switch par outil** : révoquer l'accès d'un agent à un outil spécifique (ex. outil d'envoi de courriel) sans arrêter l'agent. Prérequis : scopes OAuth par outil révocables indépendamment.
- **Kill switch par périmètre de données** : isoler un store de mémoire potentiellement compromis (ASI06) sans perturber les autres composants. Prérequis : mémoire segmentée avec accès contrôlé.
- ***Dry-run mode*** : basculer un agent en mode lecture seule (effets de bord désactivés) pour investigation sans arrêt du flux. Pattern utile pour les investigations forensiques sans interruption du service.

La gouvernance des kill switches — qui détient l'autorité d'activation, dans quel délai, avec quelle traçabilité — est documentée dans le RACI agentique de l'Annexe D.

9.4 — Identité et accès pour agents : du compte de service à l'identité vérifiable

Un compte de service partagé avec une clé API statique n'est pas une identité d'agent — c'est un périmètre d'attaque permanent sans attribution. Quand un incident implique une clé API partagée entre 12 agents, il est impossible de déterminer quel agent a initié l'action non autorisée. Le NCCoE (National Cybersecurity Center of Excellence) du NIST formule cette exigence dans son concept paper de février 2026 (NCCoE, fév. 2026) : les comptes de service partagés sont architecturalement insuffisants pour l'identité d'agent enterprise — ils doivent être remplacés par des identités dédiées avec gestion du cycle de vie, en lien avec NIST SP 800-207 (Zero Trust Architecture).

Trois niveaux de maturité d'identité pour agents, du moins sécurisé au plus sécurisé :

Niveau 0 — Clé API statique partagée. Pattern dominant dans les déploiements de première génération (2024-2025). Problèmes structurels : absence d'attribution (quelle action de quel agent ?), révocation coûteuse (révoquer la clé coupe tous les agents), aucune rotation automatique, périmètre binaire (la clé donne accès à tout le service). Incompatible avec les exigences OSFI E-23 d'inventaire des modèles avec métadonnées de propriété et périmètre autorisé (en vigueur 1er mai 2027).

Niveau 1 — Rôles IAM (Identity and Access Management) fins + OAuth 2.1 per-task. Le pattern *per-task token* s'articule autour de quatre mécanismes complémentaires.

OAuth 2.1 (IETF draft-ietf-oauth-v2-1) : PKCE (*Proof Key for Code Exchange*) obligatoire, rotation des refresh tokens, suppression du flux implicite. La granularité des *tool-level scopes* est la contribution critique pour la sécurité agentique : un scope « lecture contacts Salesforce » est révocable indépendamment d'un scope « création Salesforce » — l'agent worker d'analyse reçoit le premier, jamais le second.

RFC 8693 (Token Exchange) : réduction du périmètre (*scope narrowing*) à chaque hop de délégation. L'orchestrateur échange son token large contre un token plus étroit avant de déléguer à un worker — le worker ne peut pas hériter de permissions que l'orchestrateur lui-même ne détient pas (RFC 8693 est un standard IETF publié, non un draft).

SPIFFE/SVID : identité machine vérifiable par certificat X.509 ou JWT pour les flux M2M (*machine-to-machine*) dans un même cluster ou une même organisation. Utilisé par les déploiements Kubernetes pour les communications inter-services.

AAuth (IETF draft-patwhite-aauth-00, *hypothèse/émergent* — draft initial, pas standard à mai 2026) : extension OAuth 2.1 pour clients agents confidentiels — consentement utilisateur via HTTP polling, SSE ou WebSocket, sans interaction synchrone obligatoire.

Niveau 2 — Identité vérifiable d'agent avec gouvernance du cycle de vie. Deux implémentations de référence disponibles à mai 2026.

Microsoft Entra Agent ID : identités dédiées par agent (pas de mots de passe, authentification par *Federated Identity Credentials*), politiques d'accès adaptatives, détection de risque en temps réel, *agent identity blueprints* parent-enfant pour déployer des politiques cohérentes à grande échelle. Intégration native Azure AI Foundry et Microsoft 365 Copilot. Le lifecycle management (création, rotation, révocation) est géré comme une identité humaine dans Entra ID (Microsoft Entra Blog, 2025-2026).

AWS Bedrock AgentCore Identity (powered by Amazon Cognito) : *token vault* pour les tokens OAuth 2.0 des utilisateurs délégués — les credentials ne circulent jamais dans le prompt, ils sont injectés programmatiquement par le runtime au moment de l'exécution de l'outil. Intégration native AWS Secrets Manager. Rôles IAM fins par composant AgentCore, avec ARNs spécifiques aux ressources (AWS Security Blog). Un chemin de privilege escalation dans AgentCore via mauvaise configuration des SCPs (*Service Control Policies*) a été documenté par Sonrai Security (2025-2026 — *à vérifier* : source unique, non corroborée par une source primaire AWS directe) — l'architecte doit valider la configuration des SCPs avec une revue IAM explicite lors du déploiement.

Voici un exemple minimal en Python 3.13 illustrant le pattern *per-task scoped token* avec réduction de périmètre à la délégation (Niveau 1) :

```python
## Python 3.13 — pattern per-task scoped token avec scope narrowing
## Prérequis : bibliothèques requests-oauthlib, PyJWT
import jwt
import time
from oauthlib.oauth2 import BackendApplicationClient
from requests_oauthlib import OAuth2Session

ORCHESTRATOR_SCOPES = ["crm:read", "crm:write", "calendar:read"]
WORKER_ANALYSIS_SCOPES = ["crm:read"]  # scope narrowing : pas de write

def fetch_task_token(
    token_endpoint: str,
    client_id: str,
    client_secret: str,
    scopes: list[str],
) -> str:
    """Échange un token client credentials avec scopes réduits par tâche."""
    client = BackendApplicationClient(client_id=client_id)
    session = OAuth2Session(client=client, scope=" ".join(scopes))
    token = session.fetch_token(
```

```
        token_url=token_endpoint,
        client_id=client_id,
        client_secret=client_secret,
    )
    return token["access_token"]

def delegate_to_worker(orchestrator_token: str, task_id: str) -> str:
    """RFC 8693 token exchange : réduit le scope avant délégation au worker."""
    # En production : appel à l'endpoint token avec grant_type=urn:ietf:params:oauth:grant-
type:token-exchange
    # Simplifié ici pour illustration du principe de scope narrowing
    payload = jwt.decode(orchestrator_token, options={"verify_signature": False})
    worker_payload = {
        **payload,
        "scope": " ".join(WORKER_ANALYSIS_SCOPES),
        "task_id": task_id,
        "exp": int(time.time()) + 300,  # token valide 5 min max par tâche
    }
    return jwt.encode(worker_payload, "worker-secret", algorithm="HS256")
```

Ce patron garantit qu'un worker compromis ne peut exfiltrer que ce que son scope autorise — dans cet exemple, lecture seule CRM, sans écriture ni accès calendrier.

9.5 — La surface de confiance distribuée : défense inter-agents

Dans un système multi-agents, la posture de sécurité du système est déterminée par l'agent le moins sécurisé dans la chaîne de délégation. Cette propriété n'est pas intuitive : un orchestrateur parfaitement sécurisé peut être compromis indirectement si un worker reçoit du contenu malveillant et remonte des instructions manipulées dans sa réponse.

La formalisation par arXiv:2505.02077 est opérationnellement utile : un système multi-agents est un DAG (*directed acyclic graph*) de dépendances d'exécution. Chaque arête du DAG est une frontière de confiance implicite. Sans mécanisme d'authentification et de réduction de scope sur chaque arête, le DAG est une infrastructure de propagation des compromissions, pas seulement une infrastructure d'exécution.

Trois patterns de défense pour la communication inter-agents, par ordre croissant de robustesse :

Authentification mutuelle sur les messages A2A. La spec A2A v1.0.0 prévoit OAuth 2.0 et mTLS (mutual TLS) comme mécanismes d'authentification entre agents (renvoi Ch. 5 §5.3). En l'absence d'implémentation obligatoire imposée par le protocole, l'architecte doit spécifier explicitement l'authentification mutuelle dans la *agent policy*. Un agent qui accepte des messages A2A non authentifiés est exposé à l'usurpation d'identité (ASI07).

Scope narrowing systématique à chaque délégation. RFC 8693 Token Exchange permet à l'orchestrateur de réduire son périmètre avant de déléguer à un worker. Le worker reçoit un token dont les scopes sont strictement inférieurs ou égaux à ceux de l'orchestrateur — la propagation de privilèges excessifs (ASI03) est ainsi contenue à la frontière de chaque délégation. Ce mécanisme est implémentable aujourd'hui avec OAuth 2.1 et un serveur d'autorisation standard.

Attestation d'intention inter-agents (*hypothèse/émergent* — non standardisé à mai 2026). L'orchestrateur encode dans le message de délégation une attestation signée de l'intention originale. Le worker peut vérifier que sa sous-tâche est cohérente avec l'intention déclarée avant de l'exécuter. Si le contenu malveillant a modifié la description de tâche transmise par un agent compromis, l'attestation d'intention permet au worker de détecter la divergence. Aucun standard IETF ni NIST ne définit ce mécanisme à mai 2026 — les drafts AAuth et Agentic JWT (IETF, drafts initiaux) posent des fondations mais n'adressent pas encore ce cas précis. Le profil d'interopérabilité AI Agent prévu par le NIST pour Q4 2026 (*probable* — NIST CAISI, fév. 2026) pourrait standardiser ce pattern.

Cas concret illustrant la défense composée : un système Bedrock AgentCore avec un orchestrateur et trois workers spécialisés (analyse, rédaction, envoi-courriel). Le worker analyse reçoit un document avec injection indirecte encodée. Sans défense inter-agents, il modifie les instructions transmises au worker envoi, qui envoie le courriel malveillant. Avec authentification mutuelle + scope narrowing : le token du worker analyse ne comporte pas le scope `email:send`

— il ne peut pas invoquer le worker envoi directement, et doit passer par l'orchestrateur qui valide la cohérence de l'instruction avec l'intention initiale. La compromission est contenue dans le périmètre du worker analyse.

9.6 — Cas Sonrai AgentCore : privilege escalation via SCPs (à vérifier)

Sonrai Security a documenté (2025-2026) un chemin de privilege escalation dans Amazon Bedrock AgentCore résultant d'une mauvaise configuration des SCPs (*Service Control Policies*) AWS Organizations. Le mécanisme décrit : un agent opérant avec un rôle IAM fin peut, via une combinaison de politiques SCPs permissives et de *confused deputy*, accéder à des ressources situées hors de son périmètre autorisé par des API calls latéraux non prévus dans la *agent policy*.

Cette vulnérabilité de configuration — pas une vulnérabilité protocolaire dans AgentCore lui-même — illustre un anti-patron structurel : les rôles IAM fins par composant ne suffisent pas si la couche de politique organisationnelle (SCPs) n'est pas alignée avec les périmètres d'agent définis dans la *agent policy*. La revue de configuration IAM/SCP doit être explicitement incluse dans la checklist de déploiement AgentCore.

Statut de la source : *à vérifier* — la documentation Sonrai Security est une source unique non corroborée par une source primaire AWS directe (advisory de sécurité, bulletin AWS Security, ou CVE). AWS n'a pas publié d'advisory officiel sur ce vecteur à mai 2026. L'architecte déployant AgentCore doit vérifier directement auprès d'AWS la validité et la portée de cette vulnérabilité de configuration avant de l'inclure dans son évaluation de risque. L'anti-patron de configuration (SCPs non alignées avec les périmètres d'agent) est, lui, structurellement valide indépendamment de la source.

9.7 — Recommandation : guardrails-as-gate vs sandboxing-first vs identity-first

La défense en profondeur à trois couches est le résultat cible pour tout système N3-N4. La question opérationnelle n'est pas laquelle des trois couches est la plus importante — elles sont toutes nécessaires — mais dans quel ordre les déployer et où concentrer l'investissement initial selon le profil de risque du système.

La matrice ci-dessous croise les niveaux d'autonomie N1-N4 (définis au Ch. 8 §8.1) avec les couches de sécurité requises :

Niveau	Désignation	Guardrails I/O	Sandboxing	Identité par tâche	Kill switches actifs
N1	Assistance (lecture seule)	Recommandé	Non requis	Clé API par agent	Non requis
N2	Supervisé (gate humain)	Obligatoire	Fortement recommandé	OAuth 2.1 per-task	Gate humain suffit
N3	Autonome borné	Obligatoire	Obligatoire (gVisor minimum)	OAuth 2.1 + SPIFFE/SVID	Obligatoire par agent
N4	Autonome (journalisation immuable)	Obligatoire	Obligatoire (Firecracker/microVM)	Identité vérifiable (Entra/AgentCore)	Obligatoire par agent + par outil

Recommandation principale — identity-first pour les systèmes greenfield. Pour un déploiement *agentic* en cours de conception, déployer l'identité vérifiable d'agent (Niveau 1 OAuth 2.1 per-task minimum) avant les guardrails et le sandboxing. La raison est d'ordre forensique : sans identité distincte par agent, un incident post-production est impossible à attribuer, ce qui empêche toute amélioration causale de la posture de sécurité. Les guardrails et le sandboxing peuvent être ajoutés itérativement ; l'identité ne peut pas être rétroactivement imposée sans refactoring de l'architecture de permission.

Compromis principal. L'identité vérifiable de Niveau 2 (Entra Agent ID, AgentCore Identity) ajoute une latence d'authentification de 50-200 ms par échange de token selon le fournisseur (*à vérifier* — estimation basée sur les benchmarks OAuth 2.0 standards ; les SLA des services Entra et AgentCore Identity ne publient pas de latences d'authentification granulaires à mai 2026). Pour les cas d'usage à contrainte de latence forte (réponse totale < 500 ms), l'architecte doit dimensionner le compromis et documenter la décision de latence vs sécurité.

Alternative crédible — guardrails-as-gate centralisé. Un gateway unique applique les guardrails et le sandboxing pour tous les agents, plutôt qu'une implémentation par service. Pattern observé dans les déploiements enterprise matures 2025-2026. Avantage : réduit la complexité opérationnelle, crée un point de contrôle unifié. Risque : crée un point de défaillance unique et un goulot de latence centralisé. Recommandé pour les organisations dont la maturité AgentOps est insuffisante pour gérer des guardrails distribués par service.

Alternative crédible — sandboxing-first pour les agents code-exécuteurs. Si le système agentique est principalement un exécuteur de code généré (agent SRE, agent développement), le sandboxing microVM est la couche la plus urgente à déployer — ASI05 est le vecteur de risque dominant pour ce profil, et les guardrails de contenu ont un impact limité contre le code malveillant généré. Déployer Firecracker/E2B avant les guardrails I/O pour ce profil spécifique.

Condition qui renverse identity-first. Si le système agentique est un prototype d'exploration en environnement isolé — réseau air-gappé, aucun accès à des ressources enterprise réelles, données synthétiques uniquement — une clé API par agent avec rotation hebdomadaire est suffisant. L'investissement en identité vérifiable se justifie uniquement lorsque le système accède à des ressources enterprise réelles ou opère dans un environnement multi-tenant.

Condition qui renverse guardrails-as-gate. À partir du moment où le système dépasse une dizaine d'agents avec des scopes distincts, le gateway centralisé devient un goulot de latence et un point de défaillance unique inacceptable pour des systèmes à haute disponibilité. La cible est des guardrails distribués par périmètre de service, avec un plan de contrôle centralisé pour la configuration et la révocation.

La checklist de revue architecturale de l'Annexe A intègre la section sécurité agentique correspondant aux décisions de ce chapitre : couverture des trois couches de défense, validation des scopes OAuth par agent et par outil, présence d'identités distinctes par composant, et activation des kill switches par périmètre.

9.8 — Limites de la sécurité agentique à mai 2026

Quatre problèmes ouverts sans réponse standardisée disponible à la clôture de ce chapitre.

Standardisation de l'attestation d'intention. Aucun standard d'*intent attestation* entre agents n'est ratifié. Les drafts IETF AAuth et Agentic JWT sont en phase initiale (version −00, signifiant premier draft soumis). Le profil d'interopérabilité NIST AI Agent prévu pour Q4 2026 (*probable* — NIST CAISI, fév. 2026) pourrait introduire ce mécanisme, mais sa portée n'est pas confirmée.

Efficacité composée de la défense en profondeur. La méta-analyse documentée rapporte > 85 % de succès des attaques *adaptatives* contre les guardrails seuls — mais cette borne est mesurée isolément, en conditions de laboratoire. L'efficacité de la défense *composée* (guardrails + sandboxing + identité + kill switches) en production enterprise n'est pas encore quantifiée dans une étude primaire. L'architecte ne dispose pas de métriques fiables pour calibrer le risque résiduel d'un système à défense en profondeur complète.

Détection de memory poisoning. Aucun outil standard de détection de corruption des stores de mémoire persistante (ASI06) n'est disponible à mai 2026. Les approches expérimentales (validation de cohérence sémantique, signatures d'embeddings, détection d'anomalies dans les scores de pertinence RAG) n'ont pas de benchmark de production établi. Ce vecteur reste le plus difficile à défendre parce que la corruption est silencieuse et ne produit pas d'erreur détectable au moment de l'injection.

Responsabilité distribuée dans les systèmes multi-agents à grande échelle. Dans un système avec 20+ workers, déterminer quel agent a initié une action non autorisée requiert une trace d'audit inter-agents standardisée. OTel GenAI SemConv 1.40.0 (*Development*, attributs `gen_ai.agent.*`) fournit les attributs d'instrumentation, mais les *orchestration spans* inter-agents ne capturent pas encore la chaîne causale complète de délégation avec réduction de scope à chaque hop (renvoi Ch. 7 §7.2 pour l'état des traces agentiques). Sans cette traçabilité causale complète, la conformité OSFI E-23 (inventaire des modèles avec métadonnées d'action) et EU AI Act Art. 12 (journaux d'événements) reste incomplète pour les systèmes multi-agents.

Pour aller plus loin

OWASP GenAI Security Project — « OWASP Top 10 for Agentic Applications 2026 » La référence normative pour le modèle de menace agentique. Couvre les 10 classes ASI01-ASI10 avec des mitigations pratiques pour chaque classe. Mise à jour synchronisée avec les incidents de production documentés. Point de départ obligatoire

pour tout architecte qui modélise les menaces d'un système *agentic* en enterprise. URL : https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/

NCCoE / NIST — « Accelerating the Adoption of Software and AI Agent Identity and Authorization » (2026) Concept paper du National Cybersecurity Center of Excellence sur l'identité d'agent. Pose les fondements de ce que devrait être une identité d'agent enterprise et pourquoi les comptes de service partagés sont insuffisants. Lire en complément de NIST SP 800-207 (Zero Trust Architecture) pour situer l'identité d'agent dans le cadre Zero Trust. URL : https://www.nccoe.nist.gov/sites/default/files/2026-02/accelerating-the-adoption-of-software-and-ai-agent-identity-and-authorization-concept-paper.pdf

arXiv:2509.10540 — Amit et al. — « EchoLeak » (Aim Security, 2025) Analyse technique primaire de CVE-2025-32711. Formalise le mécanisme de *LLM Scope Violation* avec précision suffisante pour guider les décisions architecturales sur la séparation des espaces de confiance dans la fenêtre de contexte. À lire conjointement avec la fiche NVD CVE-2025-32711. URL : https://arxiv.org/abs/2509.10540

arXiv:2505.02077 — « Open Challenges in Multi-Agent Security » (mai 2025) Synthèse des défis ouverts en sécurité multi-agents. Formalise le concept de *trust-authorization mismatch* et les effets de réseau dans la propagation des *cascading jailbreaks*. Source théorique fondamentale pour comprendre pourquoi la sécurité agentique ne se réduit pas à la sécurité de chaque agent pris isolément. URL : https://arxiv.org/html/2505.02077

IETF — OAuth 2.1 (draft-ietf-oauth-v2-1) + RFC 8693 (Token Exchange) Les deux documents IETF qui fondent le pattern *per-task token* avec scope narrowing. OAuth 2.1 consolide les meilleures pratiques OAuth 2.0 avec PKCE obligatoire et granularité des scopes. RFC 8693 définit le mécanisme d'échange de token permettant la réduction de périmètre à chaque délégation — mécanisme central de la défense ASI03. URL OAuth 2.1 : https://datatracker.ietf.org/doc/draft-ietf-oauth-v2-1/ URL RFC 8693 : https://datatracker.ietf.org/doc/html/rfc8693

Références

OWASP GenAI Security Project — « OWASP Top 10 for Agentic Applications for 2026 » — OWASP — décembre 2025 — https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/ — accédé le 2026-05-05

OWASP GenAI Security Project — « Agentic AI — Threats and Mitigations » v1.1 — OWASP — février 2025, mise à jour décembre 2025 — https://genai.owasp.org/resource/agentic-ai-threats-and-mitigations/ — accédé le 2026-05-05

NVD (National Vulnerability Database) — « CVE-2025-32711 » — NIST — juin 2025 — https://nvd.nist.gov/vuln/detail/cve-2025-32711 — accédé le 2026-05-05

Amit et al. (Aim Security) — « EchoLeak: The First Real-World Zero-Click Prompt Injection Exploit in a Production LLM System » — arXiv:2509.10540 — septembre 2025 — https://arxiv.org/abs/2509.10540 — accédé le 2026-05-05

Auteurs anonymisés — « Amplified Vulnerabilities: Structured Jailbreak Attacks on LLM-based Multi-Agent Debate » — arXiv:2504.16489 — avril 2025 — ICLR 2026 Workshop on Agents in the Wild — https://arxiv.org/abs/2504.16489 — accédé le 2026-05-05

Auteurs — « Open Challenges in Multi-Agent Security: Towards Secure Systems of Interacting AI Agents » — arXiv:2505.02077 — mai 2025 — https://arxiv.org/html/2505.02077 — accédé le 2026-05-05

NCCoE / NIST — « Accelerating the Adoption of Software and AI Agent Identity and Authorization » (Concept Paper) — 5 février 2026 — https://www.nccoe.nist.gov/sites/default/files/2026-02/accelerating-the-adoption-of-software-and-ai-agent-identity-and-authorization-concept-paper.pdf — accédé le 2026-05-05

Microsoft — « Announcing Microsoft Entra Agent ID: Secure and manage your AI agents » — Microsoft Entra Blog — 2025-2026 — https://techcommunity.microsoft.com/blog/microsoft-entra-blog/announcing-microsoft-entra-agent-id-secure-and-manage-your-ai-agents/3827392 — accédé le 2026-05-05

AWS — « Introducing Amazon Bedrock AgentCore Identity: Securing agentic AI at scale » — AWS Security Blog — 2025 — https://aws.amazon.com/blogs/machine-learning/introducing-amazon-bedrock-agentcore-identity-securing-agentic-ai-at-scale/ — accédé le 2026-05-05

IETF — « The OAuth 2.1 Authorization Framework » (draft-ietf-oauth-v2-1) — IETF — 2025-2026 — https://datatracker.ietf.org/doc/draft-ietf-oauth-v2-1/ — accédé le 2026-05-05

IETF — « AAuth: Authorization for Agentic AI Clients » (draft-patwhite-aauth-00) — IETF — 2025-2026 — https://www.ietf.org/archive/id/draft-patwhite-aauth-00.html — accédé le 2026-05-05

IETF — « Agentic JWT: OAuth 2.0 JWT Authorization for Agentic Systems » (draft-goswami-agentic-jwt-00) — IETF — 2025-2026 — https://www.ietf.org/archive/id/draft-goswami-agentic-jwt-00.html — accédé le 2026-05-05

IETF — « OAuth 2.0 Token Exchange » (RFC 8693) — IETF — janvier 2020 — https://datatracker.ietf.org/doc/html/rfc8693 — accédé le 2026-05-05

Maxim AI — « Top 5 AI Guardrails Platforms for Responsible Enterprise AI in 2026 » — Maxim AI — 2026 — https://www.getmaxim.ai/articles/top-5-ai-guardrails-platforms-for-responsible-enterprise-ai-in-2026/ — accédé le 2026-05-05

AI Security in Practice — « Guardrails Engineering: Bedrock Guardrails vs NeMo Guardrails vs Lakera Guard » — 2026 — https://www.aisecurityinpractice.com/defend-and-harden/guardrails-engineering/ — accédé le 2026-05-05

Northflank — « Top AI sandbox platforms for code execution in 2026 » — 2026 — https://northflank.com/blog/top-ai-sandbox-platforms-for-code-execution — accédé le 2026-05-05

Singh, M. — « AI Agent Sandboxing Guide: Firecracker, gVisor, runtimes » — Substack — 2026 — https://manveerc.substack.com/p/ai-agent-sandboxing-guide — accédé le 2026-05-05

Palo Alto Networks Unit 42 — « New Prompt Injection Attack Vectors Through MCP Sampling » — 2025-2026 — https://unit42.paloaltonetworks.com/model-context-protocol-attack-vectors/ — accédé le 2026-05-05

Palo Alto Networks Unit 42 — « Fooling AI Agents: Web-Based Indirect Prompt Injection Observed in the Wild » — 3 mars 2026 — https://unit42.paloaltonetworks.com/ai-agent-prompt-injection/ — accédé le 2026-05-05

Cloud Security Alliance — « The Attack and Defense Landscape of Agentic AI: A Comprehensive Survey » — arXiv:2603.11088 — mars 2026 — https://arxiv.org/html/2603.11088v1 — accédé le 2026-05-05

Chapitre 10 — Croître sans verrouillage

Partie 4 — Confiance, sécurité et durabilité Chapitre 10 · Scaling Without Lock-In · ~5 200 mots · lecture ≈ 20 min

La conclusion de ce chapitre tient en une phrase : l'architecture qui évite le *lock-in* agentique ne choisit pas entre plateforme propriétaire et *stack* ouvert — elle isole la logique métier dans une couche d'abstraction (MCP + A2A) qui lui permet de cohabiter avec n'importe quel hyperscaleur sans en devenir tributaire. Les hyperscaleurs l'ont compris : en mai 2026, AWS Bedrock AgentCore, Azure AI Foundry, Google Gemini Enterprise Agent Platform, IBM watsonx Orchestrate et Salesforce Agent Fabric adoptent tous MCP et A2A comme couches standards. La convergence transforme ces protocoles d'option technique en infrastructure de fait. Pour l'architecte d'entreprise, cela modifie le problème : la question n'est plus « faut-il utiliser les protocoles ouverts ? » mais « dans quel ordre instrumenter la pile pour que le choix de plateforme reste réversible sans bloquer la livraison ? »

Ce chapitre s'appuie sur les fondations posées au Ch. 5 (MCP, A2A, AAIF — primitives et gouvernance) et au Ch. 7 (cycle de vie d'un agent comme tuple versionnable). Il ne duplique pas ces fondations. Il les applique au problème de la portabilité à l'échelle, en articulant les décisions d'architecture qui déterminent si une organisation se retrouve captive dans 24 mois ou libre de changer de fournisseur sans réécriture.

10.1 — Lock-in agentique : nature et coût

Le terme *lock-in* recouvre trois phénomènes distincts que la majorité des équipes confondent, ce qui produit des mitigations ciblées sur la mauvaise couche.

Lock-in de plateforme : l'agent dépend du transport, de l'identité et du réseau d'un hyperscaleur unique. Les outils appellent AWS SDK directement, la mémoire est stockée dans DynamoDB sans couche d'abstraction, l'observabilité repose exclusivement sur CloudWatch, l'authentification passe par IAM AWS. Coût de migration documenté : six chiffres par workload unique, 18 à 36 mois pour une plateforme partagée (Kai Waehner, avril 2026). Irréversibilité principale : les données de production et les contrats de volume rendent le départ coûteux indépendamment de la qualité technique du travail de migration.

Lock-in de *framework* : la logique d'orchestration est écrite dans les primitives natives d'un SDK — LangGraph (`StateGraph`, `ToolNode`), CrewAI (`Crew`, `Task`, `Agent`), AutoGen (`AssistantAgent`, `ConversableAgent`) — sans wrapper d'isolation. Ce lock-in est plus faible que le lock-in de plateforme : migrer de LangGraph vers CrewAI est une réécriture de 3 à 12 mois selon la complexité des graphes d'état, pas de 36 mois. Mais il est invisible jusqu'au moment où l'organisation décide d'en sortir.

Lock-in de modèle : l'agent n'invoque qu'un seul fournisseur de LLM (*grand modèle de langage*) — OpenAI-only, Claude-only, Gemini-only — sans couche d'abstraction entre la logique applicative et l'API du fournisseur. Coût de migration le plus faible des trois si une gateway (*LiteLLM*, *OpenRouter*) est intégrée : jours à semaines. Sans gateway, le remplacement impose de modifier tous les points d'appel dans le code.

Lock-in de mémoire : les données d'entraînement *fine-tuned* spécifiques au fournisseur, les embeddings stockés dans une base vectorielle propriétaire, et les schémas de mémoire épisodique couplés à un SDK particulier créent un quatrième vecteur souvent ignoré. Un agent Bedrock *fine-tuned* sur données propriétaires AWS est verrouillé à Bedrock non par contractualisation mais par dépendance de données — même si le code source est entièrement portable.

Type de lock-in	Coût de migration	Délai moyen	Irréversibilité principale	Priorité de mitigation
Plateforme (transport, identité, réseau)	6-7 chiffres (workload unique)	18-36 mois	Données de production + contrats volume	Critique
Mémoire (embeddings, fine-tuning)	5-6 chiffres	6-18 mois	Qualité modèle spécifique au fournisseur	Élevée
Framework (SDK, primitives)	4-5 chiffres	3-12 mois	Complexité graphes d'état	Modérée
Modèle (API fournisseur LLM)	3-4 chiffres	Jours-semaines	Aucune, si gateway intégrée	Faible

La matrice de priorité est directe : commencer par isoler la couche plateforme, puis protéger la mémoire, puis abstraire le *framework*, et enfin ajouter une gateway modèle. Les organisations qui inversent l'ordre — et elles sont nombreuses — investissent en abstraction LLM (facile, visible) pendant que le vrai risque s'accumule au niveau plateforme (difficile, invisible).

10.2 — Plateformes propriétaires : état de convergence (mai 2026)

La cartographie ci-dessous documente l'état des cinq plateformes majeures au 2026-05-05. Elle distingue cinq dimensions : support MCP natif, support A2A natif, présence d'un *backbone* open, structure tarifaire, et dimension principale de lock-in résiduel.

Plateforme	MCP natif	A2A natif	*Backbone* ouvert	Structure tarifaire	Lock-in résiduel principal
AWS Bedrock AgentCore	Via serveurs tiers (pas natif)	Oui — AgentCore Runtime	LangGraph, AutoGen supportés	Compute + token + appels API	Transport (IAM, CloudWatch, réseau VPC)
Azure AI Foundry	Oui — Agent Service	Oui — natif	LangGraph, AutoGen, Semantic Kernel	Compute (Azure) + token	Identité (Entra ID, subscriptions)
Google Gemini Enterprise Agent Platform	Oui — services Google (BigQuery, Maps, GKE)	Oui — natif et first-mover	ADK (*Agent Development Kit*) open	Compute (GCP) + token	Données (BigQuery, Cloud Storage, GCP projects)
IBM watsonx Orchestrate	Partiel — *à vérifier*	Oui — Agent Catalog	LangFlow, LangGraph, BeeAI Framework	Licence watsonx + compute	Orchestrateur central propriétaire
Salesforce Agent Fabric	Partiel (Agentforce natif)	Oui — Agent Broker (GA juin 2026 *à vérifier*)	Découverte agents Bedrock, Foundry, GoDaddy	Licences Salesforce + usage	Gouvernance complète sur Agentforce uniquement

Trois observations critiques ressortent de cette comparaison.

Première observation. La convergence MCP + A2A est réelle mais asymétrique. Google est le *first-mover* A2A natif — A2A est intégré dans l'ADK depuis le lancement, et 150 organisations l'utilisent en production (mai 2026, communiqué PRNewswire). Microsoft a intégré A2A dans Azure AI Foundry et Copilot Studio. AWS supporte A2A dans AgentCore Runtime mais ne l'expose pas nativement au niveau MCP — l'intégration MCP passe par des serveurs tiers. IBM supporte A2A dans l'Agent Catalog mais le support MCP natif reste *à vérifier* en source primaire.

Deuxième observation. Le lock-in résiduel n'est plus au niveau du protocole — il est au niveau des plans de données et d'identité. Azure est verrouillé par Entra ID et les subscriptions, pas par le protocole A2A. AWS est verrouillé par IAM et CloudWatch, pas par LangGraph. Google est verrouillé par les projets GCP et BigQuery, pas par l'ADK. Cette distinction est opérationnellement importante : instrumenter MCP + A2A correctement permet de changer de fournisseur LLM ou de *framework* d'orchestration sans réécriture, mais ne libère pas des plans de données et d'identité qui exigent un travail architectural distinct.

Troisième observation — limitation Salesforce Agent Fabric. L'architecture Agent Fabric est conçue pour la découverte multi-vendor : un Agent Broker central peut localiser et déléguer à des agents hébergés sur Bedrock, Foundry ou des fournisseurs tiers. La limitation architecturale est que le *enforcement* complet de la gouvernance — politiques de conformité, audit trails complets, kill switches — reste disponible uniquement pour les agents Agentforce natifs. Pour les agents tiers intégrés via Agent Broker, la gouvernance est partielle et dépend de la qualité de l'implémentation A2A côté fournisseur. Cette limitation est confirmée par l'analyse Futurum Group (2026) et constitue un risque pour les organisations qui souhaiteraient utiliser Agent Fabric comme plan de contrôle unifié réellement multi-vendor.

10.3 — Open agentic stack *: composantes et AAIF*

La *stack* ouverte d'agents en 2026 n'est pas un framework unique — c'est un assemblage de couches indépendantes, chacune substituable sans perturber les autres. L'AAIF (*Agentic AI Foundation*) sous la Linux Foundation fournit la gouvernance des deux couches de protocoles fondamentales ; les *frameworks* d'orchestration constituent la couche applicative au-dessus.

```
graph TD
    subgraph Couche_Application["Couche application (orchestration)"]
        LG["LangGraph\n(stateful graphs)"]
        CA["CrewAI\n(role-based teams)"]
        AG["AutoGen / MAF 1.0\n(multi-agent)"]
        ADK["Google ADK\n(open-source)"]
        CSDK["Anthropic Agent SDK\n(Claude-native)"]
    end

    subgraph Couche_Abstraction["Couche abstraction (protocoles AAIF)"]
        MCP["MCP — Model Context Protocol\n(tools, resources, prompts, sampling)"]
        A2A["A2A — Agent-to-Agent Protocol\n(Agent Card, Task, Message, Artifact)"]
    end

    subgraph Couche_Gateway["Couche gateway (routage multi-modèle)"]
        LiteLLM["LiteLLM\n(self-hosted, 100+ modèles)"]
        OR["OpenRouter\n(SaaS, catalogue public)"]
        BMR["Bedrock Model Router\n(cross-region AWS)"]
        AMR["Azure Model Router\n(Azure AI Foundry)"]
    end

    subgraph Couche_Plateforme["Couche plateforme (hyperscaleur)"]
        AWS["AWS Bedrock AgentCore"]
        AZU["Azure AI Foundry"]
        GCP["Gemini Enterprise Agent Platform"]
        IBM["watsonx Orchestrate"]
    end

    Couche_Application --> Couche_Abstraction
    Couche_Abstraction --> Couche_Gateway
    Couche_Gateway --> Couche_Plateforme
```

LangGraph (Apache 2.0) est le choix dominant pour les agents à état persistant complexe en 2026 : les graphes d'état (*StateGraph*) permettent de modéliser des flux de décision non linéaires, avec branchement conditionnel, cycles

et *interrupts* pour le HITL (*human-in-the-loop*). Intégré nativement dans LangSmith pour l'observabilité. Risque de *framework lock-in* : les graphes LangGraph ne se compilent pas vers d'autres *frameworks* — une migration vers CrewAI ou AutoGen implique une réécriture partielle.

CrewAI (MIT) privilégie la modélisation par rôles : chaque agent a un rôle, un but et des outils définis explicitement. Approche plus déclarative que LangGraph, moins flexible pour les flux complexes mais plus lisible pour les équipes non spécialisées en ingénierie LLM.

MAF 1.0 (*Multi-Agent Framework*, Microsoft Research, avril 2026 — *à vérifier* date exacte de disponibilité publique) et **AutoGen** partagent une architecture de type conversation multi-agents : les agents s'envoient des messages texte dans une topologie configurable (*round-robin*, *selector*, *graph*). Le pattern *debate* (débat entre agents pour converger) documenté dans arXiv:2504.16489 est un cas d'usage AutoGen ; il présente aussi les vulnérabilités de *jailbreak by delegation* documentées au Ch. 9 §9.2.

Google ADK (*Agent Development Kit*, open-source) et **Anthropic Agent SDK** (Claude-native) sont les *frameworks* first-party de leurs éditeurs respectifs. Les deux exposent nativement MCP et A2A. Ils constituent des points d'entrée naturels si l'organisation est déjà sur GCP ou si Claude est le modèle de base — sans pour autant enfermer dans la plateforme si les couches d'abstraction sont correctement instrumentées.

La gouvernance de l'AAIF — *Governing Board* avec membres Platinum (AWS, Anthropic, Block, Bloomberg, Cloudflare, Google, Microsoft, OpenAI) et processus RFC public — garantit que MCP et A2A évoluent de façon coordonnée avec les implémentations hyperscaleur. Elle ne garantit pas la stabilité de l'API à court terme : OTel GenAI SemConv est en statut *Development* (mai 2026), et la date de release officielle de la spec A2A v1.0.0 n'est pas explicitement publiée dans les sources disponibles (*à vérifier* — voir Ch. 5 §5.4).

10.4 — Portabilité par MCP/A2A : patrons d'abstraction

L'argument de portabilité repose sur un principe simple : si la couche outil et la couche coordination inter-agents sont toutes les deux exprimées en protocoles standards (MCP et A2A), alors changer le *framework* d'orchestration ou le fournisseur LLM ne nécessite pas de réécrire les intégrations.

Trois patrons concrets traduisent ce principe en décisions d'implémentation.

Patron 1 — Serveur MCP comme unité de déploiement outil. Chaque intégration avec un système externe (base de données, API métier, service cloud) est encapsulée dans un serveur MCP indépendant — déployé comme un microservice avec son propre cycle de vie. L'agent invoque le serveur MCP sans connaître l'implémentation interne. Si le fournisseur de base de données change, seul le serveur MCP est modifié ; l'agent, le *framework* et la gateway modèle ne sont pas touchés. La contrepartie est opérationnelle : chaque serveur MCP est un processus à opérer, à instrumenter (OTel), à sécuriser (authentification OAuth 2.1 *Client Credentials* pour flux *machine-to-machine*) et à versionner — charge opérationnelle non triviale pour des équipes qui débutent avec AgentOps. Voir Ch. 7 §7.6 pour le cycle de vie du tuple agent + outils MCP.

Patron 2 — Agent Card A2A comme contrat de délégation. Tout agent qui peut recevoir des tâches d'un autre agent — y compris des agents hébergés sur une autre plateforme — publie une *Agent Card* JSON signée décrivant ses capacités, son endpoint, et ses exigences d'authentification. Ce contrat est indépendant de la plateforme qui héberge l'agent. Un agent sur Bedrock AgentCore peut déléguer à un agent sur Azure AI Foundry via A2A sans que l'un ou l'autre n'ait besoin de partager le même cloud ou le même *framework*. La production confirme ce scénario : en mai 2026, les 150 organisations A2A incluent des configurations Salesforce + Google + ServiceNow sans plateforme commune (The Next Web, mai 2026). La condition de succès : les deux agents doivent implémenter le cycle de vie de tâche A2A complet (SUBMITTED → WORKING → COMPLETED / FAILED / INPUT_REQUIRED) — une implémentation partielle côté agent délégué suffit à rompre la garantie de portabilité.

Patron 3 — Observabilité OTel indépendante de la plateforme. Le *lock-in* d'observabilité est subtil et souvent découvert trop tard : CloudWatch pour Bedrock, Azure Monitor pour Foundry, Cloud Trace pour Vertex — si l'instrumentation est couplée à ces backends propriétaires, la migration de plateforme implique aussi une migration de la pile d'observabilité, avec perte de données historiques. La mitigation est directe : instrumenter en OTel (*OpenTelemetry*) avec les conventions sémantiques GenAI (SemConv 1.40.0, statut *Development*) et router les traces vers un backend neutre (Datadog, Prometheus + Grafana, ou Elastic). Cette couche OTel est également la base des évaluations en production et des *shadow runs* documentés au Ch. 7 §7.5.

Le Ch. 5 documente les surfaces d'attaque introduites par ces deux protocoles — *tool poisoning*, injection via *sampling*, RCE *supply chain*. Ces risques ne disparaissent pas avec l'adoption de MCP + A2A ; ils se déplacent vers la couche d'abstraction. L'architecte qui instrumente ces patrons doit simultanément instrumenter les contrôles de sécurité correspondants — voir Ch. 9 §9.3.

10.5 — Stratégie multi-vendor : gateway patterns et routage

La *gateway* multi-modèle est la couche la moins discutée et la plus rapide à instrumenter. Elle opère entre la couche *framework* (LangGraph, CrewAI) et les APIs fournisseurs de LLM, exposant une interface unifiée compatible OpenAI vers le *framework* et routant dynamiquement les requêtes vers le fournisseur optimal selon des critères mesurables.

LiteLLM (open-source MIT, Python) est la référence auto-hébergée en 2026 : 100+ modèles supportés (OpenAI, Anthropic, Gemini, Bedrock, Mistral, Llama via Ollama), interface compatible OpenAI, *fallback* automatique sur panne fournisseur, suivi de budget par équipe ou par projet, *virtual keys* pour isoler les accès. Déploiement : conteneur Docker ou Kubernetes avec un fichier de configuration YAML déclarant les *models*, les *router_settings* et les *model_list*. Le coût d'opération est réel : LiteLLM est un composant critique du plan de données qui requiert haute disponibilité, monitoring et gestion de configuration.

OpenRouter est l'alternative SaaS : même catalogue, sans charge opérationnelle, avec marge commerciale sur les tokens. Pertinent pour les petites équipes ou les phases exploratoires ; à reconsidérer dès que le volume dépasse 10 M tokens/mois (*hypothèse* — seuil de rentabilité dépend du différentiel de prix par modèle).

Bedrock cross-region inference (AWS) et **Azure Model Router** (Azure AI Foundry) sont les gateway propriétaires des deux hyperscaleurs. Elles résolvent le routage par région (*cross-region failover*, résidence de données) dans le périmètre de leur cloud respectif, mais ne permettent pas le routage vers des modèles hors cloud. Utiles dans un déploiement AWS-primary ou Azure-primary qui accepte le lock-in plateforme mais veut diversifier les fournisseurs LLM *à l'intérieur* du même cloud.

```
flowchart LR
    subgraph Orchestrateur
        AG["Agent LangGraph\nCrewAI / AutoGen"]
    end

    subgraph Gateway["Gateway multi-modèle"]
        LiteLLM["LiteLLM\n(self-hosted)"]
        OR["OpenRouter\n(SaaS)"]
    end

    subgraph Fournisseurs
        OAI["OpenAI GPT-4o"]
        ANT["Anthropic Claude"]
        GEM["Google Gemini"]
        BDR["Bedrock (Llama, Mistral)"]
    end

    subgraph Routage_Critères["Critères de routage"]
        COUT["Coût instantané"]
        LAT["Latence P99 par région"]
        CONF["Conformité géographique\n(résidence données)"]
        CAP["Capacité modèle\n(fenêtre contexte, multimodal)"]
    end

    AG -->|Appel OpenAI-compatible| LiteLLM
    AG -->|Appel OpenAI-compatible| OR
    LiteLLM --> OAI
    LiteLLM --> ANT
    LiteLLM --> GEM
    LiteLLM --> BDR
```

```
    OR --> OAI
    OR --> ANT
    Routage_Critères --> LiteLLM
```

Le routage par **coût** est le cas d'usage le plus immédiat : les prix des modèles LLM varient de deux à trois ordres de grandeur entre un modèle léger (*Gemini Flash*, *Claude Haiku*) et un modèle de raisonnement lourd (*o3*, *Claude Opus*). Une tâche de classification simple dirigée vers le bon modèle réduit le coût d'inférence de 80 à 95 % sans dégradation de qualité mesurable (*probable* — cohérent avec les pratiques FinOps documentées au Ch. 2).

Le routage par **latence** est critique pour les agents *client-facing* (support, interface conversationnelle) avec une contrainte P99 inférieure à 500 ms. La gateway mesure la latence par région et par modèle en temps réel et route la requête vers le fournisseur actuellement le plus rapide sur le segment géographique concerné.

Le routage par **conformité géographique** est le plus complexe opérationnellement et le plus important réglementairement en contexte UE ou canadien. Une requête contenant des données à caractère personnel au sens du RGPD (*Règlement général sur la protection des données*) ou de la Loi 25 Québec doit être traitée par un modèle hébergé dans une région conforme. Aucun hyperscaleur n'implémente ce routage nativement via A2A à mai 2026 — l'orchestrateur client doit implémenter la politique de routage et la gateway doit pouvoir l'appliquer au niveau de la requête individuelle, pas seulement au niveau du déploiement.

10.6 — Conformité géographique et résidence des données

La résidence des données est une contrainte orthogonale à la stratégie multi-vendor, mais elle détermine les options architecturales disponibles. Ce paragraphe résume les implications pour l'architecte *agentic* — le traitement normatif complet est au Ch. 8 §8.4.

L'EU AI Act (règlement UE 2024/1689) impose des obligations de transparence, de traçabilité et de documentation technique aux systèmes à *haut risque* applicables à partir du 2 août 2026. Les obligations de résidence de données stricto sensu relèvent du RGPD, pas de l'AI Act — mais les deux se combinent : un système *agentic* qui traite des données personnelles de ressortissants UE et qui s'appuie sur un modèle hébergé hors UE doit justifier le transfert (clauses contractuelles types, adéquation, ou exception applicable). Pour les institutions financières canadiennes, OSFI E-23 (en vigueur 1er mai 2027) exige un inventaire de modèles incluant les dépendances tierces — ce qui couvre explicitement les modèles LLM de fournisseurs externes.

Trois implications architecturales directes pour la *stack* agentique.

Première implication. Le routage de conformité doit être implémenté à la couche gateway, pas à la couche *framework*. Le *framework* (LangGraph, CrewAI) n'a pas de visibilité sur la région d'exécution du modèle — il délègue à la gateway. Si la gateway ne peut pas appliquer une politique de routage par région sur une base par-requête, le *framework* ne peut pas compenser.

Deuxième implication. La mémoire de l'agent est soumise aux mêmes contraintes de résidence que les données de production. Un agent qui stocke des conversations dans une base vectorielle hébergée hors de la région réglementaire crée une obligation de transfert de données distincte de l'inférence LLM. Les bases vectorielles managées (Pinecone, Weaviate Cloud, pgvector sur RDS) ont des configurations régionales — les valider explicitement avant déploiement.

Troisième implication. Aucun hyperscaleur n'implémente le routage multi-région natif via A2A à mai 2026 (*confirmé* — aucune source primaire disponible ne documente cette fonctionnalité). Pour les organisations UE et canadiennes, cela signifie que le plan de conformité géographique repose entièrement sur l'orchestrateur client et la gateway — deux composants dont la correctitude est critique et doit être auditée régulièrement. Un changement d'hyperscaleur principal (par exemple Bedrock → Foundry) peut invalider une configuration de conformité géographique validée, si les régions disponibles ou les engagements contractuels de résidence diffèrent entre les deux plateformes. Planifier cet audit dans toute migration de plateforme.

10.7 — Recommandation architecturale : modèle en couches isolées

La recommandation principale de ce chapitre est d'adopter une architecture à trois couches dont les responsabilités et les frontières sont explicites, versionnées et auditables.

Couche 1 — Agent : logique métier, prompts système, règles de décision, cycle de vie. Cette couche est indépendante de toute plateforme et de tout *framework* si elle est exprimée en termes de primitives MCP et A2A, pas en primitives SDK propriétaires. La logique de décision (quand escalader, quel outil appeler, comment interpréter les résultats) est la valeur de l'organisation — elle ne doit pas être couplée à LangGraph ou à Bedrock.

Couche 2 — Abstraction : serveurs MCP (un par domaine d'outil), contrats A2A (*Agent Cards* signées), gateway multi-modèle (LiteLLM ou équivalent), instrumentation OTel. Cette couche est le bouclier de portabilité. Son coût opérationnel est réel : plus de composants à opérer, plus de surfaces d'attaque à sécuriser (voir Ch. 9 pour les contrôles MCP supply chain). Mais c'est le coût explicite de la portabilité.

Couche 3 — Plateforme : déploiement, identité, réseau. Le *lock-in* de plateforme est accepté à cette couche, mais il est *isolé*. L'isolation signifie que les Couches 1 et 2 ne contiennent aucune référence directe aux APIs, SDKs ou services propriétaires de la Couche 3. Si la Couche 3 change (Bedrock → Foundry), les Couches 1 et 2 ne sont pas modifiées.

Compromis principal. Cette architecture ajoute une couche d'indirection (Couche 2) qui consomme de la latence — entre 20 et 80 ms par saut de gateway selon les benchmarks LiteLLM disponibles (*à vérifier* — chiffres issus de benchmarks communautaires non contrôlés). Pour des agents *client-facing* avec une contrainte de latence stricte (P99 < 200 ms de réponse utilisateur), cette surcharge peut être inacceptable. Dans ce cas, l'alternative est de déployer la Couche 1 directement sur la Couche 3 (sans Couche 2), d'accepter le *lock-in* sur ces agents spécifiques, et de réserver l'architecture à trois couches aux agents *batch* et aux agents internes où la latence n'est pas contrainte. Cette décision doit être prise agent par agent, pas à l'échelle du portefeuille.

Alternative principale. Pour les organisations qui débutent ou dont les équipes n'ont pas la maturité AgentOps pour opérer la Couche 2, accepter le *lock-in* à court terme sur un hyperscaleur et investir dans la documentation exhaustive des couplages (quels outils, quelles APIs propriétaires, quels schémas de mémoire) pour faciliter la migration future. Cette décision de *lock-in* délibéré doit être formellement inscrite dans le registre d'architecture avec une date de réévaluation.

Condition qui renverse la recommandation. Si le portefeuille d'agents est inférieur à 5 agents, tous sur un même domaine métier, avec un seul fournisseur LLM et aucune exigence de conformité géographique stricte, la complexité opérationnelle de la Couche 2 n'est pas justifiée. La recommandation s'inverse : déployer directement sur la plateforme hyperscaleur, instrumenter a minima l'OTel pour préserver l'option de migration, et réévaluer à 12 mois quand le portefeuille atteint 5 à 10 agents.

10.8 — Timing de portabilité et transition vers le Ch. 11

La recommandation de timing est la suivante : **commencer l'instrumentation MCP + A2A maintenant, sur les nouveaux agents ; attendre 2027-2028 pour les bascules majeures sur les agents en production existants.** La justification est double.

D'un côté, la maturité des protocoles et de l'écosystème justifie l'instrumentation immédiate : MCP atteint 110 M+ téléchargements SDK par mois (MCP Dev Summit, avril 2026) ; A2A est en production dans 150 organisations ; les trois hyperscaleurs principaux adoptent formellement les deux protocoles. Le risque que MCP ou A2A soit abandonné dans les 18 prochains mois est faible (*probable* — évaluation subjective fondée sur la taille de l'écosystème AAIF et les engagements des membres Platinum, non sur une projection formelle).

De l'autre côté, la gouvernance de l'AAIF est encore en stabilisation. Les conventions sémantiques OTel GenAI sont en statut *Development* (mai 2026) — un changement de spec non rétrocompatible reste possible. La date de release officielle de A2A v1.0.0 n'est pas publiée. Les registres MCP présentent des vulnérabilités *supply chain* non résolues à ce jour (voir Ch. 9 §9.2, classe ASI04). Pour des agents en production à fort volume, une migration prématurée sur une spec non stabilisée crée un risque de régression difficile à anticiper.

Deux marqueurs de maturité suffisante pour déclencher une migration majeure : (1) OTel GenAI SemConv passe en statut *Stable* (actuellement *Development*) ; (2) au moins un hyperscaleur publie un *SLA* (accord de niveau de service) formellement fondé sur la spec A2A v1.x. Aucun des deux n'est atteint à mai 2026.

Le chapitre suivant — Ch. 11 : Redesigning Work, Not Augmenting It — opère la transition de la question technique (comment architecturer la *stack* pour que la plateforme soit substituable) à la question organisationnelle (comment restructurer les rôles et les processus pour que l'organisation soit capable d'exploiter cette flexibilité). La portabilité

technique sans agilité organisationnelle produit des *stacks* abstraites que personne ne maîtrise suffisamment pour en changer. L'inverse — des équipes agiles sur une *stack* verrouillée — produit des cycles de réarchitecture coûteux qui absorbent la valeur créée. Les deux dimensions sont nécessaires.

Pour aller plus loin

Kai Waehner — « Enterprise Agentic AI Landscape 2026: Trust, Flexibility, and Vendor Lock-In » (blog, avril 2026, https://www.kai-waehner.de/blog/2026/04/06/enterprise-agentic-ai-landscape-2026-trust-flexibility-and-vendor-lock-in/). Analyse pragmatique du TCO (*coût total de possession*) du lock-in agentique, avec chiffrage des coûts de migration par type de couplage. Point d'entrée recommandé pour un *business case* de portabilité à destination d'un comité de direction.

Futurum Group — « Salesforce Stakes Out Multi-Vendor Agent Control Plane » (2026, https://futurumgroup.com/insights/salesforce-stakes-out-multi-vendor-agent-control-plane-determinism-governance-enforcement-remains-the-test/). Analyse indépendante des limites architecturales d'Agent Fabric. Utile pour tout architecte qui évalue Salesforce comme plan de contrôle multi-vendor et doit cartographier les cas où la garantie de gouvernance ne tient pas.

LiteLLM documentation officielle — « Getting Started » (https://docs.litellm.ai/docs/). Documentation de référence pour l'implémentation d'une gateway multi-modèle auto-hébergée. La section *Router* documente les stratégies de *fallback*, de *load balancing* et de *retry* disponibles en production.

AAIF — « MCP Is Now Enterprise Infrastructure » (blog, avril 2026, https://aaif.io/blog/mcp-is-now-enterprise-infrastructure-everything-that-happened-at-mcp-dev-summit-north-america-2026/). Compte-rendu du MCP Dev Summit NYC (2-3 avril 2026) : adoption à l'échelle, cas Uber (5 000+ ingénieurs, 10 000+ services, 1 500+ agents actifs mensuels). Référence pour argumenter la maturité opérationnelle de MCP dans un contexte de déploiement à grande échelle.

a2a-protocol.org — Spec v1.0.0 (https://a2a-protocol.org/latest/specification/). Spec normative de référence pour A2A. Indispensable avant toute implémentation de *Agent Card* ou de cycle de vie de tâche A2A en production.

Références

- AWS — « Amazon Bedrock now offers OpenAI models, Codex, and Managed Agents (Limited Preview) » — AWS News Blog — avril 2026 — https://aws.amazon.com/about-aws/whats-new/2026/04/bedrock-openai-models-codex-managed-agents/ — accédée le 2026-05-05
- youngju.dev — « Azure AI Foundry Agent Service Practical Guide: Enterprise Deployment Decisions for 2026 » — 12 avril 2026 — https://www.youngju.dev/blog/ai-platform/2026-04-12-azure-ai-foundry-agent-service-practical-guide.en — accédée le 2026-05-05
- TheNextWeb — « Google Cloud Next 2026: AI agents, A2A protocol, Workspace Studio, and the full-stack bet against OpenAI and Anthropic » — mai 2026 — https://thenextweb.com/news/google-cloud-next-ai-agents-agentic-era — accédée le 2026-05-05
- IBM — « Manage all your AI agents in one place with watsonx Orchestrate » — IBM Announcement — 2026 — https://www.ibm.com/new/announcements/manage-all-your-ai-agents-in-one-place-with-watsonx-orchestrate — accédée le 2026-05-05
- Salesforce — « Salesforce Advances Agent Fabric: New Guided Determinism and Governance Controls to Scale Multi-Vendor AI Faster » — Salesforce Newsroom — 2026 — https://www.salesforce.com/news/stories/agent-fabric-control-plane-announcement/ — accédée le 2026-05-05
- RelayPlane — « LLM Gateway Comparison 2026: OpenRouter, Cloudflare, LiteLLM, and RelayPlane » — 2026 — https://relayplane.com/blog/llm-gateway-comparison-2026 — accédée le 2026-05-05
- Kai Waehner — « Enterprise Agentic AI Landscape 2026: Trust, Flexibility, and Vendor Lock-In » — 6 avril 2026 — https://www.kai-waehner.de/blog/2026/04/06/enterprise-agentic-ai-landscape-2026-trust-flexibility-and-vendor-lock-in/ — accédée le 2026-05-05
- Intuz — « Top 5 AI Agent Frameworks 2026: LangGraph, CrewAI & More » — 2026 — https://www.intuz.com/blog/top-5-ai-agent-frameworks-2025 — accédée le 2026-05-05
- Tietoevry — « Multi-Agent AI Systems with Google Vertex AI, ADK, A2A, and MCP » — 2025-2026 — https://www.tietoevry.com/en/blog/2025/07/building-multi-agents-google-ai-services/ — accédée le 2026-05-05

- Futurum Group — « Salesforce Stakes Out Multi-Vendor Agent Control Plane—Determinism, Governance, Enforcement Remains the Test » — 2026 — https://futurumgroup.com/insights/salesforce-stakes-out-multi-vendor-agent-control-plane-determinism-governance-enforcement-remains-the-test/ — accédée le 2026-05-05
- AgentGavel — « Best Open-Source AI Agents 2026 » — 2026 — https://agentgavel.com/blog/best-open-source-ai-agents-2026 — accédée le 2026-05-05
- PRNewswire / A2A Project — « A2A Protocol Surpasses 150 Organizations, Lands in Major Cloud Platforms, and Sees Enterprise Production Use in First Year » — 2026 — https://www.prnewswire.com/news-releases/a2a-protocol-surpasses-150-organizations-lands-in-major-cloud-platforms-and-sees-enterprise-production-use-in-first-year-302737641.html — accédée le 2026-05-05
- AAIF — « MCP Is Now Enterprise Infrastructure: Everything That Happened at MCP Dev Summit North America 2026 » — avril 2026 — https://aaif.io/blog/mcp-is-now-enterprise-infrastructure-everything-that-happened-at-mcp-dev-summit-north-america-2026/ — accédée le 2026-05-05

Chapitre 11 — Redessiner le travail, pas l'augmenter

Partie 5 — Piloter la transition Chapitre 11 · Redesigning Work, Not Augmenting It · ~5 500 mots · lecture ≈ 22 min

Plaquer de l'*agentic AI* sur des processus conçus pour des humains ne produit pas de transformation — cela produit du « workslop » (*probable* — terme issu de Deloitte Tech Trends 2026) : des applications agentiques qui augmentent la charge opérationnelle sous couvert d'automatisation, en ajoutant une couche d'approbations, d'exceptions non anticipées et de surveillance manuelle à un processus qui n'a pas été repensé. La conclusion est contre-intuitive mais traceable : la valeur stratégique de l'*agentic AI* est inaccessible à quiconque traite le déploiement comme un problème d'outillage plutôt que comme un problème de design organisationnel.

BCG et MIT Sloan Management Review (novembre 2025, n=2 102, 21 industries, 116 pays) ont identifié la ligne de partage empiriquement : les organisations qui redesignent leurs processus depuis les premiers principes — qui redéfinissent qui fait quoi, à quel niveau d'abstraction, sur quelle unité de décision — sont celles qui atteignent 95 % de satisfaction employés et des gains mesurables en productivité. Celles qui plaquent des agents sur des processus existants s'accumulent dans la cohorte du 40 % que Gartner prédit abandonnée avant 2027, pour des raisons qui ne sont pas techniques mais organisationnelles : valeur métier floue, résistance interne, absence de nouveaux rôles capables d'opérer la surface agentique déployée.

Ce chapitre établit les trois décisions structurelles qui séparent ces deux trajectoires. Il s'appuie sur les fondations AgentOps posées au Ch. 7, sur le modèle HITL (*human-in-the-loop*) du Ch. 8, et sur la portabilité architecturale du Ch. 10 — non pour les répéter, mais pour en tirer les implications organisationnelles. L'architecture technique parfaite sans structure organisationnelle alignée produit une *stack* agentique que personne ne maîtrise. L'inverse — des équipes restructurées sur une architecture inadéquate — produit des cycles de réarchitecture qui absorbent la valeur créée.

11.1 — Pourquoi l'augmentation seule ne suffit pas

Augmenter signifie ajouter un outil à un rôle existant. Redesigner signifie redéfinir qui fait quoi, à quel niveau d'abstraction, sur quelle unité de décision — et quels rôles n'existent plus dans leur forme actuelle. La distinction n'est pas sémantique : elle détermine le modèle d'investissement, la structure d'équipe et le type de change management requis.

Les données BCG/MIT SMR établissent la posture du terrain en 2025-2026 : 35 % des organisations déploient déjà l'*agentic AI*, 44 % planifient de le faire, mais la fraction qui redesigne effectivement est minoritaire. Deloitte State of AI 2026 (n=3 235, 24 pays) documente le fossé : seulement 34 % des organisations reimaginent leurs produits, services ou modèles d'affaires autour de l'IA — les 66 % restants cherchent à intégrer l'IA dans ce qui existe. Vingt pour cent seulement estiment leurs talents « hautement préparés » pour opérer des agents en production. Vingt et un pour cent ont une gouvernance adéquate pour les agents autonomes. Ces trois chiffres ne sont pas des indicateurs de retard — ce sont les symptômes d'une approche d'augmentation qui évite de poser la question du redesign.

La mécanique du plaquage est simple à illustrer : un processus de validation de crédit en cinq étapes humaines, augmenté d'un agent IA qui assiste à l'étape trois, reste un processus de cinq étapes avec une couche supplémentaire de coordination. Le temps de cycle peut s'améliorer marginalement ; la structure de coût reste la même ; les goulets d'étranglement se déplacent mais ne disparaissent pas. Un processus redesigné pour des agents peut passer à deux étapes humaines — définition des règles de crédit et traitement des exceptions irréversibles — et N étapes agentiques exécutées en parallèle. Le rapport de productivité n'est pas marginal : il est structurel.

BCG/MIT SMR formalise quatre tensions stratégiques que toute organisation en transition doit résoudre explicitement, sous peine de laisser la décision se faire par défaut :

Tension	Pôle « plaquage »	Pôle « redesign »	Signal de position
Scalabilité vs adaptabilité	Dupliquer ce qui fonctionne	Reconstruire pour ce qui est possible	Proportion de nouveaux processus natifs agents
Investissement vs emploi	Réduire les coûts salariaux	Élever la capacité par collaborateur	Ratio coût par résultat vs coût par poste
Supervision vs autonomie	Maintenir l'approbation humaine systématique	Confier les exceptions à l'humain uniquement	Taux d'escalade vs taux d'intervention
Plaquage vs refonte	Greffer l'IA sur les processus existants	Redesigner depuis les premiers principes	Part des processus repensés vs augmentés

La tension la plus sous-estimée est supervision vs autonomie. Le modèle HITL opérationnel défini au Ch. 8 §8.2 — *humans set rules, agents execute, exceptions escalate* — exige que l'humain soit repositionné sur l'exception, pas sur chaque action. Une organisation qui maintient l'approbation humaine systématique sur des décisions routinières a opté pour le plaquage par défaut, indépendamment de la sophistication technique de ses agents.

Le signal Gartner (octobre 2025) complète ce tableau : 55 % des leaders en chaîne d'approvisionnement anticipent que l'*agentic AI* réduira le besoin de postes d'entrée de gamme. La réponse des organisations performantes n'est pas de gérer l'attrition passive — c'est de redesigner les trajectoires de carrière autour des nouveaux rôles hybrides avant que la pression ne s'accumule. La recommandation architecturale de cette section est directe : avant tout déploiement agentique, cartographier les processus cibles sur la matrice autonomie × réversibilité × tolérance-erreur définie au Ch. 3, et identifier explicitement les étapes qui deviennent agentiques, les étapes qui restent humaines, et les étapes qui disparaissent. Cette cartographie est le document fondateur du redesign — sans elle, chaque décision d'architecture technique est prise dans le vide organisationnel.

11.2 — Trois nouveaux rôles structurants

Le marché du travail *agentic* bifurque empiriquement : −17 % de postes dans les rôles à forte automatisation, +22 % dans les postes à haute collaboration humain-IA (HBR, mai 2025 — *à vérifier sur article primaire complet*). Cette bifurcation ne se gère pas par des programmes de formation génériques — elle exige la définition délibérée des rôles qui occupent la jonction entre humains et agents, et qui n'existaient pas dans les organigrammes de 2023.

Trois rôles structurants émergent de la confluence des sources disponibles (HBR, ODSC, BCG/MIT SMR) comme nécessaires dans toute organisation dont la surface agentique dépasse cinq agents en production.

AI ops manager (contrôle et fiabilité à l'échelle)

L'*AI ops manager* assure que les agents fonctionnent de façon fiable à l'échelle — infrastructure, pipelines d'automatisation, gouvernance des périmètres de permission, budgets de *retry*, *kill switches*, mise à jour des règles métier encodées dans les agents. Ce rôle est la matérialisation humaine du plan de contrôle AgentOps documenté au Ch. 7 §7.6 : quelqu'un doit piloter les *dashboards* de dérive, décider des promotions et des rollbacks, et escalader les incidents qui dépassent les seuils automatiques.

La compétence primaire n'est pas l'ingénierie LLM — c'est la connaissance du processus métier que les agents opèrent, couplée à la capacité de lire une trace d'observabilité et d'en tirer une décision opérationnelle. L'*AI ops manager* est à l'AgentOps ce que le SRE (*Site Reliability Engineer*) est aux systèmes distribués : le profil qui transforme l'instrumentation technique en fiabilité opérationnelle.

Human-AI interaction designer (interface et flux hybrides)

Le *human-AI interaction designer* conçoit les flux de travail hybrides — pas l'interface graphique au sens classique, mais la géographie de la collaboration : à quel moment l'humain intervient, sous quelle forme l'exception lui est présentée, comment le contexte accumulé par l'agent est rendu intelligible pour une décision humaine en moins de 30 secondes. Ce rôle intègre l'UX design (*user experience*), le *prompt engineering* au niveau système, et la psychologie comportementale : concevoir pour un humain qui supervise des dizaines de flux agentiques simultanément exige de comprendre les biais cognitifs de la vigilance dégradée, pas seulement les principes de l'interface intuitive.

La distinction avec le designer UX classique est précise : le designer UX modélise le comportement de l'utilisateur ; le *human-AI interaction designer* modélise simultanément le comportement de l'utilisateur et le comportement de l'agent, et optimise leur point de contact. ODSC (2026) identifie ce rôle sous l'appellation *human-AI interaction*

specialist — la monographie retient le terme *designer* pour souligner la dimension de conception de flux, pas seulement d'interface.

***Quality steward* (évaluation continue en production)**
Le *quality steward* mesure la qualité des agents IA en production — construction et maintenance des ensembles d'évaluation, simulation d'interactions limites, identification précoce des modes d'échec, analyse des cas d'escalade pour en extraire des patterns. Ce rôle donne une chair opérationnelle aux métriques *task success*, *tool correctness* et *policy compliance* définies au Ch. 4 et instrumentées au Ch. 7 : quelqu'un doit posséder ces métriques, les interpréter dans leur contexte métier, et en tirer des décisions de promotion ou de rollback.

> **Note terminologique** : la littérature industrielle (ODSC 2026) utilise le terme *AI evaluation engineer* pour ce rôle. La monographie retient *quality steward* comme terme maison pour souligner la dimension de responsabilité continue sur la qualité — pas seulement l'ingénierie des tests, mais la gardiennage de la confiance opérationnelle dans les agents déployés.

Le tableau suivant positionne les trois rôles sur quatre dimensions opérationnelles :

Rôle	Compétence primaire	Compétence secondaire	Objet quotidien	Lien AgentOps
AI ops manager	Connaissance du processus métier + lecture de traces	Gouvernance, gestion du risque	Plan de contrôle, incidents, promotions/rollbacks	Ch. 7 §7.6 : plan de contrôle
Human-AI interaction designer	UX design + psychologie comportementale	*Prompt engineering* système, design de flux	Points d'escalade, interfaces d'exception, flux hybrides	Ch. 7 §7.2 : observabilité des gates HITL
Quality steward	Métriques d'évaluation + analyse des modes d'échec	Statistiques, connaissance du domaine	Ensembles d'évaluation, analyse escalades, décisions qualité	Ch. 7 §7.5 : évaluation en production

Ces trois rôles ne remplacent pas les rôles existants — ils s'y ajoutent ou en transforment une fraction. L'*AI ops manager* émerge souvent d'un profil opérations IT ou BPO (*Business Process Outsourcing*) reconverti. Le *quality steward* peut provenir d'un rôle d'assurance qualité ou d'analyste métier. Le *human-AI interaction designer* est le plus nouveau : il n'existe pas de filière établie à mai 2026, et les organisations pionnières le construisent par hybridation interne.

```
graph TD
    subgraph Décision["Couche décision humaine"]
        AIOPS["AI ops manager\n(contrôle et fiabilité)"]
        HAID["Human-AI interaction designer\n(interface et flux)"]
        QS["Quality steward\n(évaluation et confiance)"]
    end

    subgraph Agents["Couche agentique"]
        A1["Agent A\n(back-office)"]
        A2["Agent B\n(support tier-1)"]
        AN["Agent N…"]
    end

    subgraph Gouvernance["Couche gouvernance"]
        RACI["RACI agentique\nAnnexe D"]
        HITL["Gates HITL\nCh. 8"]
        OBS["Observabilité\nCh. 7"]
    end
```

```
    AIOPS -->|"pilote plan de contrôle"| Agents
    HAID -->|"conçoit les points de contact"| Agents
    QS -->|"évalue et certifie"| Agents
    Agents --> Gouvernance
    Gouvernance --> AIOPS
    Gouvernance --> QS
```

11.3 — Hybrid workflows : design depuis les premiers principes

Un flux de travail hybride performant n'est ni un processus humain avec IA ajoutée, ni un processus entièrement automatisé avec supervision humaine décorative. C'est un système redesigné depuis les premiers principes autour d'une ligne de partage explicite : ce que l'agent fait mieux que l'humain (volume, cohérence, parallélisme, mémoire de règles complexes), et ce que l'humain fait mieux que l'agent (jugement contextuel sur des situations inédites, décisions irréversibles à fort impact, relations à enjeu élevé).

Google Cloud (2026) nomme le modèle le plus courant en production la « ligne d'assemblage numérique » (*digital assembly line*) : un flux humain-guidé orchestrant des agents multiples bout en bout, où l'humain définit les paramètres en entrée et approuve le résultat en sortie, tandis que les agents exécutent toutes les étapes intermédiaires. Ce modèle est séduisant mais contient deux erreurs symétriques de design.

Erreur par excès de supervision : laisser l'humain approuver chaque étape intermédiaire d'un flux agentique — parce que la gouvernance est mal définie, parce que l'organisation n'a pas confiance dans les agents, ou parce que les escalades ont été mal calibrées — transforme le flux hybride en goulot d'étranglement humain avec une couche d'IA qui ne sert à rien. La productivité diminue par rapport au processus entièrement humain, car l'humain perd le fil du raisonnement à chaque intervention. C'est le workslop dans sa forme la plus documentée.

Erreur par insuffisance de supervision : laisser l'agent décider sans point de contrôle humain sur les exceptions irréversibles produit le risque opérationnel documenté au Ch. 8 — l'incident Replit (suppression de 1 206 enregistrements de production malgré instruction de gel) en est l'illustration la plus précise disponible. L'erreur n'est pas technique : c'est une erreur de design de flux qui n'a pas identifié quelle classe d'action requiert un gate humain.

Les quatre principes de design qui permettent d'éviter ces deux erreurs sont directement dérivables des fondations posées aux chapitres précédents :

Principe 1 — Séparation des décisions par réversibilité. Les décisions réversibles (extraction de données, classification, rédaction de brouillons, résumé) sont déléguées à l'agent sans approbation intermédiaire. Les décisions irréversibles à fort impact (envoi d'une communication externe, modification d'un enregistrement réglementaire, exécution d'un paiement supérieur à un seuil défini) requièrent un gate humain explicite. Ce seuil est une décision métier, pas technique, et doit être documenté dans le RACI agentique (Annexe D). La matrice de référence est au Ch. 8 §8.1 (niveaux N1-N4 d'autonomie).

Principe 2 — Élévation du niveau d'abstraction humain. Dans un flux hybride bien designé, l'humain opère à un niveau d'abstraction supérieur à celui des agents : il définit les règles, fixe les critères d'exception, approuve les exceptions — il n'exécute pas les étapes. Cela exige de redesigner les critères de performance de l'humain en conséquence : un analyste crédit dans ce modèle est évalué sur la qualité de ses décisions d'exception et sur la pertinence des règles qu'il définit, pas sur le volume de dossiers traités manuellement.

Principe 3 — Boucle de feedback bidirectionnelle. L'agent remonte les exceptions inhabituelles à l'humain ; l'humain met à jour les règles sur la base des patterns d'exceptions. Ce mécanisme transforme chaque interaction d'exception en signal d'apprentissage organisationnel. Sans cette boucle, les agents accumulent les exceptions sans amélioration, et les humains perdent progressivement la compréhension du flux réel. L'instrumentation de cette boucle est une responsabilité du *quality steward* (§11.2) et doit être visible dans l'observabilité AgentOps (Ch. 7 §7.2).

Principe 4 — Instrumentation dès le design. Aucun flux hybride ne peut être piloté sans observabilité des *spans* agentiques et des *gates* humains. Ajouter l'instrumentation après coup produit des lacunes dans les traces — précisément là où les décisions critiques ont eu lieu. La règle opérationnelle : si une étape du flux produit un effet sur l'environnement (écriture, envoi, modification), elle doit générer un *span* OTel avec le contexte de décision. Ce

n'est pas négociable dans un environnement réglementaire soumis à l'EU AI Act (en vigueur pour les systèmes à haut risque au 2 août 2026) ou à la Loi 25 Québec.

Le cas Salesforce Agentforce (HBR, février 2026) illustre ce que ces principes produisent en pratique. Avant le redesign, le flux de support client était linéaire et humain : chaque ticket était assigné à un agent humain qui en gérait l'intégralité. Après redesign, l'agent IA traite 74 % des cas de façon autonome en appliquant des règles définies par l'*AI ops manager* et validées par le *quality steward*. Les 26 % restants — cas complexes, réclamations à risque réglementaire, situations émotionnelles — sont escaladés à des agents humains qui opèrent maintenant exclusivement sur les cas à valeur ajoutée. Le SDR (*Sales Development Representative*) passe de 12-15 prospects traités par jour à 350+ rendez-vous qualifiés par semaine — non parce que le même humain travaille davantage, mais parce que l'humain s'est repositionné sur la décision stratégique pendant que l'agent gère le volume.

L'anti-patron symétrique est Klarna. Entre 2022 et 2024, Klarna a réduit ses effectifs de plus de 4 000 postes en automatisant le service client avec des agents IA, passant de 7 000+ employés à environ 3 000. L'annonce officielle de 700 suppressions de postes citée au Ch. 2 §2.4 représentait une tranche de cette trajectoire plus longue. Le résultat documenté en 2025 (Fortune, 9 mai 2025) : baisse mesurable de la satisfaction client, réembauche partielle en mode hybride à partir de 2025, et la déclaration du CEO Sebastian Siemiatkowski — « We went too far ». Le problème n'était pas la technologie agentique : c'était l'absence de redesign des flux de traitement des cas complexes et l'absence de définition explicite des cas où l'humain reste irremplaçable.

11.4 — Change management : transformer sans résistance structurelle

Le *change management* agentique échoue lorsqu'il est traité comme un programme de formation. Il réussit lorsqu'il est traité comme une reconception des incitations, des structures de pouvoir et des critères de performance. Cette distinction est rarement formulée explicitement dans les feuilles de route de transformation — et sa conséquence est que 29 % des employés admettent avoir saboté activement les déploiements d'IA dans leur organisation (Writer / Workplace Intelligence, avril 2026).

Ce chiffre n'est pas un signal de mauvaise foi généralisée — c'est un signal de conception défaillante. Quand les employés ne comprennent pas l'impact de l'IA agentique sur leur poste, quand aucune trajectoire de rôle n'est communiquée, et quand leurs critères de performance restent inchangés pendant que les outils changent, la résistance est une réponse rationnelle. Les organisations qui investissent massivement dans la technologie et zéro dans la restructuration des incitations produisent exactement ce résultat.

Writer et Workplace Intelligence (2026) documentent par ailleurs que 92 % des C-suite cultivent délibérément une nouvelle classe d'« élites IA » — les super-utilisateurs qui s'approprient les agents et les exploitent pour atteindre des niveaux de productivité 5× supérieurs aux non-utilisateurs. Le risque organisationnel de cette stratégie non accompagnée d'un programme de transition : elle polarise l'organisation entre les 8 % d'élites et les 92 % restants, accélère la résistance plutôt que de la réduire, et produit une dette de compétences structurelle qui ralentit le déploiement à l'échelle.

Deloitte State of AI 2026 documente la réponse des organisations qui gèrent activement cette transition : 53 % éduquent l'ensemble de leur population (formation généralisée à l'IA, literacy de base, utilisation des outils) et 48 % conçoivent des stratégies formelles d'*upskilling* et de *reskilling* ciblées sur les rôles les plus impactés. Ces deux approches ne sont pas mutuellement exclusives — elles opèrent à des niveaux différents : la formation généralisée adresse la peur et la résistance ; la stratégie d'*upskilling* ciblée adresse la reconversion opérationnelle.

Les trois raisons d'échec du *change management* agentique sont identifiables et évitables :

Raison 1 — Formation sans redesign des incitations. Former à l'utilisation des agents IA sans modifier les critères de performance produit des employés formés qui ne changent pas leur comportement, parce que leur évaluation reste fondée sur des métriques de volume qui ne valorisent pas l'utilisation des agents. Un analyste financier évalué sur le nombre de rapports produits manuellement n'a aucun intérêt à déléguer la production à un agent.

Raison 2 — Déploiement sans communication sur la trajectoire des rôles. L'absence de visibilité sur l'impact de l'IA sur un poste spécifique — quel contenu du rôle disparaît, quel contenu se transforme, quelle nouvelle compétence devient critique — alimente l'anxiété et la résistance. La communication proactive sur la trajectoire n'est pas un exercice de relations publiques internes : c'est une condition opérationnelle pour obtenir l'adhésion nécessaire à un déploiement à l'échelle.

Raison 3 — Pilotes sans plan de transition. Déployer un agent en pilote sur un sous-ensemble du flux de travail sans définir ce que les humains font après que l'agent prend les tâches automatisables produit deux issues également mauvaises : soit les humains restent à proximité du processus sans charge de travail réelle (workslop humain), soit le pilote réussit et personne ne sait comment passer à l'échelle sans éliminer des postes sans plan de reconversion.

Un programme de *change management* efficace pour un déploiement *agentic* contient au minimum quatre composantes :

Cartographie des rôles impactés par type. Pour chaque rôle concerné par le déploiement, identifier explicitement : quel contenu de tâche disparaît (automatisé par l'agent), quel contenu se transforme (l'humain opère à un niveau d'abstraction supérieur), quel contenu reste intact (expertise de domaine, relation, jugement irréversible), et quel contenu nouveau émerge (*AI ops manager*, *quality steward*, *human-AI interaction designer*). Cette cartographie est l'entrée du plan de *reskilling* — sans elle, le plan est générique et inefficace.

Communication sur la trajectoire avec délai et critères. Pas une annonce, mais un dispositif continu : à quelle vitesse l'automatisation progressera, selon quels critères les décisions de transition de rôle seront prises, et quel est le mécanisme de recours pour les employés dont le rôle est substantiellement modifié. La transparence sur les critères de décision — même quand ils impliquent des réductions d'effectif — produit moins de résistance que l'opacité.

Upskilling ciblé sur les nouveaux rôles. Les données Deloitte (48 % des organisations conçoivent des stratégies formelles d'*upskilling*) indiquent que les organisations les plus avancées ne traitent pas l'*upskilling* comme une obligation sociale — elles le traitent comme un investissement en capital humain qui conditionne la vitesse du déploiement à l'échelle. L'*upskilling* ciblé vers les rôles de *quality steward* et d'*AI ops manager* a un retour sur investissement direct et mesurable : chaque *quality steward* opérationnel permet de maintenir X agents en production avec un niveau de risque maîtrisé.

Structures de pilotage claires. Le RACI agentique (Annexe D) n'est pas seulement un artefact de gouvernance technique — c'est l'instrument qui clarifie les responsabilités pendant et après la transition, et qui transforme les décisions de redesign en engagements nommés. Sans RACI, la question « qui décide de rollback un agent en production » n'a pas de réponse claire, et les crises produisent des paralysies.

Recommandation : redesign délibéré vs augmentation incrémentale

La recommandation de ce chapitre est de privilégier le redesign délibéré sur l'augmentation incrémentale pour les processus à fort potentiel agentique, en acceptant une période de transition plus longue en échange d'un gain structurel plus élevé.

Compromis principal. Le redesign délibéré exige un investissement amont plus élevé en cartographie, en design de flux, en définition des nouveaux rôles et en *change management* — sur un horizon de 6 à 18 mois selon la complexité du processus. L'augmentation incrémentale livre un bénéfice visible plus tôt (réduction de 20 à 40 % du temps d'exécution d'une étape spécifique) mais plafonne rapidement et accumule une dette organisationnelle sous forme de processus hybrides non conçus qui nécessiteront un redesign ultérieur de toute façon.

Alternative crédible. Pour les processus à faible réversibilité de l'action (faible risque d'effets irréversibles) et à complexité faible (flux linéaires, règles de décision explicites), l'augmentation incrémentale est une stratégie valide à court terme — elle permet d'accumuler de l'expérience opérationnelle avec les agents, de former les équipes dans le contexte réel, et de construire les données d'évaluation nécessaires à un redesign plus ambitieux en phase deux. Le Ch. 3 fournit la matrice de qualification.

Condition qui renverse la recommandation. Si l'organisation n'a pas encore de *quality steward* ni d'*AI ops manager* opérationnels, le redesign délibéré à grande échelle est prématuré : les agents redésignés pour opérer de façon autonome sur des flux critiques sans la capacité humaine de les surveiller et de les corriger créent plus de risque qu'ils n'en réduisent. Dans ce cas, l'augmentation incrémentale sur des périmètres bornés, combinée à la formation prioritaire des rôles de jonction, est la séquence correcte.

11.5 — Cas concret : redesign d'une fonction back-office (clôture financière)

La fonction de clôture financière mensuelle — réconciliation, rapprochement des comptes, consolidation, production des états financiers — est l'un des cas back-office les plus documentés de redesign agentique réussi, parce qu'elle combine des volumes de données traçables, des règles de décision explicites, et des exceptions bien définies qui exigent un jugement humain.

État de départ (processus augmenté). Un processus de clôture augmenté typique ajoute un agent d'extraction de données qui automatise la collecte depuis les ERP, et un agent de réconciliation qui signale les écarts. Le contrôleur financier reste responsable de l'ensemble de la vérification, travaille sur les signaux produits par les agents, et valide chaque poste. Résultat observé : réduction du temps de collecte de 40 %, mais temps global de clôture réduit de 15 % seulement — parce que le goulot humain de vérification n'a pas changé de nature.

État redesigné. Le processus redesigné repositionne le contrôleur financier sur trois rôles exclusifs : définition des règles de réconciliation (seuils de tolérance, règles d'exception par catégorie de compte, critères de déclenchement d'une investigation), validation des exceptions non résolues par les agents, et signature des états financiers consolidés. Les agents exécutent l'intégralité de la chaîne de traitement standard — collecte, rapprochement, validation croisée, réconciliation inter-entités, consolidation préliminaire — et escaladent uniquement les exceptions qui sortent des règles définies.

```
flowchart LR
    subgraph Humains["Couche humaine (contrôleur financier)"]
        R1["Définit les règles\nde réconciliation"]
        R3["Traite les exceptions\nnon résolues"]
        R4["Signe les états\nfinanciers consolidés"]
    end

    subgraph Agents["Couche agentique"]
        A1["Agent collecte\n(ERP, sources externes)"]
        A2["Agent réconciliation\n(rapprochement J/J)"]
        A3["Agent consolidation\n(inter-entités)"]
        A4["Agent validation\n(contrôles de cohérence)"]
    end

    subgraph Escalades["Gates HITL"]
        E1["Exception\n> seuil défini"]
        E2["Incohérence\nnon résoluble"]
    end

    R1 -->|"paramètres et règles"| A1
    A1 --> A2
    A2 --> A3
    A3 --> A4
    A4 -->|"rapport consolidé"| R4
    A2 --> E1
    A3 --> E2
    E1 --> R3
    E2 --> R3
    R3 -->|"résolution + maj règles"| A2
```

Étapes opérationnelles du redesign. La transition du premier état au second suit six étapes, dont les deux premières sont organisationnelles, non techniques :

1. **Cartographie des exceptions.** Analyser 12 mois de données de clôture pour identifier la distribution des exceptions : quelles catégories, à quelle fréquence, avec quel délai de résolution. Cette analyse détermine le volume d'escalade que l'humain devra traiter après redesign et permet de calibrer la charge résiduelle.
2. **Définition des règles d'escalade.** Traduire les heuristiques implicites du contrôleur financier en règles explicites : quels écarts sont automatiquement acceptables (< X % sur les catégories Y), lesquels déclenchent une vérification automatisée supplémentaire, lesquels exigent un gate humain. C'est le travail le plus critique et le plus sous-estimé du redesign — il requiert de rendre explicite ce qui était tacite.
3. **Design du flux hybride.** Spécifier l'architecture des agents (collecte, réconciliation, consolidation, validation) avec leurs contrats d'escalade, leurs seuils d'autonomie par étape, et leurs exigences d'observabilité. Chaque agent produit un *span* OTel pour chaque décision de classification d'exception.

4. **Pilote sur périmètre borné.** Déployer sur une entité juridique ou une catégorie de comptes, avec le contrôleur financier en supervision active, pour valider les règles d'escalade et identifier les cas non anticipés. Cette étape est non optionnelle : les données de clôture de 12 mois n'anticipent jamais 100 % des situations réelles.
5. **Calibration des escalades.** Sur la base du pilote, ajuster les seuils. L'objectif est un taux d'escalade de 5 à 15 % des transactions (*hypothèse* — calibrée sur les cas publiés, à ajuster selon le contexte) : un taux plus élevé indique des règles trop conservatrices (workslop), un taux plus faible indique des règles potentiellement trop larges (risque de sous-escalade sur des exceptions réglementaires).
6. **Extension et boucle d'amélioration.** Déployer à l'ensemble des entités avec le *quality steward* en responsabilité de la qualité des agents, l'*AI ops manager* en responsabilité de la fiabilité opérationnelle, et le contrôleur financier en responsabilité des décisions d'exception et des règles. La boucle de feedback — chaque exception résolue enrichit les règles pour le mois suivant — est le mécanisme d'amélioration continue du système hybride.

Le gain documenté dans les cas publiés les plus proches de ce modèle (*à vérifier sur données propres* — convergence de plusieurs rapports industrie sans source primaire unique) : réduction du temps de clôture de 30 à 60 %, avec réduction du taux d'erreur de réconciliation de l'ordre de 40 à 70 %. Ces chiffres ne sont pas des cibles — ce sont des ordres de grandeur qui dépendent fortement de la qualité initiale du processus et de la maturité des données sources.

11.6 — Transition vers le Ch. 12

La ligne de partage entre les organisations qui réussissent cette transformation et celles qui la ratent tient à moins de décisions qu'on ne le croit — mais ces décisions sont rarement celles que les équipes technologiques identifient comme critiques. Elles sont organisationnelles : a-t-on défini qui opère les agents, à quel niveau d'abstraction l'humain intervient, et comment les trajectoires de rôle sont communiquées avant que la pression de l'automatisation ne se manifeste ?

Les organisations qui réussissent définissent le problème comme une question de design organisationnel *avant* de le traiter comme une question technologique. Celles documentées au Ch. 12 ont fait l'inverse — elles ont optimisé la stack technique en laissant les questions de rôle, de processus et de change management s'accumuler jusqu'à ce qu'elles deviennent des crises. La frontière entre les deux trajectoires est identifiable en amont, avant l'investissement irréversible, à condition de poser les bonnes questions au bon moment.

La portabilité technique du Ch. 10 est une condition nécessaire mais insuffisante : une organisation agile sur une architecture portative mais sans *quality steward* ni *AI ops manager* opérationnels n'a pas la capacité d'exploiter la flexibilité de sa stack. L'Introduction posait la question de la rupture de 2026 — ce chapitre a posé les fondations organisationnelles de la réponse. La Gouvernance RACI en fournit l'artefact opérationnel immédiatement applicable.

Pour aller plus loin

BCG / MIT Sloan Management Review — « The Emerging Agentic Enterprise: How Leaders Must Navigate a New Age of AI » (novembre 2025, https://sloanreview.mit.edu/projects/the-emerging-agentic-enterprise-how-leaders-must-navigate-a-new-age-of-ai/). La source empirique la plus complète disponible à mai 2026 sur les tensions stratégiques du déploiement *agentic* à l'échelle de l'entreprise, avec n=2 102 sur 21 industries. Indispensable avant toute présentation exécutive sur la stratégie d'adoption.

HBR — « To Thrive in the AI Era, Companies Need Agent Managers » — Srinivasan & Wei (12 février 2026, https://hbr.org/2026/02/to-thrive-in-the-ai-era-companies-need-agent-managers). Le cas Salesforce Agentforce documenté avec les métriques de performance réelles. Point d'entrée recommandé pour tout directeur opérationnel qui veut comprendre ce que l'*agent manager* fait concrètement dans une organisation en production.

Deloitte — « The Agentic Reality Check: Preparing for a Silicon-Based Workforce » (Tech Trends 2026, https://www.deloitte.com/us/en/insights/topics/technology-management/tech-trends/2026/agentic-ai-strategy.html). Encadrement conceptuel de la main-d'œuvre agentique comme nouvelle catégorie — et introduction du concept de *workslop* avec ses mécanismes. Utile pour un CHRO ou un directeur de transformation qui cherche à cadrer le problème devant un comité de direction.

Google Cloud — « AI Agent Trends 2026: Five Shifts That Will Redefine Roles, Workflows, and Business Value » (https://cloud.google.com/resources/content/ai-agent-trends-2026). Taxonomie des flux de

travail hybrides (digital assembly lines) et des nouveaux rôles, avec données d'adoption des adopteurs précoces. Plus proche du terrain opérationnel que les rapports de recherche académique.

ODSC — « From Context Engineers to Chief AI Officers: Emerging AI Job Roles for 2026 » (https://odsc.medium.com/from-context-engineers-to-chief-ai-officers-emerging-ai-job-roles-for-2026-9f757603f547). Panorama des rôles émergents le plus complet à mai 2026 — à croiser avec HBR et BCG/MIT SMR pour valider les tendances. Utile pour les équipes RH qui construisent les fiches de poste des nouveaux rôles hybrides.

Références

- BCG / MIT Sloan Management Review — « The Emerging Agentic Enterprise: How Leaders Must Navigate a New Age of AI » — MIT SMR + BCG — novembre 2025 — https://sloanreview.mit.edu/projects/the-emerging-agentic-enterprise-how-leaders-must-navigate-a-new-age-of-ai/ — accédée le 2026-05-05
- Deloitte AI Institute — « The State of AI in the Enterprise 2026 » — mars 2026 — https://www.deloitte.com/us/en/what-we-do/capabilities/applied-artificial-intelligence/content/state-of-ai-in-the-enterprise.html — accédée le 2026-05-05
- Deloitte — « The Agentic Reality Check: Preparing for a Silicon-Based Workforce » — Tech Trends 2026 — https://www.deloitte.com/us/en/insights/topics/technology-management/tech-trends/2026/agentic-ai-strategy.html — accédée le 2026-05-05
- Writer / Workplace Intelligence — « Enterprise AI Adoption in 2026 » — 7 avril 2026 — https://writer.com/blog/enterprise-ai-adoption-survey-results-press-release/ — accédée le 2026-05-05
- Srinivasan, S. & Wei, V. — « To Thrive in the AI Era, Companies Need Agent Managers » — Harvard Business Review — 12 février 2026 — https://hbr.org/2026/02/to-thrive-in-the-ai-era-companies-need-agent-managers — accédée le 2026-05-05
- HBR — « Agentic AI Is Already Changing the Workforce » — Harvard Business Review — mai 2025 — https://hbr.org/2025/05/agentic-ai-is-already-changing-the-workforce — accédée le 2026-05-05
- Google Cloud — « AI Agent Trends 2026: Five Shifts That Will Redefine Roles, Workflows, and Business Value » — 2026 — https://cloud.google.com/resources/content/ai-agent-trends-2026 — accédée le 2026-05-05
- Gartner — « Gartner Says AI Revolution and Cost Pressures Are Two Forces Driving the Top Four Trends for Talent Acquisition in 2026 » — Gartner Newsroom — 7 octobre 2025 — https://www.gartner.com/en/newsroom/press-releases/2025-10-07-gartner-says-ai-revolution-and-cost-pressures-are-two-forces-driving-the-top-four-trends-for-talent-acquisition-in-2026 — accédée le 2026-05-05
- ODSC — « From Context Engineers to Chief AI Officers: Emerging AI Job Roles for 2026 » — Medium / Open Data Science — 2026 — https://odsc.medium.com/from-context-engineers-to-chief-ai-officers-emerging-ai-job-roles-for-2026-9f757603f547 — accédée le 2026-05-05
- Forrester — « Predictions 2026: AI Agents, Changing Business Models, And Workplace Culture Impact Enterprise Software » — 2026 — https://www.forrester.com/blogs/predictions-2026-ai-agents-changing-business-models-and-workplace-culture-impact-enterprise-software/ — accédée le 2026-05-05
- Stanford HAI — « The 2026 AI Index Report » — Stanford Human-Centered Artificial Intelligence — 2026 — https://hai.stanford.edu/ai-index/2026-ai-index-report — accédée le 2026-05-05
- Fortune — « Klarna Reverses AI Customer Service Replacement » — 9 mai 2025 — https://fortune.com/2025/05/09/klarna-ai-humans-return-on-investment/ — accédée le 2026-05-05

Chapitre 12 — Leçons des 60 % qui ont échoué

Partie 5 — Piloter la transition Chapitre 12 · Leçons tirées des 60 % ayant échoué : une anatomie de l'effondrement agentique · ~5 100 mots · lecture ≈ 20 min

Les programmes *agentic* ne meurent pas d'une panne soudaine. Ils dérivent, accumulent une dette organisationnelle invisible, et s'effondrent quand les coûts dépassent la valeur mesurable — souvent plusieurs mois après que les signaux étaient disponibles dans les traces de production. L'autopsie des projets abandonnés révèle une architecture causale tri-couche reproductible : technique (dérive des outils, dette mémoire, évaluations absentes), organisationnelle (gouvernance inexistante, ROI flou, scope incontrôlé), économique (coûts de *retry* non bornés, *infrastructure tax* invisible). Chaque couche amplifie les deux autres.

La question de ce chapitre est opérationnelle : quels signaux, présents dès les premières semaines de déploiement, distinguent un programme en trajectoire d'échec d'un programme en trajectoire de scale — et comment décider, à tout moment, de tuer, de pivoter ou d'investir ? La réponse suppose de distinguer d'abord les deux chiffres Gartner que la littérature du secteur traite souvent comme synonymes, puis de descendre dans les mécanismes causaux qui les alimentent.

12.1 — Deux chiffres Gartner, deux couches d'un même phénomène

Deux prédictions Gartner circulent simultanément en 2026, souvent confondues parce qu'elles semblent pointer dans la même direction.

La première (*confirmé*) — Gartner Newsroom, 26 février 2025, analyste Roxane Edjlali, enquête *data management leaders* juillet 2024 — établit que d'ici fin 2026, les organisations sans données AI-ready verront plus de 60 % de leurs projets IA échouer ou être abandonnés. Le périmètre est les projets IA en général ; la cause spécifique est l'incapacité à distinguer les exigences de données AI-ready des pratiques de *data management* traditionnelles. Ce 60 % fonde le titre de ce chapitre.

La seconde (*confirmé*) — Gartner Newsroom, 25 juin 2025, enquête sur 3 412 participants à des webinaires Gartner, janvier 2025 — établit que plus de 40 % des projets *agentic AI* seront annulés d'ici 2027. Le périmètre est spécifiquement les projets agentiques ; les causes citées sont les coûts escaladants, la valeur métier floue et les contrôles de risque insuffisants. Ce 40 % est documenté au Ch. 2.

Ces deux chiffres ne se contredisent pas — ils cartographient deux couches d'un même phénomène. Le 60 % est une cause structurelle (données non AI-ready) qui traverse tous les projets IA, agentiques inclus. Le 40 % est une prédiction globale d'abandon pour les projets agentiques, toutes causes combinées — dans laquelle la non-readiness des données est l'un des facteurs parmi d'autres. La relation est donc : 60 % (cause) ⊂ 40 % (prédiction d'abandon toutes causes), et les deux chiffres sont cohérents.

Un troisième signal complète le tableau de façon indépendante. S&P Global Market Intelligence (CIO Dive, 2025 — enquête 1 006 professionnels IT et LOB, Amérique du Nord et Europe) mesure que 42 % des organisations ont abandonné la majorité de leurs initiatives IA avant production en 2025, contre 17 % l'année précédente. Le saut annuel — de 17 % à 42 % — est le signal le plus alarmant : la mortalité des projets n'est pas stable, elle accélère. Quarante-six pour cent des POC (*proof of concept*) sont scrappés avant déploiement élargi, avec comme raisons principales le coût, la confidentialité des données et la sécurité.

```
graph TD
    A["60 % projets IA abandonnés\n(Gartner fév. 2025)\nCause : données non AI-ready"]
    B["40 % projets agentic IA annulés\n(Gartner juin 2025)\nCauses : coûts + ROI flou +
gouvernance"]
    C["42 % organisations ont stoppé\nla majorité de leurs initiatives IA\n(S&P Global, 2025)"]
    D["Cause technique :\nDrift + dette mémoire + evals absentes"]
    E["Cause organisationnelle :\nGouvernance absente + ROI flou + scope large"]
    F["Cause économique :\nRetry non borné + infrastructure tax"]
```

```
    A -->|cause transversale| B
    C -->|signal indépendant| B
    D --> B
    E --> B
    F --> B
```

Cette convergence entre trois sources d'analyste indépendantes — Gartner, S&P Global, et les données terrain Databricks (*State of AI Agents 2026*, n=20 000+) — établit que l'échec est systémique, pas accidentel. Les sections suivantes en dissèquent l'architecture causale.

12.2 — Anatomie technique : trois vecteurs de défaillance silencieuse

Les agents ne tombent pas en panne — ils dérivent. L'arXiv:2603.06847 (Shah et al., mars 2026) est à ce jour la source académique la plus rigoureuse sur le sujet : analyse qualitative de 385 fautes extraites de 13 602 issues et pull requests de 40 dépôts open-source agentiques. La taxonomie identifie cinq dimensions architecturales de fautes, 13 classes de symptômes et 12 catégories de causes racines — et 83,8 % des praticiens interrogés estiment la taxonomie représentative des défaillances rencontrées en production. Trois vecteurs concentrent la majorité des causes d'échec silencieux.

12.2.1 — Dérive des outils et *attention decay*

Le *tool drift* (*probable* — terme praticien, pas encore normalisé) désigne la réinterprétation progressive des paramètres d'outils par un agent dont la fenêtre de contexte s'accumule. Arize AI (jan. 2026), sur la base d'une analyse de traces LLM de déploiements en production, documente quatre vecteurs : la gestion non structurée de la fenêtre de contexte (« dump truck » sans hiérarchisation → erreur *Lost in the Middle*) ; le conflit entre la connaissance paramétrique du modèle et le contexte injecté, où la connaissance d'entraînement prime sur les instructions courantes ; les défaillances d'API externes à schéma évolutif (le changement de schéma d'une API Salesforce ou HubSpot produit un raisonnement dégradé plutôt qu'une erreur propre, l'agent continuant à opérer sur une interprétation incorrecte) ; et l'*attention decay*, la dilution progressive du *system prompt* par les tokens récents dans les longues conversations, qui produit l'ignorance d'instructions explicites.

L'incident Replit de juillet 2025 — suppression de 1 206 enregistrements de production malgré une instruction de gel explicite — est l'illustration la plus documentée de ce vecteur (*confirmé* — Fortune, 23 juillet 2025 ; earezki.com, mars 2026). L'agent n'a pas ignoré la contrainte par dysfonctionnement : il l'a ignorée parce que l'*attention decay* l'avait rendue opérationnellement invisible à ce stade de la conversation. La dérive d'outil, documentée au Ch. 7 §7.2 comme signal à instrumenter via des *tool spans*, devient ici une cause d'échec de programme entier quand elle n'est pas détectée.

La distinction critique entre Ch. 7 et ce chapitre : Ch. 7 décrit *comment instrumenter* pour détecter la dérive outil par outil ; ce chapitre examine ce qui arrive à l'échelle du portefeuille quand cette instrumentation est absente — et la réponse empirique est que 83,8 % des projets agentiques rencontrent ces symptômes à des degrés divers.

12.2.2 — Dette mémoire

La dette mémoire est une dégradation cumulative de la qualité de récupération mémoire à mesure que la complexité des tâches augmente. Chanl AI (2025) la quantifie sur une étude de décembre 2025 : le taux d'échecs mémoire passe de 0,67 par tâche simple à 2,33 sur tâche moyenne, à 3,67 sur tâche complexe — une amplification de 5,5× entre le bas et le haut de l'échelle de complexité. Plus révélateur encore : le taux de rappel effectif en scénarios complexes est de seulement 13,1 %, malgré une tâche en apparence complétée.

La distinction avec une panne est ici fondamentale. La tâche est marquée comme réussie, l'agent a produit une sortie, mais la décision sous-jacente a été prise sur une mémoire appauvrie qui a omis des contraintes critiques établies dans des interactions antérieures. Les évaluations de surface — tâche complétée = succès — ne voient pas cette dégradation. La dette de mémoire définie au Ch. 6 §6.5 et instrumentée via *memory spans* au Ch. 7 §7.2 se manifeste ici comme cause d'échec de programme quand ni l'instrumentation ni les évaluations longitudinales ne sont en place.

12.2.3 — Évaluations absentes ou inadéquates

Databricks (*State of AI Agents 2026*, n=20 000+) établit que les organisations utilisant des outils d'évaluation structurés déploient 6× plus d'agents en production avec succès que celles qui ne l'utilisent pas. Le multiplicateur 6× n'est pas une recommandation de bonne pratique — c'est une mesure de l'écart causal entre programmes qui scale et programmes qui échouent.

L'évaluation ponctuelle sur 3 à 5 tours est structurellement aveugle sur la dérive longitudinale. CIO.com documente que la dégradation commence typiquement après les 50 premières étapes d'un agent en session longue — précisément le régime que les évaluations courtes ne couvrent pas. L'absence de *shadow runs* et de *replay* déterministe sur *golden files* — techniques décrites au Ch. 7 §7.5 — rend impossible la distinction entre régression et variabilité naturelle. Le résultat est une série de décisions go/no-go fondées sur des données insuffisantes, où le programme dépasse ses seuils de coût avant que la dérive soit officiellement reconnue.

12.3 — Anatomie organisationnelle : gouvernance absente, ROI flou, scope incontrôlé

La cause organisationnelle d'échec la plus fréquente n'est pas le manque de talent technique — c'est l'absence de structure décisionnelle avant que les agents opèrent en production.

12.3.1 — Gouvernance absente et prolifération non contrôlée

HBR (juin 2025) documente le tableau de préparation organisationnelle avec une précision qui rend le problème quantifiable : 20 % seulement des organisations estiment leur infrastructure technologique « entièrement prête » ; 15 % disent idem pour leurs données ; 12 % pour leurs contrôles de risque et gouvernance. Seulement 21 % des entreprises ont des modèles de gouvernance matures pour les agents autonomes — et 6 % seulement font pleinement confiance aux agents pour les processus métier cœur (Fortune, décembre 2025).

L'absence de gouvernance ne produit pas un vide — elle produit une prolifération non contrôlée. Les outils *low-code/no-code* permettent à chaque équipe de déployer ses propres agents sans catalogue centralisé, sans RACI agentique, sans supervision cross-équipe. Le résultat documenté : agents redondants, fragmentés, sans observabilité partagée, sans coordination sur les périmètres de permission, chacun opérant sur ses propres copies de données sans version cohérente. McKinsey (2026) observe qu'environ un tiers des organisations atteignent le niveau 3 de maturité en gouvernance agentique — les deux tiers restants n'ont pas la capacité structurelle de piloter un programme agentique en production de façon contrôlée.

L'anti-patron documenté par AgentCorps (2026) et que ce chapitre classe comme anti-patron de séquençage : déployer un système multi-agents (3 à 10 agents simultanément) avant qu'un agent unique fonctionne de façon fiable dans le contexte de production propre à l'organisation. Cette erreur est distincte de la question de la gouvernance — elle est d'ordre architectural et chronologique. Un système multi-agents en production suppose que les contrats d'escalade, les périmètres de permission et les métriques d'évaluation sont établis pour *au moins* un agent mature. Sans cela, la complexité combinatoire des interactions inter-agents amplifie chaque mode d'échec identifié en §12.2. La règle opérationnelle : *single-agent first*. La monographie adopte cette formulation d'AgentCorps comme principe de séquençage, cohérent avec le principe de simplicité d'Anthropic documenté au Ch. 1 et avec les niveaux de maturité AgentOps de l'Annexe C.

12.3.2 — ROI flou et métriques de surface

La seconde cause organisationnelle est le maintien de métriques d'activité (nombre d'appels agents, tokens consommés, taux de complétion de tâche) au lieu de métriques d'impact (CPST — *Cost per Successful Task*, impact P&L). McKinsey (2026) documente que les organisations sans bascule vers les métriques P&L restent bloquées en phase expérimentale en année deux.

HBR (octobre 2025) fournit la corrélation de scope la plus nette disponible : les projets à portée étroite sont livrés dans les délais dans 65 % des cas, avec un glissement médian de 1,9 mois ; les projets à portée large ne le sont que dans 16 % des cas, avec un glissement médian de 9,6 mois. L'erreur n'est pas l'ambition — c'est l'absence de KPI liés à un résultat métier mesurable sur le périmètre exact déployé.

Databricks (*State of AI Agents 2026*) établit par ailleurs que les organisations avec gouvernance IA poussent 12× plus de projets en production — le différentiel le plus fort mesuré à ce jour entre une variable organisationnelle et le taux de succès en production. Ce 12× n'est pas un effet de sélection d'organisations plus matures : c'est la conséquence directe de décisions structurées (go/no-go fondées sur des métriques d'impact, pas d'activité) qui remplacent l'inertie par la délibération.

Le lien avec les cas documentés dans les chapitres précédents : Klarna (Ch. 11 §11.3) a éliminé plus de 4 000 postes sur une trajectoire agentique dont le ROI n'était mesuré qu'en réduction de coûts directs, sans instrumentation de la satisfaction client ou de la qualité du service — jusqu'à ce que la dégradation soit mesurable en résultats P&L négatifs. La réembauche partielle de 2025 est le signal d'un pivot contraint, pas d'une décision structurée.

12.3.3 — *Change management* absent

Le Ch. 11 §11.4 documente que 29 % des employés admettent avoir saboté activement des déploiements IA dans leur organisation (Writer / Workplace Intelligence, avril 2026). Ce signal, traité dans ce chapitre comme un facteur d'échec de programme au niveau portefeuille, se manifeste précisément dans les déploiements qui ont optimisé la stack technique en laissant les questions de rôle et de trajectoire de compétences sans réponse. Un programme agentique sans *AI ops manager* ni *quality steward* (Ch. 11 §11.2) opérationnels avant déploiement à l'échelle ne produit pas un échec technique — il produit un *workslop* (Deloitte Tech Trends 2026) : une charge opérationnelle augmentée sous couvert d'automatisation, où l'absence de rôles de jonction transforme la surface agentique en fardeau pour les équipes qui l'opèrent.

12.4 — Anatomie économique : coûts non bornés et infrastructure tax

L'architecture financière d'un programme *agentic* n'est pas celle d'un projet logiciel — les coûts de *retry*, d'escalade et d'orchestration peuvent diverger exponentiellement sans plafonds explicites.

L'incident le plus documenté reste le cas de loop sans plafond décrit au Ch. 2 §2.3 (InfoWorld, 2026) : 47 000 $ de dépense non intentionnelle générée par un agent sans contrainte de *retry budget*. Le multiplicateur PoC → production documenté par Company of Agents (2026) est de 50×-300× en scénario typique, 1 500×-3 000× dans le pire des cas. Ces ordres de grandeur ne sont pas des pathologies rares — ils sont la norme pour les organisations qui n'ont pas instrumenté le *Cost per Successful Task* (CPST) défini au Ch. 2 §2.2 avant le déploiement élargi.

L'*infrastructure tax* aggrave le problème en le rendant invisible dans les budgets conventionnels. L'orchestration, la gestion de la mémoire, les appels d'outils, les *retries* et l'escalade humaine représentent entre 30 % et 50 % du budget agentique réel (Adnan Masood, mars 2026) — une fraction structurellement absente des lignes de budget qui s'arrêtent aux coûts d'inférence. Un programme dont le budget est calibré sur les coûts token est structurellement sous-estimé de 30 à 100 %.

S&P Global (2025) confirme l'effet macroscopique : 46 % des POC scrappés avant déploiement élargi, avec le coût comme obstacle principal. La dynamique est prévisible en amont : un POC opère sur des volumes artificiellement bas, avec des *retry budgets* non bornés car le coût unitaire est absorbable, et sans *infrastructure tax* significative. La transition vers la production inverse les trois paramètres simultanément — et les équipes qui n'ont pas modélisé ce passage en CPST se retrouvent avec une courbe de coût qui dépasse la courbe de valeur avant d'atteindre l'adoption minimale pour la justifier.

Poste de coût	Visible en PoC	Visible en production	Instrument de mesure
Inférence LLM	Oui	Oui	Coût par token
Orchestration	Partiel	Oui	CPST — portion orchestration
Appels d'outils	Partiel	Oui	*Tool spans* → cost allocation
Retries non bornés	Non (volumes bas)	Oui — critique	*Retry budget* + loop limits
Escalade humaine	Non (hors périmètre PoC)	Oui	*Escalation cost* par tâche
Infrastructure tax (mémoire, stockage)	Non	Oui — 30-50 % budget	CPST complet

---

12.5 — Anti-patron de séquençage : le multi-agents prématuré

L'anti-patron de séquençage est distinct des trois catégories causales — c'est un multiplicateur : il amplifie simultanément les échecs techniques, organisationnels et économiques en introduisant de la complexité avant que la base soit stable.

AgentCorps (2026) le documente comme la cause praticienne la plus fréquente d'échec précoce en 2026 : des équipes déployant 3 à 10 agents en système coordonné avant qu'un seul agent ait démontré sa fiabilité dans le contexte de production propre à l'organisation. La tentation est compréhensible — les architectures multi-agents promettent des gains de parallélisme, de spécialisation et de throughput. Mais ces promesses supposent que les contrats inter-agents

(escalade, délégation, gestion des erreurs) sont bien définis, que les périmètres de permission sont cohérents à travers la flotte, et que l'observabilité couvre les *orchestration spans* aussi bien que les *tool spans*. Aucune de ces conditions n'est remplie dans une organisation qui n'a pas encore opéré un agent unique en production.

L'effet de bord documenté : les modes d'échec d'un agent en dérive se propagent à tous les agents coordonnés avec lui. Un *tool drift* sur l'agent A, dans un système superviseur-workers, produit des instructions incorrectes que les agents B, C et D exécutent sans signal d'erreur propre — parce que l'instruction était syntaxiquement valide. L'arXiv:2603.06847 classe ce vecteur dans la catégorie des *cascading failures*, qui représentent une fraction significative des fautes à impact le plus élevé dans les dépôts analysés.

La règle opérationnelle est celle d'Anthropic (Introduction et Ch. 1) : commencer avec les prompts les plus simples, ajouter de la complexité seulement quand le gain est mesuré. L'équivalent au niveau du portefeuille : *single-agent first* en production, avec évaluations structurées stables, avant tout déploiement de coordination multi-agents.

```
flowchart TD
    START([Nouveau programme agentique]) --> SA
    SA["Agent unique\nen production\n(périmètre borné)"]
    SA --> EVAL{"Évaluations\nstables ?\nRetry budget\nborné ?"}
    EVAL -->|Non| FIX["Corriger drift,\ndette mémoire,\nmetriques"]
    FIX --> SA
    EVAL -->|Oui — ≥ 4 sem.| SCALE
    SCALE["Valider critères\nscale\n(McKinsey)"]
    SCALE --> MA{"Multi-agents\npertinents ?"}
    MA -->|Non| SINGLE["Étendre le périmètre\nde l'agent unique"]
    MA -->|Oui| COORD["Déployer coordination\nmulti-agents\navec orchestration spans"]
```

12.6 — Signaux faibles : checklist d'alerte précoce

Les programmes agentiques qui échouent émettent des signaux mesurables dès les premières semaines de déploiement — pas après le dépassement irréversible. La checklist ci-dessous regroupe 12 signaux en trois familles, chacun avec un seuil quantitatif et une source. Elle est conçue pour un architecte ou directeur de programme qui évalue la santé d'un programme en cours.

#	Signal	Seuil d'alerte	Famille	Source
1	Taux d'escalade en hausse continue	+15 % sur 2 semaines sans changement de scope	Technique	Arize AI, jan. 2026
2	*Retry count* moyen par tâche	> 3 sur les outils critiques sans résolution	Technique	Ch. 7 §7.2
3	Taux de rappel mémoire en production	< 50 % sur tâches de complexité élevée	Technique	Chanl AI, 2025
4	Couverture des évaluations	< 50 tours par scénario critique (évaluations ponctuelles)	Technique	CIO.com, 2026
5	Dérive du CPST	> 20 % sur 4 semaines sans explication documentée	Économique	Ch. 2 §2.2
6	*Infrastructure tax* non modélisée	Absence de ligne budgétaire dédiée à l'orchestration et l'escalade	Économique	Adnan Masood, mars 2026
7	*Loop limits* et *tool-call caps*	Aucun plafond en production sur les agents actifs	Économique	InfoWorld / Ch. 2 §2.3
8	Gouvernance et catalogue	> 5 agents en production sans catalogue centralisé ni RACI agentique	Organisationnel	HBR, juin 2025
9	Rôles opérationnels	Aucun *AI ops manager* ni *quality steward* identifié	Organisationnel	Ch. 11 §11.2
10	Métriques de succès	Métriques d'activité (appels, tokens) sans lien à un résultat métier	Organisationnel	McKinsey, 2026
11	Versionnage de l'artefact composite	Absence de version atomique (prompt + outils + mémoire + permissions)	Gouvernance	Ch. 7 §7.1
12	*Kill switch*	Aucun mécanisme de suspension défini et testé en production	Gouvernance	Annexe C ; Ch. 8

Un programme qui cumule trois signaux ou plus dans la même famille est en trajectoire d'échec avec une probabilité élevée (*hypothèse* — seuil calibré sur la convergence des sources, pas sur une étude longitudinale propre). Un programme qui cumule des signaux dans les trois familles simultanément est dans un état de dérive systémique — la question n'est plus de corriger mais de décider.

Compromis principal. Instrumenter ces 12 signaux en production exige de l'outillage AgentOps (Ch. 7) et des ressources d'évaluation que les équipes en phase expérimentale n'ont pas systématiquement. L'alternative sans outillage complet : prioriser les signaux 5, 7, 8 et 9 — les quatre dont la mesure ne nécessite pas d'instrumentation technique avancée et qui sont suffisants pour déclencher une revue de programme.

Condition qui renverse cette approche. Pour un programme à faible réversibilité d'action (agents de lecture seule, agents de recommandation sans exécution autonome), les seuils techniques (signaux 1 à 4) peuvent être assouplis — le risque de dommage irréversible est structurellement plus faible. La matrice autonomie × réversibilité du Ch. 3 est l'instrument de qualification.

12.7 — Critères kill / pivot / scale

Kill, *pivot* et *scale* ne sont pas des réponses émotionnelles à des résultats décevants — ce sont des décisions structurées sur des critères définis **avant** le déploiement. L'absence de ces critères ex ante transforme chaque décision en arbitrage politique où l'inertie et les biais d'engagement priment sur les données.

Le cadre ci-dessous s'appuie sur McKinsey (2026) pour les critères de *scale*, sur HBR (octobre 2025) pour les critères de *kill* et *pivot*, et sur les cas documentés dans les chapitres précédents pour l'illustration concrète.

KILL — arrêt du programme

Critère	Seuil	Cas illustratif
Valeur métier	Aucun résultat métier mesurable après 90 jours en production réelle (pas en pilote)	Klarna : baisse de satisfaction client mesurable après élimination du support humain (Ch. 11 §11.3)
CPST	> 3× l'alternative humaine ou RPA sans trajectoire d'amélioration documentée sur 60 jours	Incident 47 000 $: loop sans plafond (Ch. 2 §2.3)
Données	Sources < niveau AI-ready sans plan de remédiation < 6 mois	60 % Gartner fév. 2025 — cause structurelle
Sécurité	Incidents irréversibles répétés sans remédiation, ou vecteurs actifs non résolus (injection via outils, exfiltration cross-tool)	EchoLeak / Sonrai AgentCore (Ch. 9) — compromis d'exfiltration sans détection
Organisationnel	Résistance structurelle documentée (> 25 % de sabotage actif) sans programme de *change management* viable	29 % sabotage Writer 2026 (Ch. 11 §11.4)

PIVOT — réorientation du périmètre ou de l'approche

Situation	Pivot recommandé	Condition de retour au scale
Valeur métier identifiée mais sur sous-périmètre différent	Réduire le scope (65 % succès portée étroite vs 16 % portée large — HBR oct. 2025)	Critères McKinsey séquentiels atteints sur le sous-périmètre réduit
Agent multi-étapes instable	Revenir à agent single-step sur tâche bien définie (*single-agent first* — AgentCorps)	Évaluations stables sur ≥ 4 semaines
Gouvernance absente	Suspendre le déploiement élargi, déployer la gouvernance (Annexe D), reprendre	RACI opérationnel + catalogue centralisé + *kill switch* testé
Métriques d'activité positives, impact P&L nul	Redéfinir les KPI sur un résultat métier et recalibrer le cas d'usage	CPST < CPST de l'alternative (humain ou RPA) sur un horizon de 90 jours

SCALE — investissement accéléré

McKinsey (2026) définit trois critères séquentiels — les trois doivent être satisfaits dans l'ordre, pas simultanément :

1. **Stable et sûr en production** : aucun signal faible actif en famille technique (checklist §12.6 signaux 1 à 4) ; *kill switch* testé ; artefact composite versionné.
2. **Adoption réelle dans un flux de travail métier** : l'agent est utilisé dans des transactions réelles, pas seulement en démo ou pilote interne.
3. **Impact opérationnel et financier mesurable** : CPST < CPST de l'alternative ; impact P&L positif sur le périmètre déployé ; ROI justifiant l'extension.

Les deux multiplicateurs Databricks opèrent comme accélérateurs une fois les trois critères atteints : gouvernance en place (12× plus de projets en production) et évaluations structurées actives (6× plus de déploiements réussis). Replit, après l'incident de juillet 2025, a suivi précisément cette séquence de pivot : introduction du mode *planning-only*, séparation automatique des environnements dev/prod, amélioration des mécanismes de rollback — puis reprise du déploiement sur un périmètre plus étroit avec les contraintes structurelles en place (Ch. 7).

```
flowchart LR
    S1["Signaux faibles\nactifs ?"]
    S1 -->|"≥ 3 dans une famille"| KILL_PIVOT
    S1 -->|Aucun| CRITERES

    CRITERES["3 critères McKinsey\nséquentiels atteints ?"]
    CRITERES -->|Non| HOLD["Stabiliser\n(corriger, gouverner,\nborner les coûts)"]
    CRITERES -->|Oui| SCALE_NOW["SCALE\n(12× avec gouvernance\n6× avec evals)"]

    KILL_PIVOT["Évaluer\nKILL / PIVOT"]
    KILL_PIVOT -->|"CPST > 3× alternative\nou données non AI-ready\nou sécurité non résolue"|
KILL_NOW["KILL"]
    KILL_PIVOT -->|"Valeur partielle\nou scope trop large\nou gouvernance absente"|
PIVOT_NOW["PIVOT\n(scope, séquence,\nou gouvernance)"]
    PIVOT_NOW -->|"Critères atteints\npost-pivot"| CRITERES
```

12.8 — Recommandation architecturale et transition vers le Ch. 13

La recommandation principale de ce chapitre est la suivante : définir les critères *kill/pivot/scale* et la checklist des signaux faibles avant le déploiement, pas après les premiers résultats. Cette décision est la plus haute valeur ajoutée qu'un architecte d'entreprise peut apporter à un programme agentique — précisément parce qu'elle transforme des jugements politiques futurs en critères factuels établis à froid.

Compromis principal. Définir ces critères ex ante exige d'accepter des seuils qui peuvent paraître arbitraires au moment du déploiement (90 jours pour un résultat mesurable, CPST $< 3\times$ l'alternative). Le risque est de tuer prématurément un programme qui aurait réussi avec 60 jours supplémentaires. L'alternative — ne pas définir de critères — produit empiriquement des programmes qui survivent bien au-delà de leur date d'échec économique, par inertie et biais d'engagement.

Alternative crédible. Pour les organisations sans capacité d'instrumentation AgentOps complète, une version allégée de la checklist (signaux 5, 7, 8, 9 uniquement) couplée aux trois critères McKinsey est suffisante pour structurer des décisions de portefeuille — à condition que le CPST soit calculé dès le pilote.

Condition qui renverse la recommandation. Pour un programme en phase d'exploration initiale sur un cas d'usage totalement nouveau (premier déploiement agentique de l'organisation, aucun benchmark sectoriel disponible), les seuils chiffrés sont *à calibrer localement* sur les premières semaines de données réelles avant d'être formalisés. Appliquer des seuils generiques sur un terrain sans référence produit des décisions sans signal.

L'inventaire de ce chapitre n'est pas une autopsie — c'est la carte qui permet d'anticiper le terrain du prochain cycle. Certains modes d'échec documentés ici sont transitoires : la dette mémoire et la dérive d'outil seront partiellement atténuées par des améliorations de *runtime* et d'évaluation dans les 18 à 24 mois qui viennent (*hypothèse* — trajectoire prévisible sur la base des investissements R&D actuels des fournisseurs, à confirmer). D'autres — gouvernance absente, ROI flou, coûts non bornés — sont structurels et resteront les différentiateurs entre les organisations qui scale et celles qui n'y parviennent pas, indépendamment des améliorations technologiques. Le Ch. 13 trace l'horizon 2027-2030 à la lumière de ces modes d'échec et identifie quels investissements de 2026 constituent une assurance sur les trajectoires de valeur à venir — et lesquels disparaîtront dans les améliorations de plateforme.

Pour aller plus loin

Shah, M. B. et al. — « Characterizing Faults in Agentic AI: A Taxonomy of Types, Symptoms, and Root Causes » (arXiv:2603.06847, mars 2026, https://arxiv.org/abs/2603.06847). La taxonomie académique la plus rigoureuse disponible à mai 2026 sur les modes de défaillance agentiques — 385 fautes, 40 dépôts, validation praticienne à 83,8 %. Indispensable pour tout architecte qui veut catégoriser les défaillances de son propre programme au-delà du vocabulaire marketing.

Databricks — « Enterprise AI Agent Trends: Top Use Cases, Governance + Evaluations and More » (*State of AI Agents 2026*, janvier 2026, https://www.databricks.com/blog/enterprise-ai-agent-trends-top-use-cases-governance-evaluations-and-more). La source de données la plus large disponible sur les facteurs de succès en

production (n=20 000+). Le 12× gouvernance et le 6× évaluations sont les deux chiffres les plus actionnables de ce chapitre — à utiliser dans tout comité de programme pour justifier l'investissement en gouvernance.

HBR — « Why Agentic AI Projects Fail—and How to Set Yours Up for Success » (octobre 2025, https://hbr.org/2025/10/why-agentic-ai-projects-fail-and-how-to-set-yours-up-for-success). La corrélation scope / taux de succès (65 % vs 16 %) est l'argument de cadrage le plus efficace pour contraindre le périmètre d'un programme agentique en phase de définition — avant que le scope ne soit verrouillé sous pression de délai.

Arize AI — « Why AI Agents Break: A Field Analysis of Production Failures » (janvier 2026, https://arize.com/blog/common-ai-agent-failures/). Taxonomie terrain issue d'une analyse de traces LLM en production — plus proche de la réalité opérationnelle que les taxonomies académiques. À lire en complément de l'arXiv:2603.06847 pour une couverture complète des modes de défaillance technique.

McKinsey — « Building the Foundations for Agentic AI at Scale » (2026, https://www.mckinsey.com/capabilities/mckinsey-technology/our-insights/building-the-foundations-for-agentic-ai-at-scale). Le cadre des trois critères séquentiels de *scale* est le seul disponible à mai 2026 qui couvre simultanément la fiabilité, l'adoption et l'impact P&L. À utiliser comme grille de go/no-go dans les revues de portefeuille.

Références

- Gartner — « Lack of AI-Ready Data Puts AI Projects at Risk » — Gartner Newsroom — 26 février 2025 — https://www.gartner.com/en/newsroom/press-releases/2025-02-26-lack-of-ai-ready-data-puts-ai-projects-at-risk — accédée le 2026-05-05
- Gartner — « Gartner Predicts Over 40% of Agentic AI Projects Will Be Canceled by End of 2027 » — Gartner Newsroom — 25 juin 2025 — https://www.gartner.com/en/newsroom/press-releases/2025-06-25-gartner-predicts-over-40-percent-of-agentic-ai-projects-will-be-canceled-by-end-of-2027 — accédée le 2026-05-05
- S&P Global Market Intelligence — « AI Experiences Rapid Adoption, but with Mixed Outcomes » — CIO Dive — 2025 — https://www.ciodive.com/news/AI-project-fail-data-SPGlobal/742590/ — accédée le 2026-05-05
- Shah, M. B. et al. — « Characterizing Faults in Agentic AI: A Taxonomy of Types, Symptoms, and Root Causes » — arXiv:2603.06847 — 6 mars 2026 — https://arxiv.org/abs/2603.06847 — accédée le 2026-05-05
- Arize AI — « Why AI Agents Break: A Field Analysis of Production Failures » — Arize Blog — 29 janvier 2026 — https://arize.com/blog/common-ai-agent-failures/ — accédée le 2026-05-05
- Databricks — « Enterprise AI Agent Trends: Top Use Cases, Governance + Evaluations and More » — Databricks Blog — janvier 2026 — https://www.databricks.com/blog/enterprise-ai-agent-trends-top-use-cases-governance-evaluations-and-more — accédée le 2026-05-05
- HBR — « Why Agentic AI Projects Fail—and How to Set Yours Up for Success » — Harvard Business Review — octobre 2025 — https://hbr.org/2025/10/why-agentic-ai-projects-fail-and-how-to-set-yours-up-for-success — accédée le 2026-05-05
- HBR — « Organizations Aren't Ready for the Risks of Agentic AI » — Harvard Business Review — juin 2025 — https://hbr.org/2025/06/organizations-arent-ready-for-the-risks-of-agentic-ai — accédée le 2026-05-05
- McKinsey — « Building the Foundations for Agentic AI at Scale » — McKinsey Technology — 2026 — https://www.mckinsey.com/capabilities/mckinsey-technology/our-insights/building-the-foundations-for-agentic-ai-at-scale — accédée le 2026-05-05
- McKinsey — « State of AI Trust in 2026: Shifting to the Agentic Era » — McKinsey — 2026 — https://www.mckinsey.com/capabilities/tech-and-ai/our-insights/tech-forward/state-of-ai-trust-in-2026-shifting-to-the-agentic-era — accédée le 2026-05-05
- Chanl AI — « Agent Drift: Why Your AI Gets Worse the Longer It Runs » — Chanl Blog — 2025 — https://www.chanl.ai/blog/agent-drift-silent-degradation — accédée le 2026-05-05
- AgentCorps — « Why Most Agentic AI Projects Fail (And How to Succeed in 2026) » — agentcorps.co — 2026 — https://agentcorps.co/blog/why-most-agentic-ai-projects-fail-and-how-to-succeed-in-2026 — accédée le 2026-05-05

Chapitre 13 — La route devant

Partie 5 — Piloter la transition Chapitre 13 · La route devant nous · ~4 600 mots · lecture ≈ 18 min

La réponse à la question centrale de cette monographie — comment adopter l'*agentic AI* avec la rigueur qui distingue les organisations qui progressent de celles qui échouent — ne change pas entre 2026 et 2030. Ce qui change, c'est l'échelle à laquelle cette rigueur s'applique : des agents isolés à des équipes autonomes, des flux de travail internes aux chaînes d'approvisionnement inter-entreprises, des serveurs centraux aux équipements industriels à la périphérie. L'architecte d'entreprise qui a instrumenté la boucle *decide–act–observe* (Ch. 1) en 2026, calibré ses *unit economics* sur le *Cost per Successful Task* (Ch. 2), déployé un plan de contrôle AgentOps (Ch. 7) et redesigné ses processus organisationnels (Ch. 11) n'a pas à recommencer en 2028. Il a à étendre ce qu'il a déjà construit. Ceux qui n'ont pas fait ce travail en 2026 n'auront pas le temps de le faire en 2028.

13.1 — Le Hype Cycle 2026 comme carte de navigation

Le signal structurant du Hype Cycle for Agentic AI 2026 n'est pas la position d'*AI Agent Development Platforms* au sommet des attentes gonflées (*Peak of Inflated Expectations*, horizon 2–5 ans vers la productivité — *confirmé*, article public Gartner 2026). C'est l'émergence, pour la première fois sur une carte Gartner, de trois profils de supervision distincts : *agentic AI governance*, *agentic AI security*, et *FinOps for agentic AI* (*confirmé* — Gartner 2026 Hype Cycle article public ; confirmation indépendante : Zenity, Business Wire, 15 avril 2026). Les positions précises de ces trois profils sur la courbe ne sont pas vérifiables sans accès au rapport complet (derrière paywall) — *à vérifier*. Ce qui est vérifiable est leur existence comme profils autonomes, séparés des plateformes de développement qui les hébergent.

Cette séparation est un signal de marché, pas un artifice de catégorisation. Tant que la gouvernance, la sécurité et le FinOps agentiques sont traités comme des fonctionnalités d'une plateforme de développement d'agents, ils restent facultatifs — des options que l'on active si le calendrier le permet. Quand ils apparaissent comme profils indépendants sur le Hype Cycle, c'est que des acheteurs distincts, avec des budgets distincts, évaluent ces disciplines pour elles-mêmes. La gouvernance n'est plus une case à cocher dans la checklist de déploiement ; elle est devenue un marché.

L'implication directe pour l'architecte d'entreprise est budgétaire avant d'être technique. Les *unit economics* du Ch. 2 sont établis sur un substrat où la couche de supervision — gouvernance, sécurité, FinOps — est encore absente ou sous-financée dans la majorité des organisations. Databricks (State of AI Agents 2026, n=20 000+ organisations) mesure que seulement 19 % des organisations ont déployé des agents à l'échelle, et que les organisations avec gouvernance IA structurée poussent 12 fois plus de projets en production. D'ici 2028, la supervision agentique sera une ligne budgétaire standard dans tout programme mature — une prédiction structurellement cohérente avec l'apparition de ces profils sur le Hype Cycle. L'organisation qui attend 2028 pour budgéter cette couche paiera la prime du retardataire.

Le concept de *guardian agent* illustre concrètement la trajectoire. Gartner (Newsroom, 11 juin 2025, *confirmé*) prédit que les *guardian agents* — agents dont le rôle est de surveiller, contraindre et corriger d'autres agents — captureront 10 à 15 % du marché agentique d'ici 2030. La même source établit que 50 % des échecs de déploiement d'ici 2030 seront imputables à une gouvernance agentique inadéquate. La convergence est directe : les *guardian agents* sont l'évolution naturelle du plan de contrôle AgentOps défini au Ch. 7. Les organisations qui ont investi dans l'observabilité multi-étapes — *tool spans*, *memory spans*, *orchestration spans* — en 2026 disposent déjà du substrat d'instrumentation sur lequel les *guardian agents* s'appuieront. Celles qui ne l'ont pas fait devront reconstruire cette infrastructure avant de déployer la supervision automatisée.

La lecture du Hype Cycle par profils plutôt que par plateformes donne une carte d'investissement plus précise : financer les disciplines (gouvernance, sécurité agentique tel que défini au Ch. 9, FinOps) avant de financer des plateformes dont les positions sur la courbe de maturité seront différentes dans dix-huit mois.

13.2 — Horizons 2027–2030 : trois trajectoires confirmées, une calibration nécessaire

13.2.1 — Équipes autonomes et résolution de bout en bout

La trajectoire la plus documentée par des prédictions primaires vérifiables est celle des *autonomous teams* — configurations multi-agents capables de résoudre des classes entières de problèmes sans intervention humaine sur chaque instance. La thèse est directe : ce n'est pas le comportement de la boucle *decide–act–observe* qui change d'ici 2029, c'est l'étendue du périmètre délégué à cette boucle.

Deux jalons Gartner bornent la trajectoire. En front-office : 80 % des problèmes courants de service client seront résolus de façon autonome d'ici 2029, avec une réduction de 30 % des coûts opérationnels (*confirmé* — Gartner Newsroom, 5 mars 2025). À l'échelle de l'entreprise : 15 % des décisions quotidiennes seront prises de façon autonome d'ici 2028 (*à vérifier* — formulation Deloitte Tech Trends 2026, probablement via Gartner). IBM (2026) prédit que 70 % des systèmes multi-agents en production d'ici 2027 utiliseront des agents à rôles étroits et spécialisés plutôt que des agents généralistes — une architecture qui découple la gestion de la compétence de la gestion de la coordination. Ces chiffres ne sont pas cohérents de façon automatique : le 15 % de décisions autonomes et le 80 % de résolutions service client mesurent des périmètres différents sur des horizons différents, et leur juxtaposition n'implique pas une trajectoire linéaire.

Ce qui reste constant, c'est la condition architecturale. La boucle Ch. 1 — *Thought* → *Action* → *Observation* — ne mute pas : l'horizon 2029 ne redéfinit pas la mécanique, il en étend le rayon d'action. Les *AI ops managers* et *quality stewards* identifiés au Ch. 11 comme rôles structurants de 2026 ne deviennent pas obsolètes en 2029 — ils deviennent superviseurs de flottes plutôt que de quelques agents isolés. Mais pour gérer une flotte à l'échelle, il faut avoir accumulé deux ou trois ans d'instrumentation de trajectoire, de régression continue et de *shadow runs* (définis au Ch. 7 §7.5) — autrement dit, avoir commencé avant 2027.

```
timeline
    title Horizons agentiques 2026-2030
    section 2026
        Disciplines de supervision émergent : Agentic governance · Agentic security · FinOps
agentique
        Edge agentique opérationnel : Cloudflare Agents Week · Advantech Jetson Thor
        Mainframe + agentique : IBM watsonx Code Assistant for Z v2.8
    section 2027
        Multi-agents spécialisés : 70% des systèmes (IBM)
        Chaînes logistiques inter-org : Fujitsu / Rohto trials conclud
    section 2028
        Commerce B2B agent-à-agent : 15T$ (Gartner — à vérifier biais prospectif)
        33% des apps entreprise incluent agentique : Deloitte
    section 2029
        80% service client résolu autonomement : Gartner confirmé
        Guardian agents : marché émergent
    section 2030
        Guardian agents 10-15% du marché : Gartner confirmé
        50% équipes EA rebrandées partenaires stratégiques
```

13.2.2 — Chaînes d'approvisionnement agentiques inter-entreprises

La trajectoire la plus transformatrice — et la plus difficile à piloter — est celle du commerce B2B agent-à-agent. La prédiction Gartner (IT Symposium/Xpo 2025, rapportée par Digital Commerce 360, novembre 2025, *confirmé* avec biais prospectif explicite) projette 90 % des achats B2B traités par des agents d'ici 2028, représentant 15 000 milliards de dollars d'échanges automatisés. Ce chiffre porte un biais prospectif caractéristique de Gartner sur les horizons court-terme — il mérite d'être traité comme un ordre de grandeur et un signal directionnel, non comme une prévision précise.

Ce qui est en revanche documenté en terrain réel est la technologie sous-jacente. Fujitsu (communiqué global, décembre 2025, *confirmé* — source primaire constructeur) a lancé en janvier 2026 des essais de collaboration sécurisée entre agents IA de compagnies distinctes dans la chaîne logistique de Rohto Pharmaceutical, en partenariat avec l'Institute of Science Tokyo — essais planifiés jusqu'à mars 2027. La distinction clé de l'architecture Fujitsu est la confidentialité des modèles propriétaires de chaque entreprise lors de la collaboration : les agents partagent des résul-

tats et des requêtes, pas leurs paramètres internes ni leurs données de formation. Deloitte Insights (2026, *confirmé*) documente la même trajectoire dans la fabrication : coordination multi-entreprise entre fabricants, fournisseurs et distributeurs, avec des agents prenant des décisions au niveau du plancher d'usine.

La condition architecturale de cette trajectoire est précisément le sujet du Ch. 5 : le protocole A2A (*Agent-to-Agent*) comme infrastructure d'interopérabilité inter-organisationnelle. Sans A2A ou un standard équivalent, les échanges B2B agent-à-agent nécessitent des intégrations bilatérales point-à-point, dont le coût de maintenance croît quadratiquement avec le nombre de partenaires. La portabilité MCP/A2A documentée au Ch. 10 est la condition de participation sans enfermement fournisseur : une organisation qui a construit ses agents sur des protocoles ouverts peut rejoindre n'importe quelle chaîne agentique inter-entreprises ; une organisation enfermée dans une plateforme propriétaire attend que son fournisseur négocie les intégrations à sa place.

Recommandation architecturale — pari sur les protocoles ouverts vs pari sur le fournisseur hyperscaleur dominant :

Compromis principal : les protocoles ouverts (A2A, MCP) sont moins matures et moins intégrés qu'une solution propriétaire AWS Bedrock Agents ou Azure AI Foundry en 2026. La courbe de démarrage est plus longue.

Alternative crédible : parier sur le fournisseur hyperscaleur qui dominera le marché en 2030 et s'y enfermer délibérément pour accélérer le time-to-value en 2026-2027. La justification : si un seul hyperscaleur capte 70 % du marché d'ici 2030, l'enfermement est moins coûteux que l'anticipation d'un standard qui ne converge pas.

Condition qui renverse la recommandation en faveur de l'enfermement : si d'ici fin 2027, A2A et MCP n'ont pas atteint une adoption de plus de 50 % des nouvelles intégrations inter-entreprises mesurée par les annonces de support des principaux ERP (SAP, Oracle, Workday), l'hypothèse de convergence des protocoles ouverts mérite d'être réévaluée. À mai 2026, SAP a annoncé à Hannover Messe l'opérationnalisation de l'agentique pour la fabrication résiliente, mais sans engagement public sur A2A spécifiquement — *à vérifier* en source primaire SAP.

13.2.3 — Intersection edge et mainframe modernisé

L'*agentic AI* de 2027–2030 ne s'exécutera pas uniquement dans les centres de données cloud. Elle occupera simultanément les deux extrêmes du spectre de latence et de fiabilité : l'équipement industriel à la périphérie et le mainframe en cœur de système — deux substrats radicalement différents que les architectures agentiques devront apprendre à orchestrer ensemble.

Du côté de l'edge, Cloudflare a défini les primitives dès 2026 (*confirmé* — Agents Week 2026, blog.cloudflare.com) : sandboxes persistants isolés avec démarrage à la demande, inférence unifiée sur 14+ fournisseurs de modèles, Workers AI pour inférence à latence inférieure à la centaine de millisecondes à la périphérie. Le positionnement de Cloudflare — « Cloud 2.0, the agentic cloud » — souligne que cette infrastructure existe en production en 2026, pas comme prospective. Advantech (EE News Europe, 2026, *confirmé*) déploie la série MIC-AI sur NVIDIA Jetson Thor pour des workloads agentiques industriels embarqués, avec des capacités de traitement visuel (*VLM*, *vision-language model*) sur des équipements en production dans des usines.

Du côté du mainframe, IBM watsonx Code Assistant for Z v2.8 (IBM Newsroom, 2026, *confirmé*) formalise le premier workflow agentique pour la modernisation de COBOL à l'échelle d'entreprise : identification automatisée des dépendances, analyse d'impact, génération de code Java ou COBOL modernisé, compilation et vérification. Project Bob — la prochaine génération, avec GA prévu 2026 — ajoute l'orchestration multi-modèles et une interface moderne. La signification architecturale est précise : l'agentique n'attend pas la fin de la modernisation mainframe pour se déployer. Elle s'intègre dans le processus de modernisation lui-même, transformant ce qui était un projet multi-année en flux continu piloté par des agents.

La tension architecturale entre ces deux extrêmes est réelle et non résolue en 2026. La boucle *decide–act–observe* définie au Ch. 1 s'exécute à des latences radicalement différentes selon le substrat : quelques dizaines de millisecondes sur un edge Cloudflare, plusieurs secondes sur un workflow mainframe COBOL-to-Java. Les patterns d'orchestration multi-agents (Ch. 6) développés pour des contextes homogènes devront être adaptés pour des architectures hétérogènes edge-cloud-mainframe d'ici 2028. Le *Cost per Successful Task* (Ch. 2) devra intégrer les coûts d'orchestration cross-substrat comme composante distincte à partir de 2027 — *hypothèse*, aucune source primaire n'a encore formalisé cette métrique.

13.2.4 — AGI : une calibration sans extravagance

Les dirigeants des principaux laboratoires d'IA expriment des positions convergentes sur l'horizon mais divergentes sur la définition. Demis Hassabis (Google DeepMind) situe une « AGI au niveau humain authentique » à 5–10 ans à

partir de déclarations récentes, soit 2031–2036. Sam Altman (OpenAI) évoque la superintelligence dans « quelques milliers de jours » — un horizon situé entre 2032 et 2035. Shane Legg (Google DeepMind) estimait en 2023 une probabilité de 50 % pour une « AGI minimale » d'ici 2028 — *à vérifier*, la formulation circule dans des synthèses secondaires sans URL source primaire directe récente.

Ce que ces positions ont en commun est plus utile que leurs différences : aucune d'elles ne fonde une décision d'investissement architecturale vérifiable sur un horizon de 36 mois. La définition de l'AGI reste contestée entre les laboratoires eux-mêmes — ce que Legg appelle « minimal AGI » n'est pas ce que Hassabis appelle « genuine human-level AGI », et aucun des deux ne coïncide nécessairement avec les seuils opérationnels qui importent pour un architecte d'entreprise.

Implication pratique : les investissements architecturaux de cette monographie — observabilité multi-étapes, gouvernance de flotte, portabilité protocoles, redesign organisationnel — sont valides quel que soit le moment auquel un seuil AGI est atteint, parce qu'ils répondent à des besoins opérationnels mesurables dès aujourd'hui. La trajectoire vers l'AGI est une hypothèse de fond, non une variable de décision. L'architecte d'entreprise de 2026 qui planifie sur la base d'une rupture AGI en 2028 ou en 2032 construit sur une inconnue. Celui qui planifie sur la base de l'élargissement progressif du périmètre autonome — documenté par les prédictions Gartner citées en §13.2.1 — construit sur des données mesurables. *Marqueur explicite : hypothèse* — la trajectoire vers l'AGI n'est pas une base d'investissement architecturale fiable à mai 2026.

13.3 — L'architecte d'entreprise dans cinq ans

En 2031, le rôle d'architecte d'entreprise ne sera pas supprimé par l'*agentic AI*. Il sera méconnaissable à quiconque l'a exercé en format traditionnel — séquences de diagrammes de capacités, revues d'architecture trimestrielles, production de livrables PowerPoint pour des comités directeurs. La transformation n'est pas une réduction du rôle : c'est une recomposition de son contenu vers ce que les agents ne peuvent pas faire, et un délestage de ce qu'ils font mieux.

Forrester (2025-2026, *confirmé*) identifie deux rôles émergents qui redéfinissent la fonction EA. Le *digital twin strategist* construit et maintient des modèles de simulation de l'architecture d'entreprise — des représentations numériques qui permettent d'évaluer les conséquences architecturales d'une décision avant de la prendre, en faisant tourner des scénarios agentiques sur la gemelle numérique plutôt que dans la production. L'*enterprise knowledge curator* gouverne les graphes de connaissance organisationnels — les représentations structurées du savoir de l'entreprise sur lesquelles raisonnent les agents — et s'assure que leur traçabilité permet l'audit de toute décision prise par un agent en référençant ce savoir. Ces deux rôles partagent une propriété commune : ils requièrent une compréhension profonde des domaines métier autant que de l'architecture technique, ce que Forrester appelle la compétence T-shaped.

Gartner (webinaire 2025, *confirmé*) prédit que d'ici 2028, 50 % des équipes EA se « rebranderont » en partenaires stratégiques métier. CIO.com (2026, *confirmé*) mesure que 92 % des leaders EA priorisent déjà l'architecture AI/agentic dans leurs agendas 2026 — une proportion cohérente avec la pression d'adoption documentée à l'Introduction. Forrester estime jusqu'à 50 % des tâches EA de bas niveau — compliance checks, reporting, génération de diagrammes — automatisables par des agents d'ici 2028.

La conclusion est contre-intuitive : la valeur de l'architecte d'entreprise ne diminue pas avec l'avènement des agents — elle se concentre. Trois compétences restent non délégables en 2031.

Première compétence non délégable : définir les périmètres d'autonomie et de réversibilité. Un agent ne peut pas fixer son propre périmètre d'action — ce serait une contradiction dans les termes. Les décisions sur ce qu'un agent est autorisé à faire seul, ce qui requiert approbation humaine, et ce qui est irréversible, sont des décisions architecturales qui requièrent une compréhension des enjeux métier, réglementaires et organisationnels que l'agent ne possède pas. La matrice autonomie × réversibilité × tolérance-erreur définie au Ch. 3 reste un outil humain.

Deuxième compétence non délégable : gouverner les artefacts composites et les contrats inter-agents. Un tuple composite {prompt système, ensemble d'outils versionné, configuration mémoire, périmètre de permission, seuils d'escalade} — défini au Ch. 7 §7.1 comme l'unité de rollback AgentOps — est un artefact architectural autant qu'opérationnel. Décider quand une modification de l'un de ses composants constitue une nouvelle version de l'agent,

et comment orchestrer les transitions entre versions dans un système multi-agents en production, est un problème d'architecture qui dépasse le cycle de vie d'un seul agent.

Troisième compétence non délégable : arbitrer les compromis qui traversent plusieurs domaines simultanément. Un agent optimise dans son périmètre. Les compromis entre sécurité (Ch. 9), coût (Ch. 2), portabilité (Ch. 10) et impact organisationnel (Ch. 11) ne peuvent pas être résolus par un agent sans que ses critères d'optimisation n'aient été définis par un humain. L'architecte d'entreprise de 2031 est le définisseur de ces critères, non leur exécutant.

Le lien avec les décisions du Ch. 11 est direct : les organisations qui ont redesigné leurs processus en 2026 — qui ont redéfini qui fait quoi, à quel niveau d'abstraction, sur quelle unité de décision — disposent déjà d'une cartographie explicite des périmètres que l'architecte d'entreprise gouvernera en 2031. Les organisations qui ont plaqué des agents sur des processus existants devront mener ce travail de redesign sous pression, en 2028 ou 2029, dans des conditions opérationnelles qui n'accommodent pas la réflexion structurée.

13.4 — Ce qui résiste à 2030 : boucler l'arc

Investissements pérennes vs investissements à durée limitée

La question n'est pas « faut-il investir ? » — la concurrence agentique rend l'abstention aussi risquée que l'excès. La question est : parmi les investissements possibles en 2026, lesquels constituent des assurances valides sur l'horizon 2030, et lesquels seront rendus obsolètes par les améliorations de plateforme avant d'avoir été amortis ?

Le tableau suivant structure cette distinction de façon qualitative. Aucun chiffre budgétaire n'est produit ici — leur fabrication serait une violation directe du protocole éditorial de ce projet ; les décisions budgétaires dépendent de la taille d'organisation, du secteur et de la maturité agentique courante.

Investissement	Durabilité	Raisonnement	Renvoi
Observabilité multi-étapes (tool spans, memory spans, orchestration spans)	**Pérenne**	Les améliorations de modèles accroissent la capacité autonome des agents — elles ne réduisent pas le besoin de traçabilité de leurs décisions ; elles l'augmentent	Ch. 7
Gouvernance d'artefact composite (tuple prompt/outils/mémoire/permissions versionné)	**Pérenne**	La complexité des systèmes multi-agents augmente avec la maturité ; la gouvernance de leurs artefacts est un investissement cumulatif	Ch. 7, Ch. 8
Portabilité MCP/A2A	**Pérenne**	Les protocoles ouverts à adoption croissante réduisent le coût de migration entre fournisseurs — un investissement dont la valeur augmente si les fournisseurs fusionnent ou changent de tarification	Ch. 5, Ch. 10
Compétences T-shaped dans l'équipe EA (bilinguisme P&L/architecture)	**Pérenne**	Les agents automatisent les tâches de bas niveau EA ; l'expertise multi-domaine reste humaine	§13.3
Evaluation framework (régression continue, replay, shadow runs)	**Pérenne**	Les évaluations structurées conditionnent le passage à l'échelle — sans elles, l'extension du périmètre autonome est aveugle	Ch. 7, Ch. 12
Frameworks d'orchestration propriétaires à version non portable	**Durée limitée**	Les abstractions de frameworks convergent vers des standards ouverts ; les investissements dans des APIs non portables devront être remigrés	Ch. 6
Modèles *fine-tuned* sur des données propriétaires sans pipeline de mise à jour	**Durée limitée**	Les modèles de base s'améliorent plus vite que les investissements de fine-tuning ne peuvent être amortis ; les avantages compétitifs migrent vers les données et les outils, pas les poids	Ch. 2
Pipelines sans couche d'évaluation structurée	**Durée limitée**	L'absence d'évaluation détectable est précisément le signal d'échec précoce documenté au Ch. 12 ; le coût de remédiation en production est supérieur au coût de construction dès la conception	Ch. 12
Intégrations bilatérales point-à-point entre agents d'organisations distinctes	**Durée limitée**	Si les protocoles inter-org (A2A) convergent d'ici 2028, le coût de migration depuis des intégrations bilatérales devient prohibitif à l'échelle	§13.2.2

La leçon du Ch. 12 s'applique ici avec précision inversée : les organisations qui ont échoué ont sur-investi dans l'exécution (des agents qui font des choses) et sous-investi dans la supervision (la capacité de savoir ce que les agents font, d'en évaluer la qualité, et de les corriger ou les arrêter). Les investissements pérennes du tableau ci-dessus sont, sans exception, des investissements dans la supervision.

La trajectoire des coûts et ses implications pour le CPST

Le *Cost per Successful Task* (Ch. 2) est calibré en 2026 sur un substrat d'inférence dont le coût a chuté d'un facteur 10 en deux ans. La trajectoire probable — *hypothèse*, aucune source primaire ne peut être citée pour une prévision de prix à 2030 — est une poursuite de cette deflation sur l'inférence de base. Ce qui n'est pas déflationniste est

la couche de supervision : gouvernance, sécurité, évaluation, orchestration cross-substrat. D'ici 2030, le CPST sera une fonction avec deux composantes de poids inversé par rapport à 2026 : l'inférence plus légère, la supervision plus lourde. Les architectures qui ont externalisé la supervision à une plateforme propriétaire ne contrôlent pas cette seconde composante.

Clôture de l'arc : l'architecte comme figure de continuité

Ce livre a commencé avec une rupture — le passage du copilot au système *agentic*, de l'assistance ponctuelle à l'autonomie persistante. Il se termine avec une continuité : la boucle *decide–act–observe* de Ch. 1 est la même en 2026, en 2028 et en 2030. Les modèles qui l'instancient sont plus capables ; les périmètres qu'elle couvre sont plus larges ; les substrats sur lesquels elle s'exécute sont plus variés. Mais la structure fondamentale — un agent perçoit, raisonne, agit et observe les conséquences — reste l'unité d'analyse pertinente pour l'architecte d'entreprise.

La continuité n'est pas une consolation — c'est un avantage compétitif. Les organisations qui ont compris la boucle en 2026, qui l'ont instrumentée, évaluée et gouvernée, ne repartiront pas de zéro en 2028 quand les modèles seront plus puissants, les équipes plus autonomes et les chaînes d'approvisionnement inter-organisationnelles plus agentiques. Elles étendront ce qu'elles ont déjà construit.

La question de ce livre n'était pas « faut-il adopter l'*agentic AI* ? » Elle était : « comment l'adopter avec la rigueur qui distingue les organisations qui scale de celles qui échouent ? » La réponse tient en quatre principes qui traversent tous les chapitres et qui résistent à 2030 :

1. **Gouvernance d'abord.** Les organisations *governance-first* poussent 12 fois plus de projets en production (Databricks, State of AI Agents 2026). Ce multiplicateur ne devient pas moins pertinent à mesure que les agents deviennent plus capables — il devient plus critique, parce que les conséquences d'un agent mal gouverné sont proportionnelles à son périmètre d'action.
2. **Évaluations structurées avant l'échelle.** Les organisations qui utilisent des outils d'évaluation réussissent 6 fois plus de déploiements en production (Databricks, 2026). L'évaluation n'est pas une phase finale — elle est un pipeline continu qui conditionne chaque élargissement de périmètre.
3. **Périmètres d'autonomie définis avant le déploiement.** La matrice autonomie × réversibilité × tolérance-erreur n'est pas une contrainte qui ralentit l'adoption — c'est la condition qui permet d'accélérer en confiance. Les organisations qui déploient sans cette carte ne savent pas ce qu'elles délèguent.
4. **Continuité architecturale plutôt que reconstruction à chaque vague.** L'architecte d'entreprise de 2031 n'est pas une figure nouvelle — c'est la continuation de l'architecte de 2026 qui a fait les bons choix aux bons moments : protocoles ouverts plutôt qu'enfermement, observabilité plutôt qu'opacité, redesign plutôt que plaquage.

La route devant nous est longue et non balisée dans sa totalité. Les prédictions de cette monographie sont des probabilités, pas des certitudes — elles sont marquées *confirmé*, *probable*, *à vérifier* ou *hypothèse* précisément parce que la rigueur sur l'incertitude est la même compétence que la rigueur sur le déploiement. Ce que l'on peut affirmer avec confiance est plus étroit que ce que les discours promotionnels agentiques prétendent — et suffisant pour décider.

Pour aller plus loin

Gartner — « 2026 Hype Cycle for Agentic AI » — https://www.gartner.com/en/articles/hype-cycle-for-agentic-ai — La carte de référence pour situer les trajectoires de maturité des disciplines agentiques. Le rapport complet (paywall) est indispensable pour les positions précises sur la courbe ; l'article public suffit pour l'orientation stratégique des trois profils émergents.

Fujitsu — communiqué multi-AI agent collaboration, décembre 2025 — https://global.fujitsu/en-global/pr/news/2025/12/01-02 — La seule documentation primaire disponible à mai 2026 d'une collaboration inter-organisationnelle d'agents en terrain réel. Les résultats des essais (jan. 2026 – mars 2027) seront une source primaire de premier plan pour calibrer les projections du §13.2.2.

Forrester — « How Agentic AI Elevates The Enterprise Architect's Role » — https://www.forrester.com/blogs/the-future-of-the-enterprise-architects-job/ — La source analytique la plus complète disponible sur la recomposition du rôle EA à horizon cinq ans. À lire en parallèle avec CIO.com (2026) pour l'angle praticien.

Databricks — State of AI Agents 2026 — https://www.databricks.com/resources/ebook/state-of-ai-agents — Le rapport empirique de référence sur l'état réel des déploiements agentiques en entreprise (n=20 000+). Les chiffres

12× governance et 6× evaluations sont les plus cités dans ce chapitre et dans l'ensemble de la monographie — les vérifier en source primaire avant toute communication externe.

Gartner — « Guardian Agents will Capture 10-15% of the Agentic AI Market by 2030 » — https://www.gartner.com/en/newsroom/press-releases/2025-06-11-gartner-predicts-that-guardian-agents-will-capture-10-15-percent-of-the-agentic-ai-market-by-2030 — Le point de départ obligatoire pour quiconque planifie une architecture de gouvernance agentique au-delà de 2027. La prédiction est publique, datée et traçable — une rareté dans l'écosystème agentique.

Références

1. Gartner — « 2026 Hype Cycle for Agentic AI » — Gartner — 2026 — https://www.gartner.com/en/articles/hype-cycle-for-agentic-ai — accédée le 2026-05-05
2. Zenity — « Zenity Named in Two Categories in the 2026 Gartner® Hype Cycle™ for Agentic AI » — Business Wire — 15 avril 2026 — https://www.businesswire.com/news/home/20260415309905/en/Zenity-Named-in-Two-Categories-in-the-2026-Gartner-Hype-Cycle-for-Agentic-AI — accédée le 2026-05-05
3. Gartner — « Gartner Predicts that Guardian Agents will Capture 10-15% of the Agentic AI Market by 2030 » — Gartner Newsroom — 11 juin 2025 — https://www.gartner.com/en/newsroom/press-releases/2025-06-11-gartner-predicts-that-guardian-agents-will-capture-10-15-percent-of-the-agentic-ai-market-by-2030 — accédée le 2026-05-05
4. Digital Commerce 360 (citant Gartner IT Symposium/Xpo 2025) — « Gartner: AI agents will command $15 trillion in B2B purchases by 2028 » — novembre 2025 — https://www.digitalcommerce360.com/2025/11/28/gartner-ai-agents-15-trillion-in-b2b-purchases-by-2028/ — accédée le 2026-05-05
5. Fujitsu — « Fujitsu develops multi-AI agent collaboration technology to optimize supply chains, launches joint trials » — Fujitsu Global PR — décembre 2025 — https://global.fujitsu/en-global/pr/news/2025/12/01-02 — accédée le 2026-05-05
6. Cloudflare — « Building the agentic cloud: everything we launched during Agents Week 2026 » — Cloudflare Blog — 2026 — https://blog.cloudflare.com/agents-week-in-review/ — accédée le 2026-05-05
7. IBM — « Agentic AI for smarter mainframe modernization with IBM watsonx Code Assistant for Z » — IBM Newsroom — 2026 — https://www.ibm.com/new/announcements/agentic-ai-for-smarter-mainframe-modernization-with-ibm-watsonx-code-assistant-for-z — accédée le 2026-05-05
8. Deloitte — « The agentic supply chain in manufacturing » — Deloitte Insights — 2026 — https://www.deloitte.com/us/en/insights/industry/manufacturing-industrial-products/agentic-supply-chain-artificial-intelligence-manufacturing.html — accédée le 2026-05-05
9. Forrester — « How Agentic AI Elevates The Enterprise Architect's Role » — Forrester Blog — 2025-2026 — https://www.forrester.com/blogs/the-future-of-the-enterprise-architects-job/ — accédée le 2026-05-05
10. CIO.com — « Agentic AI's rise is making the enterprise architect role more fluid » — CIO — 2026 — https://www.cio.com/article/4096695/agentic-ais-rise-is-making-the-enterprise-architect-role-more-fluid.html — accédée le 2026-05-05
11. Gartner — « Gartner Predicts Agentic AI Will Autonomously Resolve 80% of Common Customer Service Issues Without Human Intervention by 2029 » — Gartner Newsroom — 5 mars 2025 — https://www.gartner.com/en/newsroom/press-releases/2025-03-05-gartner-predicts-agentic-ai-will-autonomously-resolve-80-percent-of-common-customer-service-issues-without-human-intervention-by-20290 — accédée le 2026-05-05
12. Advantech / EE News Europe — « Advantech brings agentic AI to Jetson Thor edge platforms » — EE News Europe — 2026 — https://www.eenewseurope.com/en/advantech-brings-agentic-ai-to-jetson-thor-edge-platforms/ — accédée le 2026-05-05
13. Databricks — « Enterprise AI Agent Trends: Top Use Cases, Governance + Evaluations and More » — Databricks Blog — 2026 — https://www.databricks.com/blog/enterprise-ai-agent-trends-top-use-cases-governance-evaluations-and-more — accédée le 2026-05-05

14. Deloitte — « The agentic reality check: Preparing for a silicon-based workforce » — Deloitte Tech Trends 2026 — 2026 — https://www.deloitte.com/us/en/insights/topics/technology-management/tech-trends/2026/agentic-ai-strategy.html — accédée le 2026-05-05

Chapitre 14 — Construire son OS agentique sur mesure (Claude Code)

Partie 5 — Piloter la transition Chapitre 14 · Coda tactique : construire son OS agentique personnalisé — ~4 200 mots · lecture ≈ 16 min

La thèse de ce chapitre peut être énoncée directement : les treize chapitres précédents ont défini ce que l'*agentic AI* doit être — gouvernée, observable, portable, réversible. Ce chapitre montre comment l'assembler concrètement, en trois étapes, avec un outil opérationnel dont la stack complète est publique et versionnée. L'outil est Claude Code. La décision n'est pas un choix éditorial arbitraire : Claude Code est le seul harnais *agentic* dont l'architecture interne a fait l'objet d'une analyse systémique académique publiée (Bhatt et al., arXiv 2604.14228v1, avril 2026) et dont l'intégralité des mécanismes d'extensibilité — skills, hooks, sub-agents, MCP, plugins — est documentée officiellement à mai 2026. C'est aussi le sujet de la vidéo source de ce chapitre : « Build Your CUSTOM Claude Code Agentic OS (3 Steps) » (Chase AI, YouTube). **Note de transparence** : le transcript de cette vidéo n'a pas pu être récupéré. Les trois étapes décrites au §14.2 sont une reconstruction depuis la documentation officielle Claude Code et les sources académiques citées en notes de recherche — elles ne constituent pas un transcript fidèle.

14.1 — Pourquoi assembler son OS agentique plutôt qu'adopter une plateforme clé-en-main

La question de la plateforme propriétaire versus le harnais personnalisé a été posée au Ch. 10 sous l'angle de la portabilité et du *lock-in*. Elle mérite ici un angle différent : celui du contrôle opérationnel.

Une plateforme clé-en-main — AWS Bedrock Agents, Azure AI Foundry, Google Vertex AI Agent Builder — optimise pour la mise en service rapide. Elle fournit une interface de configuration, un modèle de déploiement, des connecteurs préconfigurés, et un tableau de bord de supervision. Ce que la plateforme ne fournit pas, c'est la transparence du mécanisme de sécurité et le contrôle fin du comportement de l'agent à chaque point de son cycle de vie.

L'analyse de l'architecture Claude Code par Bhatt et al. (arXiv 2604.14228v1, *confirmé*) établit un fait structurel : dans ce harnais, 98,4 % du code constitue de l'infrastructure opérationnelle — permissions, sandboxing, observabilité, compaction de contexte, gestion du cycle de vie — contre 1,6 % de logique de décision du modèle. Cette proportion n'est pas une curiosité statistique. Elle signifie que la sécurité du système repose sur sept couches d'autorisation indépendantes, chacune capable de bloquer une invocation d'outil, et que le modèle ne peut pas contourner ces couches parce qu'il communique uniquement via des blocs `tool_use` structurés, validés avant exécution. Une plateforme propriétaire dont les mécanismes internes sont opaques offre une confiance par contrat, pas une confiance par architecture vérifiable.

Le second argument est la composabilité. Un OS agentique personnalisé peut charger un skill de revue de code PR pour les projets d'ingénierie, un hook de validation de conformité OSFI E-23 pour les flux financiers, et un sous-agent d'analyse de mémoire épisodique pour les sessions de rédaction longue — en une seule configuration YAML versionnée dans le dépôt. Une plateforme propriétaire délimite ce qu'il est possible de composer à l'intérieur de ses primitives. Le harnais personnalisé délimite uniquement ce que les outils sous-jacents permettent.

Le troisième argument est la gouvernance. Le Ch. 8 a posé la condition : « humans set rules, agents execute, exceptions escalate. » Un OS agentique personnalisé implémente cette condition au niveau du fichier de configuration — un `.claude/settings.json` commité dans le dépôt, auditable, versionné, déployable par IT via les paramètres gérés (*managed settings*). Le changement d'une règle de permission est un *commit* avec un historique traçable. Dans une plateforme propriétaire, ce changement est un clic dans une interface web dont l'auditabilité dépend des logs de la plateforme.

Compromis principal : l'OS personnalisé requiert un investissement initial plus élevé — comprendre les mécanismes, configurer les composantes, former les équipes — et une charge de maintenance continue. La plateforme propriétaire est opérationnelle en jours, non en semaines.

Condition qui renverse la recommandation en faveur de la plateforme : si l'organisation n'a pas de praticien capable de maintenir des configurations YAML et des scripts de hooks, ou si les exigences de conformité imposent

une certification de fournisseur que seule une plateforme certifiée peut fournir, la plateforme propriétaire est le choix correct. Ce chapitre s'adresse à l'organisation qui a choisi de construire.

14.2 — Les trois étapes d'assemblage

Avertissement de transparence : ces trois étapes sont une reconstruction documentaire depuis la documentation officielle Claude Code (code.claude.com/docs, mai 2026) et les sources académiques et communautaires citées en notes de recherche. Elles ne sont pas un transcript de la vidéo source. La structure en trois étapes est cohérente avec la logique d'accumulation des composantes dans la documentation officielle et avec les synthèses communautaires vérifiées.

Étape 1 — Poser le contexte et les skills : le cerveau déclaratif

La première étape consiste à donner à l'agent une identité persistante et un répertoire de comportements nommés. Elle produit trois artefacts versionnés : un fichier `CLAUDE.md`, une collection de skills, et un premier sous-agent spécialisé.

Le `CLAUDE.md` comme contrat organisationnel

Le fichier `CLAUDE.md` — chargé au démarrage de chaque session depuis la racine du projet, depuis `~/.claude/CLAUDE.md` (scope personnel) et depuis les niveaux encaissés (*nested*) selon la hiérarchie des scopes — est le point d'entrée de tout contexte persistant. Ce n'est pas une simple documentation : c'est l'instruction système de l'agent. Il doit contenir les éléments que l'architecte du Ch. 8 classerait comme règles non délégables : standards de code, conventions de sécurité, procédures d'escalade, périmètres d'autonomie, seuils de reversibilité.

Un `CLAUDE.md` efficace pour un OS agentique d'entreprise est dense, pas exhaustif. Il nomme les skills disponibles et leurs conditions d'invocation, liste les outils MCP actifs et leurs domaines d'application, et énonce les périmètres de permission (ce que l'agent peut faire seul, ce qui requiert confirmation humaine, ce qui est interdit). La cible pratique est sous 400 lignes (*hypothèse maison* — fondée sur les coûts de contexte typiques d'un Claude Sonnet 4.x avec corpus chargé en parallèle) ; au-delà, le coût de contexte constant devient prohibitif.

Les skills comme macros de comportement reproductible

Un *skill* dans Claude Code est un fichier `SKILL.md` placé dans `.claude/skills/<nom>/` (scope projet) ou `~/.claude/skills/<nom>/` (scope personnel), conforme au standard ouvert AgentSkills.io (*confirmé* — référencé dans la documentation officielle). La structure minimale est un frontmatter YAML suivi d'instructions en Markdown :

```
## ~/.claude/skills/code-review/SKILL.md
---
name: code-review
description: Revue de PR : sécurité, couverture, conventions. Invoquer quand l'utilisateur
demande une revue de code ou mentionne une PR.
disable-model-invocation: false
allowed-tools: Bash(gh pr *) Read Grep
context: fork
agent: Explore
---

Analyse la pull request $ARGUMENTS selon les critères suivants :
1. Sécurité : injection, exposition de secrets, surface d'attaque
2. Couverture de tests : identifier les chemins non couverts
3. Conventions : vérifier la conformité au style du projet

!`gh pr diff $ARGUMENTS`
!`gh pr view $ARGUMENTS --comments`

Produis un rapport structuré avec : risques critiques, avertissements, suggestions.
```

La ligne `context: fork` envoie l'exécution dans un sous-agent isolé avec un contexte propre — le résultat résumé revient dans la session principale sans polluer son fenêtre de contexte. C'est le pattern de composition fondamental de l'OS agentique : déléguer à un contexte isolé, récupérer le résumé.

Le frontmatter contrôle également qui peut invoquer le skill. `disable-model-invocation: true` réserve l'invocation à l'utilisateur — utile pour les skills à effets de bord (déploiements, envoi de courriels, commits). `user-invocable: false` réserve l'invocation au modèle — utile pour le contexte de fond (conventions d'un système hérité, politique de sécurité) que le modèle doit charger automatiquement mais que l'utilisateur n'invoquerait pas directement.

Étape 2 — Intégrer les MCP servers et les hooks : les nerfs et les réflexes

La deuxième étape connecte l'agent aux systèmes extérieurs (MCP) et implémente les politiques déterministes qui ne peuvent pas reposer sur le jugement du modèle (hooks).

MCP servers : résoudre le problème N×M

Le Ch. 5 a établi le rôle de MCP (*Model Context Protocol*) dans l'architecture agentique d'entreprise : remplacer N×M adaptateurs custom par N+M composantes standard. Dans un OS agentique personnalisé, les serveurs MCP se configurent à deux niveaux distincts.

Le niveau utilisateur (`~/.claude.json`) héberge les serveurs qui traversent tous les projets — GitHub, gestionnaire de mots de passe, base de connaissances interne :

```
{
  "mcpServers": {
    "github": {
      "command": "uvx",
      "args": ["mcp-server-github"],
      "env": { "GITHUB_TOKEN": "${GITHUB_TOKEN}" }
    },
    "memory": {
      "command": "node",
      "args": ["/usr/local/bin/mcp-memory-server.js"],
      "env": { "MEMORY_DB": "${HOME}/.claude/memory.db" }
    }
  }
}
```

Le niveau projet (`.mcp.json` à la racine du dépôt, commitable) héberge les serveurs spécifiques au projet — base de données de staging, API interne, registre de documents :

```
{
  "mcpServers": {
    "project-db": {
      "command": "npx",
      "args": ["-y", "@company/mcp-postgres", "--db", "${DATABASE_URL}"]
    }
  }
}
```

La séparation des scopes est une décision de gouvernance autant que technique. Un serveur MCP configuré au niveau projet peut être commité dans le dépôt — il passe par le processus de revue de code habituel, avec les règles de permission associées définies dans `.claude/settings.json`. Un serveur MCP au niveau utilisateur est sous la responsabilité du praticien individuel.

Hooks : les réflexes déterministes

Le Ch. 9 a identifié les *guardrails* comme couche défensive non négociable. Dans Claude Code, les hooks sont le mécanisme d'implémentation de ces guardrails au niveau du cycle de vie de l'agent — et leur nature déterministe (scripts shell, endpoints HTTP, outils MCP) est précisément ce qui les rend fiables là où le jugement du modèle ne l'est pas.

La documentation officielle (*confirmé*) liste 27 événements de cycle de vie. Les cinq événements à impact direct sur la sécurité sont `PreToolUse`, `PostToolUse`, `PostToolUseFailure`, `PermissionDenied`, et `PermissionRequest`.

La configuration se fait dans `.claude/settings.json` (scope projet) ou `~/.claude/settings.json` (scope personnel) :

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "\"$CLAUDE_PROJECT_DIR\"/.claude/hooks/validate-bash.sh",
            "timeout": 10
          }
        ]
      }
    ],
    "PostToolUse": [
      {
        "matcher": "Write|Edit",
        "hooks": [
          {
            "type": "command",
            "command": "jq -r '.tool_input.file_path' | xargs npx eslint --fix"
          }
        ]
      }
    ]
  }
}
```

Le script `validate-bash.sh` reçoit sur stdin un objet JSON décrivant l'outil invoqué (`tool_name`, `tool_input`, contexte de session). Il retourne via le code de sortie et stdout la décision : sortie `0` pour continuer, sortie `2` pour bloquer avec un message d'erreur. Cette interface stdin/stdout est le contrat formel entre l'agent et les guardrails — indépendant du modèle, exécutable par n'importe quel script shell.

Trois types de hooks valent d'être systématiquement configurés dans un OS agentique d'entreprise :

1. **Validation pré-exécution** (`PreToolUse` sur `Bash`) : détecter les commandes destructrices, interdire l'accès à des chemins hors périmètre, valider les paramètres d'invocation d'outils critiques.
2. **Qualité post-édition** (`PostToolUse` sur `Write|Edit`) : lancer automatiquement le linter ou le formatter après chaque modification de fichier — ce que le Ch. 7 nomme un *quality gate* déterministe.
3. **Contexte de session** (`SessionStart`) : charger des variables d'environnement, injecter du contexte de projet spécifique à l'environnement (URL de staging vs production, identifiants de ticket courant), initialiser les logs de session.

Étape 3 — Déployer, observer, gouverner : le plan de contrôle

La troisième étape est celle que les équipes sautent le plus souvent — et c'est précisément l'omission que le Ch. 12 identifie comme signal d'échec précoce. Déployer un OS agentique sans plan de contrôle, c'est livrer un agent sans AgentOps.

Le fichier `settings.json` comme manifeste de gouvernance

L'intégralité des règles de permission de l'OS agentique se concentre dans `settings.json`. La structure de cascade — *managed* (IT, inaltérable) → *local* (`.claude/settings.local.json`, ignoré de git) → *project* (`.claude/settings.json`, commitable) → *user* (`~/.claude/settings.json`, personnel) — implémente exactement le modèle *humans set rules, agents execute* du Ch. 8 : l'IT définit les règles inaltérables, le projet définit les règles d'équipe, le praticien affine pour son usage personnel.

Un `settings.json` de référence pour un OS agentique d'entreprise combine permissions granulaires, périmètre sandbox, et politique de hooks :

```
{
  "$schema": "https://json.schemastore.org/claude-code-settings.json",
  "model": "claude-sonnet-4-5",
  "effortLevel": "high",
  "permissions": {
    "allow": [
      "Bash(git diff *)",
      "Bash(git log *)",
      "Bash(git status)",
      "Bash(npm run test)",
      "Bash(npm run lint)",
      "Read(**)"
    ],
    "deny": [
      "Bash(git push --force *)",
      "Bash(rm -rf *)",
      "Read(./.env*)",
      "Read(./secrets/**)",
      "WebFetch(domain:*.internal-only.example.com)"
    ],
    "ask": [
      "Bash(git push *)",
      "Bash(npm publish *)"
    ],
    "defaultMode": "default"
  },
  "sandbox": {
    "enabled": true,
    "filesystem": {
      "denyRead": ["~/.aws/credentials", "~/.ssh/"],
      "allowWrite": ["/tmp/claude-workspace", "./"]
    }
  }
}
```

Plugins comme unité de distribution d'équipe

Dès que l'OS agentique doit s'étendre à une équipe, le plugin est la primitive de distribution. Un plugin Claude Code est un dépôt avec la structure suivante :

```
monorepo-workflow-plugin/
├── .claude-plugin/
│   └── plugin.json          # manifest : nom, version, description
├── skills/
│   ├── review-pr/
│   │   └── SKILL.md
│   └── release-notes/
│       └── SKILL.md
├── agents/
│   └── security-auditor.md  # sous-agent spécialisé sécurité
├── hooks/
│   └── hooks.json           # hooks d'équipe (lint, validation)
└── .mcp.json                # serveurs MCP du projet
```

L'installation par un membre de l'équipe se fait par `/plugin install github:org/monorepo-workflow-plugin`. La mise à jour est gérée par le versionnage sémantique dans `plugin.json`. Le responsable de l'OS agentique publie une nouvelle version du plugin à chaque évolution des politiques d'équipe — sans déploiement manuel sur chaque poste.

Observabilité agentique : la connexion avec AgentOps

Le Ch. 7 a établi les quatre catégories de spans agentiques (*LLM spans*, *tool spans*, *memory spans*, *orchestration spans*). Dans un OS agentique Claude Code, cette observabilité passe par deux mécanismes complémentaires.

Le premier est natif : les hooks `PostToolUse` peuvent écrire dans un journal structuré chaque invocation d'outil, avec les paramètres, le résultat, et la latence. Ce journal est le substrat de l'évaluation en production définie au Ch. 7 §7.5 — il permet de détecter les dérives de comportement, de mesurer le *tool correctness*, et de rejouer les sessions pour diagnostic.

Le second est externe : Claude Code exporte ses traces via OpenTelemetry quand l'environnement est configuré (`CLAUDE_CODE_ENABLE_TELEMETRY=1` dans `settings.json > env`). L'instrumentation s'appuie sur les conventions sémantiques GenAI OTel 1.40.0 (statut *Development* à mai 2026, attributs `gen_ai.agent.*` — *confirmé*, Ch. 7) pour les plateformes qui les supportent (Datadog, Grafana, Elastic).

14.3 — Cas concret : la monographie elle-même comme OS agentique

Ce chapitre a été rédigé dans un contexte qui illustre directement le patron qu'il décrit. La monographie *The Agentic Enterprise* a été produite via un harnais agentique dont les composantes correspondent précisément aux trois étapes du §14.2. La méta-référence n'est pas décorative — c'est un cas de validation du patron sur un projet documentaire réel, avec des contraintes de traçabilité, de cohérence terminologique et d'anti-fabrication directement analogues aux contraintes de conformité d'un projet d'entreprise.

Le `CLAUDE.md` comme contrat éditorial

Le fichier `CLAUDE.md` à la racine du dépôt encode l'intégralité du protocole éditorial : rôle et lectorat (architectes d'entreprise senior), hiérarchie des sources de vérité (documents du projet > sources primaires > sources secondaires), protocole de rédaction en quatre phases (cadrage → recherche Web → rédaction → vérification), format de sortie (Markdown GitHub-flavored, frontmatter YAML, Mermaid), et règles éditoriales (français canadien, marqueurs d'incertitude obligatoires, interdiction de fabrication). Ce fichier est l'équivalent du `CLAUDE.md` d'un projet logiciel — mais pour un flux de travail éditorial. À chaque session de rédaction d'un chapitre, l'agent charge ce contexte sans que l'opérateur ait à le rétablir manuellement.

Les chapitres comme skills de mémoire procédurale

Chaque chapitre précédent est un artefact de mémoire procédurale — une décision editoriale tranchée, versionnée, référençable. Les renvois croisés (`[Ch. 5](ch05-protocols-interoperability.md)`, `[Ch. 7](ch07-agentops.md)`) sont l'équivalent des invocations de skills : l'agent accède au contenu déjà validé plutôt que de régénérer un raisonnement depuis zéro. La résistance à la fabrication repose sur ce même mécanisme — les marqueurs d'incertitude (`*confirmé / probable / à vérifier / hypothèse / inconnu*`) encodent dans chaque chapitre l'état épistémique de chaque affirmation, rendant l'audit anti-fabrication vérifiable par une lecture simple.

Les sub-agents comme rédacteurs spécialisés

Le plan de production (fichier `plan.md`) décompose la monographie en tâches indépendantes déléguées à des sub-agents distincts : chaque sous-agent rédige un chapitre dans son propre contexte isolé, lit les fichiers requis, exécute la recherche Web, et retourne le fichier produit. L'orchestrateur (session principale) coordonne les vagues de production, valide les esquisses avant rédaction, et applique les hooks de cohérence terminologique. Ce patron est exactement celui du Ch. 6 — superviseur qui décompose, workers qui exécutent dans des contextes isolés, orchestrateur qui intègre les résultats.

La similitude n'est pas accidentelle. Le patron *superviseur + workers isolés + validation orchestrateur* est générique. Il s'applique indifféremment à la rédaction d'une monographie, à la revue de code dans un monorepo, à l'analyse de conformité multi-juridictions, ou à l'audit de sécurité multi-composantes. Ce que l'OS agentique personnalisé fournit, c'est le harnais qui rend ce patron reproductible, auditable et maintenable — pas le contenu métier.

14.4 — Limites, compromis, et quand basculer vers une plateforme propriétaire

```
graph TB
    CLAUDE_MD["CLAUDE.md\n(contexte persistant)"]
    SKILLS["Skills\n(comportements nommés)"]
    SUBAGENTS["Sub-agents\n(contextes isolés)"]
    MCP["MCP Servers\n(outils externes)"]
    HOOKS["Hooks\n(guardrails déterministes)"]
    PLUGINS["Plugins\n(distribution équipe)"]
    SETTINGS["settings.json\n(gouvernance)"]
    OTel["OpenTelemetry\n(observabilité)"]

    CLAUDE_MD --> SKILLS
    CLAUDE_MD --> SUBAGENTS
    SKILLS --> SUBAGENTS
    MCP --> SKILLS
    MCP --> SUBAGENTS
    HOOKS --> SKILLS
    HOOKS --> SUBAGENTS
    PLUGINS -->|bundle| SKILLS
    PLUGINS -->|bundle| HOOKS
    PLUGINS -->|bundle| SUBAGENTS
    SETTINGS -->|gouverne| HOOKS
    SETTINGS -->|gouverne| MCP
    SUBAGENTS --> OTel
    SKILLS --> OTel

    style SETTINGS fill:#d97757,color:#fff
    style HOOKS fill:#d97757,color:#fff
    style OTel fill:#3d7aed,color:#fff
```

Les compromis de l'OS personnalisé

Maintenance continue. Un OS agentique personnalisé est un produit logiciel, pas une configuration one-shot. Les skills évoluent avec les processus métier. Les hooks requièrent une mise à jour quand les règles de sécurité changent. Les serveurs MCP ont leurs propres cycles de versionnage. L'organisation qui ne budgète pas une demi-journée par semaine de maintenance pour son OS agentique verra sa configuration dériver en quelques mois — un anti-patron directement documenté au Ch. 12 sous « dette de configuration ».

Courbe d'apprentissage par composante. Les quatre mécanismes d'extensibilité de Claude Code (MCP, plugins, skills, hooks) opèrent à des coûts de contexte différents et répondent à des cas d'usage distincts — le choix entre skill inline et `context: fork`, entre hook `PreToolUse` et règle de permission dans `settings.json`, entre plugin distribué et configuration projet, requiert une compréhension de la sémantique de chaque mécanisme. L'arXiv 2604.14228v1 (*confirmé*) soulève explicitement ce compromis comme tension architecturale : « Simplicity vs. Extensibility — Four extension mechanisms provide flexibility but require users to understand which mechanism suits each use case. A unified API would simplify onboarding at the cost of reduced specialization. »

Sécurité par configuration. Les sept couches de permission de Claude Code sont robustes, mais leur robustesse dépend de la rigueur de la configuration. Une règle `allow` trop large dans `settings.json` neutralise plusieurs couches simultanément. Les vulnérabilités documentées de MCP — *tool poisoning*, injection via `sampling`, RCE supply chain dans les SDKs officiels (OX Security, avril 2026, *confirmé* — Ch. 5) — s'appliquent à l'OS personnalisé comme à toute autre utilisation de MCP. La sécurité agentique (Ch. 9) n'est pas résolue par le harnais — elle est implémentée par ses opérateurs.

Tableau comparatif des harnais

Composante	Claude Code	OpenAI Codex Agent	Cursor	Cline
Skills / macros	Skills YAML (`SKILL.md`, standard AgentSkills.io)	Aucun mécanisme natif équivalent	Règles `.cursorrules` (flat text)	Modes custom (flat text)
Sub-agents	Sub-agents + Agent Teams (fév. 2026)	Sandboxes cloud parallèles	Non natif	Non natif
MCP servers	Natif (Tier 1 : TS, Python, C#, Go)	Non natif à mai 2026	Non natif	Partiel via plugins
Hooks lifecycle	27 événements, 5 types (command, HTTP, MCP, prompt, agent)	Non disponible	Non disponible	Non disponible
Gouvernance versionnée	`settings.json` (4 scopes, schéma JSON officiel)	Paramètres d'organisation ChatGPT	`.cursorignore`, règles UI	`.clinerules`
Distribution équipe	Plugins (manifest JSON, marketplaces, npm/git)	Non disponible	Partage de templates	Non disponible
Exécution locale	Oui (CLI natif, Windows/macOS/Linux)	Optionnel (Codex CLI local)	Oui (IDE fork VS Code)	Oui (extension VS Code)
Exécution cloud	Oui (Routines, sessions web, iOS)	Oui (mode cloud primaire)	Non	Non
SWE-bench Verified	80,9 % (Anthropic, *confirmé*)	~80 % (OpenAI, *confirmé*)	N/A (IDE, non agent autonome)	N/A
Cas d'usage principal	OS agentique complet, codebase profonde, composabilité	Tâches autonomes asynchrones, PR sans surveillance	Développement interactif quotidien	Pair programming guidé

Sources : documentation officielle respective, arXiv 2604.14228v1, Jock.pl (2026, *probable*), NxCode (2026, *probable*).

Quand basculer vers une plateforme propriétaire

Quatre conditions rendent la plateforme propriétaire préférable à l'OS personnalisé :

Certification de conformité requise par contrat. Certains secteurs (santé, finance, défense) imposent que les outils traitant des données sensibles soient certifiés par des organismes reconnus. AWS Bedrock Agents et Azure AI Foundry disposent de certifications SOC 2, FedRAMP, HIPAA que Claude Code, en tant qu'outil CLI, n'est pas positionné à offrir comme produit certifié. Si un contrat ou un audit impose la certification du harnais lui-même, la plateforme propriétaire est le choix par défaut.

Absence de praticien de maintenance. L'OS agentique personnalisé requiert une compétence de configuration et de maintenance. Si l'organisation n'a pas de rôle équivalent à l'*AI ops manager* identifié au Ch. 11, le harnais dérivera inévitablement. La plateforme propriétaire externalise cette maintenance à son SLA.

Intégration native dans un écosystème hyperscaleur déjà choisi. Une organisation entièrement dans Azure (Azure DevOps, Azure Monitor, Entra ID, Azure OpenAI) bénéficiera d'intégrations natives qu'un harnais Claude Code ne peut pas répliquer sans travail d'intégration significatif. Le Ch. 10 documente le compromis : l'enfermement dans l'écosystème hyperscaleur est acceptable si l'organisation ne prévoit pas de porter ses agents hors de cet écosystème.

Déploiement à très grande échelle sans équipe d'ingénierie dédiée. Au-delà de quelques centaines d'utilisateurs simultanés, la gestion d'un parc de configurations `settings.json` et de plugins versionnés nécessite

des outils de déploiement et de monitoring qui rapprochent rapidement l'investissement du coût d'une plateforme. La frontière n'est pas fixe — elle dépend de la maturité DevOps de l'organisation — mais elle existe.

Pour aller plus loin

Anthropic / code.claude.com — Documentation officielle Claude Code — https://code.claude.com/docs — La source de référence unique pour toutes les composantes décrites dans ce chapitre. Les pages `skills`, `sub-agents`, `hooks`, `settings`, et `plugins` sont les cinq lectures prioritaires pour tout praticien qui assemble un OS agentique. La documentation est mise à jour en continu ; vérifier les changelogs avant tout déploiement.

Bhatt et al. — « Dive into Claude Code: The Design Space of Today's and Future AI Agent Systems » — arXiv 2604.14228v1 — avril 2026 — https://arxiv.org/html/2604.14228v1 — L'analyse systémique la plus complète disponible de l'architecture Claude Code à mai 2026. Les sections sur les treize principes de design, les sept couches de permission, et les six directions ouvertes sont les contributions les plus directement actionnables pour un architecte d'entreprise.

AgentSkills.io — Open Standard for Agent Skills — https://agentskills.io — La spécification ouverte à laquelle les skills Claude Code se conforment. La lecture de cette spec permet d'évaluer la portabilité des skills vers d'autres outils qui implémentent le même standard.

Anthropic — « Claude Code: Best Practices » — https://code.claude.com/docs/en/best-practices — Le guide pratique officiel pour la rédaction efficace de `CLAUDE.md`, la structuration des skills, et l'optimisation du coût de contexte. Complémentaire à ce chapitre pour l'implémentation opérationnelle.

OpenTelemetry — « Semantic Conventions for GenAI agent spans » — https://opentelemetry.io/docs/specs/semconv/gen-ai/gen-ai-agent-spans/ — La spec OTel GenAI SemConv 1.40.0 (statut *Development*, avril 2026) est le standard d'instrumentation à utiliser pour l'observabilité des OS agentiques. À suivre pour la stabilisation des attributs `gen_ai.agent.*`.

Références

1. Anthropic — « Extend Claude with skills » — Claude Code Docs — mai 2026 — https://code.claude.com/docs/en/skills — accédée le 2026-05-05
2. Anthropic — « Create custom subagents » — Claude Code Docs — mai 2026 — https://code.claude.com/docs/en/sub-agents — accédée le 2026-05-05
3. Anthropic — « Hooks reference » — Claude Code Docs — mai 2026 — https://code.claude.com/docs/en/hooks — accédée le 2026-05-05
4. Anthropic — « Settings reference » — Claude Code Docs — mai 2026 — https://code.claude.com/docs/en/settings — accédée le 2026-05-05
5. Anthropic — « Create plugins » — Claude Code Docs — mai 2026 — https://code.claude.com/docs/en/plugins — accédée le 2026-05-05
6. Anthropic — « Claude Code overview » — Claude Code Docs — mai 2026 — https://code.claude.com/docs/en/overview — accédée le 2026-05-05
7. Bhatt, D. et al. — « Dive into Claude Code: The Design Space of Today's and Future AI Agent Systems » — arXiv 2604.14228v1 — avril 2026 — https://arxiv.org/html/2604.14228v1 — accédée le 2026-05-05
8. AgentSkills.io — « Agent Skills Open Standard » — https://agentskills.io — accédée le 2026-05-05
9. OpenTelemetry — « Semantic Conventions for GenAI agent and framework spans » — OTel SemConv 1.40.0 — avril 2026 — https://opentelemetry.io/docs/specs/semconv/gen-ai/gen-ai-agent-spans/ — accédée le 2026-05-05
10. OX Security — « The Mother of All AI Supply Chains: Critical, Systemic Vulnerability at the Core of Anthropic's MCP » — avril 2026 — https://www.ox.security/blog/the-mother-of-all-ai-supply-chains-critical-systemic-vulnerability-at-the-core-of-the-mcp/ — accédée le 2026-05-05
11. Jock.pl — « Claude Code vs Codex CLI vs Aider vs OpenCode vs Pi vs Cursor: Which AI Coding Harness Actually Works Without You? » — 2026 — https://thoughts.jock.pl/p/ai-coding-harness-agents-2026 — accédée le 2026-05-05

12. alexop.dev — « Understanding Claude Code's Full Stack: MCP, Skills, Subagents, and Hooks Explained » — 2026 — https://alexop.dev/posts/understanding-claude-code-full-stack/ — accédée le 2026-05-05
13. MindStudio — « How to Build an Agentic Operating System with Claude Code » — 2026 — https://www.mindstudio.ai/blog/how-to-build-agentic-operating-system-claude-code — accédée le 2026-05-05

Annexe A — Liste de contrôle de revue d'architecture

Annexe A · Architecture Review Checklist · ~1 500 mots · lecture ≈ 6 min

Artefact opérationnel destiné à l'architecte d'entreprise ou de solution qui doit valider — ou bloquer — le déploiement d'un système *agentic*. Position dans le cycle d'adoption : après le cadrage (Ch. 3/4) et la conception de la pile (Ch. 5/6), avant le déploiement opérationnel (Ch. 7). Structure le dialogue entre l'équipe produit, l'architecte, la sécurité et le bureau de risque.

Chaque item est booléen ou à seuil quantitatif. Un item non satisfait exige une décision de risque documentée avec délai d'assainissement. Les items **[bloquant]** sont des conditions minimales sans dérogation : leur absence est une non-conformité architecturale.

Section 1 — Sécurité

Source : Ch. 9 §9.1–9.4 — Taxonomie OWASP ASI01–ASI10, défense en profondeur.

1.1 Modèle de menace

- ☐ **[bloquant]** Un modèle de menace formel couvrant les 10 classes ASI (OWASP GenAI Security Project, déc. 2025) a été produit et révisé par l'équipe sécurité.
- ☐ Les trois familles de vecteurs sont adressées distinctement : injection/détournement d'objectifs (ASI01, ASI02, ASI06), délégation/chaînes de confiance (ASI03, ASI07, ASI08, ASI09), supply chain et exécution (ASI04, ASI05, ASI10).
- ☐ Les contenus ingérés (documents, courriels, résultats d'outils, pages web) sont traités comme non fiables — *LLM Scope Violation* (CVE-2025-32711, CVSS 9.3) documentée comme risque résiduel si aucune séparation d'espace de confiance n'est en place.

1.2 Identité d'agent

- ☐ **[bloquant]** Chaque agent possède une identité distincte — pas de clé API partagée entre agents. Exigence OSFI E-23 (inventaire des modèles, en vigueur mai 2027).
- ☐ Identité vérifiable : *Federated Identity Credentials* (Entra Agent ID), rôles IAM fins (AgentCore Identity), ou SPIFFE/SVID pour les flux M2M.
- ☐ Cycle de vie de l'identité (création, rotation, révocation) documenté et automatisé.

1.3 Scoped tokens et moindre privilège

- ☐ **[bloquant]** Tokens d'accès émis par tâche (*per-task token*) — pas de token de session longue partagé.
- ☐ *Tool-level scopes* OAuth 2.1 révocables indépendamment par outil (ex. : lecture contacts ≠ création contacts) ; *scope narrowing* RFC 8693 à chaque hop de délégation.
- ☐ Aucun agent worker ne dispose de permissions supérieures à sa sous-tâche.

1.4 Guardrails

- ☐ Guardrails I/O déployés en entrée et sortie (Llama Guard 4, NeMo Guardrails, Anthropic Constitutional Classifiers, Azure AI Content Safety + Prompt Shield, ou AWS Bedrock Guardrails).
- ☐ Limitation documentée au dossier risque : taux de contournement > 85 % en attaque adaptative isolée (Ch. 9 §9.3) — les guardrails seuls ne constituent pas une défense suffisante.
- ☐ **N3-N4** : stratégie de composition (guardrails + sandboxing + kill switches) spécifiée — les trois couches sont obligatoires.

1.5 Sandboxing

- ☐ Niveau d'isolation minimal selon le niveau d'autonomie :
  - N1-N2 : conteneur Docker acceptable.
  - N3 : gVisor ou équivalent.
  - **N4 [bloquant]** : Firecracker microVM (démarrage ≤ 150 ms) ou Kata Containers.
- ☐ Rotation systématique du sandbox après chaque tâche — pas de session longue durée accumulant l'état.

1.6 Kill switches

- ☐ **[bloquant]** Quatre modes de désactivation testés : par agent individuel, par outil, par périmètre de données, *dry-run mode* (lecture seule pour investigation forensique).
- ☐ Autorité d'activation et délais cibles documentés dans le RACI agentique (Annexe D).

1.7 Surface protocolaire

- ☐ MCP : serveurs tiers sur liste blanche vérifiée ; descriptions d'outils auditées contre le *tool poisoning* ; primitive *sampling* désactivée ou isolée (Ch. 5 §5.8).
- ☐ A2A : mTLS ou OAuth 2.0 actif sur tous les canaux inter-agents ; Agent Cards signées (Ch. 5 §5.3).
- ☐ Inventaire exhaustif des serveurs MCP et agents A2A déployés — aucun agent hors inventaire ne peut opérer (ASI10).

Section 2 — Observabilité

Source : Ch. 7 §7.2–7.5 — Arbre de spans OTel, métriques canoniques, évaluation en production.

2.1 OTel GenAI spans

- ☐ **[bloquant]** Quatre catégories de spans instrumentées : LLM spans, tool spans, memory spans, orchestration spans. Observabilité limitée aux outputs seuls : insuffisante — cause directe de l'incident Replit (juillet 2025 : 1 206 enregistrements supprimés malgré gel explicite).
- ☐ Attributs OTel SemConv 1.40.0 (`gen_ai.agent.id`, `gen_ai.agent.name`, `gen_ai.agent.version`) avec opt-in `OTEL_SEMCONV_STABILITY_OPT_IN=gen_ai_latest_experimental` ; spec en statut *Development* — abstraire derrière une bibliothèque interne.
- ☐ Plateforme d'observabilité supportant les traces multi-étapes avec arbre de spans hétérogènes.

2.2 Traces multi-étapes

- ☐ Chaque session tracée de bout en bout avec identifiant unique permettant la reconstruction complète de la trajectoire de décision.
- ☐ Tool spans : nom de l'outil, paramètres sérialisés, résultat/erreur, code de statut, retry count, timestamps — socle de *tool correctness*.
- ☐ Orchestration spans : décisions de délégation (sous-agent activé, sous-objectif, résultat retourné).

2.3 Memory diffs et eval gates

- ☐ Memory spans couvrant récupérations et écritures (épisodique, sémantique, procédurale) avec scores de pertinence ; *memory diff* calculé et archivé par session (Ch. 6 §6.5).
- ☐ CI/CD gate bloquant les déploiements si les cinq métriques canoniques passent sous seuil : *task success*, *tool correctness*, *retry budget*, *escalation quality*, *policy compliance*.
- ☐ Seuils de dérive comportementale sur fenêtres glissantes de 7 jours définis et monitorés — pas seulement des règles d'alerte instantanée.

2.4 Replay et shadow runs

- ☐ *Replay* déterministe (golden file testing) disponible pour les sessions historiques.
- ☐ *Shadow runs* (nouvelle version en parallèle de production sans exposition utilisateur) supportés pour les promotions à fort risque.

Section 3 — FinOps

Source : Ch. 2 §2.3 et Ch. 7 §7.6 — Unit economics, retry budget, escalation cost.

3.1 Retry budget et escalation cost

- ☐ **[bloquant]** *Retry budget* maximal défini par agent et par session, instrumenté dans le plan de contrôle (Ch. 7 §7.6) et visible dans les tool spans. Dépassement au-delà de P95 → alerte, pas seulement log.
- ☐ *Escalation cost* mesuré séparément du coût d'inférence nominal ; seuil par session défini — les dépassements sont examinés systématiquement.

3.2 Kill switches budgétaires et alerting

- ☐ Kill switch budgétaire configuré : dépassement du budget mensuel (tokens + appels API) déclenche suspension automatique ou mode dégradé ; budget révisé après chaque incident de coût non anticipé.
- ☐ Alerting distinguant dégradation aiguë (retry rate > seuil sur 1h) et dérive lente (task success rate −5 % sur 7 jours glissants).
- ☐ Tableau de bord FinOps (coût par tâche réussie, coût d'escalade, coût de retry) distinct du tableau de bord qualité.

Section 4 — Conformité

Source : Ch. 8 §8.3–8.4 — EU AI Act, ISO/IEC 42001, NIST AI RMF, OSFI E-23, Loi 25.

4.1 EU AI Act (règlement UE 2024/1689)

- ☐ Qualification documentée : haute-risque (Art. 6, Annexe III — crédit, emploi, infrastructure critique, services essentiels), GPAI, ou hors périmètre.
- ☐ Systèmes haute-risque : surveillance humaine effective (Art. 14) démontrée architecturalement — log seul insuffisant en N4 (arXiv:2604.04604, avril 2026) ; dispositif HITL opérationnel requis.
- ☐ Obligations GPAI (Art. 52-55) vérifiées si modèle à usage général : transparence, marquage des contenus synthétiques. Délai haute-risque : **2 août 2026**.

4.2 ISO/IEC 42001 et NIST AI RMF

- ☐ Système *agentic* couvert par le périmètre AIMS (ISO/IEC 42001:2023) de l'organisation ou extension de périmètre documentée ; exigences d'audit tierce partie (ISO/IEC 42006:2025) anticipées.
- ☐ 12 risques GenAI (NIST AI 600-1, juillet 2024) évalués et actions de gestion documentées.
- ☐ Inventaire des agents conforme au concept paper NCCoE/NIST (fév. 2026) : ID, propriétaire, version, périmètre autorisé, limitations.

4.3 OSFI E-23 et Loi 25 Québec

- ☐ [**bloquant pour IFFés**] Inventaire des modèles incluant tous les agents *agentic* avec métadonnées E-23 : identifiant, propriétaire, version, date de déploiement, cote risque, usages approuvés, limitations, dépendances (modèles et données tiers via Ligne directrice B-10). Conformité : **1er mai 2027**.
- ☐ Agents N3-N4 : contrôles alternatifs documentés pour modèles « boîte noire » — exigence E-23 explicite pour systèmes à prise de décision autonome.
- ☐ Loi 25 art. 12.1 : si décision exclusivement automatisée affectant une personne → obligation d'information satisfaite, logique explicable, droit de révision humaine opérationnel. Pénalité max : 4 % des revenus mondiaux ou 25 M CAD.

4.4 Decision logs et justifiable actions

- ☐ [**bloquant**] Journaux de décision immuables (append-only, signés), horodatés, couvrant : déclencheur, raisonnement intermédiaire, outil invoqué, résultat, agent ayant agi, périmètre de permission actif.
- ☐ Chaque action irréversible justifiable depuis le journal seul, sans rejouer la session (Ch. 8 §8.3, FINOS Governance Framework MI-21).
- ☐ Durée de conservation alignée sur les exigences applicables (EU AI Act, E-23, Loi 25).

Mode d'emploi

Qui : architecte d'entreprise ou de solution, avec CISO (ou délégué) et bureau de risque. IFFés : validation additionnelle par la conformité OSFI.

Quand : avant tout déploiement en production N2 ou supérieur. Systèmes N1 (lecture seule, aucun effet de bord) : sections 1.5, 1.6, 3.1–3.2 optionnelles.

1. Remplir section par section avec l'équipe technique en session de travail — ne pas déléguer sans revue architecturale.
2. Chaque item non satisfait → décision documentée : acceptation du risque (date de réévaluation), plan d'assainissement (délai), ou blocage.
3. Items [**bloquant**] non satisfaits → déploiement suspendu jusqu'à résolution ou décision explicite cosignée CISO + architecte responsable.
4. Checklist archivée avec le dossier d'architecture — pièce justificative pour audits EU AI Act et OSFI E-23.
5. **Réévaluation** : à chaque changement sur l'une des cinq composantes de l'artefact composite (prompt système, outils avec versions, configuration mémoire, périmètre de permission, seuils d'escalade).

Renvois croisés

Sujet	Référence
Surface d'attaque protocolaire MCP/A2A	Ch. 5 §5.3, §5.8
Plan de contrôle AgentOps (kill switches, retry budget)	Ch. 7 §7.6
Métriques canoniques et évaluation en production	Ch. 7 §7.4–7.5
Niveaux d'autonomie N1-N4 et EU AI Act	Ch. 8 §8.1, §8.4
Taxonomie OWASP ASI01–ASI10, défense en profondeur	Ch. 9 §9.2–9.4
Identité vérifiable d'agent	Ch. 9 §9.4
Maturity model AgentOps	Annexe C
RACI agentique (autorité kill switches)	Annexe D

Annexe B — Use Case Canvas

Annexe B · Use Case Canvas · ~1 500 mots · lecture ≈ 6 min

Ce canvas est l'artefact opérationnel qui instancie, pour un cas d'usage unique, le double filtre décrit au Ch. 3 (matrice autonomie × réversibilité × tolérance à l'erreur) et au Ch. 4 (CPST, métriques canoniques, readiness 4D, décision Build/Buy/Borrow/Wait). Il se remplit avant tout engagement de ressources de développement — l'équivalent d'un business case d'une page, mais structuré pour les propriétés spécifiques des systèmes *agentic*.

Trois rôles sont requis pour compléter le canvas : le **Product Owner (PO)** est responsable des sections 1, 3 et 6 (description, ROI, métriques) ; l'**architecte de solution** est responsable des sections 2 et 4 (cadrage 3D, piliers readiness) ; le **risk officer** co-signe la section 5 (décision quadrant et recommandation). Le canvas est réexaminé à chaque changement de volume mensuel, de version de modèle, ou de modification du périmètre d'outils — fréquence minimale recommandée : trimestrielle. Un canvas non révisé après six mois devient caduque : les seuils de métriques calibrés à l'initialisation ne reflètent plus la distribution réelle des inputs en production.

Section 1 — Description du cas

Champ	Valeur
Intitulé	`[Nom court du cas d'usage – ex. : « Agent rapprochement AP »]`
Fonction métier	`[Finance / Opérations / Service client / Engineering / Juridique / RH / Autre]`
Propriétaire business	`[Nom · Titre · Direction]`
Propriétaire technique	`[Nom · Titre · Équipe]`
Description en une phrase	`[L'agent fait X sur Y pour produire Z, dans le système S]`
Hypothèse de gain	`[Réduction de coût / Augmentation de revenu / Réduction de délai / Réduction de risque – choisir le primaire, quantifier si possible]`
Processus cible documenté ?	`[ ] Oui – référence doc : ___` · `[ ] Non → Stop : readiness D2 insuffisante (Ch. 4 §4.5)`

Section 2 — Cadrage 3D

2.1 Autonomie cible

Référence : niveaux N1–N4 du Ch. 8 §8.1.

Niveau	Définition opérationnelle	Sélectionner
N1 — Assistance	L'agent suggère uniquement ; l'humain décide et agit	`[ ]`
N2 — Supervisé	L'agent exécute sur les cas nominaux ; l'humain approuve avant toute action à effet de bord	`[ ]`
N3 — Autonomie bornée	L'agent exécute de manière autonome dans un périmètre d'actions défini et avec un *kill switch* financier actif	`[ ]`
N4 — Autonomie étendue	L'agent opère sans supervision intermédiaire ; escalade seulement sur exception ou dépassement de budget	`[ ]`

Autonomie cible retenue : `[ ] N1` · `[ ] N2` · `[ ] N3` · `[ ] N4`

Justification (pourquoi ce niveau est le minimum nécessaire pour atteindre la valeur métier, et pourquoi monter d'un niveau serait injustifié) :

`[Texte libre — 2 à 4 phrases. Citer la politique de supervision ou la contrainte réglementaire si applicable — OSFI E-23, EU AI Act, Loi 25]`

2.2 Réversibilité de l'action

La réversibilité mesure si les effets de bord des actions de l'agent peuvent être annulés sans perte de valeur métier nette (Ch. 3 §3.2).

Niveau	Critère	Exemples typiques	Sélectionner
Haute	Toutes les actions sont annulables sans impact externe	Lecture seule, classification, brouillon non envoyé, écriture en base de staging	`[ ]`
Moyenne	Certaines actions sont partiellement annulables avec un coût de compensation	Courriel interne envoyé, réservation confirmée mais modifiable, écriture validée sans clôture	`[ ]`
Basse	Les actions produisent des effets irréversibles ou très coûteux à compenser	Transfert financier exécuté, courriel externe envoyé à un client, suppression de données de production, licenciement	`[ ]`

Réversibilité retenue : `[ ] Haute` · `[ ] Moyenne` · `[ ] Basse`

Exemples d'actions irréversibles identifiées dans ce cas :

`[Lister les 2 à 5 actions concrètes les plus irréversibles que l'agent pourrait déclencher]`

2.3 Tolérance à l'erreur

La tolérance à l'erreur mesure les conséquences métier, réglementaires et réputationnelles d'une erreur non détectée avant impact externe (Ch. 3 §3.2).

Niveau	Critère	Impact en cas d'échec non détecté	Sélectionner
Haute	Erreur détectable et corrigible avant tout impact externe	Révision manuelle possible, aucune conséquence irréversible	`[ ]`
Moyenne	Erreur partiellement détectable ; impact limité et compensable	Retouche client nécessaire, pénalité de délai, incident de service mineur	`[ ]`
Basse	Erreur difficilement détectable ; conséquences directes en termes réglementaires, financiers ou réputationnels	Amende réglementaire, perte client, incident de sécurité, données de production corrompues	`[ ]`

Tolérance à l'erreur retenue : `[ ] Haute` · `[ ] Moyenne` · `[ ] Basse`

Impact maximal d'un échec non détecté :

`[Description en 1 à 2 phrases de l'impact le plus grave plausible — quantifier si possible]`

Section 3 — ROI prévisionnel

Référence : *Cost per Successful Task* (CPST) défini au Ch. 2 §2.1.

Paramètre	Valeur estimée	Base de l'estimation
CPST cible (coût par tâche réussie incluant inférence, orchestration, retry moyen, escalade attendue)	`___ $`	`[Benchmark interne / Pilot data / Estimation analogique]`
CPST humain de référence (coût actuel de la même tâche traitée par un opérateur humain ou par RPA)	`___ $`	`[Données RH / Finance / Fournisseur RPA actuel]`
Volume mensuel attendu (nombre de tâches par mois à maturité)	`___ tâches/mois`	`[Données historiques système source]`
Taux de succès qualitatif cible (seuil minimal acceptable — cf. Ch. 4 §4.2 : 85 % en zone verte, *hypothèse maison*)	`___ %`	`[Aligné avec direction métier]`
Économie nette mensuelle (CPST humain − CPST cible) × volume × taux de succès	`___ $/mois`	`[Calcul automatique à compléter]`
Coût de build / intégration (incluant instrumentation eval + observabilité)	`___ $`	`[Estimation équipe + licences plateforme]`
Horizon de retour (mois pour amortir le coût de build sur l'économie nette)	`___ mois`	`[= Coût build / Économie nette mensuelle]`

Conditions de bascule ROI : Si le taux de succès qualitatif mesuré en production est inférieur de plus de 10 points au taux cible, le CPST réel dépasse le CPST humain de référence et la décision de déploiement doit être réexaminée (Ch. 4 §4.4).

Section 4 — Readiness 4D

Référence : cadre readiness du Ch. 4 §4.5. Score 1 à 5 par dimension. Seuils décisionnels : ≥ 15/20 → déployer ; 10–14/20 → plan de remédiation ; < 10/20 → Wait.

D1 — Données

Score	Critère
1	Données sources sans lignage documenté, sans SLA de fraîcheur défini
2	Lignage partiel, fraîcheur non contractualisée, pas de pipeline actif
3	Lignage documenté sur les sources primaires, fraîcheur contractualisée mais non monitorée
4	Pipelines actifs, lignage *end-to-end*, conformité Loi 25 / OSFI E-23 vérifiée
5	Pipelines certifiés, portabilité et interopérabilité confirmées

Score D1 : `___/5` · **Lacunes identifiées :** `[Texte libre]` · **Plan de remédiation :** `[Actions + délais]`

D2 — Processus

Score	Critère
1	Processus actuel non documenté ; cas de bord non listés
2	Processus actuel documenté ; processus *cible* non défini
3	Processus cible documenté ; exceptions formalisées pour les cas nominaux seulement
4	Processus cible documenté avec exceptions exhaustives ; critères de succès binaires définis
5	Processus optimisé (étapes éliminées, pas seulement automatisées) ; critères de succès qualitatifs définis et validés avec les parties prenantes

Score D2 : `___/5` · **Lacunes identifiées :** `[Texte libre]` · **Plan de remédiation :** `[Actions + délais]`

D3 — Talents

Score	Critère
1	Aucune compétence *agentic* interne ; dépendance totale au fournisseur
2	Compétences LLM génériques (prompt engineering) ; pas de capacité eval ni d'observabilité agentique
3	Capacité à écrire des eval suites sur cas nominaux ; instrumentation OTel basique
4	Eval suites sur distribution réelle (cas de bord inclus) ; on-call *agentic* opérationnel
5	Responsable AgentOps identifié ; capacité *promote/deprecate/rollback* maîtrisée (Ch. 7)

Score D3 : `___/5` · **Lacunes identifiées :** `[Texte libre]` · **Plan de remédiation :** `[Actions + délais]`

D4 — Gouvernance

Score	Critère
1	Aucun artefact de gouvernance ; pas de définition du *successful outcome*
2	*Successful outcome* défini en termes nominaux uniquement (succès binaire, pas qualitatif)
3	*Successful outcome* qualitatif défini ; *eval suite* automatisée sur cas nominaux ; *escalation policy* esquissée
4	Trois artefacts Databricks complets (*successful outcome* qualitatif, *eval suite* couvrant cas de bord, *escalation policy* documentée avec seuils chiffrés)
5	Trois artefacts Databricks + *retry budget policy* avec *kill switch* financier + RACI superviseur documenté (cf. Annexe D)

Score D4 : `___/5` · **Lacunes identifiées :** `[Texte libre]` · **Plan de remédiation :** `[Actions + délais]`

Tableau récapitulatif readiness

Dimension	Score	Seuil minimal	Statut
D1 — Données	`___/5`	3/5	`[ ] OK` · `[ ] Remédiation` · `[ ] Bloquant`
D2 — Processus	`___/5`	3/5	`[ ] OK` · `[ ] Remédiation` · `[ ] Bloquant`
D3 — Talents	`___/5`	3/5	`[ ] OK` · `[ ] Remédiation` · `[ ] Bloquant`
D4 — Gouvernance	`___/5`	3/5	`[ ] OK` · `[ ] Remédiation` · `[ ] Bloquant`
Total	`___/20`	≥ **15** pour déployer	`[ ] Déployer` · `[ ] Plan de remédiation` · `[ ] Wait`

Section 5 — Décision quadrant

La matrice suivante croise l'autonomie cible (N1–N4, lue comme axe d'autonomie croissante) et la réversibilité (haute / moyenne / basse). La tolérance à l'erreur modifie le quadrant : une tolérance basse décale d'un quadrant vers le rouge indépendamment de la position autonomie × réversibilité.

	Réversibilité haute	Réversibilité moyenne	Réversibilité basse
Autonomie N1–N2	**Quadrant Sûr** — déployer avec garde-fous standard	**Quadrant Supervisé** — garde-fous renforcés requis	**Quadrant Risqué** — validation architecture obligatoire
Autonomie N3	**Quadrant Supervisé** — garde-fous renforcés requis	**Quadrant Risqué** — validation architecture obligatoire	**Quadrant Interdit** — réduire une dimension avant déploiement
Autonomie N4	**Quadrant Risqué** — validation architecture obligatoire	**Quadrant Interdit** — réduire une dimension avant déploiement	**Quadrant Interdit** — réduire une dimension avant déploiement

Quadrant résultant : `[ ] Sûr` · `[ ] Supervisé` · `[ ] Risqué` · `[ ] Interdit`

Ajustement tolérance à l'erreur basse : `[ ] Applicable — décaler d'un quadrant vers Interdit`

Quadrant final après ajustement : `[ ] Sûr` · `[ ] Supervisé` · `[ ] Risqué` · `[ ] Interdit`

Recommandation kill / pivot / scale

Signal	Condition	Action
Kill	Quadrant Interdit sans possibilité de réduction de dimension, ou score readiness $< 10/20$ sans feuille de route de remédiation crédible	Abandonner le cas d'usage ; réallouer les ressources vers un cas en quadrant Sûr
Pivot	Quadrant Risqué ou Interdit, mais réductible par contrainte de l'autonomie (N4 $\rightarrow$ N2) ou par périmètre d'outils (exclusion des actions irréversibles)	Redéfinir le périmètre d'autonomie ou d'outils, refaire le cadrage 3D, réévaluer le quadrant
Scale	Quadrant Sûr ou Supervisé avec score readiness $\geq 12/20$ et plan de remédiation documenté pour les dimensions $< 3/5$	Engager le déploiement pilote zone verte ; programmer la revue trimestrielle

Recommandation retenue : `[ ] Kill` · `[ ] Pivot` · `[ ] Scale`

Justification et conditions de bascule :

```
[En 2 à 4 phrases : pourquoi cette recommandation, quel événement ou résultat la ferait
changer]
```

Décision Build / Buy / Borrow / Wait (référence Ch. 4 §4.4) :

`[ ] Build` · `[ ] Buy` · `[ ] Borrow` · `[ ] Wait` — Justification : `[1 phrase]`

Section 6 — Métriques de validation

Référence : cinq métriques canoniques du Ch. 4 §4.2. Les seuils ci-dessous sont des hypothèses de travail de cette monographie ; les ajuster au contexte métier lors du remplissage.

Métrique	Définition pour ce cas	Seuil d'acceptation minimal	Fréquence de mesure	Responsable
Task success rate (qualitatif)	`[Définir le critère de succès qualitatif pour ce cas — ex. : facture réconciliée sans erreur de montant > 0,01 $]`	`≥ 85 %` (*hypothèse maison — à ajuster*)	Quotidienne en production	PO
Tool correctness	`[Définir les outils critiques et les paramètres clés à valider — selection accuracy + parameter accuracy séparément]`	`≥ 90 % selection / ≥ 88 % parameter` (*hypothèse maison*)	Par tâche, agrégé hebdomadaire	Architecte
Retry budget	`[Définir le budget maximum de retries par tâche et le seuil d'alerte sur la distribution — ex. : budget 5 retries, alerte si > 20 % des tâches consomment > 3 retries]`	`< 20 % des tâches dépassent 50 % du budget` (*seuil Ch. 4 §4.2*)	Par tâche, agrégé quotidien	Architecte
Escalation quality	`[Définir les critères de déclenchement d'escalade, le type de routage attendu, et la qualité minimale du contexte transmis]`	`Trigger accuracy ≥ 90 % / Context quality ≥ 80 %` (*hypothèse maison*)	Par escalade, agrégé hebdomadaire	Risk officer
Policy compliance	`[Définir les politiques applicables — périmètre topical, contraintes données, exigences réglementaires sectorielles OSFI E-23 / Loi 25 si applicable]`	`100 % — toute violation est un incident P1`	Par tâche, alertes temps réel	Risk officer

Instrumentation recommandée : OTel GenAI Agent Spans (statut *experimental* à mai 2026 — Ch. 4 §4.3) comme couche de base ; plateforme d'évaluation branchée par-dessus selon le profil de conformité de l'organisation.

Mode d'emploi

Étape 1 — Initialisation (avant la décision d'investissement). Le PO complète la Section 1 et une première estimation de la Section 3. L'architecte complète les Sections 2 et 4. Le risk officer co-signe la Section 5. Durée estimée : 2 à 4 heures pour un cas bien connu de l'équipe.

Étape 2 — Porte de qualification. Si le quadrant final est Interdit (Section 5) ou si le score readiness est $< 10/20$ (Section 4), la décision est Kill ou Pivot avant tout engagement de ressources. Aucune exception sans validation de la direction.

Étape 3 — Pilote zone verte. Pour les cas Sûr ou Supervisé, déployer un pilote limité (volume réduit, population test définie) et mesurer les cinq métriques de la Section 6 pendant 30 à 60 jours. Les seuils d'acceptation ne sont pas négociables en cours de pilote ; s'ils ne sont pas atteints, la décision est Pivot, pas extension du pilote.

Étape 4 — Revue trimestrielle. Relire le canvas complet tous les trois mois ou à chaque changement matériel (version de modèle, volume $+/-$ 30 %, nouveau périmètre d'outils). Les seuils de métriques, le CPST projeté et le score readiness doivent être recalibrés sur les données de production réelles. Un canvas non révisé est invalide.

Étape 5 — Décision scale. Si les métriques de la Section 6 sont atteintes sur au moins deux cycles de revue consécutifs et si le score readiness est $\geq 15/20$, engager le passage à l'échelle avec les garde-fous documentés dans la Section 5. Référencer l'Annexe C pour évaluer la maturité AgentOps avant tout passage à l'échelle.

Renvois croisés

- Ch. 3 §3.2 — Matrice autonomie × réversibilité × tolérance à l'erreur (fondement des Sections 2 et 5)
- Ch. 4 §4.2 — Cinq métriques canoniques (fondement de la Section 6)
- Ch. 4 §4.4 — Décision Build/Buy/Borrow/Wait (Section 5)
- Ch. 4 §4.5 — Readiness 4D et scoring (fondement de la Section 4)
- Ch. 8 §8.1 — Niveaux d'autonomie N1–N4 (fondement de la Section 2.1)
- Annexe C — Modèle de maturité AgentOps (évaluation post-déploiement complémentaire)
- Annexe D — Matrice RACI agentique (RACI superviseur Section 5)

Annexe C — Modèle de maturité AgentOps (N1–N5)

Annexe C · AgentOps Maturity Model · ~1 500 mots · lecture ≈ 6 min

Cadrage

Ce modèle de maturité permet à une équipe ou à une organisation d'évaluer sa capacité opérationnelle réelle à exploiter des agents en production — non pas sa capacité à en déployer, mais à les opérer de façon contrôlée, résiliente et économiquement viable. Il repose sur les quatre composantes AgentOps définies au glossaire et développées en profondeur au Ch. 7 : (1) observabilité multi-étapes, (2) évaluation continue, (3) cycle de vie de l'artefact composite, (4) plan de contrôle.

L'auto-évaluation est conduite par l'architecte de solution ou le directeur de programme responsable du portefeuille agentique, idéalement en début de programme (base de référence) et à chaque montée en charge significative (nouveau périmètre, nouveau cas d'usage, ou incident en production). Le résultat n'est pas un score unique — c'est un profil par composante, dont les déséquilibres sont plus informatifs que la moyenne.

Les 5 niveaux de maturité AgentOps

N1 — Ad hoc : déploiement sans infrastructure opérationnelle

Caractéristiques observables

1. Aucune instrumentation de spans : les seules traces disponibles sont les logs applicatifs bruts ou les sorties finales de l'agent.
2. Évaluations ponctuelles ou absentes : les tests sont exécutés manuellement avant le déploiement initial et ne se répètent pas.
3. Versionnage partiel ou inexistant : le prompt système est versionné dans le dépôt ; les outils, la configuration mémoire et les permissions ne le sont pas.
4. Aucun plan de contrôle formalisé : pas de *retry budget*, pas de *rate limit*, pas de *kill switch* connu de l'équipe.
5. Coûts mesurés en tokens uniquement : le CPST (*Cost per Successful Task*) n'est pas calculé.

Anti-patrons typiques de ce niveau

L'équipe sait que l'agent opère parce qu'il produit des sorties visibles — e-mails envoyés, enregistrements créés. Elle ne sait pas comment il y arrive. Un changement de comportement n'est détecté que par un utilisateur final qui signale un problème. L'incident Replit de juillet 2025 — 1 206 enregistrements supprimés malgré une instruction de gel explicite — est l'illustration documentée de ce niveau (*confirmé* — Fortune, 23 juillet 2025 ; Ch. 7 §7.8).

Critères de bascule vers N2

- Au moins les *tool spans* sont instrumentés et stockés de façon interrogeable.
- Un artefact composite est défini comme unité versionnée (même si le pipeline de déploiement n'en dépend pas encore).

Renvoi Ch. 7 : §7.1 (delta AgentOps vs MLOps), §7.8 (anti-patrons).

N2 — Reproductible : observabilité de base et versionnage atomique

Caractéristiques observables

1. *Tool spans* et *LLM spans* instrumentés et centralisés dans une plateforme (Arize Phoenix, Langfuse, Datadog LLM Observability ou équivalent).
2. Artefact composite versionné comme unité atomique : tuple (prompt système, outils avec versions, configuration mémoire, périmètres de permission, seuils d'escalade) — un changement sur l'un des cinq éléments produit une nouvelle version.
3. *Retry budget* configuré sur les outils critiques (pas nécessairement sur tous les agents).
4. *Task success rate* mesuré sur les sessions récentes, même sans gate CI/CD automatisé.
5. *Kill switch* défini et documenté — pas nécessairement automatisé, mais l'équipe sait comment arrêter un agent en production.

Anti-patrons typiques de ce niveau

Les métriques existent mais ne déclenchent rien. Le dashboard montre que le *retry rate* dépasse le P95 ; personne n'a défini le seuil qui déclenche une action. Les évaluations sont lancées manuellement avant les déploiements majeurs mais pas à chaque *pull request*. Les *memory spans* et *orchestration spans* ne sont pas instrumentés — la dérive mémoire est invisible.

Critères de bascule vers N3

- Les quatre catégories de spans sont instrumentées (*LLM spans*, *tool spans*, *memory spans*, *orchestration spans*).
- Un CI/CD gate bloque le merge si le *task success rate* descend sous un seuil défini.

Renvoi Ch. 7 : §7.2 (quatre catégories de spans), §7.3 (plateformes), §7.7 (versionnage de l'artefact composite).

N3 — Instrumenté : quatre spans actifs et gate CI/CD
Caractéristiques observables

1. Les quatre catégories de spans sont opérationnelles et interrogeables : *LLM spans*, *tool spans*, *memory spans*, *orchestration spans*.
2. CI/CD gate actif sur *task success rate* — chaque *pull request* déclenche une évaluation sur le dataset canonique ; un score dégradé bloque le merge.
3. Dashboard opérationnel temps réel (métriques court terme : *retry rate*, taux d'erreur par outil, latence P50/P95/P99) distinct du dashboard qualité asynchrone (fenêtres 24h et 7 jours).
4. *Permission boundaries* déclarées explicitement pour chaque agent — liste des outils disponibles, périmètre de données, séparation dev/prod.
5. Les cinq métriques canoniques sont mesurées depuis leurs spans respectifs : *task success*, *tool correctness*, *retry budget*, *escalation quality*, *policy compliance* (Ch. 7 §7.4).

Anti-patrons typiques de ce niveau

Le *task success rate* est mesuré sur 3 à 5 tours de conversation — précisément la fenêtre qui ne détecte pas la dérive longitudinale. Les évaluations courtes sont suffisantes pour un déploiement initial, mais aveuglent sur la dégradation qui commence typiquement après 50+ interactions (Ch. 12 §12.2). Le *shadow run* n'est pas budgété : l'équipe veut valider les nouvelles versions avant promotion mais supprime l'étape faute de provision FinOps.

Critères de bascule vers N4

- Le *replay* déterministe sur *golden files* est opérationnel (scénarios issus de traces de production annotées).
- Les *shadow runs* sont budgétés et exécutés sur au moins une promotion par trimestre.
- La règle de dérive comportementale est active : alerte si le *task success rate* baisse de X % sur 7 jours glissants (pas seulement les alertes de seuil aigu).

Renvoi Ch. 7 : §7.4 (cinq métriques), §7.5 (régression continue, replay, shadow runs).

N4 — Évalué : évaluation longitudinale et plan de contrôle actif
Caractéristiques observables

1. *Replay* déterministe sur corpus de *golden files* maintenu à jour : tout changement d'outil s'accompagne d'une mise à jour du corpus.
2. *Shadow runs* systématiques pour chaque promotion vers la production — budget d'inférence provisionné explicitement.
3. Règle de dérive comportementale active : détection des dégradations lentes (pas seulement des pics aigus) sur des fenêtres glissantes de 7 jours minimum.
4. Plan de contrôle complet et appliqué : *permission boundaries*, *rate limits* sur tous les outils exposés, *retry budgets* par agent et par session, *kill switch* automatisé (arrêt en < 2 minutes sans cycle de déploiement).
5. Boucle de feedback production → dataset opérationnelle : les traces sous le seuil alimentent automatiquement le dataset de tests de développement.

Anti-patrons typiques de ce niveau

Le *kill switch* existe mais n'est pas testé régulièrement — il est découvert défaillant lors de l'incident qui devait le déclencher. La gouvernance des *golden files* est informelle : un ingénieur maintient le corpus sans processus documenté ; lors de son départ, le corpus se périme sans que l'équipe s'en aperçoive pendant plusieurs semaines. Les

agents sont promus via le pipeline CI/CD sans critères formels de promotion (*task success rate* ≥ version précédente ± marge de bruit statistique, *retry rate* ≤ seuil, zéro violation *policy compliance*).

Critères de bascule vers N5

- La promotion vers la production est entièrement automatisée sur critères formels (pas de validation manuelle dans la boucle critique).
- Le *kill switch* est testé en production sur une cadence formalisée (mensuel ou à chaque déploiement majeur).
- Les données de coût (CPST complet incluant l'*infrastructure tax*) informent les décisions de portefeuille agentique.

Renvoi Ch. 7 : §7.5 (évaluation en production), §7.6 (plan de contrôle), §7.7 (promote/deprecate/roll back).

N5 — Optimisé : cycle de vie automatisé et gouvernance de portefeuille
Caractéristiques observables

1. Promotion automatisée sur critères formels : canary → production sans validation manuelle, avec auto-revert si le taux d'erreur dépasse le seuil dans la fenêtre post-déploiement.
2. *Kill switch* testé à cadence formalisée ; auto-revert déclenché automatiquement en cas d'incident de *policy compliance* répété.
3. CPST complet (inférence + orchestration + outils + *retries* + escalade + *infrastructure tax*) calculé par agent, intégré aux décisions de portefeuille (*kill/pivot/scale* — Ch. 12 §12.7).
4. Sunset (*deprecate*) des agents géré comme une API : délai d'annonce aux consommateurs (y compris aux agents consommateurs dans les systèmes multi-agents), archivage de la version retirée comme cible de rollback d'urgence pendant une période définie.
5. Le modèle de maturité lui-même est réévalué : les seuils des cinq métriques canoniques sont recalibrés sur les données de production réelles, pas sur les estimations initiales.

Anti-patrons typiques de ce niveau

Le risque de ce niveau est l'automatisation sans supervision humaine sur les décisions à fort impact. Un auto-revert sur un incident de sécurité est correct ; un auto-revert sur une fausse alarme statistique pendant un pic de trafic légitime génère une interruption de service injustifiée. La règle de gouvernance : les décisions automatisées sont bornées aux actions réversibles ; les actions irréversibles (suppression de données, transactions financières, communications externes) maintiennent un HITL (*human-in-the-loop*) même à N5.

Renvoi Ch. 7 : §7.7, §7.9 (recommandation et question d'auto-évaluation en 5 minutes).

Tableau synthétique — 5 niveaux × 4 composantes AgentOps

Niveau	Observabilité multi-étapes	Évaluation continue	Cycle de vie artefact composite	Plan de contrôle
N1 Ad hoc	Logs bruts ou sorties finales seulement	Absente ou ponctuelle manuelle	Prompt versionné, outils non	Aucun (*retry budget*, *kill switch* inexistants)
N2 Reproductible	*Tool spans* + *LLM spans* centralisés	Manuelle pré-déploiement	Artefact composite défini comme unité atomique	*Retry budget* sur outils critiques ; *kill switch* documenté non automatisé
N3 Instrumenté	4 catégories de spans actives	CI/CD gate sur *task success* ; datasets canoniques	Pipeline de déploiement fondé sur l'artefact versionné	*Permission boundaries* + *rate limits* + seuils actifs
N4 Évalué	Dérive comportementale détectée (fenêtres glissantes)	*Replay* déterministe + *shadow runs* budgétés ; boucle feedback production	Promote/deprecate/roll back formalisés	Plan de contrôle complet ; *kill switch* automatisé
N5 Optimisé	Alerting multi-couche : seuil aigu + dérive lente + anomalie statistique	Seuils recalibrés sur données réelles ; évaluation longitudinale	Auto-revert sur critères ; sunset comme API	CPST complet intégré aux décisions de portefeuille

---

Mode d'emploi

Étape 1 — Auto-évaluation par composante. Pour chacune des quatre composantes (observabilité, évaluation, cycle de vie, plan de contrôle), identifier le niveau le plus élevé dont *toutes* les caractéristiques observables sont satisfaites. Ne pas moyenner : un profil N3/N2/N4/N1 est plus informatif qu'un « N2,5 moyen ». Cette évaluation prend 30 à 45 minutes avec les responsables techniques et opérationnels du programme.

Étape 2 — Identifier le goulot. La composante au niveau le plus bas détermine le risque opérationnel réel. Un plan de contrôle à N1 avec une observabilité à N4 reste un programme à risque critique : on voit le problème sans pouvoir l'arrêter. Prioriser la montée en niveau de la composante la plus basse avant d'investir dans les composantes avancées.

Étape 3 — Définir les critères de bascule au niveau suivant. Pour la composante goulot, les critères de bascule listés ci-dessus sont les critères de succès du prochain cycle de travail. Les transformer en *user stories* ou en tâches d'infrastructure avec un responsable nommé et une date.

Étape 4 — Intégrer les signaux faibles. Les 12 signaux d'alerte précoce de la checklist Ch. 12 §12.6 sont les indicateurs opérationnels de régression de maturité — un programme à N3 qui accumule trois signaux de la famille technique est en régression vers N2. Relier les signaux au profil de maturité dans les revues de programme.

Étape 5 — Fréquence de réévaluation. Base de référence en début de programme. Réévaluation à chaque montée en charge significative (nouveau périmètre, nouveau cas d'usage, incident en production, ou doublement du volume de sessions actives). Pour les programmes en scale actif : réévaluation trimestrielle. La stabilité du profil sur deux cycles consécutifs est le signal que la maturité est effectivement ancrée — pas que l'auto-évaluation s'est dégradée.

Renvois croisés

- Ch. 7 — AgentOps : opérer des agents longue durée — source de toutes les définitions opérationnelles des quatre composantes et des cinq métriques canoniques ; §7.9 formule la question d'auto-évaluation à 5 minutes.

- Ch. 12 — Leçons tirées des 60 % ayant échoué — §12.6 (checklist des 12 signaux faibles), §12.7 (critères *kill/pivot/scale*) ; les niveaux N1 et N2 correspondent aux profils de programmes en trajectoire d'échec documentés dans ce chapitre.
- Annexe A — Architecture Review Checklist — la checklist d'architecture AgentOps couvrant sécurité, observabilité et FinOps ; à utiliser en complément du modèle de maturité pour qualifier un déploiement spécifique.

Annexe D — Gouvernance RACI agentique

Annexe D — Governance RACI agentique Artefact opérationnel · ~1 500 mots · lecture ≈ 6 min

La matrice RACI présentée ici est l'artefact de gouvernance qui traduit les exigences du Ch. 8 et les rôles organisationnels du Ch. 11 en responsabilités nominatives par activité. Elle répond à la question que l'architecture technique ne peut pas résoudre seule : qui décide, qui exécute, qui valide, qui est informé — pour chaque acte de gouvernance d'un agent en production. Sans cette précision, les questions critiques (qui autorise le *kill switch*, qui signe l'inventaire OSFI E-23, qui traite l'escalade Loi 25) restent flottantes jusqu'à ce qu'une crise les rende urgentes.

Ce document s'adresse aux architectes d'entreprise, aux *AI ops managers*, aux responsables conformité et aux responsables de la sécurité qui mettent en place ou révisent un programme de gouvernance agentique. Il est conçu pour être adapté à l'organisation cible : les neuf rôles colonnes correspondent aux fonctions distinctes les plus courantes dans une organisation de taille intermédiaire à grande ; les douze activités lignes couvrent le cycle de vie complet d'un agent, de la qualification du cas d'usage à la formation continue des équipes. L'Annexe A (Architecture Review Checklist) et l'Annexe C (AgentOps Maturity Model) alimentent plusieurs de ces activités avec leurs propres artefacts.

Matrice RACI

Légende. R = Responsable (exécute) · A = Approbateur (rend compte, signe) · C = Consulté (apporte un avis avant la décision) · I = Informé (reçoit le résultat). Chaque ligne comporte exactement un A.

Abréviations de colonnes. CIO = Chief Information Officer · AE = Architecte d'entreprise · AIOM = *AI ops manager* (Ch. 11) · QS = *Quality steward* (Ch. 11) · HAID = *Human-AI interaction designer* (Ch. 11) · PO = Product owner · RO/DPO = Risk officer / Délégué à la protection des données personnelles (Loi 25) · SEC = Sécurité (RSSI ou équipe sécurité) · AUD = Audit interne.

Activité	CIO	AE	AIOM	QS	HAID	PO	RO/DPO	SEC	AUD
1. Qualification du cas d'usage (matrice autonomie × réversibilité × tolérance-erreur, Ch. 3)	A	R	C	C	C	R	C	I	I
2. Conception de l'architecture agentique (niveaux N1–N4, *agent policy*, flux HITL, Ch. 8 §8.1–8.2)	I	A	C	C	R	C	C	C	I
3. Décision build/buy/borrow (évaluation ROI, portabilité, Ch. 4 §4.4, Ch. 10)	A	R	C	I	I	R	C	I	I
4. Déploiement en production (instrumentation OTel, gates HITL, *permission boundaries*, Ch. 7)	I	C	R	C	C	I	I	C	I
5. Observabilité et plan de contrôle (*dashboards*, *retry budgets*, alertes dérive, Ch. 7 §7.2)	I	I	A	R	I	I	I	C	I
6. Évaluation continue en production (*task success*, *tool correctness*, *policy compliance*, Ch. 7 §7.5)	I	I	C	A	I	C	I	I	C
7. Activation du *kill switch* (arrêt d'urgence d'un agent ou d'une flotte d'agents)	C	I	R	C	I	I	C	C	I
8. Audit de la gouvernance agentique (journaux de décision, *agent policy*, Ch. 8 §8.3–8.4)	I	C	C	I	I	I	C	C	A
9. Traitement des incidents (actions hors périmètre, violations de politique, escalades non résolues)	I	C	R	C	I	I	C	C	I
10. Conformité réglementaire (EU AI Act, OSFI E-23 inventaire, Loi 25 Art. 12.1, ISO/IEC 42001, Ch. 8 §8.4)	C	C	C	I	I	I	A	C	C
11. Évolution du niveau d'autonomie (passage N1→N2→N3→N4 ou rétrogradation, *agent policy* versionnée)	C	R	C	C	I	C	C	C	I
12. Formation continue des équipes (nouveaux rôles, pratiques HITL, mises à jour réglementaires)	A	I	C	C	C	I	C	I	I

Notes d'application

Le A reste humain, quel que soit le niveau d'autonomie de l'agent. Le modèle RACI est construit sur le principe que l'approbation (*Accountability*) n'est jamais déléguée à un agent, même en N4 (*Autonome*). Un agent N4 exécute sans gate préalable dans son périmètre, mais la responsabilité de définir ce périmètre, de le versionner et de le signer reste nominativement humaine. Cette règle n'est pas une contrainte légale spécifique — c'est la condition structurelle sans laquelle la gouvernance devient fictive : si l'agent peut lui-même approuver ses propres périmètres d'action, le A de la *agent policy* n'a plus de contenu. L'Annexe A inclut cette vérification dans sa checklist de revue d'architecture.

Loi 25 Art. 12.1 — risque de *compliance washing.* L'activité 10 (Conformité réglementaire) assigne le A au RO/DPO, qui est la personne légalement responsable de la satisfaction de l'Art. 12.1 de la Loi modernisant des dispositions législatives en matière de protection des renseignements personnels (Québec, LQ 2021, c 25). L'obligation est claire : pour toute décision prise exclusivement par traitement automatisé produisant des effets significatifs sur une personne, l'organisation doit informer l'intéressé, lui expliquer la logique employée, et lui permettre de demander une révision par une personne physique. L'anti-patron est de positionner un humain nominal dans la boucle — un approbateur qui valide des centaines de décisions par heure sans examen réel — pour cocher la case légale sans satisfaire l'obligation. Ce *compliance washing* est identifiable par le taux de refus humain sur les escalades : si ce taux est inférieur à 1 % sur un volume élevé, l'humain n'est pas effectif. Le RO/DPO qui signe l'activité 10 a la responsabilité de mesurer et de documenter ce taux — pas seulement de désigner un approbateur nominal. Ce point est développé au Ch. 8 §8.2.

OSFI E-23 — inventaire AI/ML par l'*AI ops manager.* La Ligne directrice E-23 du Bureau du surintendant des institutions financières (en vigueur 1er mai 2027) exige des institutions financières fédérales canadiennes un inventaire de tous les modèles AI/ML à risque non négligeable, avec les métadonnées obligatoires : identifiant, propriétaire, version, date de déploiement, cote de risque, usages approuvés, limitations, dépendances, modèles et données tiers (Ligne directrice B-10). Dans la matrice ci-dessus, l'*AI ops manager* est le rôle R de l'activité 10 pour la production et la tenue à jour de cet inventaire — il détient la connaissance opérationnelle de l'état exact de chaque agent en production. Le RO/DPO (A sur l'activité 10) valide que l'inventaire est complet, à jour et conforme aux métadonnées exigées par E-23 avant soumission au régulateur. L'audit interne (C sur l'activité 10) vérifie indépendamment la complétude de l'inventaire lors de ses cycles annuels. Cette tripartition R/A/C prévient le scénario où l'inventaire est délégué entièrement à la conformité sans que l'opérationnel n'y contribue — ce qui produit des inventaires exacts sur la forme mais obsolètes sur le contenu.

Mode d'emploi

Étape 1 — Identifier les rôles dans l'organisation cible. Mapper les neuf colonnes de la matrice aux fonctions existantes. Dans une organisation sans *AI ops manager* formellement créé, le rôle est typiquement absorbé par le responsable des opérations IT ou l'équipe SRE ; sans *quality steward*, par l'assurance qualité ou l'équipe data. Les colonnes sans titulaire identifié sont des lacunes de gouvernance à combler avant déploiement à l'échelle — l'Annexe C fournit les critères de maturité pour évaluer si les rôles sont suffisamment définis.

Étape 2 — Adapter les activités au contexte. Les douze lignes couvrent le cycle de vie complet, mais toutes ne sont pas pertinentes à chaque déploiement. Un agent N1 (assistance, lecture seule) n'expose pas le même profil que N4 (autonome, écriture libre) : supprimer les lignes hors périmètre ou les marquer comme non applicables. Ne pas supprimer les activités 7 (*kill switch*), 9 (traitement des incidents) et 10 (conformité) — elles s'appliquent à tout agent en production, quelle que soit son autonomie.

Étape 3 — Valider avec chaque titulaire de A. Chaque approbateur de la matrice doit explicitement accepter sa responsabilité avant le déploiement. Un A non validé est un A fictif. Pour les activités à fort enjeu réglementaire (10, conformité) et opérationnel (7, *kill switch*), obtenir une validation écrite — courriel ou document signé — qui vaut engagement organisationnel, pas simple connaissance du rôle.

Étape 4 — Versionner la matrice avec la *agent policy.* La matrice RACI est un artefact de gouvernance soumis au même régime de versionnement que la *agent policy* (Ch. 8 §8.2). Toute modification des rôles ou des responsabilités déclenche une mise à jour versionnée, horodatée, avec l'identité des validateurs. Cette exigence n'est pas bureaucratique : elle est la condition pour qu'un auditeur puisse reconstituer qui était responsable de quoi au moment d'un incident.

Étape 5 — Réviser à chaque cycle AIMS PDCA. La révision annuelle imposée par ISO/IEC 42001 (phase *Check*) est le moment naturel de révision de la matrice RACI. Vérifier : les rôles ont-ils changé, les activités sont-elles encore pertinentes, les titulaires de A ont-ils les moyens effectifs d'exercer leur responsabilité ? Une matrice RACI non révisée depuis plus de 12 mois dans un environnement agentique actif est probablement déjà décalée par rapport à la réalité opérationnelle.

Renvois croisés

- Ch. 8 — Bâtir des systèmes dignes de confiance : fondements de la *agent policy*, niveaux d'autonomie N1–N4, HITL opérationnel, auditabilité, corpus réglementaire. La matrice RACI opérationnalise les exigences de ce chapitre en assignant des responsabilités nominatives.
- Ch. 11 — Redesigning Work, Not Augmenting It : définition des rôles *AI ops manager*, *quality steward*, *human-AI interaction designer*. Les colonnes AIOM, QS et HAID de la matrice correspondent directement aux rôles définis en §11.2.
- Annexe A — Architecture Review Checklist : checklist de revue d'architecture qui précède le déploiement (activité 4) et vérifie que les conditions de gouvernance sont satisfaites avant mise en production.
- Annexe C — AgentOps Maturity Model : les cinq niveaux de maturité AgentOps fournissent les critères d'évaluation de la capacité organisationnelle à exercer les responsabilités R de la matrice — notamment les activités 5 (observabilité), 6 (évaluation continue) et 9 (traitement des incidents).

Annexe E — Glossaire et lectures recommandées

Annexe E — Glossaire et lectures recommandées

Cette annexe remplit deux fonctions complémentaires. Premièrement, elle offre une porte d'entrée terminologique pour le lecteur qui aborde la monographie hors séquence ou qui doit aligner rapidement son équipe sur le lexique adopté : les définitions ci-dessous sont des définitions opérationnelles, calibrées pour décider et non pour enseigner. Deuxièmement, elle fournit une bibliographie thématique de 35 sources vérifiées, extraites des chapitres 1 à 13 et organisées par préoccupation : un CIO cherchera d'abord le thème « Gouvernance et conformité » ; un architecte commencera par « Fondations » et « Protocoles ouverts » ; un *risk officer* ira directement à « Sécurité » et « Gouvernance ». Le glossaire complet fait autorité sur toute divergence terminologique.

Section A — Glossaire récapitulatif

Les entrées suivantes condensent les définitions opérationnelles du glossaire.md. En cas de divergence entre une source externe et ces définitions, la convention de la monographie prévaut.

Agent — Entité logicielle unique exécutant la boucle *decide–act–observe* avec accès à des outils et à une mémoire persistante. Distinguer de l'assistant (pas d'outils), du *workflow* (chemin prédéfini) et du système multi-agents (plusieurs agents coordonnés).

Agentic (adjectif) — Qualifie tout système, comportement ou propriété d'un LLM exécutant la boucle *decide–act–observe*, qu'il soit à agent unique ou multiple. Convention Anthropic, adoptée dans cette monographie.

Agentic AI (nom) — Système composite multi-agents orchestré. Convention IBM. Synonyme de « système multi-agents » dans ce texte. La distinction avec « agent » isolé est architecturalement significative : le premier introduit des problèmes de coordination, de cohérence d'état et de sécurité inter-agents absents du second.

AgentOps — Discipline opérationnelle d'agents en production. Quatre composantes : (1) observabilité multi-étapes (LLM spans, *tool spans*, *memory spans*, *orchestration spans* selon OTel GenAI SemConv 1.40.0) ; (2) évaluation continue (régression, *replay* déterministe, *shadow runs*) ; (3) cycle de vie de l'artefact composite versionné comme unité atomique ; (4) plan de contrôle (*permission boundaries*, *retry budgets*, *kill switches*). Voir Ch. 7 et Annexe C.

MCP (Model Context Protocol) — Protocole ouvert d'interfaçage entre LLM et outils ou données, créé par Anthropic, gouverné par l'AAIF depuis décembre 2025. Quatre primitives : *Resources* (lecture), *Tools* (action), *Prompts* (templates), *Sampling* (LLM côté serveur). 110 M+ téléchargements SDK/mois à mai 2026. Surface d'attaque documentée : *tool poisoning*, injection via *sampling*, RCE supply chain. Voir Ch. 5.

A2A (Agent-to-Agent Protocol) — Protocole de coordination entre agents, sous Linux Foundation depuis juin 2025. Quatre primitives : *Agent Card* (capability discovery), *Task* (cycle de vie SUBMITTED → WORKING → terminal/HITL), *Message*, *Artifact*. Transports : JSON-RPC 2.0/HTTP, gRPC, REST. 150+ organisations adoptantes à mai 2026. Voir Ch. 5.

AAIF (Agentic AI Foundation) — *Directed fund* sous Linux Foundation, annoncé le 9 décembre 2025. Membres Platinum : AWS, Anthropic, Block, Bloomberg, Cloudflare, Google, Microsoft, OpenAI. MCP est projet fondateur. Anthropic conserve l'autonomie technique des mainteneurs MCP.

WebMCP — Extension navigateur de MCP, statut W3C *Candidate Recommendation* (non ratifié, mai 2026). API `navigator.modelContext`. Périmètre : *tools only*. Support : Chrome 146, Edge 147 ; Firefox et Safari sans date confirmée. Voir Ch. 5.

OSFI E-23 — Ligne directrice *Model Risk Management* du Bureau du surintendant des institutions financières (Canada), en vigueur le 1er mai 2027. Référence réglementaire applicable aux systèmes agentiques dans le secteur financier canadien, pas E-21. Voir Ch. 8.

EU AI Act — Règlement européen sur l'IA (entré en vigueur 2024, actes délégués 2025–2026). Les agents autonomes à impact élevé tombent dans les catégories à risque élevé ; les obligations de journalisation, d'auditabilité et de supervision humaine s'appliquent directement aux architectures agentiques. Voir Ch. 8.

ISO 42001 — ISO/IEC 42001:2023 — Système de management de l'IA. Certification organisationnelle (contrôles, politiques, revues) distincte de la conformité produit. ISO/IEC 42006:2025 définit les exigences pour les organismes d'audit. Voir Ch. 8.

NIST AI RMF — *AI Risk Management Framework* 1.0 + profil IA générative (NIST AI 600-1, juillet 2024). Quatre fonctions : Gouverner, Cartographier, Mesurer, Gérer. Cadre de référence pour structurer la gouvernance des risques agentiques, complémentaire à ISO 42001. Voir Ch. 8.

CPST (Cost per Successful Task) — Métrique financière agentique centrale : coût total (inférence + mémoire + outils + orchestration + retries + escalade humaine) divisé par le nombre de tâches réussies. Distingue le coût d'ingénierie (par token) du coût de décision (par résultat). Voir Ch. 2.

Retry budget — Plafond d'itérations ou de coût qu'un agent peut consommer avant escalade obligatoire vers un opérateur humain. Paramètre de contrôle du plan de contrôle AgentOps, distinct du *circuit breaker* (coupure immédiate sur seuil d'erreur). Sans *retry budget* borné, un agent peut accumuler des dépenses non intentionnelles de plusieurs dizaines de milliers de dollars en une seule session (incident documenté au Ch. 2).

Section B — Lectures recommandées par thème

1. Fondations agentiques

Sources constitutives du cadre conceptuel de la monographie.

- Yao, S. et al. (Princeton / Google Brain) — « ReAct: Synergizing Reasoning and Acting in Language Models » — arXiv / ICLR 2023 — https://arxiv.org/abs/2210.03629 — Papier fondateur de la boucle *decide–act–observe* ; les résultats empiriques (ALFWorld, WebShop) restent la référence quantitative pour justifier l'approche ReAct face aux alternatives.
- Anthropic — « Building Effective Agents » — Anthropic, décembre 2024 — https://www.anthropic.com/research/building-effective-agents — Source primaire des cinq patrons de composition agentique et du principe de simplicité comme stratégie de risque.
- Anthropic Engineering — « Effective Harnesses for Long-Running Agents » — Anthropic, 2025 — https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents — Persistance d'état, points de reprise, observabilité des agents multi-étapes.
- Russell, S. & Norvig, P. — *Artificial Intelligence: A Modern Approach*, 4e éd. — Pearson, 2020 — https://aima.cs.berkeley.edu/ — Fondement conceptuel sur les agents rationnels, PEAS et boucles de perception-action.
- Intellyx — « Why State Management Is the #1 Challenge for Agentic AI » — Intellyx, 24 février 2025 — https://intellyx.com/2025/02/24/why-state-management-is-the-1-challenge-for-agentic-ai/ — Cinq modes de défaillance *stateful* ; lecture préalable à tout dossier de risque agent.

2. Protocoles ouverts (MCP / A2A / WebMCP / AAIF)

- Linux Foundation — « Linux Foundation Announces the Formation of the Agentic AI Foundation (AAIF) » — Linux Foundation Press, 9 décembre 2025 — https://www.linuxfoundation.org/press/linux-foundation-announces-the-formation-of-the-agentic-ai-foundation — Annonce officielle de la gouvernance MCP/A2A.
- modelcontextprotocol.io — « SDKs — Model Context Protocol » — documentation officielle, mai 2026 — https://modelcontextprotocol.io/docs/sdk — Référence primaire pour l'implémentation MCP.
- A2A Protocol — « Agent2Agent Protocol Specification v1.0.0 » — a2a-protocol.org, mai 2026 — https://a2a-protocol.org/latest/specification/ — Spécification complète des quatre primitives A2A.
- DigitalApplied — « AI Agent Protocol Ecosystem Map 2026 » — DigitalApplied, mars 2026 — https://www.digitalapplied.com/blog/ai-agent-protocol-ecosystem-map-2026-mcp-a2a-acp-ucp — Cartographie comparative MCP / A2A / ACP / UCP ; source secondaire, croiser avec les spécifications primaires.
- Brosset, P. (Microsoft Edge) — « WebMCP updates, clarifications, and next steps » — patrickbrosset.com, 23 février 2026 — https://patrickbrosset.com/articles/2026-02-23-webmcp-updates-clarifications-and-next-steps/ — État du support navigateur et roadmap W3C.
- NIST / CAISI — « Announcing the AI Agent Standards Initiative for Interoperable and Secure Innovation » — NIST, 17 février 2026 — https://www.nist.gov/news-events/news/2026/02/announcing-ai-agent-standards-initiative-interoperable-and-secure — Initiative de standardisation fédérale américaine ; signale la convergence réglementaire avec les protocoles ouverts.

3. AgentOps — opérations en production

- Joshi, S. — « LLMOps, AgentOps, and MLOps for Generative AI: A Comprehensive Review » — *International Journal of Computer Applications Technology and Research*, vol. 14 n° 7, 2025 — https://ijcat.com/archieve/volume14/issue7/ijcatr14071001.pdf — Seul article académique à jour sur le delta opérationnel MLOps → AgentOps.

- OpenTelemetry — « Semantic Conventions for GenAI agent and framework spans » — SemConv 1.40.0, 17 avril 2026 — https://opentelemetry.io/docs/specs/semconv/gen-ai/gen-ai-agent-spans/ — Standard d'instrumentation de référence ; statut *Development*, API non stabilisée à mai 2026.
- Anthropic Engineering — « Demystifying evals for AI agents » — Anthropic, 2025-2026 — https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents — Régression continue et *replay* déterministe expliqués par les équipes de l'éditeur.
- N-iX — « AI agent observability: The new standard for enterprise AI in 2026 » — N-iX Engineering, 2026 — https://www.n-ix.com/ai-agent-observability/ — Synthèse praticienne sur les outils d'observabilité (LangSmith, Langfuse, Arize).
- Microsoft — « From Zero to Hero: AgentOps — End-to-End Lifecycle Management for Production AI Agents » — Azure AI Foundry Blog, 2026 — https://techcommunity.microsoft.com/blog/azure-ai-foundry-blog/from-zero-to-hero-agentops—end-to-end-lifecycle-management-for-production-ai-a/4484922 — Perspective plateforme (Azure) sur le cycle de vie *promote/deprecate/rollback*.

4. Sécurité agentique

- OWASP GenAI Security Project — « OWASP Top 10 for Agentic Applications for 2026 » — OWASP, décembre 2025 — https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/ — Taxonomie de référence des vecteurs d'attaque ; lecture obligatoire pour tout architecte sécurité.
- Cloud Security Alliance — « The Attack and Defense Landscape of Agentic AI: A Comprehensive Survey » — arXiv:2603.11088, mars 2026 — https://arxiv.org/html/2603.11088v1 — Revue systématique des mécanismes d'attaque et de défense, couvrant l'état de l'art de la recherche.
- Amit et al. (Aim Security) — « EchoLeak: The First Real-World Zero-Click Prompt Injection Exploit in a Production LLM System » — arXiv:2509.10540, septembre 2025 — https://arxiv.org/abs/2509.10540 — Exploitation en conditions réelles ; démontre la viabilité des attaques indirectes à grande échelle.
- Palo Alto Networks Unit 42 — « New Prompt Injection Attack Vectors Through MCP Sampling » — Unit 42, 2025-2026 — https://unit42.paloaltonetworks.com/model-context-protocol-attack-vectors/ — Vecteurs spécifiques à MCP ; indispensable pour toute architecture MCP en production.
- OX Security — « The Mother of All AI Supply Chains: Critical, Systemic Vulnerability at the Core of Anthropic's MCP » — OX Security, avril 2026 — https://www.ox.security/blog/the-mother-of-all-ai-supply-chains-critical-systemic-vulnerability-at-the-core-of-the-mcp/ — RCE supply chain dans les SDKs officiels MCP ; source primaire de risque documenté.
- NCCoE / NIST — « Accelerating the Adoption of Software and AI Agent Identity and Authorization » (Concept Paper) — NIST, 5 février 2026 — https://www.nccoe.nist.gov/sites/default/files/2026-02/accelerating-the-adoption-of-software-and-ai-agent-identity-and-authorization-concept-paper.pdf — Cadrage fédéral sur l'identité des agents ; fondement des architectures *least-privilege* dynamique.
- IETF — « The OAuth 2.1 Authorization Framework » (draft-ietf-oauth-v2-1) — IETF, 2025-2026 — https://datatracker.ietf.org/doc/draft-ietf-oauth-v2-1/ — Draft en cours de ratification ; socle des schémas d'autorisation per-task pour agents.

5. Gouvernance et conformité

- ISO — « ISO/IEC 42001:2023 — Artificial Intelligence Management Systems » — ISO, décembre 2023 — https://www.iso.org/standard/42001 — Norme de certification organisationnelle IA ; la plus opérationnelle pour les programmes agentiques d'entreprise.
- NIST — « AI 600-1 — Generative Artificial Intelligence Profile » — NIST, juillet 2024 — https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.600-1.pdf — Extension du AI RMF 1.0 aux systèmes génératifs et agentiques ; quatre fonctions : Gouverner, Cartographier, Mesurer, Gérer.
- OSFI — « Guideline E-23 — Model Risk Management » — OSFI, en vigueur 1er mai 2027 — https://www.osfi-bsif.gc.ca/en/guidance/guidance-library/guideline-e-23-model-risk-management-2027 — Référence réglementaire canadienne obligatoire pour le secteur financier.
- Reuel, A. et al. — « AI Agents Under EU Law: A Compliance Architecture for AI Providers » — arXiv:2604.04604, avril 2026 — https://arxiv.org/html/2604.04604v1 — Traduction juridique de l'EU AI Act en exigences d'architecture ; la source académique la plus opérationnelle disponible à mai 2026.
- FINOS — « AI Governance Framework v2.0 — MI-21 Agent Decision Audit and Explainability » — FINOS, 2026 — https://air-governance-framework.finos.org/mitigations/mi-21_agent-decision-audit-and-explainability.html — Cadre sectoriel finance ; exigences d'auditabilité des décisions agentiques.

- Raymond Chabot Grant Thornton — « Loi 25 : l'enjeu des décisions automatisées » — RCGT, 2024-2026 — https://www.rcgt.com/fr/conseils/avis-d-experts/loi-25-enjeu-decisions-automatisees/ — Application québécoise aux décisions prises par ou via des agents ; contexte canadien francophone.

6. Cas concrets et retours d'expérience

- Databricks — *State of AI Agents 2026* — Databricks, 2026 — https://www.databricks.com/resources/ebook/state-of-ai-agents — Source primaire du multiplicateur 12× (organisations *governance-first*) ; base empirique la plus large sur les pratiques de déploiement.
- Fortune — « Klarna Reverses AI Customer Service Replacement » — Fortune, 9 mai 2025 — https://fortune.com/2025/05/09/klarna-ai-humans-return-on-investment/ — Contre-exemple documenté au ROI agentique naïf : élimination puis réembauche d'agents humains.
- Shah, M. B. et al. — « Characterizing Faults in Agentic AI: A Taxonomy of Types, Symptoms, and Root Causes » — arXiv:2603.06847, mars 2026 — https://arxiv.org/abs/2603.06847 — Taxonomie académique des défaillances ; lecture structurante avant la définition d'un programme de tests de régression.
- Earezki.com — « AI Agent Observability: Lessons from the Replit Production Data Loss Incident » — mars 2026 — https://earezki.com/ai-news/2026-03-18-the-ai-agent-that-defied-a-code-freeze-deleted-1200-customer-records-and-then-lied-about-it/ — Incident documenté de suppression de 1 200 enregistrements clients ; analyse causale sur l'absence de *kill switch* et de *retry budget*.
- InfoWorld — « FinOps for agents: Loop limits, tool-call caps and the new unit economics of agentic SaaS » — InfoWorld, 2026 — https://www.infoworld.com/article/4138748/finops-for-agents-loop-limits-tool-call-caps-and-the-new-unit-economics-of-agentic-saas.html — Incident documenté de 47 000 $ de dépense non intentionnelle par boucle sans plafond ; base pour calibrer un *retry budget*.

7. Dimension organisationnelle et travail

- Deloitte AI Institute — « The State of AI in the Enterprise 2026 » — Deloitte, mars 2026 — https://www.deloitte.com/us/en/what-we-do/capabilities/applied-artificial-intelligence/content/state-of-ai-in-the-enterprise.html — Enquête principale sur l'adoption agentique en entreprise, incluant les données sur la réorganisation du travail.
- BCG / MIT Sloan Management Review — « The Emerging Agentic Enterprise: How Leaders Must Navigate a New Age of AI » — MIT SMR + BCG, novembre 2025 — https://sloanreview.mit.edu/projects/the-emerging-agentic-enterprise-how-leaders-must-navigate-a-new-age-of-ai/ — Cadre stratégique exécutif sur la transition organisationnelle ; perspective complémentaire à la dimension technique.
- Srinivasan, S. & Wei, V. — « To Thrive in the AI Era, Companies Need Agent Managers » — *Harvard Business Review*, 12 février 2026 — https://hbr.org/2026/02/to-thrive-in-the-ai-era-companies-need-agent-managers — Définit le rôle d'*AI ops manager* ; source de référence sur les nouveaux rôles organisationnels.
- Stanford HAI — « The 2026 AI Index Report » — Stanford Human-Centered AI, 2026 — https://hai.stanford.edu/ai-index/2026-ai-index-report — Panorama annuel quantitatif de l'adoption, de l'emploi et des politiques IA ; source secondaire de référence pour contextualisation macro.

8. Perspectives et horizon 2027–2030

- Gartner — « 2026 Hype Cycle for Agentic AI » — Gartner, 2026 — https://www.gartner.com/en/articles/hype-cycle-for-agentic-ai — Positionnement de l'*agentic AI governance*, de l'*agentic AI security* et du *FinOps agentique* sur le cycle 2026.
- McKinsey — « Building the Foundations for Agentic AI at Scale » — McKinsey Technology, 2026 — https://www.mckinsey.com/capabilities/mckinsey-technology/our-insights/building-the-foundations-for-agentic-ai-at-scale — Cadre d'exécution à l'échelle ; biais commercial McKinsey à signaler, mais les checklists de préparation organisationnelle sont actionnables.
- Forrester — « How Agentic AI Elevates The Enterprise Architect's Role » — Forrester, 2025-2026 — https://www.forrester.com/blogs/the-future-of-the-enterprise-architects-job/ — Repositionnement du rôle d'architecte d'entreprise face à l'*agentic AI* ; lecture recommandée pour les équipes d'architecture.
- Fujitsu — « Fujitsu develops multi-AI agent collaboration technology to optimize supply chains » — Fujitsu Global PR, décembre 2025 — https://global.fujitsu/en-global/pr/news/2025/12/01-02 — Signal industriel : déploiement de coordination multi-agents en chaînes d'approvisionnement manufacturing.

Mode d'emploi — Parcours par rôle

Étape 1 — Identifier son rôle prioritaire parmi les quatre profils du livre (CIO, architecte, *product owner*, *risk officer*). Chaque rôle a un point d'entrée différent dans cette annexe et dans la monographie.

- *CIO* : lire d'abord les thèmes 6 (cas concrets) et 7 (organisationnel), puis 5 (gouvernance). Les chapitres de référence sont Ch. 2, Ch. 11 et Ch. 13.
- *Architecte d'entreprise ou de solution* : commencer par les thèmes 1 (fondations) et 2 (protocoles), puis 3 (AgentOps). Les chapitres de référence sont Ch. 1, Ch. 5, Ch. 6 et Ch. 7.
- *Product owner* : aller directement aux thèmes 6 (cas concrets) et 8 (horizon), avec les chapitres Ch. 3 et Ch. 4 comme base de priorisation.
- *Risk officer* : thèmes 4 (sécurité) et 5 (gouvernance) d'abord, puis 6 pour les incidents documentés. Chapitres de référence : Ch. 8, Ch. 9 et Ch. 12.

Étape 2 — Consulter le glossaire.md complet avant de lire les chapitres techniques (Ch. 5 à Ch. 9) : les conventions terminologiques — notamment la distinction *agent* / *agentic AI* et la convention réglementaire OSFI E-23 — conditionnent la lecture correcte des recommandations architecturales.

Étape 3 — Tracer les renvois croisés au fur et à mesure : chaque chapitre de la monographie contient une section « Pour aller plus loin » avec 3 à 5 sources sélectives commentées, et la section `## Références` complète. Les sources de la présente annexe en sont la synthèse transversale ; retourner au chapitre source pour le contexte complet de chaque citation.

Renvois croisés : tous les chapitres (Ch. 1 à Ch. 13), Annexe A, Annexe B, Annexe C, Annexe D, glossaire.md.

Glossaire principal

Glossaire — *The Agentic Enterprise*

Termes techniques retenus, avec définition opérationnelle et politique de traduction. Ajouter une entrée à la première occurrence d'un terme dans un chapitre.

Conventions

- **Anglais conservé** (italique à la première occurrence par chapitre) : *event-driven*, *idempotency*, *exactly-once*, *backpressure*, *circuit breaker*, *saga*, *agentic*, *prompt injection*, *tool calling*, *retry budget*, *escalation cost*.
- **Traduit** : *agent* → agent, *workflow* → flux de travail, *deployment* → déploiement.

Convention éditoriale — terme « agentic »
Les sources primaires divergent (IBM : *agentic AI* = système multi-agents orchestré ; Anthropic : *agentic* = adjectif qualifiant tout LLM semi-autonome, y compris à agent unique). La monographie adopte une **convention hybride** :

- « **agentic** » (adjectif, italique) — qualifie tout système, comportement ou propriété d'un LLM exécutant la boucle *decide–act–observe* avec mémoire et outils, qu'il soit à agent unique ou multiple. Suivit Anthropic.
- « ***agentic AI*** » (nom, italique) — désigne spécifiquement un système composite multi-agents orchestré. Suivit IBM.
- « **agent** » (nom, romain) — entité unique dans la boucle.
- « **système multi-agents** » — explicite la pluralité ; synonyme de *agentic AI* dans cette monographie.

À introduire au §2 de l'introduction et à respecter dans tous les chapitres.

Convention réglementaire — OSFI
La référence OSFI pertinente pour le contexte agentique en 2026 est **OSFI E-23** (*Model Risk Management*, en vigueur 1er mai 2027), pas E-21. Les sources primaires (BLG, Blakes) ne confirment que E-23 dans le périmètre IA.

Entrées (alphabétique)

A2A (Agent-to-Agent Protocol)
Protocole ouvert de coordination entre agents, sous Linux Foundation depuis juin 2025. Spec v1.0.0 (date de release officielle *à vérifier*). Quatre primitives : *Agent Card* (capability discovery), *Task* (cycle de vie SUBMITTED → WORKING → terminal/intervention HITL), *Message*, *Artifact*. Transports : JSON-RPC 2.0/HTTP, gRPC, REST. Adoption mai 2026 : 150+ organisations, déployé dans Azure AI Foundry, Copilot Studio, Bedrock AgentCore Runtime, Salesforce, SAP, ServiceNow.

AAIF (Agentic AI Foundation)
Directed fund annoncé 9 décembre 2025 sous l'égide de la Linux Foundation. Membres Platinum fondateurs : AWS, Anthropic, Block, Bloomberg, Cloudflare, Google, Microsoft, OpenAI. MCP est projet fondateur. Distinction critique : MCP n'a pas été « donné à la Linux Foundation » au même titre que le noyau Linux — l'AAIF est l'entité de gouvernance, et Anthropic conserve l'autonomie technique des mainteneurs MCP.

AgentOps
Discipline opérationnelle d'agents en production — distincte de MLOps et LLMOps par la nature de l'objet opéré : un système qui agit dans l'environnement réel, pas seulement un modèle qui prédit. Quatre composantes : (1) **observabilité multi-étapes** — arbre de *spans* hétérogènes (LLM spans, *tool spans*, *memory spans*, *orchestration spans*) ; (2) **évaluation continue** — *régression continue* (CI/CD gate), *replay* déterministe (golden file testing), *shadow runs* (parallèle de production sans exposition utilisateur) ; (3) **cycle de vie de l'artefact composite** — tuple (prompt système, outils avec versions, configuration mémoire, périmètres de permission, seuils d'escalade) versionné comme unité atomique (*promote/deprecate/roll back*) ; (4) **plan de contrôle** — *permission boundaries*, *rate limits*, *retry budgets*, *kill switches* — couche d'application distincte de l'observabilité. Standard d'instrumentation : OTel GenAI SemConv 1.40.0 (statut *Development*, attributs `gen_ai.agent.*`, API non stabilisée). Référence : Ch. 7 ; modèle de maturité à 5 niveaux : Annexe C.

CPST (Cost per Successful Task)
Métrique économique unitaire centrale du livre : coût total (tokens d'inférence + outils + escalade humaine éventuelle + *retry*) divisé par le nombre de tâches **menées à terme avec succès** selon la définition opérationnelle du *successful outcome*. À distinguer du coût par appel ou par token. Référence : Ch. 2 §2.2, Ch. 4 §4.6. Composante *infrastructure tax* (orchestration, mémoire, escalade, *retry*) : 30 à 50 % du budget agentique réel selon les praticiens FinOps 2026.

Retry budget
Plafond explicite — par tâche, par tenant ou par fenêtre temporelle — du nombre de tentatives qu'un agent peut consommer avant escalade ou abandon. Sans *retry budget* opposable, un agent en boucle peut générer un coût non borné (cas InfoWorld 2026 : 47 000 $ d'appels non plafonnés). À coupler à un *kill switch* financier dans le plan de contrôle AgentOps. Référence : Ch. 2 §2.3, Ch. 7 §7.6, Annexe A.

MCP (Model Context Protocol)
Protocole ouvert d'interfaçage entre LLM et outils/données, créé par Anthropic et **projet fondateur de l'AAIF** lors de la création de la fondation le 9 décembre 2025 (à distinguer d'une donation au sens d'un transfert complet de gouvernance technique : Anthropic conserve l'autonomie des mainteneurs). Quatre primitives : *Resources* (lecture), *Tools* (action), *Prompts* (templates), *Sampling* (LLM côté serveur). Architecture client-hôte-serveur. SDKs Tier 1 (TypeScript, Python, C#, Go) ; Tier 2 (Java, Rust) ; Tier 3 (Swift, Ruby, PHP) ; TBD (Kotlin). 110M+ téléchargements/mois de SDK mai 2026. Surface d'attaque documentée : *tool poisoning*, injection via *sampling* (Palo Alto Unit 42), RCE supply chain dans SDKs officiels (OX Security, avril 2026).

WebMCP
Extension navigateur de MCP, statut W3C *Candidate Recommendation* (non ratifié, mai 2026). API : `navigator.modelContext`. Périmètre : *tools only* (pas *resources* ni *prompts*). Support : Chrome 146 stable, Edge 147 stable, Firefox 8-12 semaines (*à vérifier*), Safari sans engagement public. Co-développé Microsoft / Google via W3C WebML Working Group.

Bibliographie consolidée

Bibliographie globale — *The Agentic Enterprise*

Bibliographie consolidée. Chaque chapitre maintient sa propre section `## Références` exhaustive ; ce fichier est un **index thématique** des sources les plus structurantes, pointant vers le chapitre où chacune est citée et discutée.

Format
auteur — titre — éditeur — date — URL — accédée le YYYY-MM-DD

Sources primaires éditeurs (Anthropic, OpenAI, Google, Microsoft, IBM, AWS)

- Anthropic — « Building Effective Agents » — 2024 — https://www.anthropic.com/research/building-effective-agents — accédée le 2026-05-05 — *cité dans Intro, Ch. 1, Ch. 6, Ch. 14*
- Anthropic Engineering — « Demystifying evals for AI agents » — 2025-2026 — https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents — accédée le 2026-05-05 — *cité dans Ch. 4, Ch. 7*
- Anthropic Engineering — « Effective context engineering for AI agents » — 2025 — https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents — accédée le 2026-05-05 — *cité dans Ch. 6, Ch. 14*
- Anthropic Engineering — « Writing effective tools for AI agents » — 2025 — https://www.anthropic.com/engineering/writing-tools-for-agents — accédée le 2026-05-05 — *cité dans Ch. 6, Ch. 14*
- OpenAI — « Introducing AgentKit » et « New tools for building agents » — 2025-2026 — https://openai.com/index/new-tools-for-building-agents/ — accédée le 2026-05-05 — *cité dans Intro, Ch. 6*
- Microsoft — « Powering Frontier Transformation with Copilot and agents » — 9 mars 2026 — https://www.microsoft.com/en-us/microsoft-365/blog/2026/03/09/powering-frontier-transformation-with-copilot-and-agents/ — accédée le 2026-05-05 — *cité dans Intro, Ch. 10*
- Microsoft Foundry — « Introducing Microsoft Agent Framework » — avril 2026 — https://devblogs.microsoft.com/foundry/introducing-microsoft-agent-framework-the-open-source-engine-for-agentic-ai-apps/ — accédée le 2026-05-05 — *cité dans Ch. 6, Ch. 10*
- Microsoft TechCommunity — « From Zero to Hero: AgentOps — End-to-End Lifecycle Management » — 2026 — https://techcommunity.microsoft.com/blog/azure-ai-foundry-blog/from-zero-to-hero-agentops—end-to-end-lifecycle-management-for-production-ai-a/4484922 — accédée le 2026-05-05 — *cité dans Ch. 7, Ch. 10*
- AWS — « Introducing Amazon Bedrock AgentCore » — 2025-2026 — https://aws.amazon.com/blogs/aws/introducing-amazon-bedrock-agentcore-securely-deploy-and-operate-ai-agents-at-any-scale/ — accédée le 2026-05-05 — *cité dans Ch. 2, Ch. 7, Ch. 9, Ch. 10*
- IBM — « The 2026 Guide to AI Agents » — 2026 — https://www.ibm.com/think/ai-agents — accédée le 2026-05-05 — *cité dans Intro, Ch. 1*
- Google Cloud — « AI Agent Trends 2026 » — 2026 — https://cloud.google.com/resources/content/ai-agent-trends-2026 — accédée le 2026-05-05 — *cité dans Ch. 11, Ch. 13*
- Google Cloud — « Agent2Agent protocol is getting an upgrade » — 31 juillet 2025 — https://cloud.google.com/blog/products/ai-machine-learning/agent2agent-protocol-is-getting-an-upgrade — accédée le 2026-05-05 — *cité dans Ch. 5*

Standards et protocoles ouverts

- modelcontextprotocol.io — Documentation officielle MCP (SDK, primitives) — 2024-2026 — https://modelcontextprotocol.io/ — accédée le 2026-05-05 — *cité dans Ch. 5, Ch. 14*
- a2a-protocol.org — « A2A Protocol Specification v1.0.0 » — 2026 — https://a2a-protocol.org/latest/specification/ — accédée le 2026-05-05 — *cité dans Ch. 5, Ch. 9, Ch. 10*
- Linux Foundation / AAIF — « Formation of the Agentic AI Foundation » — 9 décembre 2025 — https://www.linuxfoundation.org/press/linux-foundation-announces-the-formation-of-the-agentic-ai-foundation — accédée le 2026-05-05 — *cité dans Ch. 5, Ch. 13*

- AAIF Blog — « MCP Is Now Enterprise Infrastructure » — avril 2026 — https://aaif.io/blog/mcp-is-now-enterprise-infrastructure-everything-that-happened-at-mcp-dev-summit-north-america-2026/ — accédée le 2026-05-05 — *cité dans Ch. 5*
- Patrick Brosset (Microsoft Edge) — « WebMCP updates, clarifications, and next steps » — 23 février 2026 — https://patrickbrosset.com/articles/2026-02-23-webmcp-updates-clarifications-and-next-steps/ — accédée le 2026-05-05 — *cité dans Ch. 5*
- OpenTelemetry — « Semantic Conventions for GenAI agent and framework spans » (SemConv 1.40.0) — avril 2026 — https://opentelemetry.io/docs/specs/semconv/gen-ai/gen-ai-agent-spans/ — accédée le 2026-05-05 — *cité dans Ch. 4, Ch. 7, Ch. 10*
- IETF — AAuth (draft-patwhite-aauth-00) — https://www.ietf.org/archive/id/draft-patwhite-aauth-00.html — accédée le 2026-05-05 — *cité dans Ch. 9*

Sécurité agentique — primaires

- Aim Security / arXiv:2509.10540 — « EchoLeak (CVE-2025-32711) » — sept. 2025 — https://arxiv.org/abs/2509.10540 — accédée le 2026-05-05 — *cité dans Ch. 9*
- NVD — CVE-2025-32711 (EchoLeak Microsoft 365 Copilot) — patché juin 2025 — https://nvd.nist.gov/vuln/detail/cve-2025-32711 — accédée le 2026-05-05 — *cité dans Ch. 9*
- Palo Alto Networks Unit 42 — « Fooling AI Agents: Web-Based Indirect Prompt Injection Observed in the Wild » — 3 mars 2026 — https://unit42.paloaltonetworks.com/ai-agent-prompt-injection/ — accédée le 2026-05-05 — *cité dans Intro, Ch. 5, Ch. 9*
- Palo Alto Networks Unit 42 — « New Prompt Injection Attack Vectors Through MCP Sampling » — 2025-2026 — https://unit42.paloaltonetworks.com/model-context-protocol-attack-vectors/ — accédée le 2026-05-05 — *cité dans Ch. 5, Ch. 9*
- OX Security — « The Mother of All AI Supply Chains » (RCE MCP) — avril 2026 — https://www.ox.security/blog/the-mother-of-all-ai-supply-chains-critical-systemic-vulnerability-at-the-core-of-the-mcp/ — accédée le 2026-05-05 — *cité dans Ch. 5, Ch. 9, Ch. 14*
- OWASP GenAI — « Top 10 for Agentic Applications 2026 » — déc. 2025 — https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/ — accédée le 2026-05-05 — *cité dans Ch. 9*
- arXiv:2505.02077 — « Open Challenges in Multi-Agent Security » — mai 2025 — https://arxiv.org/html/2505.02077 — accédée le 2026-05-05 — *cité dans Ch. 9*
- arXiv:2603.11088v1 — « The Attack and Defense Landscape of Agentic AI: A Comprehensive Survey » — mars 2026 — https://arxiv.org/html/2603.11088v1 — accédée le 2026-05-05 — *cité dans Intro*

Gouvernance et conformité

- artificialintelligenceact.eu — Implementation Timeline EU AI Act — 2024-2026 — https://artificialintelligenceact.eu/implementation-timeline/ — accédée le 2026-05-05 — *cité dans Ch. 8*
- ISO/IEC 42001:2023 — AI Management System (AIMS) — 2023 — https://www.iso.org/standard/42001 — accédée le 2026-05-05 — *cité dans Ch. 8*
- NIST AI 600-1 — Generative AI Profile — 2024 — https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.600-1.pdf — accédée le 2026-05-05 — *cité dans Ch. 8*
- NIST CAISI — « Announcing the AI Agent Standards Initiative » — 17 février 2026 — https://www.nist.gov/news-events/news/2026/02/announcing-ai-agent-standards-initiative-interoperable-and-secure — accédée le 2026-05-05 — *cité dans Intro, Ch. 5, Ch. 8*
- OSFI — « Guideline E-23 — Model Risk Management » (en vigueur 1er mai 2027) — https://www.osfi-bsif.gc.ca/en/guidance/guidance-library/guideline-e-23-model-risk-management-2027 — accédée le 2026-05-05 — *cité dans Intro, Ch. 8, Ch. 10*
- BLG — « AI in Canada in 2026 across economic sectors » — mars 2026 — https://www.blg.com/insights/2026/03/a-turning-point-for-ai-in-canada-in-2026 — accédée le 2026-05-05 — *cité dans Intro, Ch. 8*
- FINOS AIGF v2.0 — « MI-21 Agent Decision Audit » — 2026 — https://air-governance-framework.finos.org/mitigations/mi-21_agent-decision-audit-and-explainability.html — accédée le 2026-05-05 — *cité dans Ch. 8*

Analystes et études (Gartner, Deloitte, BCG, MIT, McKinsey, Forrester, IDC)

- Gartner — « 2026 Hype Cycle for Agentic AI » — 2026 — https://www.gartner.com/en/articles/hype-cycle-for-agentic-ai — accédée le 2026-05-05 — *cité dans Intro, Ch. 5, Ch. 7, Ch. 8, Ch. 12, Ch. 13*

- Gartner — « Over 40% of Agentic AI Projects Will Be Canceled by End of 2027 » — 25 juin 2025 — https://www.gartner.com/en/newsroom/press-releases/2025-06-25-gartner-predicts-over-40-percent-of-agentic-ai-projects-will-be-canceled-by-end-of-2027 — accédée le 2026-05-05 — *cité dans Ch. 2, Ch. 12*
- Gartner — « Lack of AI-Ready Data Puts AI Projects at Risk » — 26 février 2025 — https://www.gartner.com/en/newsroom/press-releases/2025-02-26-lack-of-ai-ready-data-puts-ai-projects-at-risk — accédée le 2026-05-05 — *cité dans Ch. 4, Ch. 12*
- Deloitte — « State of AI in the Enterprise 2026 » — 2026 — https://www.deloitte.com/us/en/what-we-do/capabilities/applied-artificial-intelligence/content/state-of-ai-in-the-enterprise.html — accédée le 2026-05-05 — *cité dans Intro, Ch. 11*
- Deloitte — « Tech Trends 2026 : The agentic reality check » — 2026 — https://www.deloitte.com/us/en/insights/topics/technology-management/tech-trends/2026/agentic-ai-strategy.html — accédée le 2026-05-05 — *cité dans Intro, Ch. 11*
- Writer / Workplace Intelligence — « Enterprise AI Adoption 2026 » — avril 2026 — https://writer.com/blog/enterprise-ai-adoption-survey-results-press-release/ — accédée le 2026-05-05 — *cité dans Intro, Ch. 11*
- BCG / MIT SMR — « The Emerging Agentic Enterprise » — nov. 2025 — https://sloanreview.mit.edu/projects/the-emerging-agentic-enterprise-how-leaders-must-navigate-a-new-age-of-ai/ — accédée le 2026-05-05 — *cité dans Ch. 11*
- McKinsey — « Seizing the Agentic AI Advantage » — 2025 — *cité dans Ch. 3, Ch. 12*
- McKinsey — « Building the Foundations for Agentic AI at Scale » — 2026 — https://www.mckinsey.com/capabilities/mckinsey-technology/our-insights/building-the-foundations-for-agentic-ai-at-scale — accédée le 2026-05-05 — *cité dans Ch. 12*
- HBR — « To Thrive in the AI Era, Companies Need Agent Managers » — 12 février 2026 — https://hbr.org/2026/02/to-thrive-in-the-ai-era-companies-need-agent-managers — accédée le 2026-05-05 — *cité dans Ch. 11*
- HBR — « Why Agentic AI Projects Fail » — octobre 2025 — https://hbr.org/2025/10/why-agentic-ai-projects-fail-and-how-to-set-yours-up-for-success — accédée le 2026-05-05 — *cité dans Ch. 12*
- Databricks — « State of AI Agents 2026 » — 2026 — https://www.databricks.com/blog/enterprise-ai-agent-trends-top-use-cases-governance-evaluations-and-more — accédée le 2026-05-05 — *cité dans Ch. 2, Ch. 12*
- Fivetran — « 2026 Agentic AI Readiness Index » — 5 mai 2026 — https://www.fivetran.com/blog/85-of-enterprises-are-running-agentic-ai-on-a-data-foundation-that-isnt-ready — accédée le 2026-05-05 — *cité dans Ch. 4*
- KPMG — « Agentic AI Untangled: Build, Buy, or Borrow Decision » — 2026 — https://kpmg.com/us/en/articles/2026/agentic-ai-untangled.html — accédée le 2026-05-05 — *cité dans Ch. 4*
- MIT CISR — « Update on the Enterprise AI Maturity Model » — 2025 — https://cisr.mit.edu/publication/2025_0801_EnterpriseAIMaturityUpdate_WoernerSebastianWeillKaganer — accédée le 2026-05-05 — *cité dans Ch. 4*
- Stanford HAI — « 2026 AI Index Report » — 2026 — https://hai.stanford.edu/ai-index/2026-ai-index-report — accédée le 2026-05-05 — *cité dans Ch. 11*

Sources académiques

- arXiv:2210.03629 — « ReAct: Synergizing Reasoning and Acting » — Yao et al., 2022-2023 — https://arxiv.org/abs/2210.03629 — accédée le 2026-05-05 — *cité dans Ch. 1*
- arXiv:2512.12791 — « Beyond Task Completion: An Assessment Framework for Evaluating Agentic AI Systems » — Akshathala et al. (SERC IIIT-Hyderabad / MontyCloud), décembre 2025 — https://arxiv.org/abs/2512.12791 — accédée le 2026-05-05 — *cité dans Ch. 4*
- arXiv:2512.13564 — « Memory in the Age of AI Agents: A Survey » — Hu et al., décembre 2025 — https://arxiv.org/abs/2512.13564 — accédée le 2026-05-05 — *cité dans Ch. 6*
- arXiv:2604.04604 — « AI Agents Under EU Law » — 2026 — https://arxiv.org/html/2604.04604v1 — accédée le 2026-05-05 — *cité dans Ch. 8*

Cas concrets et incidents (post-mortems publics)

- Fortune — Klarna AI reversal — mai 2025 — *cité dans Ch. 2, Ch. 11*
- Fortune — Replit AI agent incident — 23 juillet 2025 — *cité dans Ch. 1, Ch. 7, Ch. 12*

- earezki.com — « AI Agent Observability: Lessons from the Replit Production Data Loss Incident » — 18 mars 2026 — https://earezki.com/ai-news/2026-03-18-the-ai-agent-that-defied-a-code-freeze-deleted-1200-customer-records-and-then-lied-about-it/ — accédée le 2026-05-05 — *cité dans Ch. 7*
- Bank of America Newsroom — Erica métriques Q1 2026 — mars 2026 — https://newsroom.bankofamerica.com/content/newsroom/press-releases/2026/03/bofa-ai-and-digital-innovations-fuel-30-billion-client-interacti.html — accédée le 2026-05-05 — *cité dans Ch. 3, Ch. 4*
- Salesforce Customer Stories — Wiley 213 % ROI — 2024-2025 — https://www.salesforce.com/customer-stories/wiley/ — accédée le 2026-05-05 — *cité dans Ch. 3, Ch. 4*

Plateformes et outils (frameworks, observabilité, mémoire, sandbox)

- Mem0 — « State of AI Agent Memory 2026 » — 2026 — https://mem0.ai/blog/state-of-ai-agent-memory-2026 — accédée le 2026-05-05 — *cité dans Ch. 6*
- DigitalApplied — « Agent Observability: LangSmith, Langfuse, Arize 2026 » — 2026 — https://www.digitalapplied.com/blog/agent-observability-platforms-langsmith-langfuse-arize-2026 — accédée le 2026-05-05 — *cité dans Ch. 7*
- Northflank — « AI Agent Sandboxing Guide 2026 » — 2026 — https://northflank.com/blog/top-ai-sandbox-platforms-for-code-execution — accédée le 2026-05-05 — *cité dans Ch. 9*

Pour la bibliographie exhaustive d'un chapitre, se référer à sa section `## Références` dédiée. L'Annexe E propose une sélection commentée par thème.